HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Information and communications technology

Software Design Document

Version 1.3

AIMS Project

Subject: ITSS Software Development

Group 16

Nguyễn Huy Diễn – 20235910

Nguyễn Tiến Đạt – 20239715

Lê Hoàng Kiên – 20235958

Phạm Đức Phước – 20235988

Hanoi, June 2026

Table of Contents

List of Figures

List of Tables

1 Introduction

1.1 Objective

The purpose of this Software Design Document (SDD) is to describe the architectural design, detailed design, and implementation strategies of the AIMS (An Internet Media Store) system. This document presents the software structure, design decisions, data organization, class design, interaction mechanisms, and architectural patterns used to develop the system.

The SDD serves as a technical blueprint for the implementation and maintenance of the AIMS system. It provides detailed descriptions of the system architecture, database design, user interface design, security mechanisms, and software components to ensure that the software is developed consistently and systematically according to the requirements specified in the Software Requirements Specification (SRS).

In addition, this document explains the design considerations adopted to improve software quality attributes such as maintainability, extensibility, scalability, low coupling, and high cohesion. It also presents the application of SOLID principles and software design patterns to support future requirement changes, including the integration of new product types, payment gateways, shipping fee calculation strategies, and notification delivery methods.

The intended audiences of this document include:

- Developers and maintainers who implement, extend, and maintain the system;
- Testers and quality assurance personnel who validate the correctness of the implementation;
- Project supervisors and instructors who evaluate the software architecture and design quality;
- Future developers and stakeholders who need to understand the internal structure and design rationale of the system.

This document acts as a reference throughout the software development lifecycle and supports future system enhancement and refactoring activities.

1.2 Scope

The software product to be described in this document is called AIMS (An Internet Media Store), an e-commerce software that supports the buying and selling of physical media products. The system supports several types of physical media products including Books, Newspapers, CDs, and DVDs. Customers can browse for products, add products to a virtual shopping cart, and place orders for their purchases. Besides, the system also provides administrative features for users to manage products, orders, and users. In particular, the Product Manager can add, update, remove products, or manage stock, whereas the Administrator can manage user accounts and system roles.

The system aims to provide a convenient platform for purchasing media products while ensuring efficient product management and order processing. It also integrates payment services to support secure transaction processing.

1.3 Glossary

<Listing and explaining the terms appearing in the software's profession and this document. Any assumption of the reader's prior knowledge or experience on the subject is ill advised>

#	**Term**	**Explanation**
1	AIMS	An Internet Media Store, the e-commerce system described in this document, supporting the retail of physical media products.
2	SDD	Software Design Document, this document. Describes the architectural and detailed design of AIMS, bridging the SRS and the implementation.
3	SRS	Software Requirements Specification, the preceding document that defines the functional and non-functional requirements of AIMS.
4	UC	Use Case, a description of a specific interaction between an actor and the system to achieve a goal (e.g., UC001 Place Order).
5	API	Application Programming Interface, the set of HTTP endpoints exposed by the AIMS backend that the frontend and external systems communicate with.
6	REST	Representational State Transfer, the architectural style used for the AIMS HTTP API, where resources are addressed by URL and operations are expressed via HTTP verbs (GET, POST, PUT, DELETE).
7	DTO	Data Transfer Object, a plain object used to carry data between the frontend and backend across the HTTP boundary, distinct from the domain entity it represents.
8	JPA	Java Persistence API, the Java specification for object-relational mapping, used in AIMS via its Hibernate implementation to map domain entities to PostgreSQL tables.

9	ORM	Object-Relational Mapping, the technique of mapping Java objects to relational database tables, handled in AIMS by Hibernate.
10	AOP	Aspect-Oriented Programming, a programming paradigm used in AIMS to implement cross-cutting concerns (specifically audit logging) via advice without embedding that logic in business methods.
11	SOLID	An acronym for five object-oriented design principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion. These principles are a primary evaluation criterion for this project.
12	SRP	Single Responsibility Principle, the first of the SOLID principles, stating that a class should have only one reason to change.
13	OCP	Open/Closed Principle, the second of the SOLID principles, stating that a class should be open for extension but closed for modification.
14	LSP	Liskov Substitution Principle, the third of the SOLID principles, stating that subtypes must be substitutable for their base types without altering program correctness.
15	ISP	Interface Segregation Principle, the fourth of the SOLID principles, stating that no class should be forced to implement methods it does not use.
16	DIP	Dependency Inversion Principle, the fifth of the SOLID principles, stating that high-level modules should depend on abstractions, not on concrete implementations.
17	GoF	Gang of Four, refers to the four authors of the seminal book Design Patterns: Elements of Reusable Object-Oriented Software (Gamma et al., 1994), which catalogues the classical design patterns referenced in Section 5.4.
18	Product Manager	A human actor in AIMS responsible for managing the product catalog (creating, updating, deleting products, adjusting stock) and processing orders (approving or rejecting).

19	Administrator	A human actor in AIMS responsible for managing user accounts, assigning roles, and performing system administration tasks.
20	Customer	An anonymous human actor in AIMS who browses products, manages a cart, and places orders. No authentication is required.
21	VAT	Value Added Tax, a fixed 10% tax applied to all product prices in AIMS when computing order totals.
22	VietQR	A Vietnamese QR-code-based bank transfer payment standard integrated into AIMS as one of two supported payment methods.
23	PayPal	An international online payment platform integrated into AIMS via its Sandbox API as the second supported payment method, handling credit card payments.
24	Sandbox	A test environment provided by a third-party service (VietQR, PayPal) that simulates real API behaviour without processing actual financial transactions. Used exclusively in AIMS for development and demonstration.
25	JWT	JSON Web Token, a signed, self-contained token issued by the AIMS backend on successful login and attached by the frontend to subsequent requests to authenticate Product Manager and Administrator sessions.
26	Entity	A persistent domain object mapped to a database table via JPA (e.g., Product, Order, User).
27	Aggregate	A cluster of domain entities treated as a single unit for data consistency purposes. In AIMS, Order is the primary aggregate root, owning OrderItem, DeliveryInformation, Invoice, and PaymentTransaction.
28	Registry	A design mechanism used throughout AIMS where a map keyed by a type discriminator (enum value or class reference) resolves the correct strategy, factory, or handler at runtime without switch statements.
29	Strategy	A GoF design pattern in which a family of interchangeable algorithms is encapsulated behind a common interface, allowing the algorithm to be selected at runtime. Used in AIMS for delivery fee calculation, product type dispatch, notification routing, and refund dispatch.

30	Template Method	A GoF design pattern in which an abstract class defines the skeleton of a shared behaviour and provides utility methods that subclasses call at the appropriate points in their own flow. Used in AIMS in PaymentService.
31	Builder	A GoF design pattern in which a separate builder object accumulates the parameters needed to construct a complex object. Used in AIMS for Product entity construction and partial update.
32	Observer	A GoF design pattern in which a subject publishes events and independent observers react to them. Used in AIMS via Spring's ApplicationEventPublisher and @TransactionalEventListener for order lifecycle side effects.
33	Factory	A GoF design pattern in which a dedicated object is responsible for constructing instances of another class. Used in AIMS for notification message creation.
34	Flyway	A database migration tool used in AIMS to manage PostgreSQL schema evolution through versioned SQL scripts.
35	MapStruct	A Java annotation processor used in AIMS to generate type-safe DTO-to-entity mappers at compile time for non-polymorphic mappings.
36	Supabase	A cloud-based backend-as-a-service platform. Used in AIMS exclusively for its object storage capability to host product images.
37	YAGNI	You Aren't Gonna Need It, a software development principle advising against implementing functionality speculatively before a concrete requirement exists. Applied consistently in AIMS to defer abstractions until two real cases exist to justify them.
38	DRY	Don't Repeat Yourself, a principle stating that every piece of knowledge should have a single, unambiguous representation in the codebase. Applied in AIMS via the Template Method pattern and shared registry infrastructure.
39	Snapshot	A copy of a value taken at a specific point in time, independent of future changes to the source. Used in AIMS for OrderItem

		(product price and title at checkout) and Invoice (financial totals at the time of order placement).
40	Post-commit	Referring to logic that executes after a database transaction has been successfully committed. In AIMS, notification and refund side effects run post-commit.

1.4 References

Centers for Medicare & Medicaid Services. (n.d.). *System Design Document Template.* Retrieved from Centers for Medicare & Medicaid Services: https://www.cms.gov/Research-Statistics-Data-and-Systems/CMS-Information-Technology/XLC/Downloads/SystemDesignDocument.docx

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

NGUYEN, T. T. T. (2026). *ITSS Software Development Lab Assignment 11: Coupling and Cohesion*. School of Information and Communications Technology, Hanoi University of Science and Technology.

NGUYEN, T. T. T. (2026). *ITSS Software Development Lab Assignment 12: Design Principles and Design Patterns*. School of Information and Communications Technology, Hanoi University of Science and Technology.

NGUYEN, T. T. T. (2026). *AIMS: An Internet Media Store Problem Statement (ver. 3.1.1)*. School of Information and Communications Technology, Hanoi University of Science and Technology.

NGUYEN, T. T. T. (2026). *AIMS: An Internet Media Store Additional Requirements (ver. 3.1.1)*. School of Information and Communications Technology, Hanoi University of Science and Technology.

Group 16. (2026). *AIMS Software Requirements Specification (ver. 1.2)*. School of Information and Communications Technology, Hanoi University of Science and Technology.

2 Overall Description

2.1 General Overview

AIMS (An Internet Media Store) is a web-based e-commerce platform for the retail of physical media products – Books, Newspapers, CDs, and DVDs. The system supports two primary user-facing roles: the Customer, who browses, purchases, and tracks orders; and the Product Manager, who maintains the product catalog and processes orders. A third role, the Administrator, manages user accounts, roles, and system access. Two external payment service actors are also integrated: VietQR (QR-code-based bank transfer) and PayPal (credit card processing).

Software Architecture Overview

The system follows a client-server architecture with a clear separation between the frontend and backend tiers:

- Frontend: A single-page application built with React and TypeScript, responsible for rendering the user interface and communicating with the backend via RESTful HTTP APIs.
- Backend: A Java Spring Boot application organized into layered packages (controller → service → repository), handling all business logic, data persistence (PostgreSQL via Spring Data JPA), and integration with external payment gateways.

The backend is structured around the key domain entities: Product (with subtypes Book, CD, DVD, Newspaper), Order, OrderItem, Cart, and User. Design patterns, including Strategy, Registry, Factory, Builder, and AOP-based audit logging, are applied throughout the backend to enforce extensibility and separation of concerns, as detailed in Section 5.

Design Goals

The key design goals driving architectural decisions are:

- Extensibility: the system must accommodate new product types and new payment methods without requiring changes to existing components (Open/Closed Principle).
- Separation of concerns: controllers own only the HTTP boundary; all business logic resides in the service layer.
- Maintainability: high cohesion within classes and loose coupling between modules, with design patterns applied to eliminate switch/if-else chains over product type hierarchies.
- Auditability: all significant state changes (product creations, edits, deletions) are logged with actor identity and timestamp.

System Context – Use Case Diagram

The following Use Case Diagram, carried over from the SRS (with no structural changes), summarizes all system capabilities and the actors that interact with them.

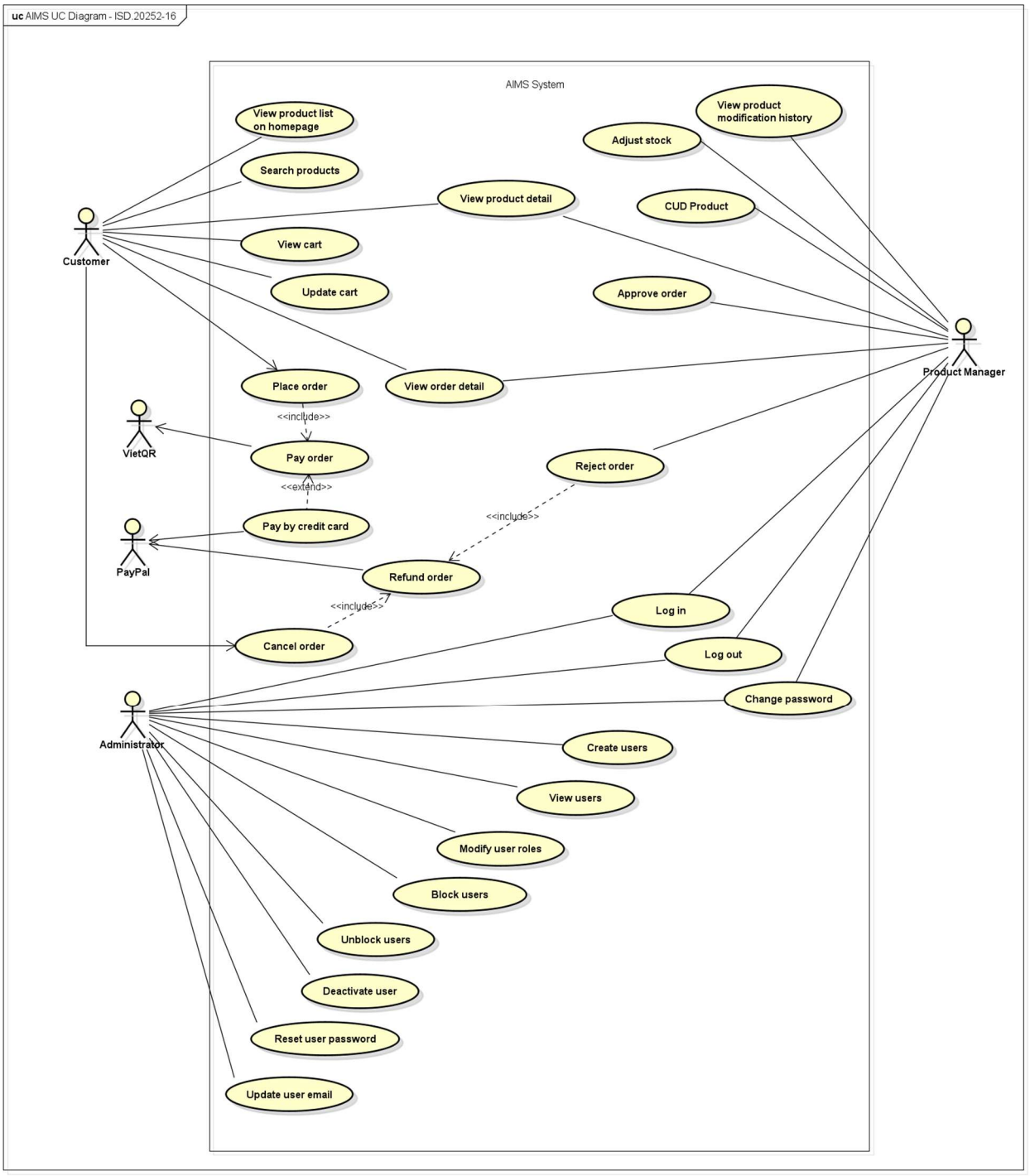


Figure 1. AIMS Use Case Diagram

The diagram identifies three human actors (Customer, Product Manager, Administrator) and two system actors (VietQR, PayPal), and maps them to the full set of use cases across the purchase, product management, and administration processes.

Business Process Activity Diagrams

The SRS identified three core business processes. These are reproduced below unchanged, as the implementation design does not alter the high-level process flows.

- Purchase Process

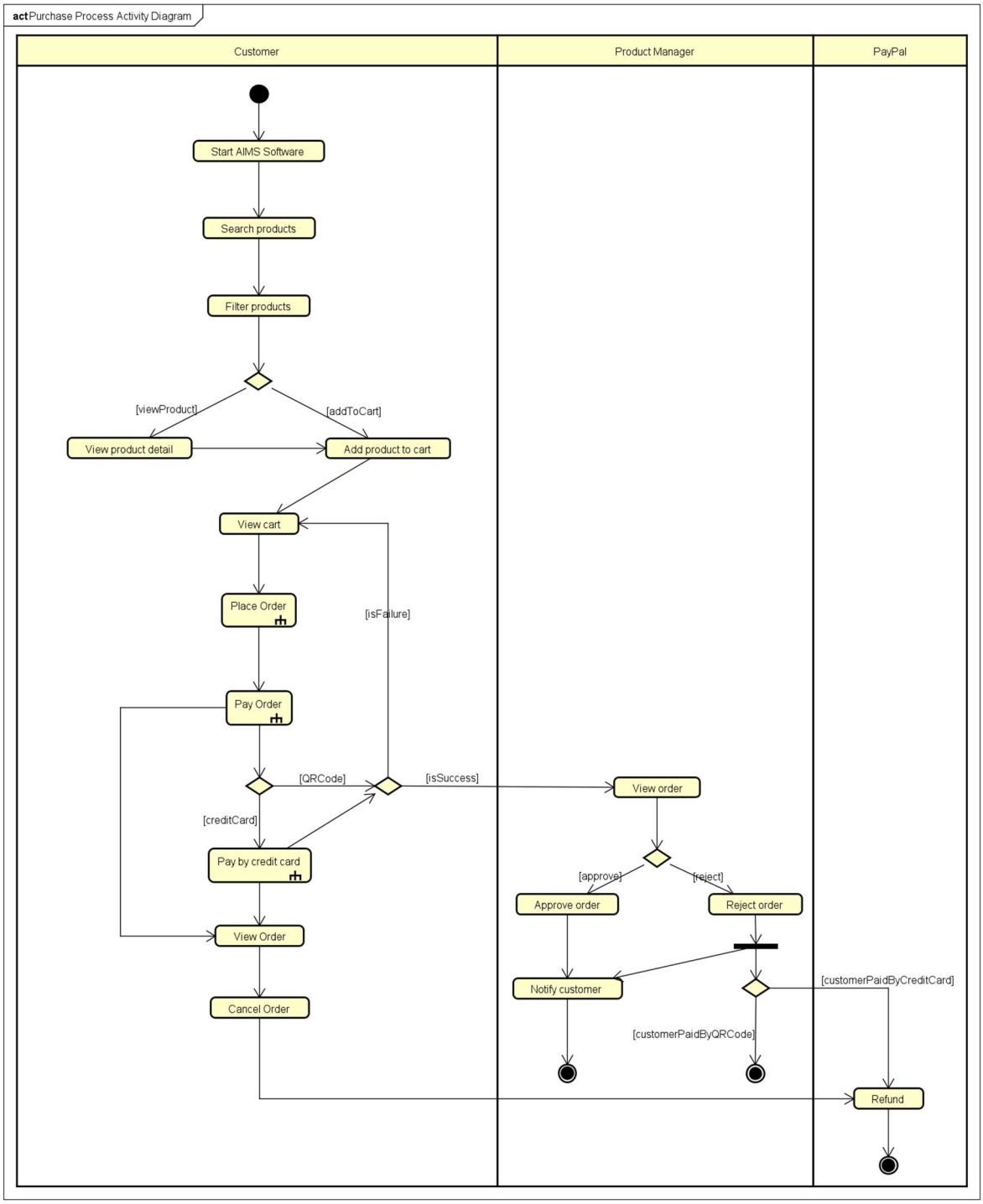


Figure 2. AIMS Purchase Process

The Purchase Process covers the end-to-end customer journey from product discovery through payment and order confirmation. After placing an order, the Product Manager reviews and either approves or rejects it; rejection via credit card triggers a PayPal refund.

- Product Management Process

The Product Management Process covers all catalog maintenance activities available to the Product Manager after login: searching, creating, updating, and deleting products, adjusting stock levels, and viewing product modification history.

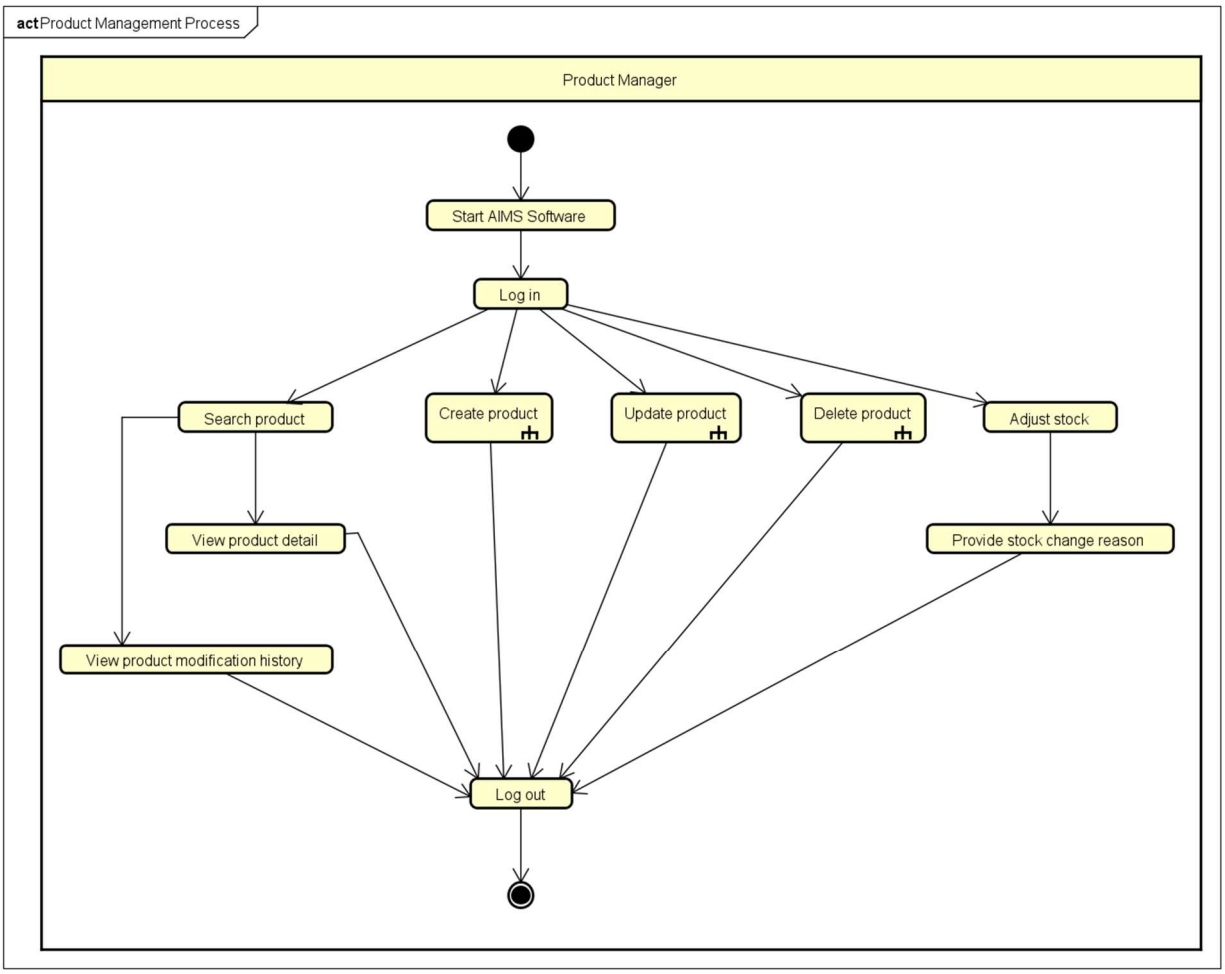


Figure 3. AIMS Product Management Process

- Administration Process

The Administration Process covers user account lifecycle management by the Administrator: creating users, viewing users, modifying user status (block, unblock, deactivate), managing user roles (assign, modify), and resetting passwords.

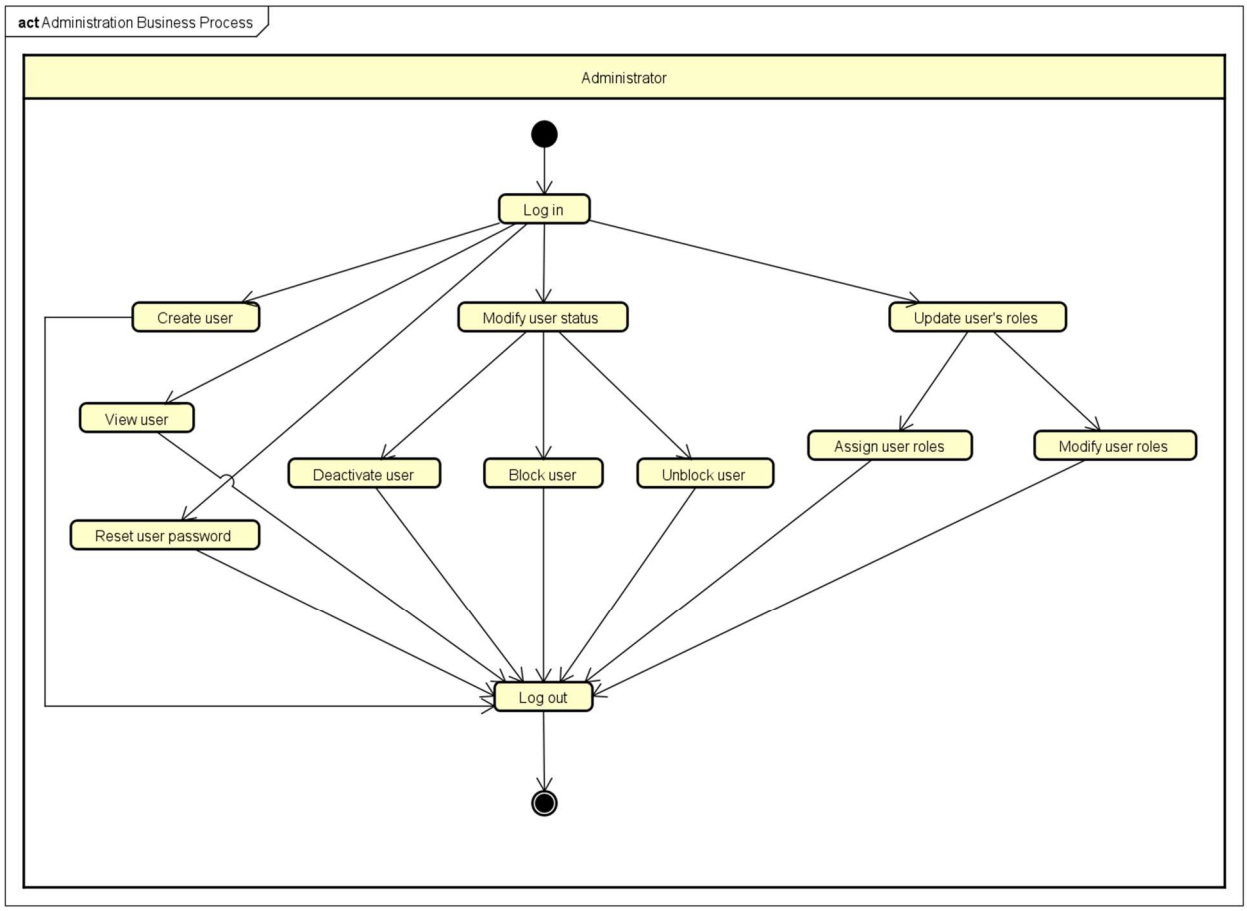


Figure 4. AIMS Administration Process

A note on diagram updates: Little structural changes have been made to the use case diagram or business process activity diagrams from the SRS, only some minor refinements have been made to better reflect the problem statement. The only refinements in the SDD arise at the detailed design level, specifically the addition of the Approve/Reject Order flow visible in the Purchase Process diagram, and the audit logging requirement that links Product Management back to the product modification history use case.

2.2 Assumptions/Constraints/Risks

2.2.1 Assumptions

The following assumptions have been made during the design of the AIMS system. These assumptions form the basis upon which design decisions were made; if any of them prove incorrect, some design decisions may need to be revisited.

- End-user environment

Customers are assumed to access the system through a modern web browser (Chrome, Firefox, Edge, Safari) on desktop or mobile devices. No native mobile application is required within the scope of this project.

Customers are assumed to have basic familiarity with e-commerce workflows: adding products to a cart, entering a delivery address, and completing a payment. No onboarding tutorial is in scope.

Product Managers and Administrators are assumed to be internal staff operating from a trusted network, on a desktop browser. They are not assumed to be technically skilled beyond what is required to operate a standard business web interface.

No login is required for customers. All customer-facing browsing, cart management, and order placement features are available to anonymous users.

- Related software and infrastructure

The system is assumed to be deployed on a server environment capable of supporting up to 1,000 concurrent users with a response time under 2 seconds under normal load and under 5 seconds during peak hours.

A PostgreSQL relational database is assumed to be available and is the sole persistent data store. No secondary caches, message brokers, or external data stores are assumed to be in scope for the initial version.

The VietQR and PayPal Sandbox APIs are assumed to be available and reachable from the backend server during development and demonstration. Their behavior is assumed to match the specifications provided in the official API documentation referenced in the Problem Statement.

Email delivery is assumed to be handled by an external SMTP service. The system generates and dispatches notification emails (order confirmation, approval/rejection, cancellation) but does not own the email infrastructure.

- Payment and refund behavior

VietQR payment is assumed to follow the sandbox callback model: a test callback API call simulates a successful bank transfer. No real fund movement occurs in the demonstration environment.

PayPal Sandbox is assumed to support the full order capture and refund API cycle. Automated refunds on order rejection or customer cancellation are only implemented for PayPal-paid orders. For VietQR-paid orders, no automated refund is triggered; the Product Manager is assumed to handle these manually outside the system.

- Product and business rules

The four product types in scope, Book, CD, DVD, and Newspaper, are assumed to have stable data schemas for the duration of this project. The architecture is designed to accommodate new types without modifying existing components, but no new types are required to be implemented.

All product prices are assumed to be stored and displayed in Vietnamese Dong (VND). A 10% VAT is always applied on top of the current price when presenting totals to the customer. Shipping fees are not subject to VAT.

Delivery addresses are assumed to be within Vietnam. The delivery fee calculation rules (Hanoi/HCMC vs. elsewhere, weight tiers) are assumed to remain fixed as specified in the Problem Statement.

2.2.2 Constraints

The following constraints impose fixed boundaries on the design and implementation of the AIMS system. Unlike assumptions, these are not open to interpretation but rather they are hard requirements, external mandates, or environmental facts that the design must work within.

- Hardware and software environment

The backend must be implemented in Java with Spring Boot. The frontend must be implemented in React with TypeScript. These technology choices are mandated by the course and can only be substituted with other technologies that support Object-Oriented Programming.

PostgreSQL is the required database management system. The data model and persistence layer must be designed accordingly, including reliance on JPA/Hibernate for ORM and Spring Data JPA for repository abstraction.

The system must run on a standard server environment accessible via the public internet or a local network during demonstration. No specialized hardware (GPU, dedicated load balancer, custom appliance) is assumed or required.

- Performance requirements

The system must support up to 1,000 concurrent users without significant performance degradation.

Response time must not exceed 2 seconds under normal conditions and 5 seconds during peak load. This constrains the design to avoid computationally expensive operations on the request thread; in particular, notification dispatch and audit logging must not block the primary transaction path.

The system must be capable of continuous operation for at least 300 consecutive hours without failure, and must recover to normal operation within 1 hour of any incident.

- Interoperability and interface/protocol requirements

Payment processing must integrate exclusively with VietQR (QR-code bank transfer) and PayPal Sandbox (credit card). Both integrations communicate over HTTPS REST APIs using the authentication and request formats defined in their respective official documentation. No other payment gateways are in scope.

The VietQR integration requires obtaining a short-lived access token (valid for 300 seconds) before each QR generation request. The system must handle token expiry correctly and re-request tokens as needed.

The frontend and backend communicate exclusively via RESTful HTTP APIs (JSON over HTTPS). No WebSocket, gRPC, or other communication protocol is in scope.

Order and transaction notification emails must be sent to the customer's provided email address. The system depends on an external SMTP provider for delivery; the internal design is constrained to producing well-formed email content and delegating dispatch.

- Security requirements

All user passwords must be securely hashed before storage. Plaintext passwords must never be persisted or accessible to any user, including Administrators. Administrators may trigger a password reset process but cannot retrieve or view a user's existing password.

Sensitive administrative actions, such as password resets and email updates, must be logged with the actor's user ID and timestamp, and the affected user must be automatically notified.

Product Managers and Administrators must authenticate via login before accessing any management features. Customer-facing features require no authentication.

- Data integrity and business rule constraints

Every product barcode must be globally unique across the entire database. The system must enforce this at both the application and database constraint levels.

The current price of any product must always remain between 30% and 150% of its original value. The system must reject any update that would violate this range.

Product Managers may not delete more than 20 products per day. Products with stock greater than zero cannot be hard-deleted; they must be deactivated instead. The system must enforce both rules and include the count of deactivated products in operation summaries.

Up to 10 products may be selected for deletion in a single operation.

Orders in the Pending Processing state are the only ones eligible for customer cancellation or Product Manager approval/rejection. State transition rules must be enforced by the system and cannot be bypassed via direct API calls.

Refunds are only automated for PayPal-paid orders. For VietQR-paid orders that are cancelled or rejected, the system must notify the Product Manager that a manual refund is required; no automated fund transfer is triggered.

- Data repository and distribution

All persistent data resides in a single PostgreSQL instance. No distributed database, replication topology, or read replica is in scope. This constrains horizontal scalability but is acceptable within the course project context.

Product audit history (creation, update, deletion events) must be permanently stored and queryable by Product Managers through the system interface. This is a retention requirement; audit records must not be purged on a rolling basis.

- Capacity constraints

Product Manager order review pages display 30 pending orders per page. This is a fixed display constraint defined in the Problem Statement.

The homepage displays 20 random products on initial load for customers. This is a fixed presentation constraint.

- Licensing and academic constraints

The PayPal and VietQR integrations are implemented against sandbox/test environments only and are for demonstration purposes. No real financial transactions occur. The system must not be deployed in a production context with live payment credentials within the scope of this course.

All design decisions must be defensible in terms of SOLID principles, coupling and cohesion levels, and design pattern application, as these are explicit course evaluation criteria. This constrains the design toward well-structured, documented patterns even in cases where a simpler implementation might suffice.

2.2.3 Risks

The following risks have been identified as having a meaningful probability of impact on the system design, implementation quality, or project delivery. Each entry describes the nature of the risk, its potential impact, and the mitigation strategy applied or planned.

Table 1. Risk Category & Mitigation Strategy

Risk Category	Description	Mitigation strategy
External API Dependencies	The system depends on two third-party sandbox APIs (VietQR and PayPal) for payment processing.	Payment gateway integrations are isolated behind a gateway interface in the service layer. If a sandbox is

(Payment Gateways)	These services may be unavailable during development, testing, or the final demonstration due to network issues, sandbox downtime, or API changes. → Core purchase flow cannot be demonstrated end-to-end. Order placement and payment confirmation are blocked.	unreachable, the team can substitute a local stub that simulates a successful response, without touching any other part of the system. Token expiry handling for VietQR (300-second lifetime) is explicitly implemented to reduce one common failure mode.
Data Concurrency & Inventory	The system is required to support up to 1,000 concurrent users. Under concurrent load, two customers may simultaneously attempt to purchase the last unit of a product, or a Product Manager may update a price concurrently with a customer placing an order. → Overselling (orders accepted beyond available stock), or price inconsistency between the displayed invoice and the recorded order.	Stock availability is checked within a database transaction immediately before order confirmation. Spring's @Transactional combined with PostgreSQL's default row-level locking provides protection against the most common race conditions at the expected concurrency level. The invoice is generated and temporarily saved at the point the customer proceeds to payment, capturing the price at that moment. JPA @Version annotations are implemented on core entities. If a concurrent modification occurs, the transaction will fail safely, prompting the user to refresh their cart rather than processing an invalid financial transaction.
Database Performance	Inefficient database access patterns, excessive object loading, or unnecessary service dependencies may reduce system performance as the application grows. → Impact: Under peak load of 1000 concurrent users, lazy-loading relational data could severely	Query Optimization: The Data Access (Repository) layer utilizes JPA JOIN FETCH queries and carefully mapped boundaries for aggregate roots. Heavy, bulk read operations will bypass deep ORM entity mapping where necessary to preserve memory and connection pool limits.

	degrade response times beyond the 5-second maximum.	
Team coordination	The project is developed collaboratively within a limited academic schedule. Inconsistent coding styles, architectural misunderstandings, or merge conflicts may affect project quality and progress.	Establish coding conventions and architectural guidelines; Use version control and regular code reviews; Maintain consistent documentation and communication among team members.
VietQR refund gap	VietQR does not provide a refund API. When a VietQR-paid order is cancelled by the customer or rejected by the Product Manager, no automated refund can be issued. → Customers who paid via VietQR receive no automated reimbursement. Without a clear process, this creates a poor customer experience and a potential dispute.	The system explicitly detects the payment method at the point of cancellation or rejection and, for VietQR orders, sends a notification to the Product Manager flagging that a manual refund is required. This is documented as a known limitation in both the Problem Statement and the SRS, and is an accepted constraint within the course scope rather than a design defect.

3 System Architecture and Architecture Design

The architectural design of AIMS followed an iterative, requirements-driven process. The starting point was the use case model and business process diagrams from the SRS, from which the major functional subsystems (product catalog, order lifecycle, payment processing, notification, and audit logging) were identified. For each subsystem, the team analyzed the volatility of requirements (e.g., new product types, new payment methods, new notification channels), identified the extension points that needed to be protected, and selected architectural patterns accordingly. The resulting architecture was then validated against SOLID principles and refactored where violations were found. The patterns applied, and the reasoning behind each, are described in Section 3.1 below. Detailed class-level design arising from these patterns is documented in Section 4.4.

3.1 Architectural Patterns

AIMS applies architectural patterns at two levels: the macro-level (overall system organization) and the micro-level (individual subsystem structure). The following patterns are in use.

3.1.1 Layered Architecture (Macro-level)

The system is organized into four horizontal layers, applied uniformly across both the frontend and the backend.

On the backend, the layers are:

- Presentation layer: Spring MVC REST controllers. Responsible solely for parsing HTTP requests, invoking the service layer, and serializing responses. No business logic resides here.
- Service layer: Spring @Service components. Owns all business rules, orchestration, and transaction management. This is where design patterns (Strategy, Template Method, Factory, Registry) are applied.
- Repository layer: Spring Data JPA repositories. Responsible for all database access. No query logic escapes this layer into the service layer.
- Domain layer: JPA entity classes, value objects, and enumerations representing the core business concepts (Product hierarchy, Order, Cart, User, etc.).

On the frontend, the layers are:

- Screen layer: React page components acting as thin orchestrators. They hold no business logic and delegate all data operations to the service layer.
- Component layer: Reusable UI components (ProductFormSection hierarchy, shared order components) with no direct API knowledge.

- Service layer: TypeScript service objects (ProductManagementService, OrderService, etc.) that own all HTTP communication with the backend.

Rationale: Layered architecture enforces a single direction of dependency (each layer depends only on the one below it), makes responsibilities unambiguous, and allows each layer to be tested or replaced independently. The constraint that controllers own only the HTTP boundary and all business logic lives in the service layer is a direct consequence of this pattern and is applied without exceptions throughout the codebase.

3.1.2 Vertical Slice Architecture (for Payment subsystems & services)

Within the payment subsystem, rather than forcing both payment methods into a single unified strategy interface, each payment method (VietQR, PayPal) is organised as a self-contained vertical slice under its own package (`*.payment.vietqr, *.payment.paypal`). Each slice owns its own controller endpoint, service logic, and gateway client.

A pair of shared contracts (`IQRPaymentGateway` and `IRedirectPaymentGateway`) are defined in the `services.payment.contract` package, representing the two structurally distinct interaction models (poll-based QR vs. redirect-and-capture). These interfaces are defined by the application layer and implemented by each slice, following the Dependency Inversion Principle.

Rationale: VietQR and PayPal differ structurally: VietQR generates a QR code and polls for a callback confirmation, while PayPal redirects the user to an external page and captures payment on return. Forcing a single interface over both flows would require one or both implementations to expose methods they do not use, a violation of the Interface Segregation Principle. Vertical slices provide package-level OCP (each new payment method is a new slice, not a modification of existing ones) and SRP (each slice is responsible for exactly one payment method). This trade-off is discussed further in Section 5.2.

3.1.3 Event-driven Decoupling

Post-state-transition side effects (notification emails and refund triggers) are decoupled from the order state transition itself via Spring application events (e.g. `OrderSuccessEvent`, `OrderCancelledEvent`). These events are published after the originating transaction commits, and handled by Transactional Event Listener methods.

Rationale: Without event-driven decoupling, the Service class would need to directly invoke the notification service and the refund service, accumulating dependencies and fan-out coupling. Events invert this relationship: the Service publishes a fact about what happened; downstream handlers react independently. This satisfies OCP at the service level, adding a new side effect (e.g., an SMS notification) requires only a new listener, not a modification to the service class.

3.1.4 AOP-based cross-cutting concerns (Audit Logging)

Audit logging for product and admin operations is implemented as an AOP advice triggered by custom annotations. The advice intercepts annotated service methods, captures the method arguments and return value, constructs an Audit Log record, and persists it, entirely outside the annotated method's own logic.

Rationale: Audit logging is a cross-cutting concern: it is required on every crucial product and admin operation but has nothing to do with the business logic of creating or updating a product. Weaving it in via AOP avoids polluting every service method with logging boilerplate, keeps the service methods focused on their primary responsibility (SRP), and allows the logging behavior to be modified or extended without touching business logic.

3.1.5 Micro-level design patterns (summary)

Beyond the structural patterns above, several classical GoF design patterns are applied within individual subsystems to eliminate type-switch dispatch and enforce extensibility at the class level. These include the Strategy and Registry pattern (delivery fee calculation, product type dispatch, notification channel routing, refund dispatch), the Template Method pattern (shared payment finalization sequence), the Builder pattern (Product entity construction and selective mutation), and the Factory pattern (polymorphic product mapping and notification message building). The full rationale and implementation details for each are covered in Section 5.4 Design Patterns.

3.2 Interaction Diagrams

3.2.1 Sequence Diagrams

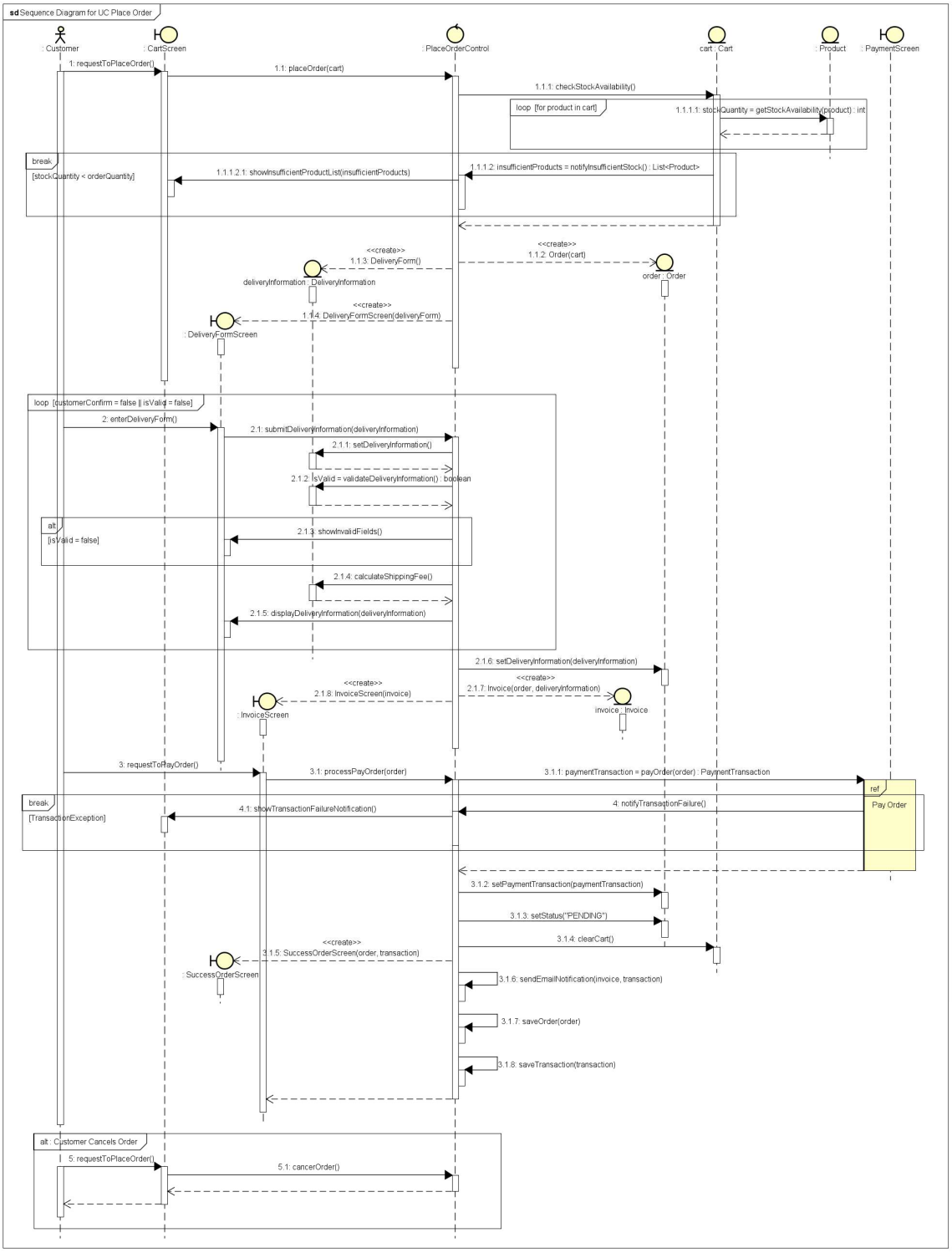


Figure 5. UC Place Order Sequence Diagram

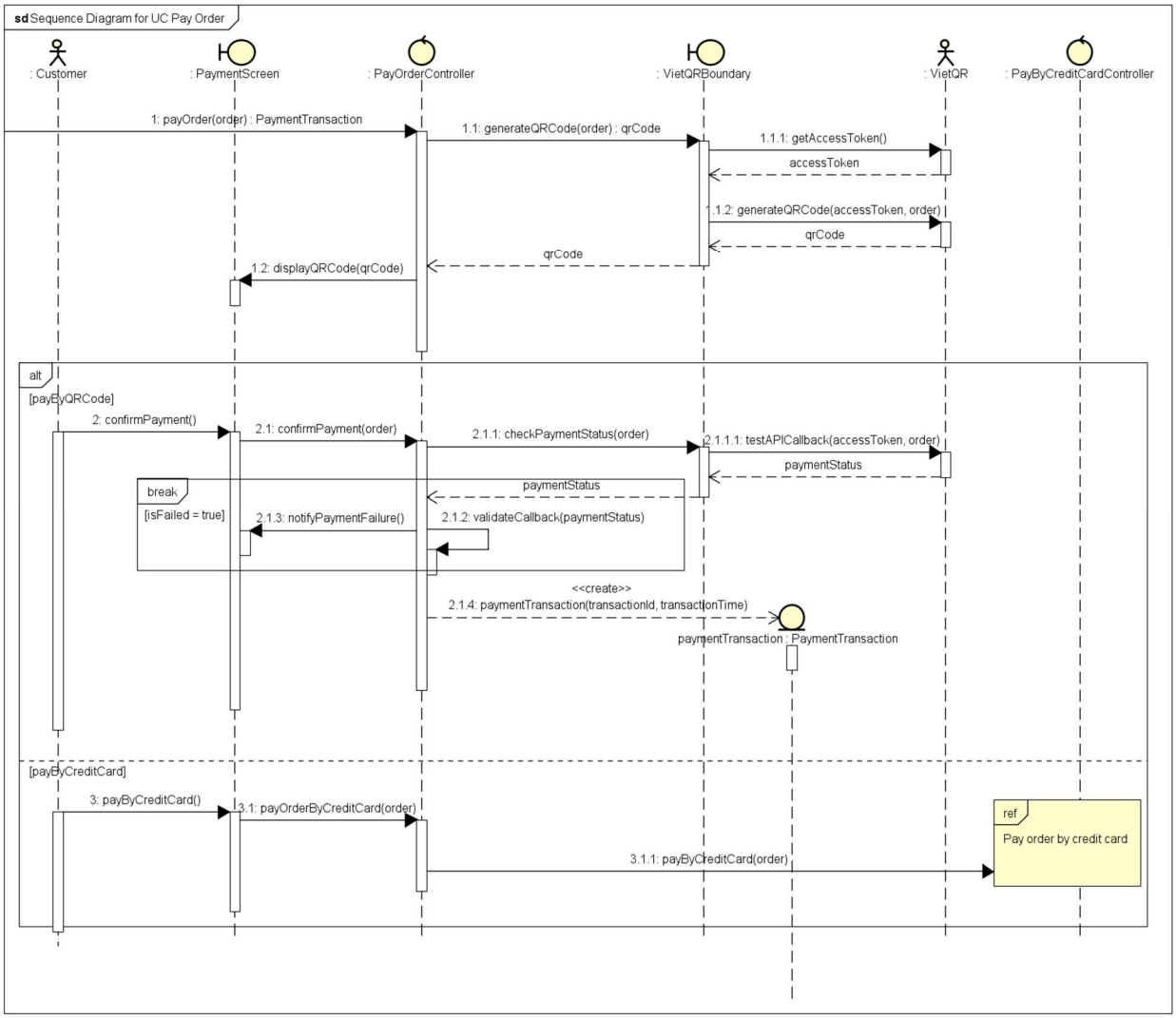


Figure 6. UC Pay Order Sequence Diagram

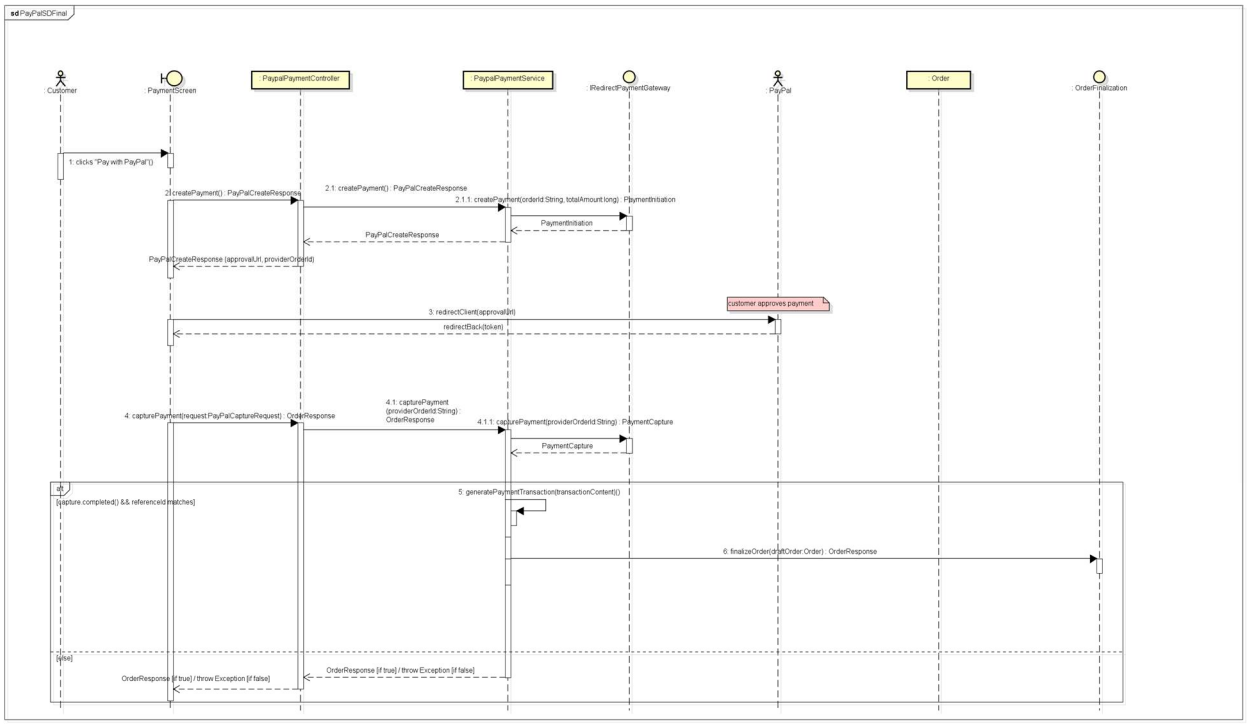


Figure 7. UC Pay Order by Credit Card Sequence Diagram

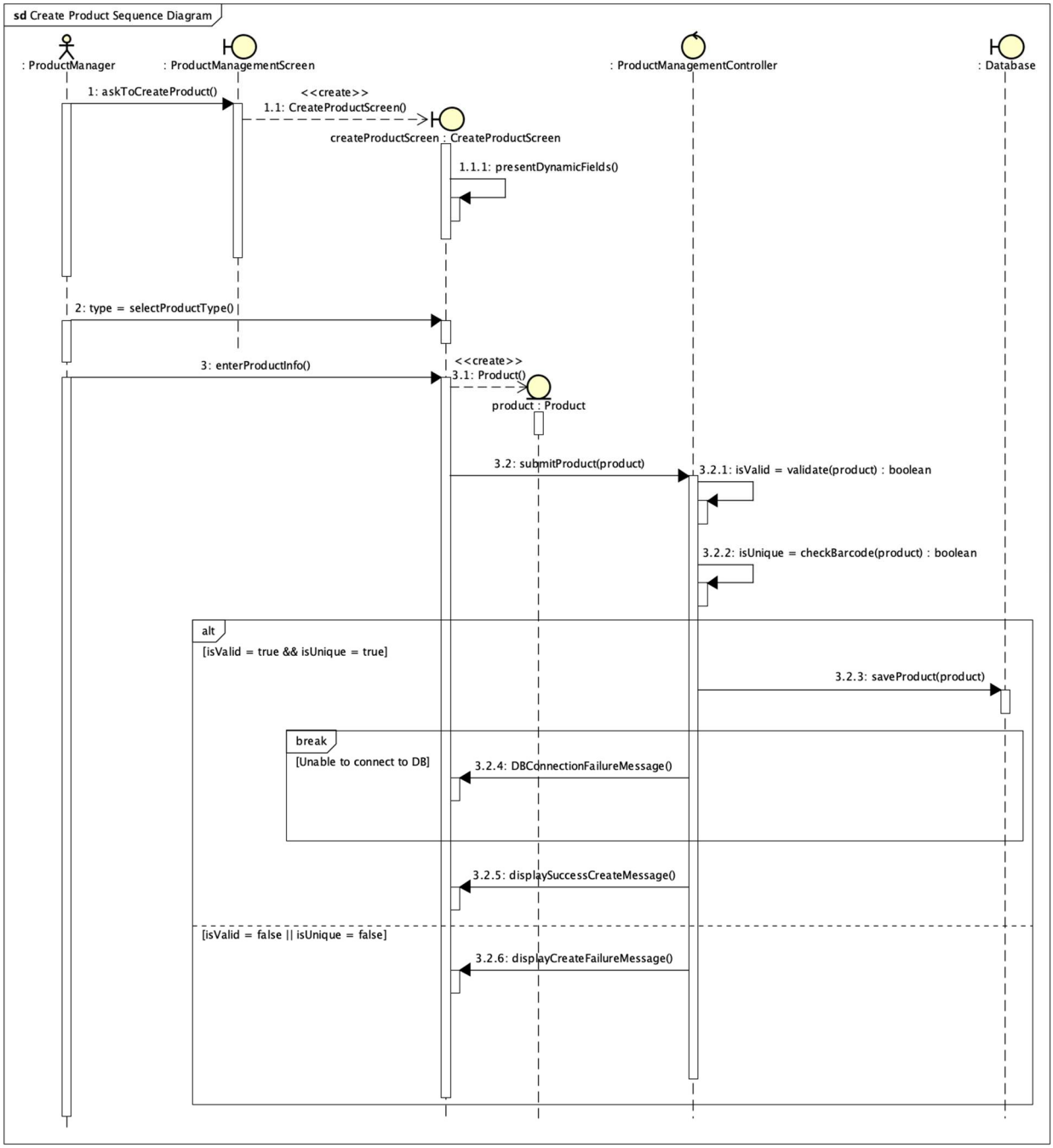


Figure 8: UC Create Product Sequence Diagram

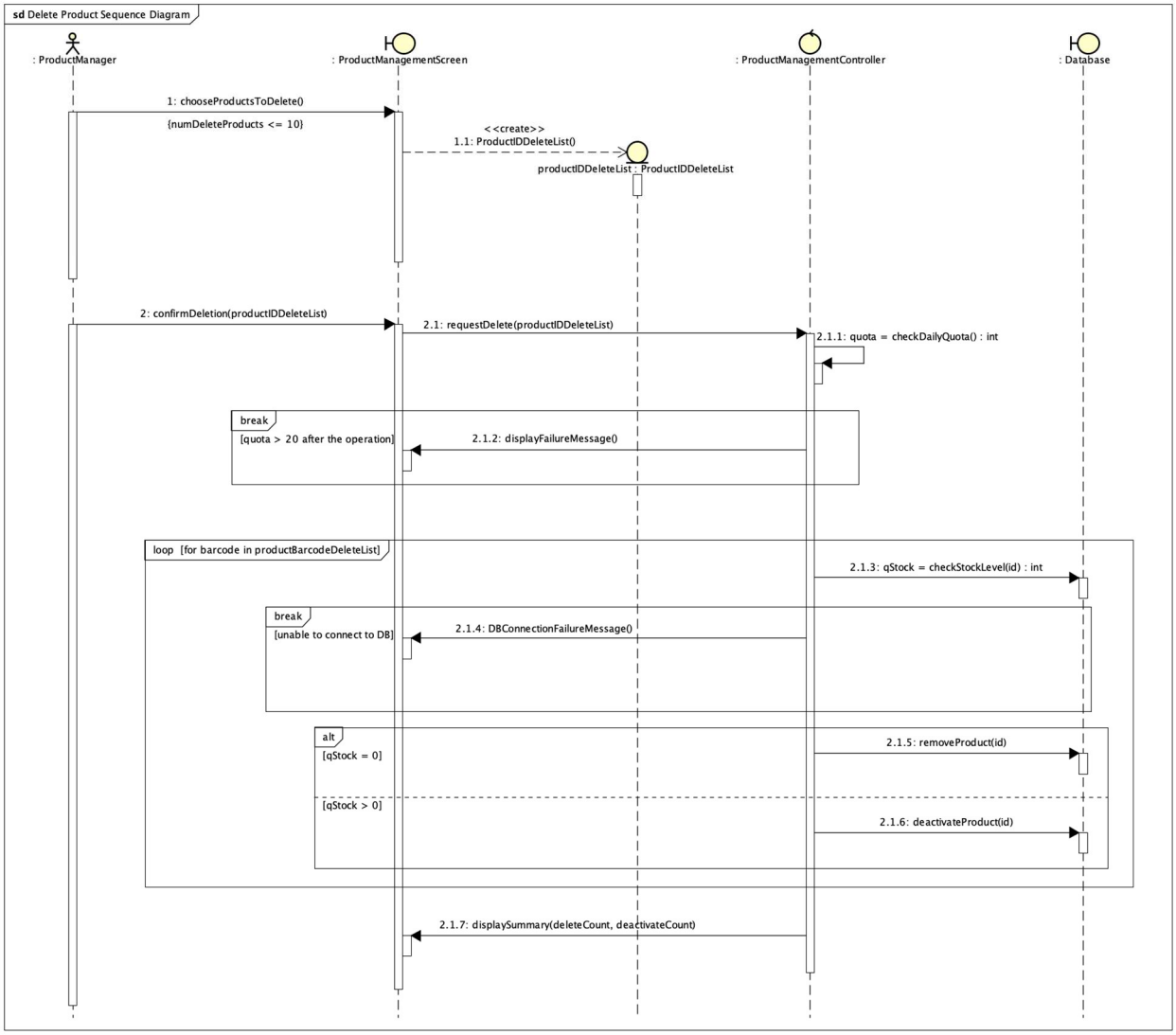


Figure 9: UC Delete Product Sequence Diagram

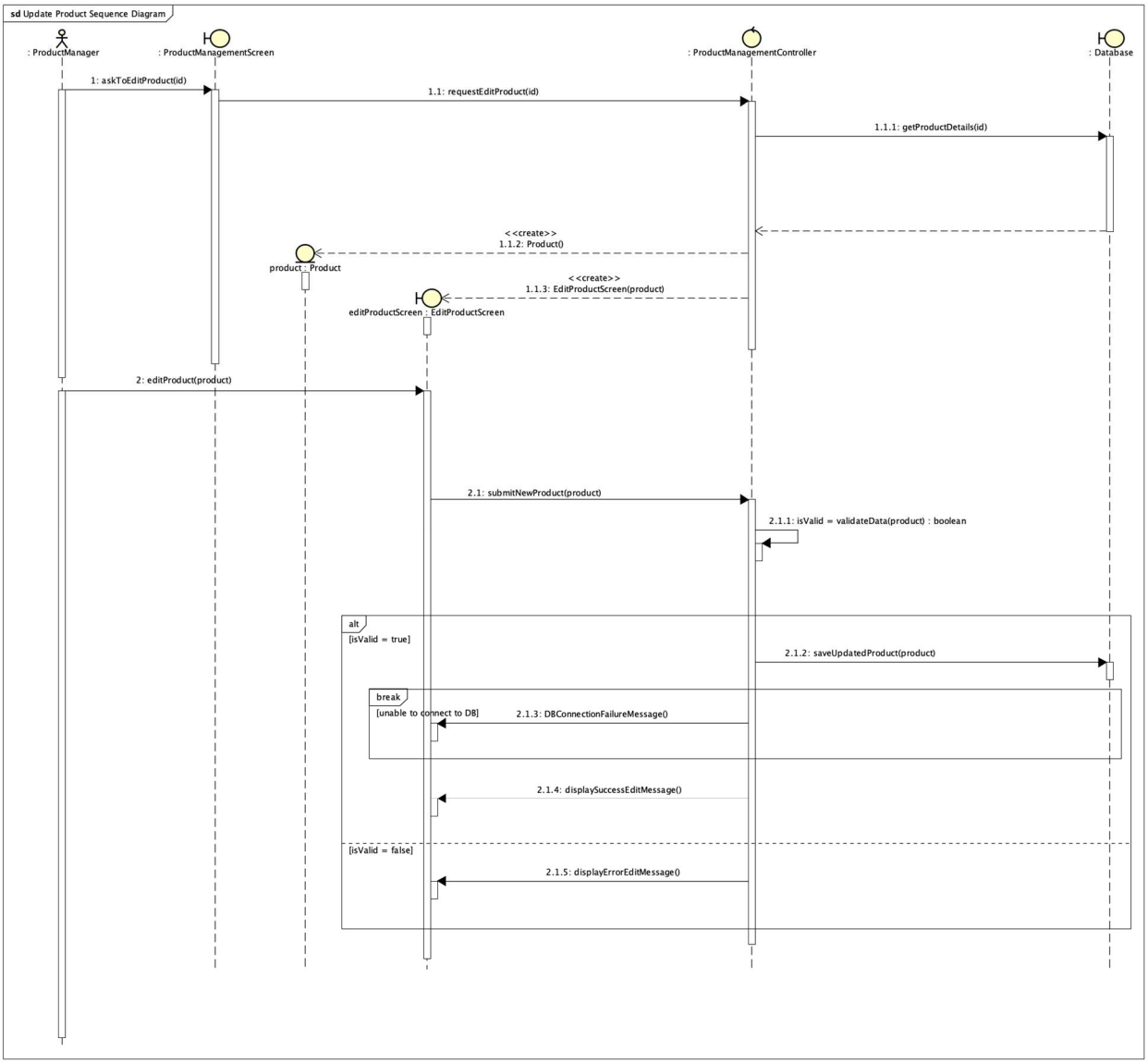


Figure 10: UC Update Product Sequence Diagram

3.2.2 Communication Diagrams

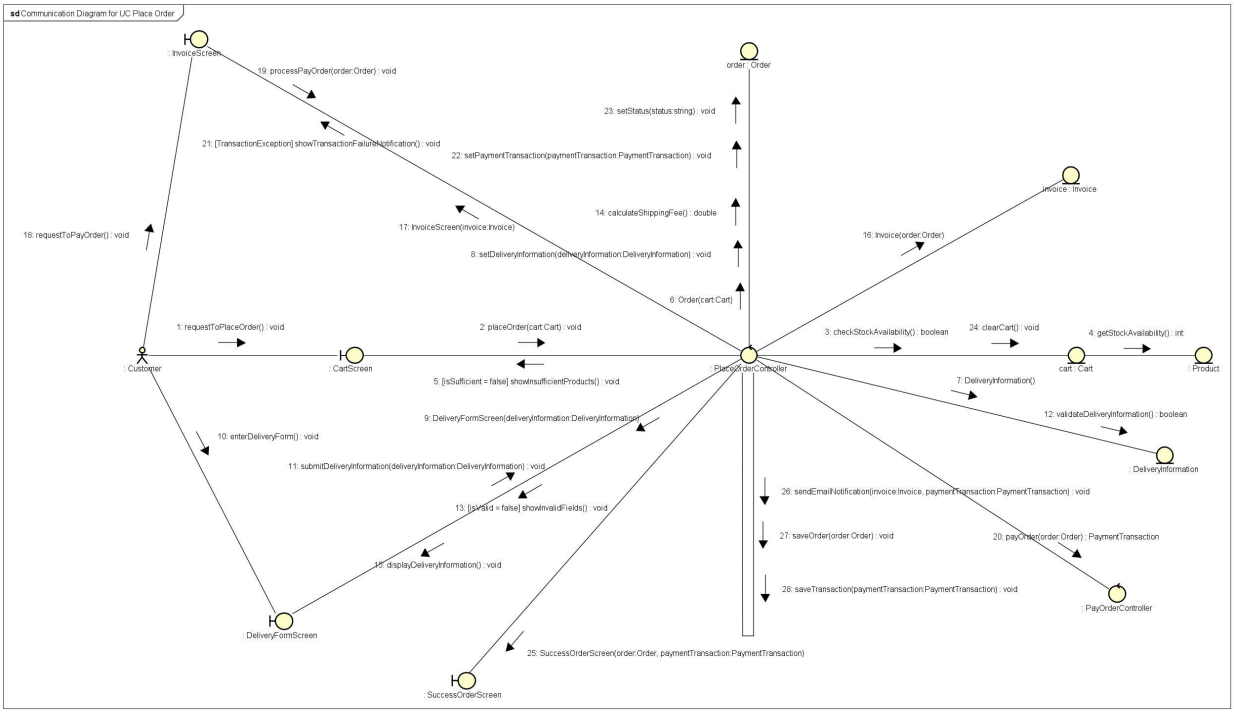


Figure 11. UC Place Order Communication Diagram

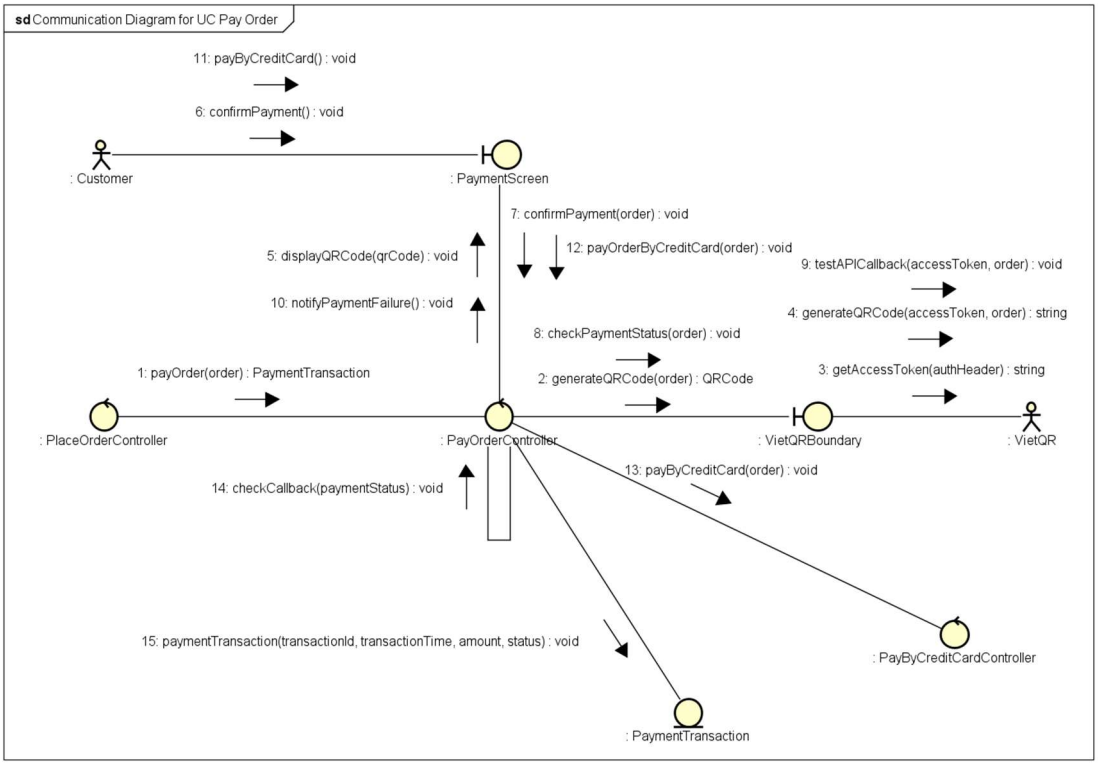


Figure 12. UC Pay Order Communication Diagram

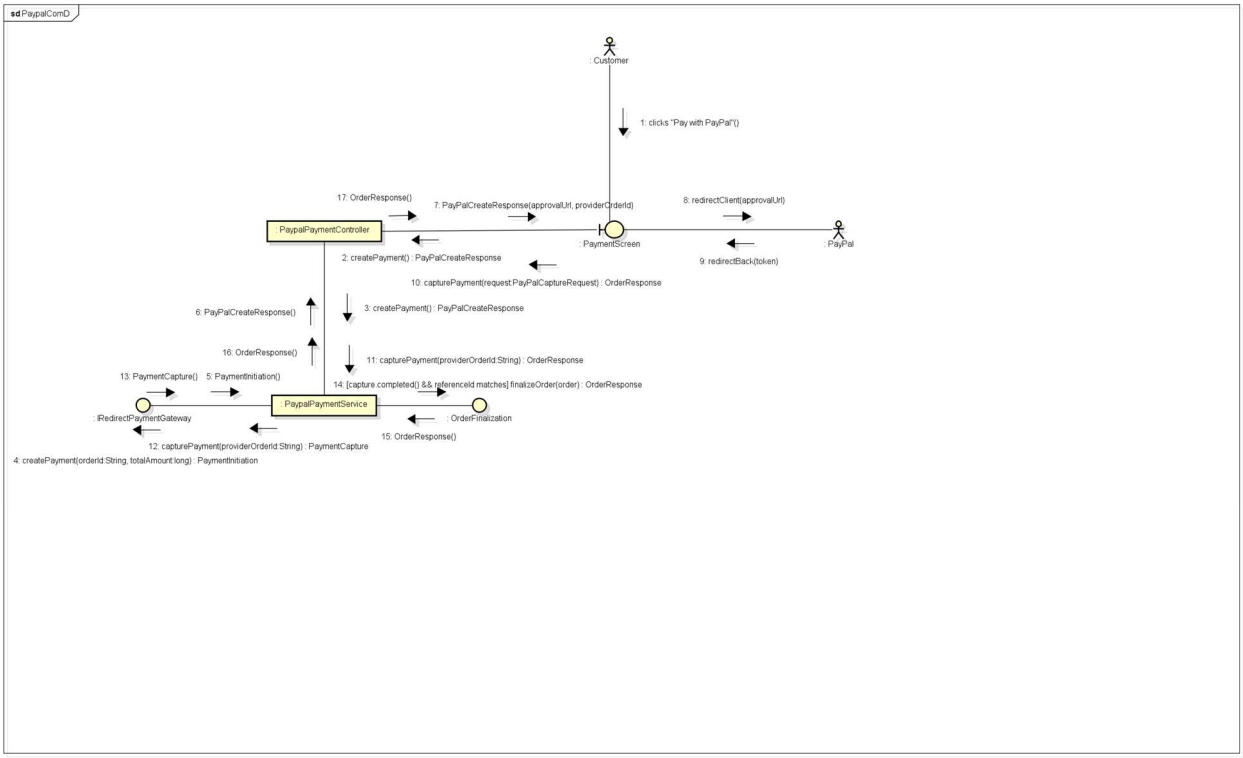


Figure 13. UC Pay Order by Credit Card Communication Diagram

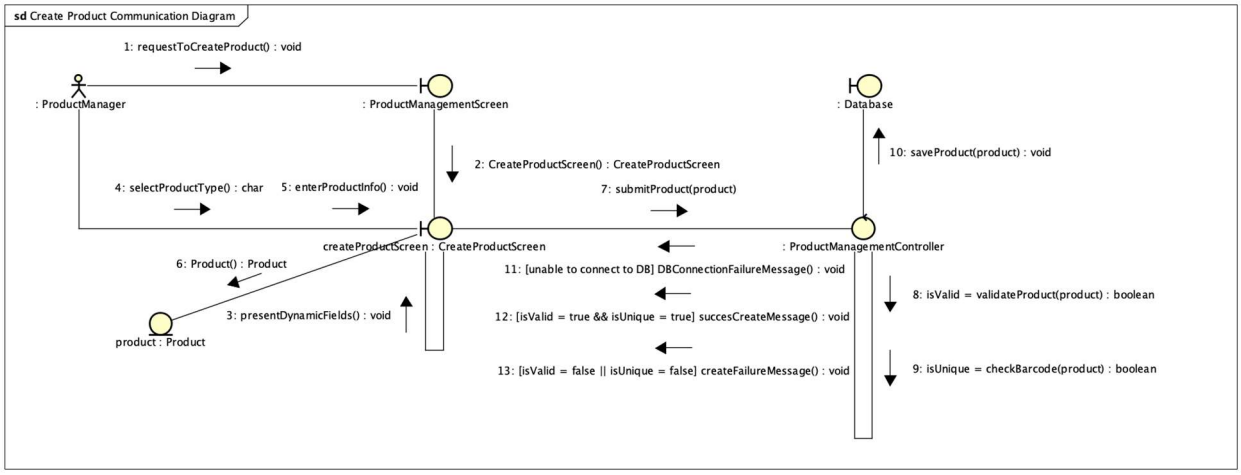


Figure 14. UC Create Product Communication Diagram

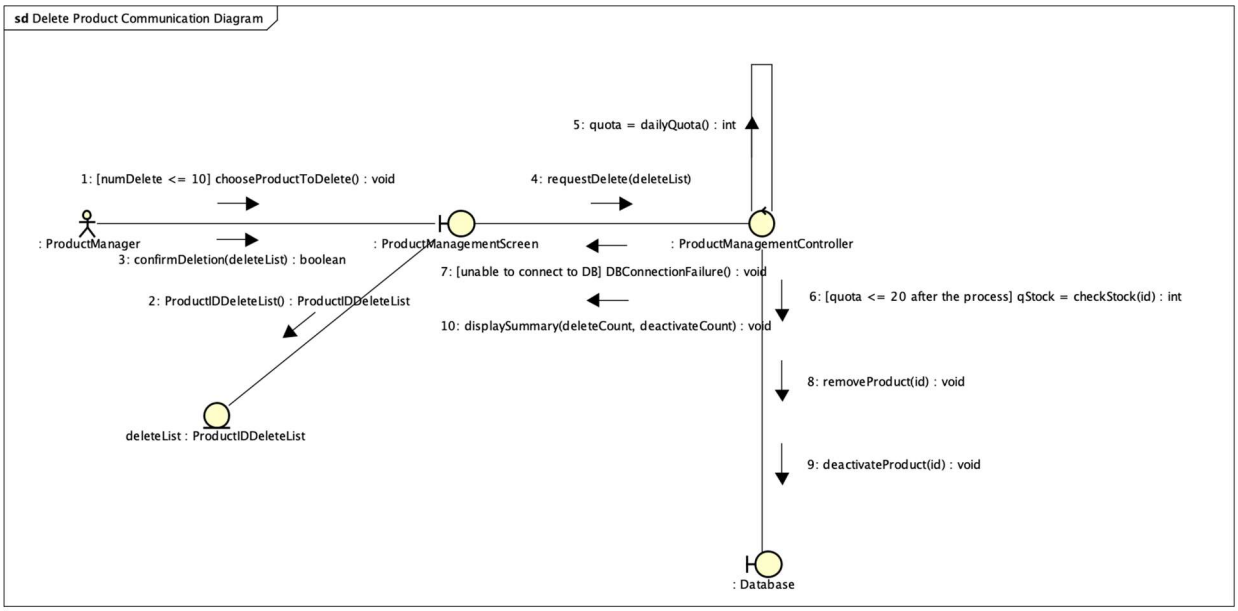


Figure 15. UC Delete Product Communication Diagram

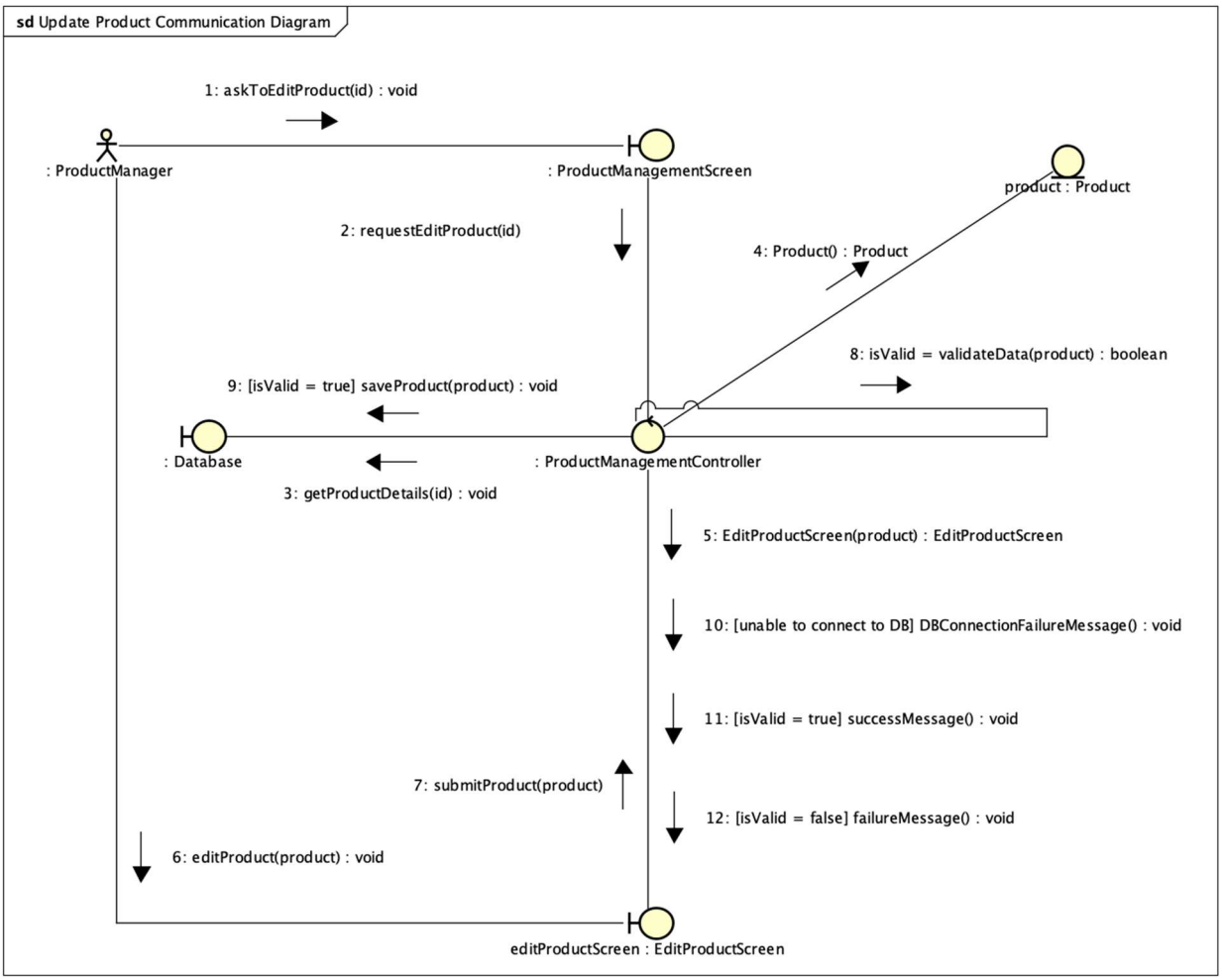


Figure 16. UC Update Product Communication Diagram

3.3 Analysis Class Diagrams

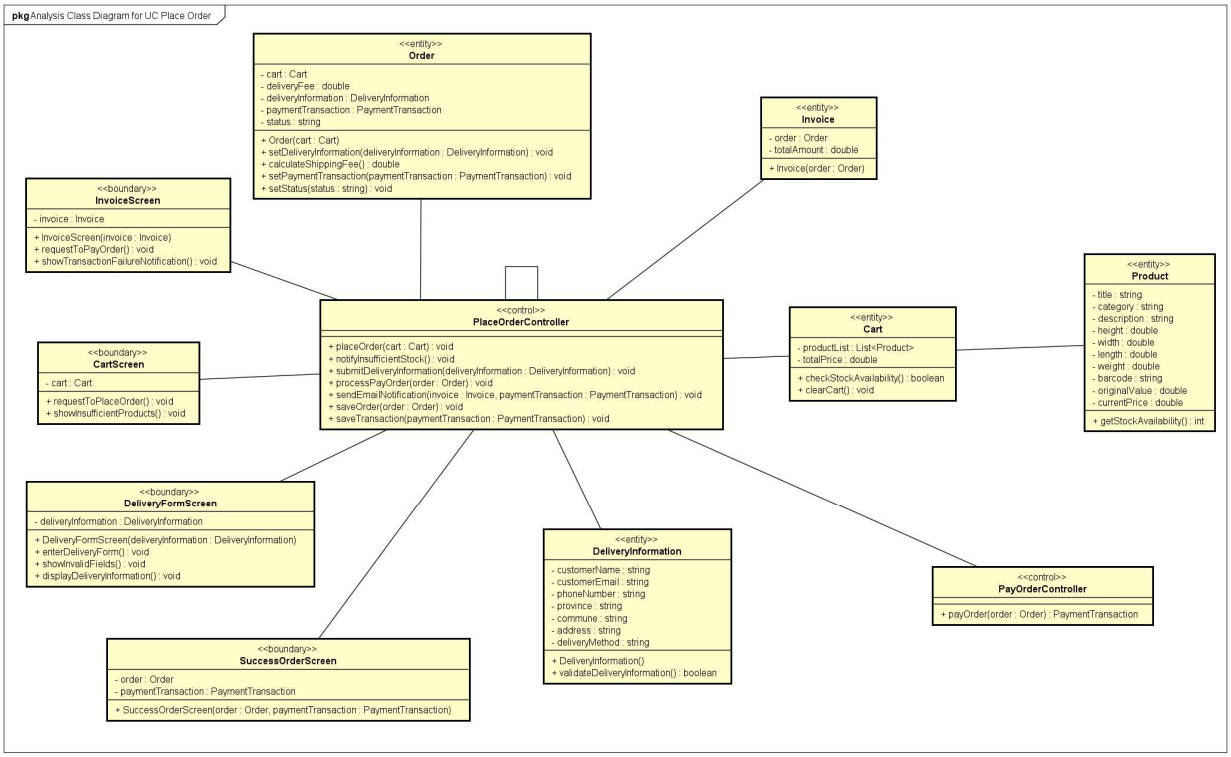


Figure 17. UC Place Order Analysis Class Diagram

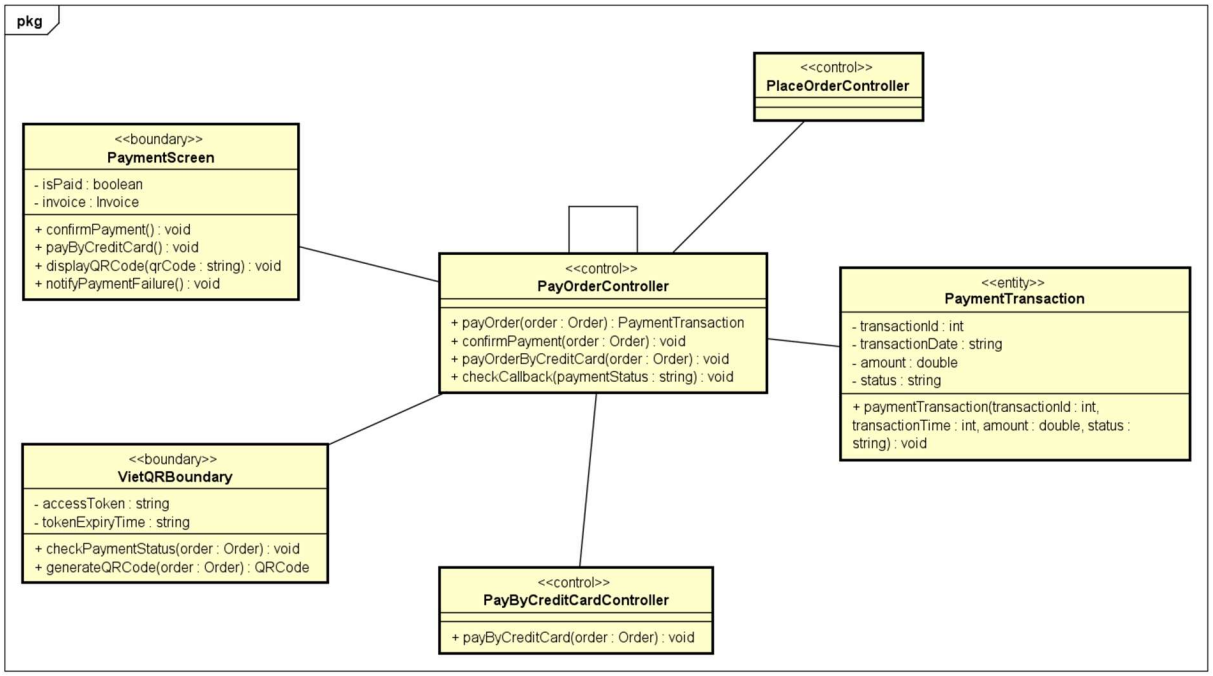


Figure 18. UC Pay Order Analysis Class Diagram

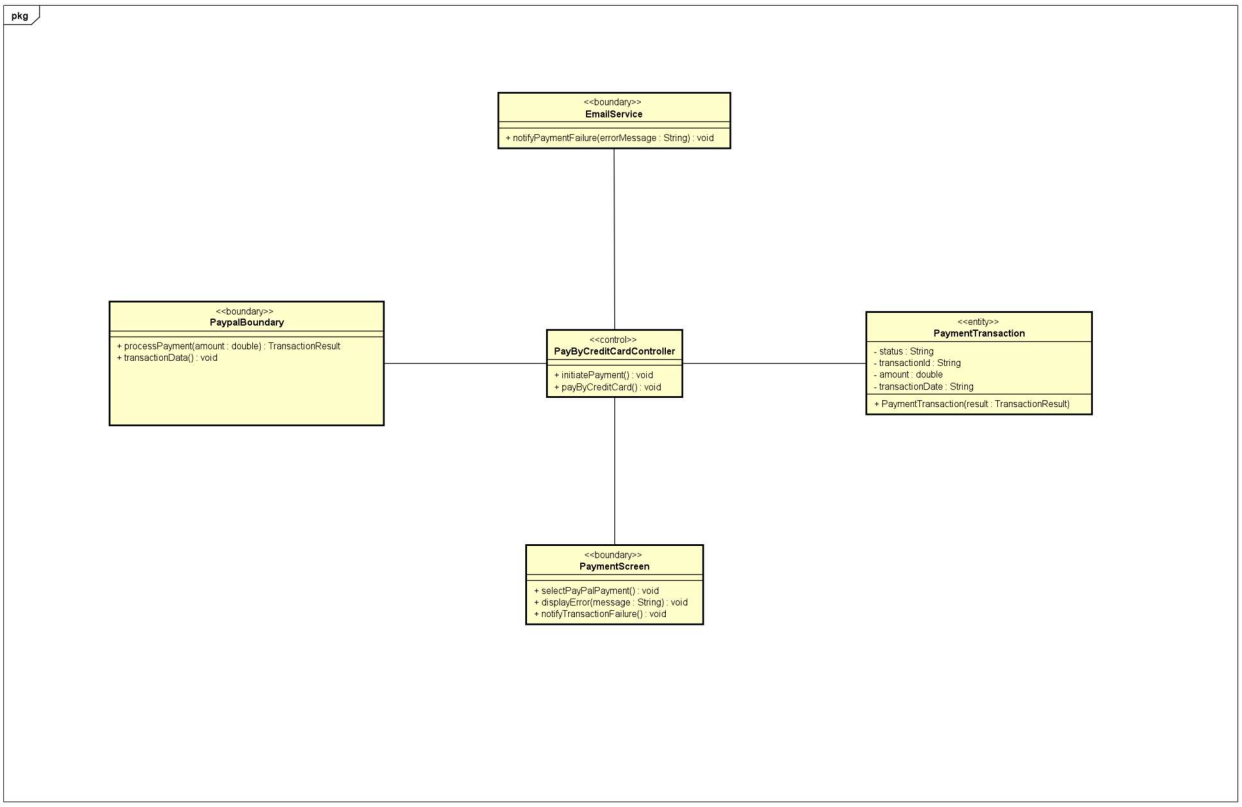


Figure 19. UC Pay Order by Credit Card Analysis Class Diagram

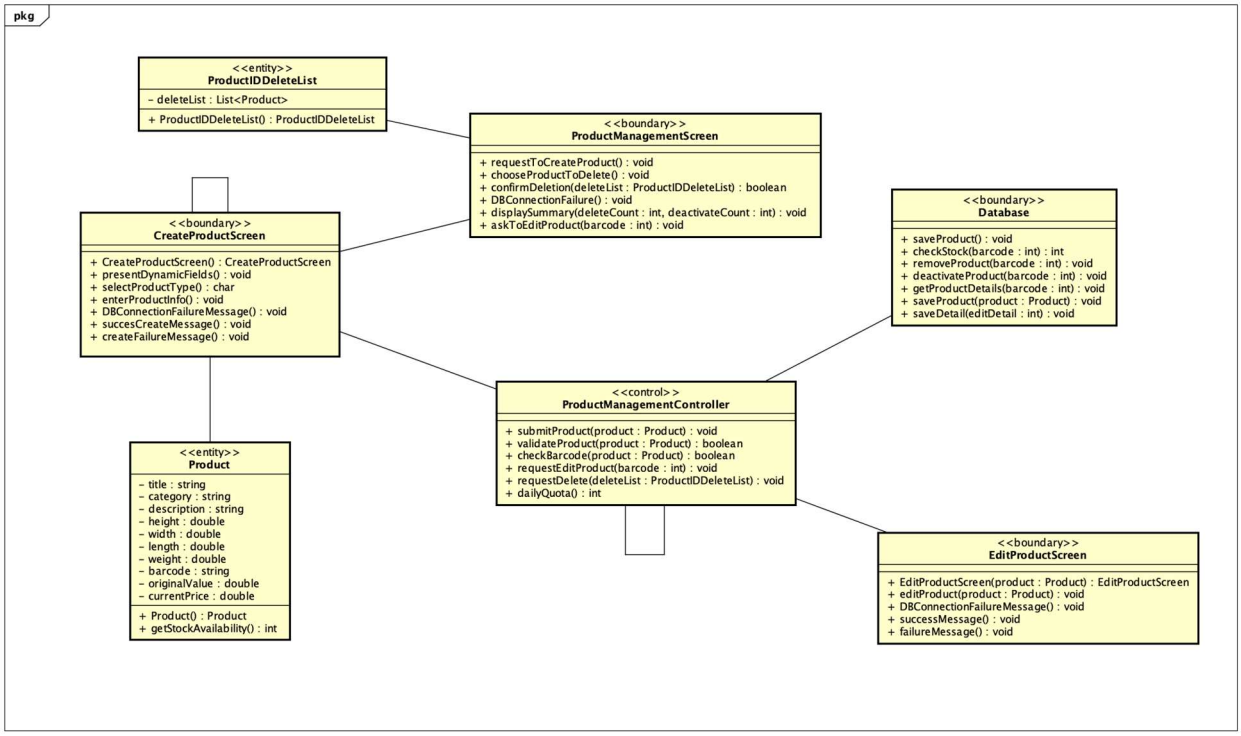


Figure 20. UC CUD Product Analysis Class Diagram

3.4 Unified Analysis Class Diagram

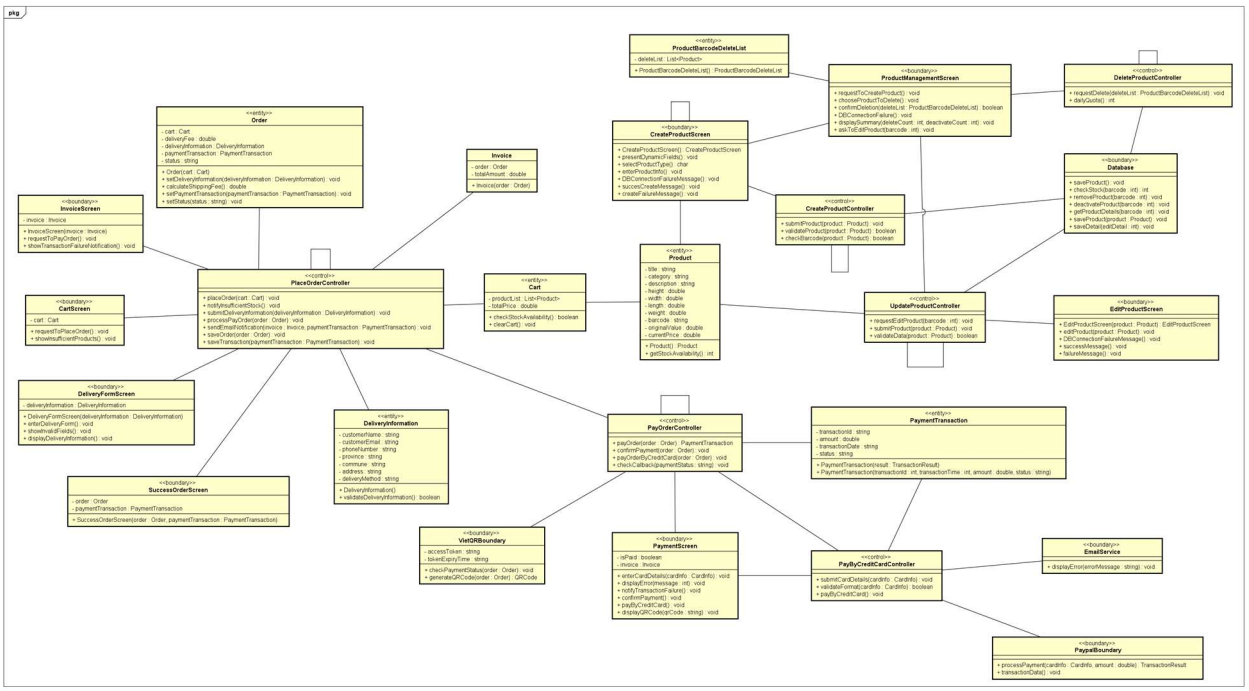


Figure 21. Unified Analysis Class Diagram of AIMS

3.5 Security Software Architecture

The AIMS security architecture is designed to protect sensitive user data, enforce the Principle of Least Privilege, and ensure accountability for administrative actions. The system employs a stateless, token-based security model utilizing Spring Security and JSON Web Tokens (JWT) for the backend, integrated with a React frontend.

3.5.1 Authentication (User Identity Validation)

Authentication is handled via a stateless JWT mechanism to validate user identities before granting access to the AIMS portal (specifically for Administrators and Product Managers).

- **Credential Verification:** Users submit their credentials (email and password) to the login endpoint. Spring Security intercepts the request and verifies the credentials against the PostgreSQL database.
- **Account Status Evaluation:** Before issuing a token, the system evaluates the user's account status. Deactivated users are rejected immediately at the authentication level (DisabledException).
- **Token Generation:** Upon successful verification, the backend generates a signed JWT. The token payload contains the user's ID, email, assigned roles, and critical security flags (including the account status and password reset requirement).

- **Client-Side Storage:** The React frontend securely stores the JWT and attaches it as a Bearer token in the Authorization header for all subsequent API requests.

3.5.2 Authorization (Functional Activity Access Control)

Authorization in AIMS is enforced using strict Role-Based Access Control (RBAC) combined with a "Restricted JWT" pattern to handle edge cases without relying on continuous database queries.

- **Filter-Level Enforcement:** Every incoming request passes through a custom JwtAuthFilter. The filter decodes the JWT, validates its signature, and extracts the user's roles and claims to build the SecurityContext.
- **Role-Based Routing:** Endpoints are protected using Spring's method security. Only Admins can access admin related endpoints (specifically for user creation, blocking, role assignment), while Managers access product and order-related services.
- **Restricted JWT Pattern for Edge Cases:**
    - *Blocked Accounts:* If the JWT contains a flag showing that the account is blocked: the filter automatically rejects all business operations with an HTTP 403 Forbidden status, while still allowing the frontend to route the user to a specific "Blocked Notice" screen.
    - *Password Enforcement:* If the JWT contains must_change_password: true status, the system restricts access to all endpoints *except* /api/profile/password, forcing the user to secure their account before proceeding.

3.5.3 Encryption Protocol (Data Protection)

Encryption protocols are deployed to secure data both in transit and at rest, mitigating business risks associated with credential theft or interception.

- **Data in Transit:** All communications between the React frontend client and the Spring Boot backend server are encrypted, ensuring that JWTs, temporary passwords, and sensitive business data cannot be intercepted.
- **Data at Rest (Passwords):** Passwords are never stored in plain text. The system utilizes the **BCrypt** hashing algorithm (via Spring's BCryptPasswordEncoder) to hash user passwords before storing them in the PostgreSQL database. This protects user credentials even in the event of a database breach.

4 Detailed Design

4.1 User Interface Design

4.1.1 Screen Configuration Standardization

Table 2. General Layout Requirement

Category	Rules
Minimum screen size	At least 1366×768 pixels. Layouts must be able to automatically adapt to any screen sizes, resolutions, and aspect ratios.
Page titles	Place the main title at the top-center of every screen. Use only one main title per screen
Action buttons	Standard action buttons must be positioned in a consistent, recognizable location, typically at the bottom vertically and aligned to the center or right horizontally. Primary buttons (Upload, Save, Publish) go at the bottom-right or top-right. Secondary buttons (Cancel, Filter) sit next to them but look less prominent.
Menu Bar / Navigation	It is recommended to place global navigation at the top-right or in a left sidebar. Highlight the current page clearly.
Content area	Main content sits in the center. Side panels (filters, metadata) go on the left or right. Keep consistent spacing between sections.
Responsive behaviour	On smaller screens: stack columns vertically, shrink padding, and make buttons full-width. Never hide essential actions.

Table 3. Customer Flow Layout Requirement

Category	Rules
Header	A top-anchored, sticky navigation bar, allowing page content to scroll smoothly underneath. It houses the AIMS logo (left) and interactive actions like Theme Toggle, Login, and Cart (right)

Content area	Centered with a maximum width for maximum readability
Footer	Anchored to the bottom of the viewport, providing standard copyright and contact information.

Table 4. Manager / Admin Flow Layout Requirement

Category	**Rules**
Sidebar Navigation	A fixed left-hand sidebar serving as the primary navigation hub, housing the logo, navigational tabs, and the user profile/logout actions.
Header	A top header for the active viewport containing a mobile menu toggle, page title, and contextual actions.
Content area	Implements a grid system to cleanly separate complex data into easily digestible, independent modules.

Table 5. UI Kit

Category	**Rules**
Fonts & Typography	Use 2 cohesive font families consistently throughout the entire application. - For Headings: Use Montserrat - For UI / Body Text: Be Vietnam Pro A clear hierarchy of text sizes must be defined, distinguishing between main titles, sub-titles, and standard paragraph text.
Color Palette	Use defined color roles instead of hardcoding colors for theme compatibility - Primary color: Deep Violet / Indigo - Backgrounds: Pearl White - Status colors: Fractional opacity layers are used instead of harsh solid colors for Success, Destructive for Errors / Failures

Icons & Graphics	Use Lucide icon library universally Images must maintain correct aspect ratio and quality
Spacing	Follow a steady rhythm: 4px, 8px, 12px, 16px, 24px, 32px for gaps and padding. Use border radii and deep, soft shadows to create depth for shapes.

Table 6. Data formatting

Category	**Rules**	**Examples**
Numeric values	Use a comma separator every 3 digits. For decimal numbers, use dot separator to separate whole part and fractional part. There should only be exactly 2 digits after the dot separator.	1,450,800.00 93,491.30 89.33 …
Currency	Follow Vietnamese standard currency format: a number adhering to numeric value rules (with no fractional part) followed by the currency symbol ₫.	1,000,000₫ 730,000₫ 45,000₫
Date & Time	Use standardized pattern commonly used in Vietnam (Day of the week is optional) - Date: [Day of the week] DD/MM/YYYY - Time: HH:mm:SS (24-hour format) Never show raw timestamp (YYYY-MM-DDTHH:mm:SSZ) For recent timestamps of up to 1 week before now, show relative time.	18/04/2026 17:45:49 15 minutes ago 4 hours ago 3 days ago
Text handling	Support all Unicode characters using UTF-8 format	
Media Duration	Show CD and DVD duration as HHhMMm or MMmSSs	143 minutes = 2h23m

		231 seconds = 3m51s
Product dimensions	Each dimension: width, height, length must follow the above numeric value rule and formatted as width×height×length	13.4×1.6×9.2

Table 7. Control rule

Category	**Rules**
Input validation	Check inputs in this order: - Is the field empty? - Is the format correct? Only show errors after the user leaves the field or clicks submit. Messages must explain what's wrong and how to fix it.
Error handling	Show friendly, specific messages. Log technical details for developers, but never show raw errors to users. If a request fails, offer a Retry button.
Navigation & focus	When users press Tab, fields should follow a logical top-to-bottom, left-to-right order. Popups (like help screens) should block interaction with the main screen until closed. Users are consistently provided with "Go Back" or "Cancel" buttons to safely return to the previous state without committing changes. No complex shortcuts are required
Secondary screens	Secondary interfaces, such as help or manual screens, should appear as popups to prevent interaction with the main screen until the popup is closed.
Loading states	Show a skeleton screen or spinner while data loads. Never leave a blank white space.
Empty states	When a list has no items, show: a helpful message + simple illustration + a button to take action.

4.1.2 Screen Transition Diagrams

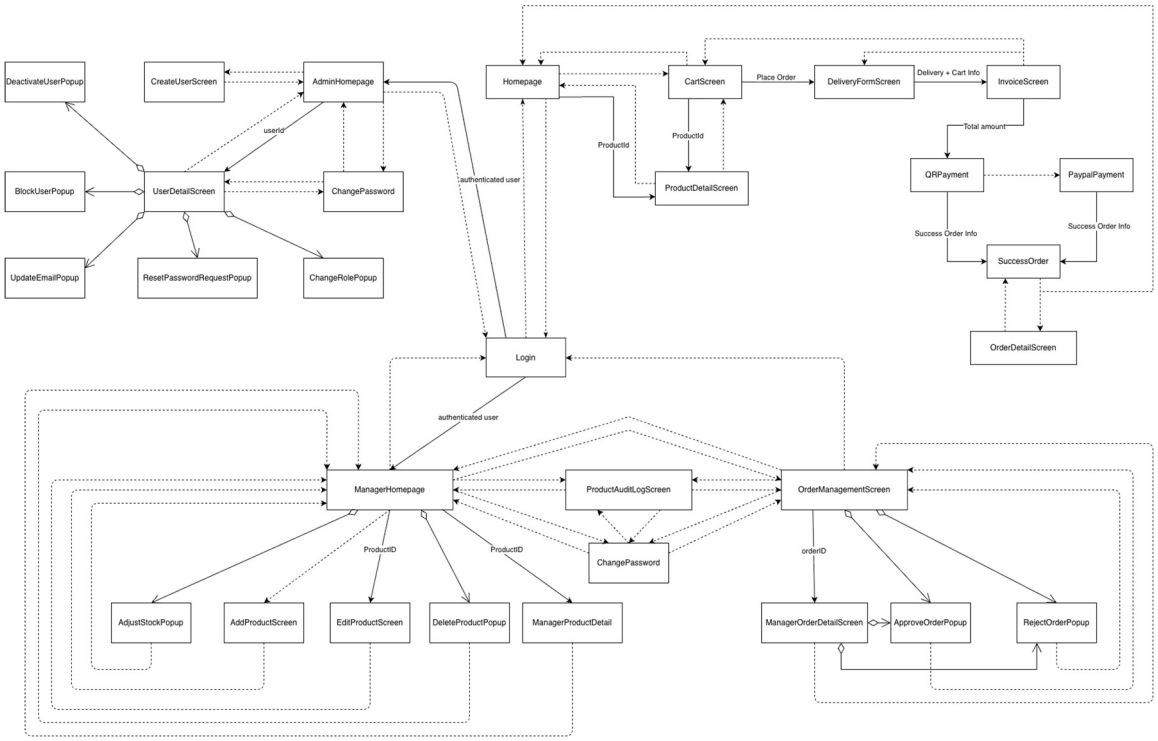
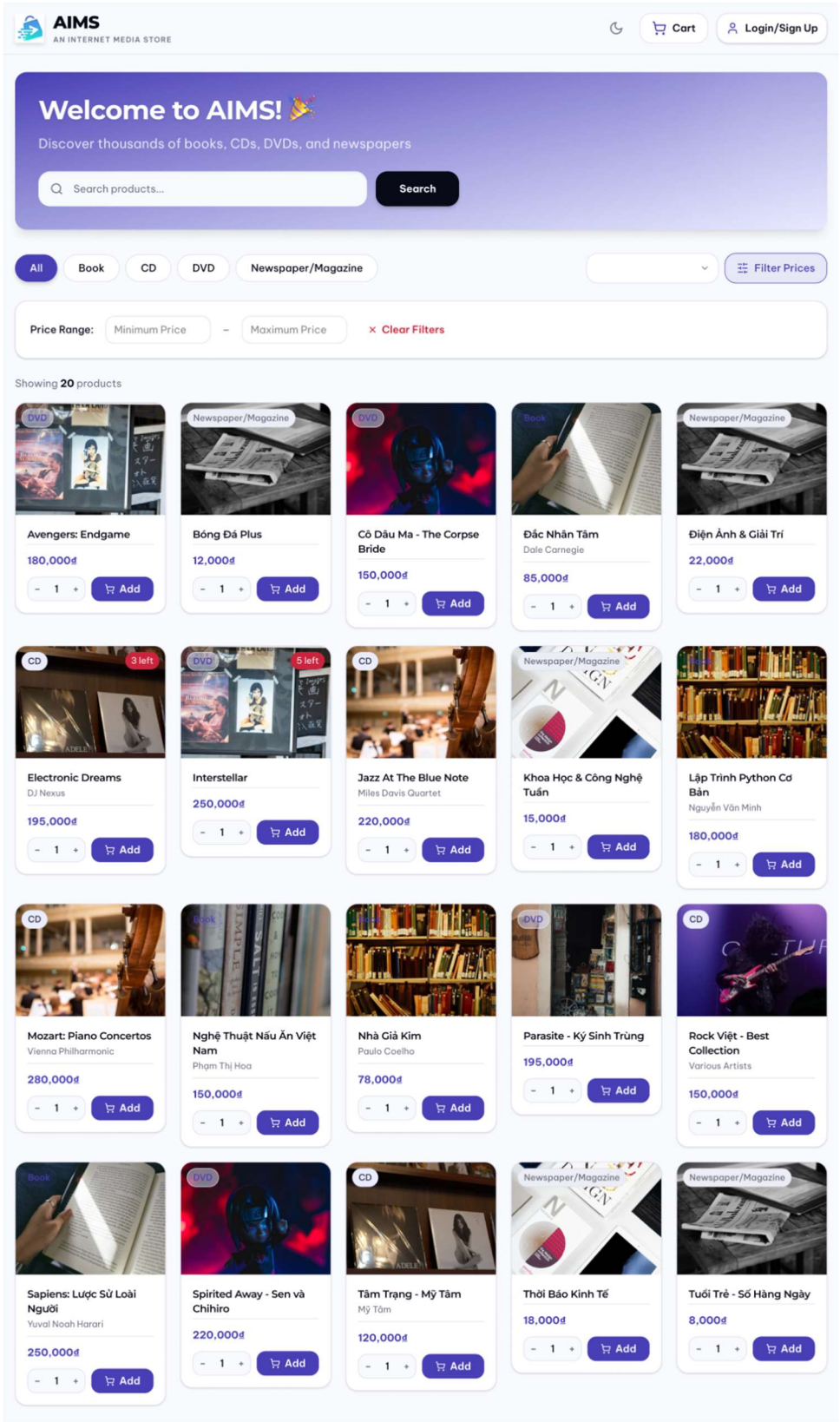


Figure 22. Screen Transition Diagram for AIMS System

4.1.3 Screen Specifications

Screen Specification for "Homepage"


AIMS
AN INTERNET MEDIA STORE
Cart
Login/Sign Up
Welcome to AIMS!
Discover thousands of books, CDs, DVDs, and newspapers
Search products...
Search
All
Book
CD
DVD
Newspaper/Magazine
Filter Prices
Price Range:
Minimum Price
–
Maximum Price
Clear Filters
Showing 20 products
Avengers: Endgame
180,000đ
Bóng Đá Plus
12,000đ
Cô Dâu Ma - The Corpse Bride
150,000đ
Đắc Nhân Tâm
Dale Carnegie
85,000đ
Điện Ảnh & Giải Trí
22,000đ
Electronic Dreams
DJ Nexus
195,000đ
3 left
Interstellar
250,000đ
5 left
Jazz At The Blue Note
Miles Davis Quartet
220,000đ
Khoa Học & Công Nghệ Tuần
15,000đ
Lập Trình Python Cơ Bản
Nguyễn Văn Minh
180,000đ
Mozart: Piano Concertos
Vienna Philharmonic
280,000đ
Nghệ Thuật Nấu Ăn Việt Nam
Phạm Thị Hoa
150,000đ
Nhà Giả Kim
Paulo Coelho
78,000đ
Parasite - Ký Sinh Trùng
195,000đ
Rock Việt - Best Collection
Various Artists
150,000đ
Sapiens: Lược Sử Loài Người
Yuval Noah Harari
250,000đ
Spirited Away - Sen và Chihiro
220,000đ
Tâm Trạng - Mỹ Tâm
Mỹ Tâm
120,000đ
Thời Báo Kinh Tế
18,000đ
Tuổi Trẻ - Số Hàng Ngày
8,000đ
Add
© 2026 AIMS - An Internet Media Store. All rights reserved.
Hotline: 1800-AIMS
support@aims.vn

Date of creation	Approved by	Reviewed by	Person in charge
17/04/2026	Lê Hoàng Kiên	Nguyễn Tiến Đạt	Nguyễn Huy Diễn

1. Controls, Operations, and Functions

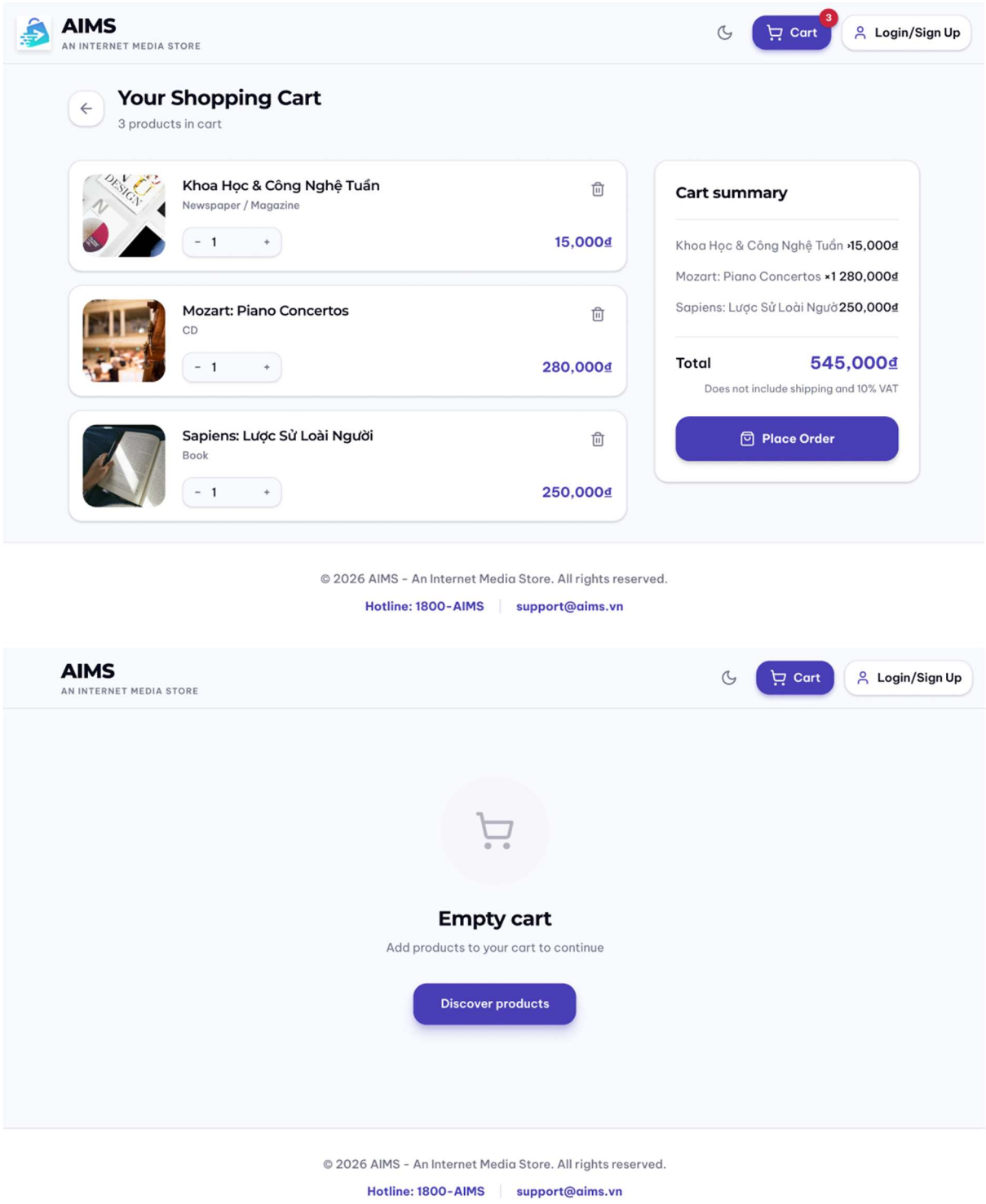
Control	**Operation**	**Function**
Area for displaying product list	Initial	Display product list
Product item card, showing product type, picture, title, author(s)/artist(s), and price	Initial/Click	Show product detail
[Each product item] Area for adjusting quantity	Click +/- or Input	Adjust the quantity of the product item to be added to cart
[Each product item] Add button with cart symbol	Click	Add the product with specified quantity to cart
Area with welcome message and search bar	Initial	Greets customer
Search bar & Search button	Input & Enter	Search items in store
Button area “All”, “Book”, …	Click	Filters items by product type
Dropdown selection box	Click & Select	Sort items by alphabet or by price
Filter price Box	Click	Show an area to filter by price
Price Range area	Input	Define price range to filter

Moon Symbol	Click	Toggle Light/Dark mode
Cart button with cart symbol	Click	Show cart
Login button	Click	Show login screen for product manager and admin.

2. Field attributes

Item	**Length**	**Type**	**Color**	**Remarks**
Media title	40 characters	Text	Black	Left-Justified
Author(s) / Artist(s)	30 characters	Text	Gray	Left-Justified
Price	8 digits + ₫	Numeral	Purple / Indigo	Left-Justified

Screen Specification for "CartScreen"



(cart screen when empty)

Date of creation	Approved by	Reviewed by	Person in charge

18/04/2026	Nguyễn Tiến Đạt	Lê Hoàng Kiên	Nguyễn Huy Diễn

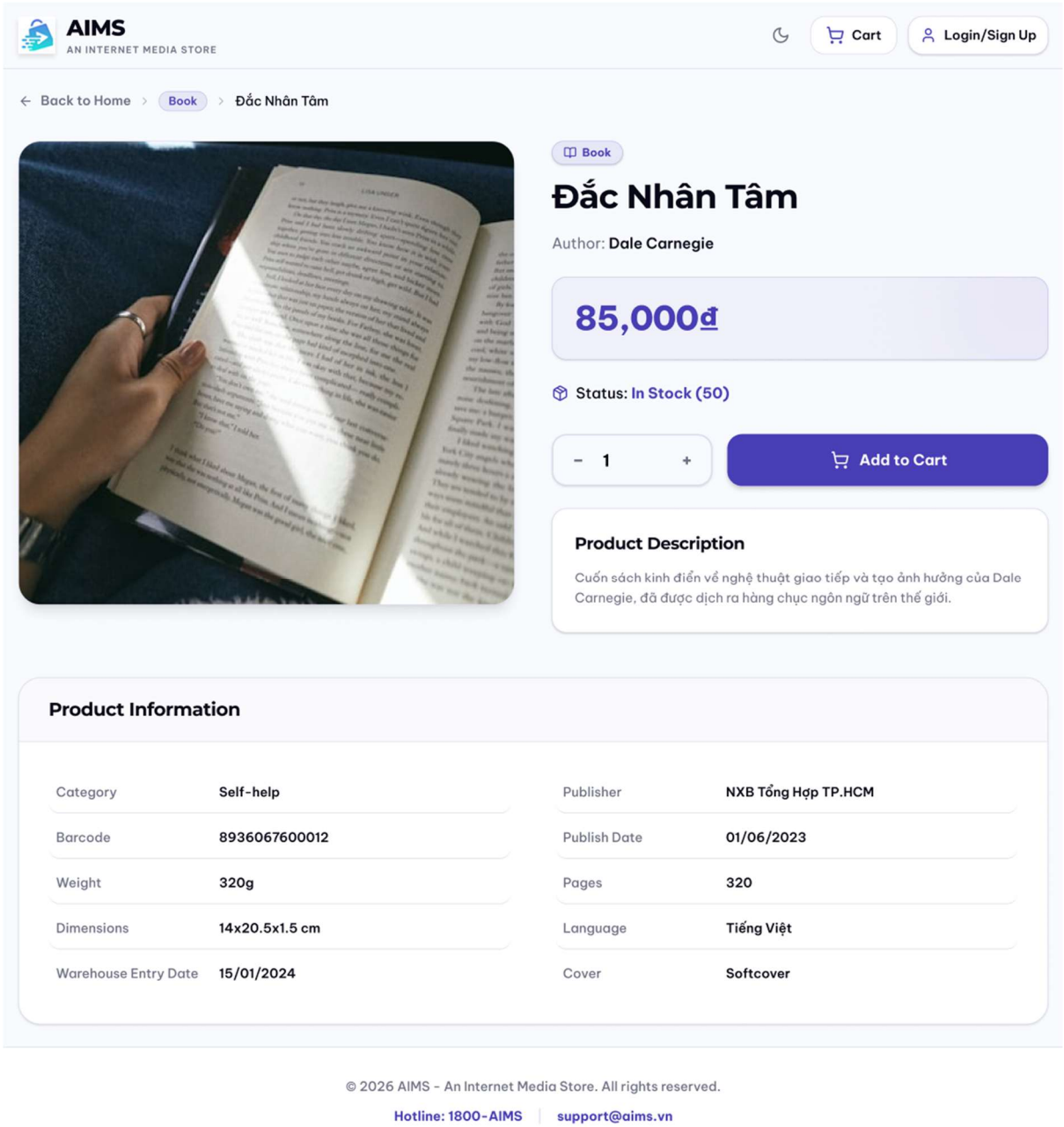
1. Controls, Operations, and Functions

Control	**Operation**	**Function**
Area for displaying cart content	Initial	Display cart content
Area for displaying cart summary	Initial	Display cart summary
Place order	Click	Display / Transition to Delivery Form screen
Back button	Click	Back to Homepage
Shopping cart title with number of products summary	Initial	Show number of products in cart
[Each cart item] Area for adjusting quantity	Click +/- or Input	Adjust the quantity of the cart item
[Each cart item] Trashbin button	Click	Remove item from cart
[When cart is empty] Empty cart content area	Initial	Show notification that cart is empty
[When cart is empty] Discover products Button	Click	Back to Homepage

2. Field attributes

Item	Length	Type	Color	Remarks
Media title	40 characters	Text	Black	Left-Justified
Author(s) / Artist(s)	30 characters	Text	Gray	Left-Justified
Price	8 digits + ₫	Numeral	Purple / Indigo	Right-Justified
Media title with multiplier	30 characters + x[digits]	Text	Black	Left-Justified
Total	10 digits + ₫	Numeral	Purple / Indigo	Right-Justified

Screen Specification for “ProductDetailScreen”



Date of creation	Approved by	Reviewed by	Person in charge
18/4/2026	Nguyễn Tiến Đạt	Phạm Đức Phước	Lê Hoàng Kiên

1. Controls, Operations, and Functions

Control	Operation	Function
Image (left)	Initial	Display representative image of the product
Area for navigation-like display with “Back to Home” button (top)	Initial Click	Display the button to go back to Homepage. Show the type of product and title of product
Product summary area	Initial	Display general information of the product (type, title, author(s)/artist(s), price, description, stock status)
Area for adjusting quantity	Click +/- or Input	Adjust the quantity of the product item to be added to cart
Add button with cart symbol	Click	Add the product with specified quantity to cart
Product information area	Initial	Show more detailed information about the product

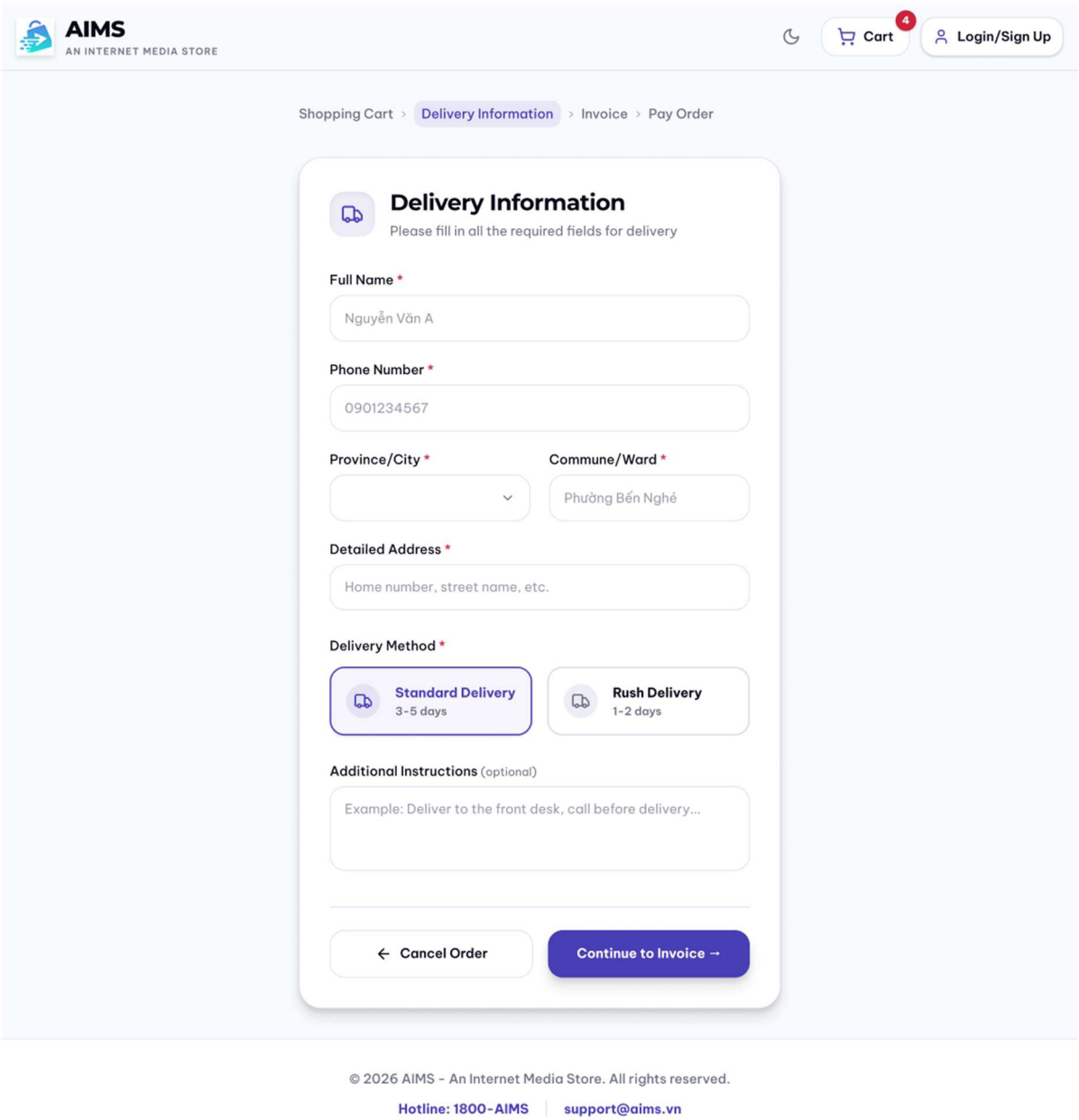
2. Field attributes

Item	Length	Type	Color	Remarks
Title	40 characters	Text	Black	Left-justified
Barcode	13 characters	Text	Gray	Left-justified
Category	30 characters	Text	Black	Left-justified
Price	8 digits + ₫	Numeral	Purple	Left-justified
Stock Quantity	3 digits	Numeral	Purple	Left-justified

Weight	5 digits	Numeral	Black	Left-justified
Dimensions	3 digits x 3 digits x 3 digits	Numeral	Black	Left-justified
Warehouse Entry Date		Date	Black	Left-justified
Product Description	1000 characters	Text	Black	Left-justified
Author	30 characters	Text	Black	Left-justified
Publisher	30 characters	Text	Black	Left-justified
Publication Date		Date	Black	Left-justified
Pages	5 digits	Numeral	Black	Left-justified
Language	10 characters	Text	Black	Left-justified
Genre	20 characters	Text	Black	Left-justified
Cover Type	10 characters	Text	Black	Left-justified
Artist/Band	30 characters	Text	Black	Left-justified
Record Label	30 characters	Text	Black	Left-justified
Release Date		Date	Black	Left-justified
Number of tracks	2 digits	Numeral	Black	Left-justified
Studio	30 characters	Text	Black	Left-justified
Director	30 characters	Text	Black	Left-justified
Runtime	4 digits	Numeral	Black	Left-justified

Disc Type	15 characters	Text	Black	Left-justified
Main Topic	30 characters	Text	Black	Left-justified

Screen Specification for “DeliveryFormScreen”



Date of creation	Approved by	Reviewed by	Person in charge

19/4/2026	Phạm Đức Phước	Lê Hoàng Kiên	Nguyễn Huy Diễn

1. Controls, Operations, and Functions

Control	Operation	Function
Area for navigation	Initial	Display steps in the Order Placement flow and highlight the current step
Area for inputting delivery information	Initial	Display area for inputting delivery information
Full name input textbox	Input	Box for inputting customer's full name
Phone number input textbox	Input	Box for inputting customer's phone number
Province / City dropdown selection box	Click & Input	Box for choosing customer's province or city
Commune / Ward input textbox	Input	Box for inputting customer's commune or ward
Detailed address input textbox	Input	Box for inputting customer's detailed address
Delivery method selection area	Click & Select	Area for choosing delivery method
Additional instructions input textbox	Input	Box for inputting customer's additional instructions

Screen Specification for "InvoiceScreen"

AIMS
AN INTERNET MEDIA STORE

Cart 4 | Login/Sign Up

Cart › Delivery Information › Invoice › Payment

INVOICE

Please review the information carefully before proceeding with the payment

Delivery Information

Recipient
Nguyễn Huy Diễn

Phone Number
0901234567

Delivery Address
Số 1 Đại Cồ Việt, Bách Khoa, Hà Nội

Delivery Method
Standard Delivery (3-5 days)

Product List

PRODUCT	QUANTITY	UNIT PRICE	TOTAL
 Bóng Đá Plus — Newspaper	1	12,000₫	12,000₫
 Sapiens: Lược Sử Loài Người — Book	1	250,000₫	250,000₫
 Tâm Trạng - Mỹ Tâm — Cd	1	120,000₫	120,000₫
 Interstellar — Dvd	1	250,000₫	250,000₫

Subtotal (excluding VAT)	632,000₫
VAT (10%)	+63,200₫
Subtotal (including VAT)	695,200₫
Shipping Fee (Standard Delivery (3-5 days))	+30,000₫
Grand Total	**725,200₫**

← Edit Delivery Information | Edit Cart | Proceed to Payment →

Hotline: 1800-AIMS | support@aims.vn

Date of creation	Approved by	Reviewed by	Person in charge
19/4/2026	Phạm Đức Phước	Nguyễn Tiến Đạt	Nguyễn Huy Diễn

1. Controls, Operations, and Functions

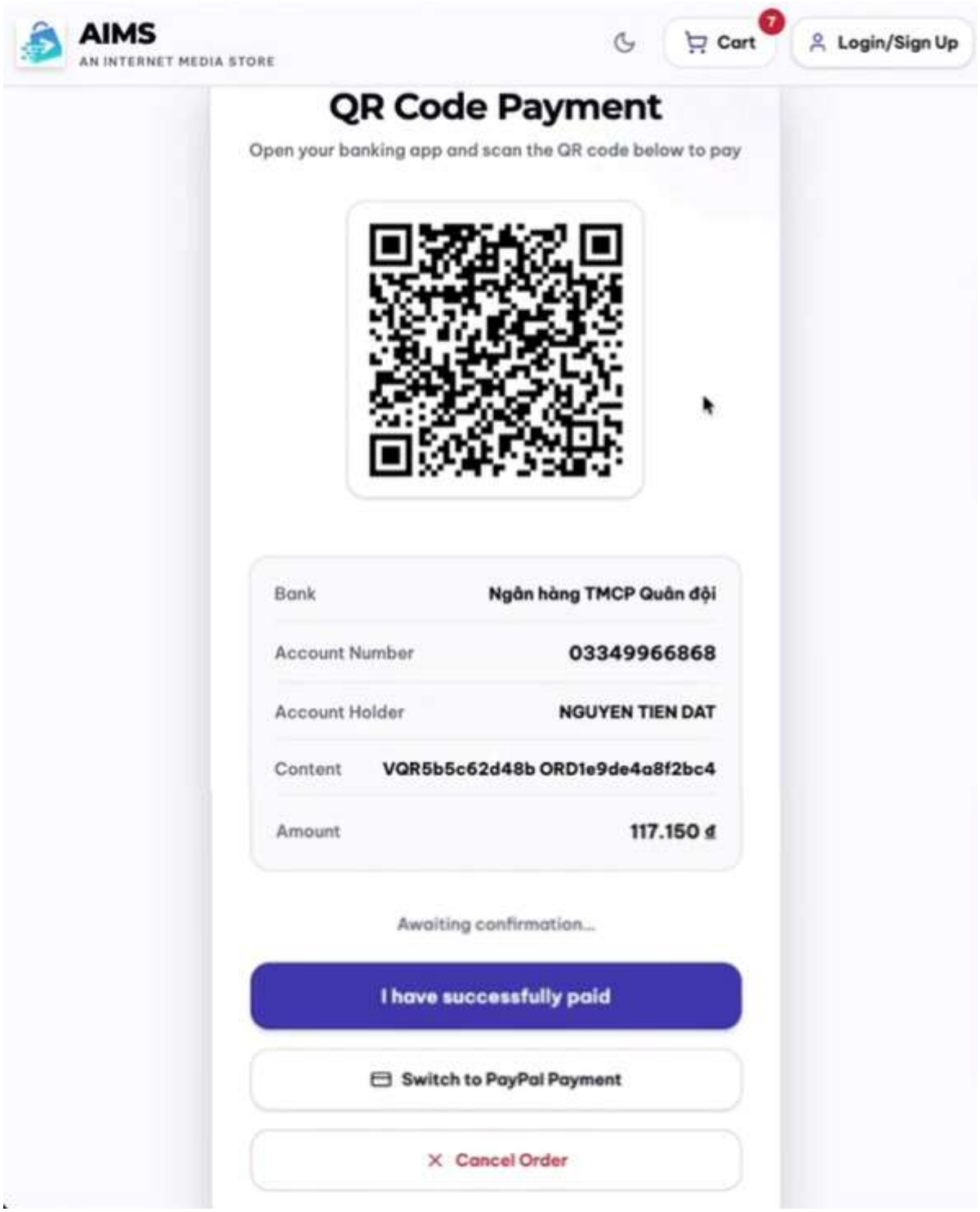
Control	**Operation**	**Function**
Area for navigation	Initial	Display steps in the Order Placement flow and highlight the current step
Invoice area	Initial	Display invoice information, including delivery information
Delivery information area	Initial	Show some general information about delivery form the customer has just submitted
Product list area	Initial	Show the product list the customer places
Price calculation area	Initial	Show subtotal, VAT, subtotal with VAT, shipping fee, and the grand total
Edit delivery information button	Click	Switch back to Delivery Form screen
View cart button	Click	Switch back to Cart Screen
Proceed to Payment button	Click	Switch to QR Payment Screen

2. Field attributes

Item	**Length**	**Type**	**Color**	**Remarks**

Recipient	50 characters	Text	Black	Left-justified
Phone number	10 digits	Text	Black	Left-justified
Delivery address	100 characters	Text	Black	Left-justified
Media title	40 characters	Text	Black	Left-justified
Media type	10 characters	Text	Gray	Left-justified
Quantity	3 digits	Numeral	Black	Left-justified
Unit price	8 digits + ₫	Numeral	Gray	Right-justified
Total price	8 digits + ₫	Numeral	Black	Right-justified
Subtotal	8 digits + ₫	Numeral	Black	Right-justified
VAT	8 digits + ₫	Numeral	Black	Right-justified
Subtotal with VAT	8 digits + ₫	Numeral	Black	Right-justified
Shipping Fee	8 digits + ₫	Numeral	Black	Right-justified
Grand Total	8 digits + ₫	Numeral	Purple / Indigo	Right-justified

Screen Specification for “QRPaymentScreen”



Date of creation	Approved by	Reviewed by	Person in charge

18/4/2026	Nguyễn Huy Diễn	Phạm Đức Phước	Nguyễn Tiến Đạt

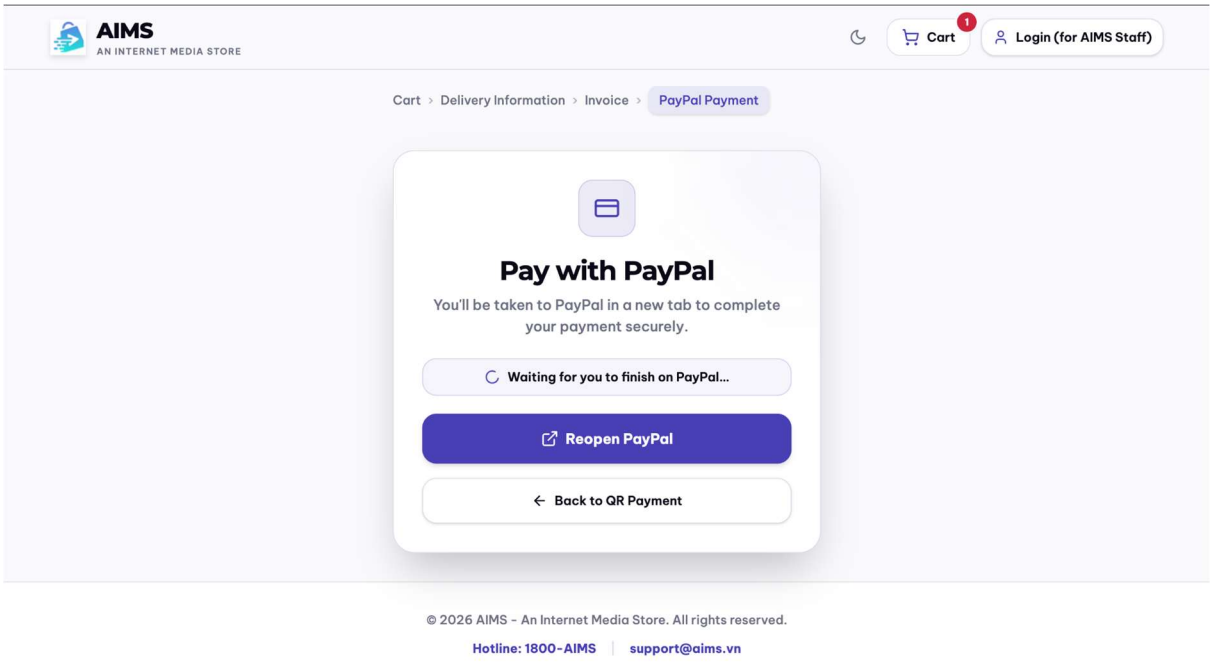
1. **Controls, Operations, and Functions**

Control	**Operation**	**Function**
QR Code Display Area	Initial	Display the payment QR code.
Bank Information Area	Initial	Display receiving bank information.
Account Number Area	Initial	Display merchant bank account number.
Account Holder Area	Initial	Display account holder name.
Payment Content Area	Initial	Display payment description/content.
Amount Area	Initial	Display total payment amount.
Payment Status Area	Initial	Display current payment status.
Button "I have successfully paid"	Click	Verify payment and complete the order.
Button "Switch to PayPal Payment"	Click	Redirect to PayPal payment screen.
Button "Cancel Order"	Click	Cancel the order and return to the previous screen.

2. Field attributes

Item	Length	Type	Color	Remarks
QR Code Image	N/A	Image	Default	Displays payment QR code.
Payment Title	30 characters	Text	Black	"QR Code Payment".
Bank	100 characters	Text	Black	Receiving bank name.
Account Number	20 digits	Numeral	Black	Merchant account number.
Account Holder	100 characters	Text	Black	Account owner name.
Payment Content	50 characters	Text	Black	Payment reference content.
Amount	20 digits	Numeral	Black	Total payment amount.
Payment Status	50 characters	Text	Gray	Current payment status.
Button "I have successfully paid"	N/A	Button	Purple	Verify payment.
Button "Switch to PayPal Payment"	N/A	Button	Gray	Switch payment method.
Button "Cancel Order"	N/A	Button	Red	Cancel current order.

Screen Specification for “PayPalPayment”



Date of creation	Approved by	Reviewed by	Person in charge
18/4/2026	Nguyễn Huy Diễn	Nguyễn Tiến Đạt	Phạm Đức Phước

1. **Controls, Operations, and Functions**

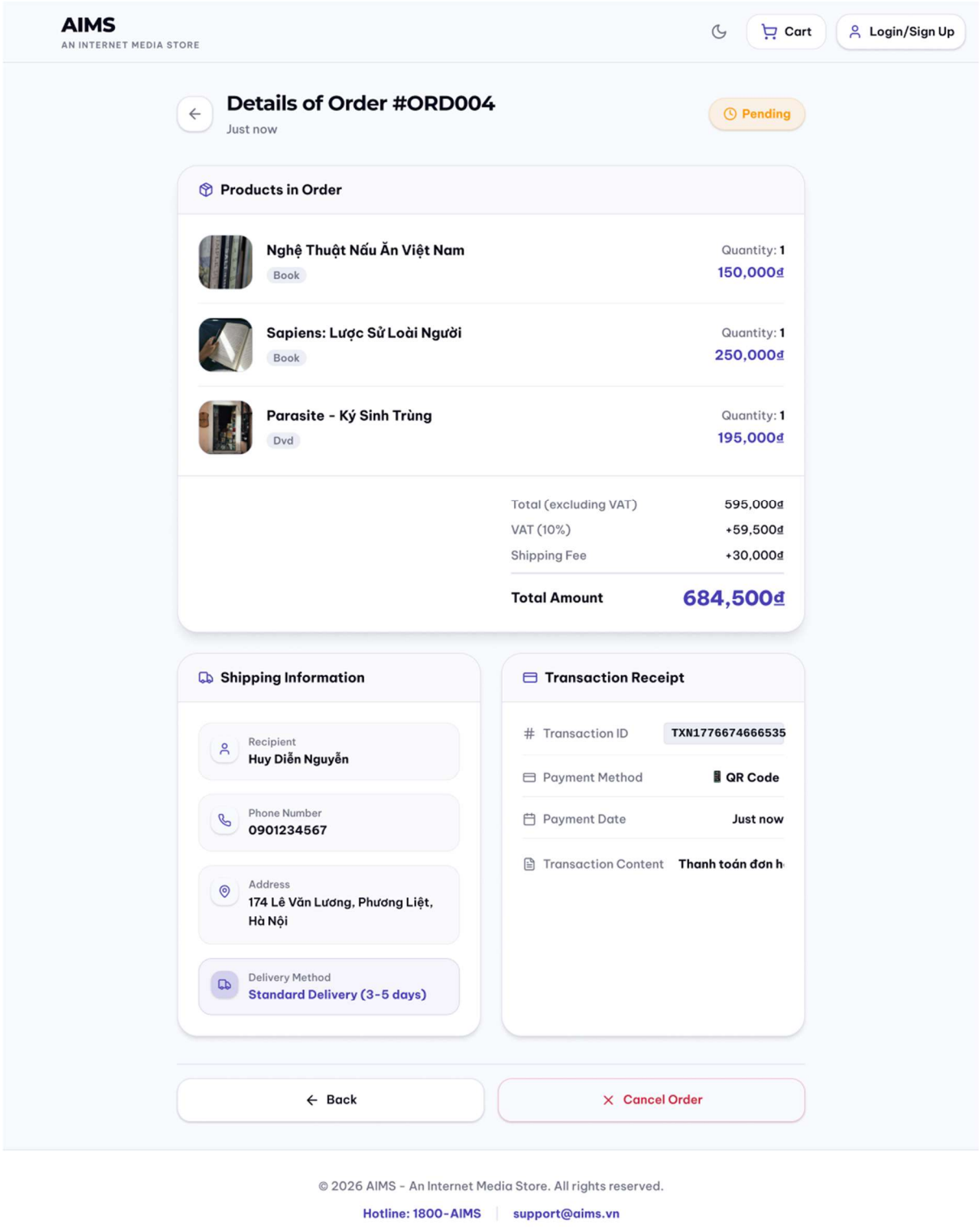
Control	**Operation**	**Function**
Button Reopen PayPal	Click	Redirect to PayPal website
Button Back to QR Payment	Click	Return to QRPaymentScreen

2. **Field attributes**

Item	**Length**	**Type**	**Color**	**Remarks**
Breadcrumb: Previous Steps	N/A	Text	Gray	Displays the previous steps in the checkout progression. Read-only, not clickable.

Breadcrumb: Current Step	N/A	Text	Gray + Indigo	Highlights the current step the user is currently on.
Screen Title	N/A	Text	Black	Main heading of the screen, bold formatting.
Instruction Text	N/A	Text	Gray	Instructs the user that the system will open a new tab for payment processing.
Status Banner: Idle*("Secured by PayPal Sandbox")*	N/A	Icon + Text	Black	Default initial state (state === 'idle'). Features a ShieldCheck icon.
Status Banner: Awaiting*("Waiting for you to finish...")*	N/A	Icon + Text	Black	State while waiting for the user to complete payment in the new tab (state === 'awaiting'). Features a spinning Loader icon.
Primary Button: Reopen PayPal	N/A	Button	Black	Alternative label for the primary button when state is awaiting, cancelled, or error. Allows the user to try opening the tab again.
Primary Button: Pay with PayPal	N/A	Button	Black	Main action button. Triggers handlePay() to open the PayPal approval URL in a new tab.
Primary Button: Connecting...	N/A	Button	Black	Loading state of the primary button while fetching the approval URL (isBusy === true). Click is disabled.
Secondary Button: Back to QR	N/A	Button	Black	Navigation button to return to the VietQR payment screen (/checkout/payment/qr). Disabled if isBusy === true.

Screen Specification for “OrderDetailScreen”



Date of creation	Approved by	Reviewed by	Person in charge
20/4/2026	Nguyễn Tiến Đạt	Phạm Đức Phước	Lê Hoàng Kiên

1. Controls, Operations, and Functions

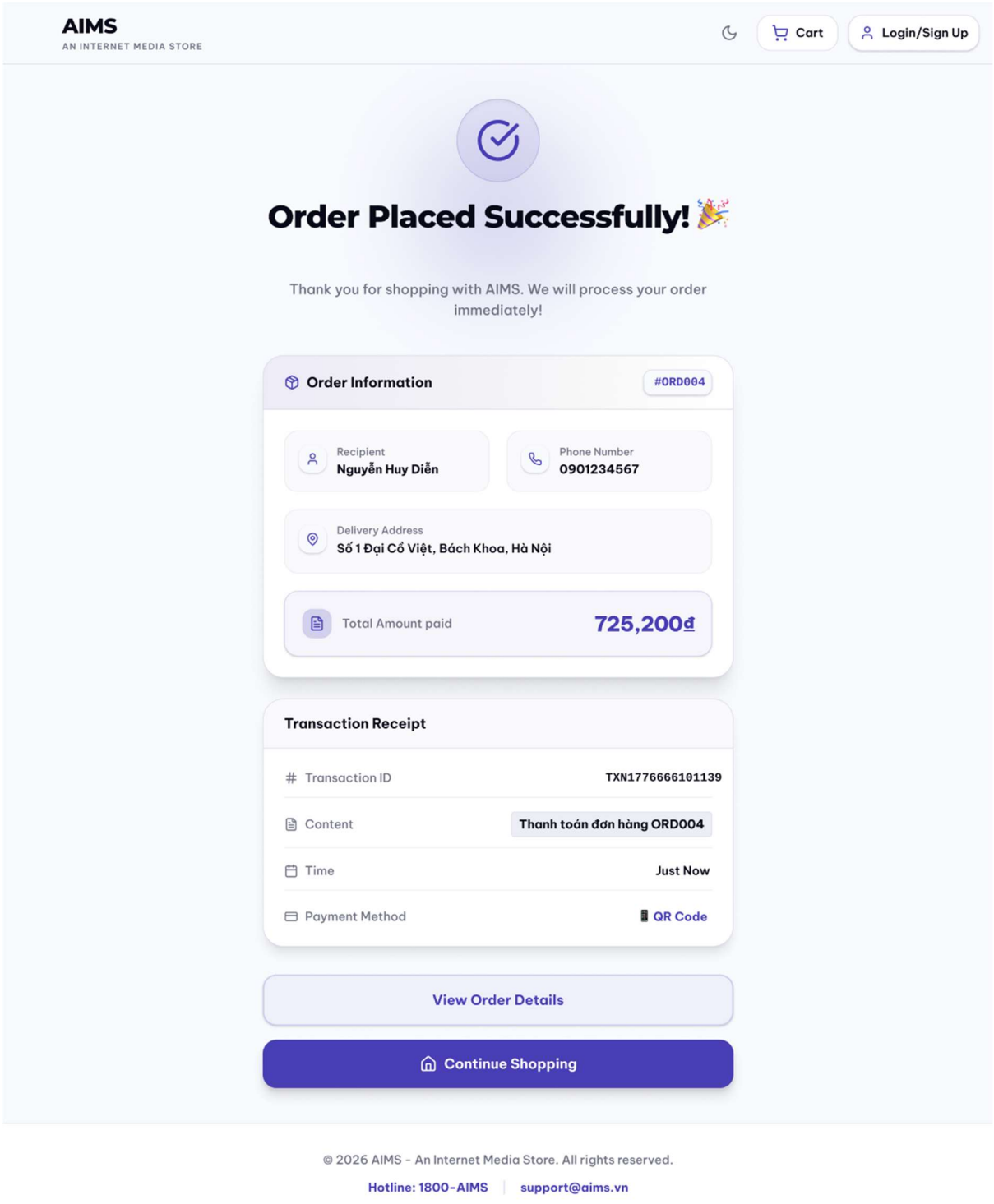
Control	**Operation**	**Function**
Button to return to order success screen	Click "←" or "← Back"	Return to SuccessOrderScreen
Cancel order button	Click	Confirm cancelling order
Area to display order information	Initial	Display order information

2. Field attributes

Item	**Length**	**Type**	**Color**	**Remarks**
Product title	40 characters	Text	Black	Left-justified
Product type	12 characters	Text	Gray	Left-justified
Unit price	8 digits + ₫	Numeral	Purple	Right-Justified
Quantity	3 digits	Numeral	Black	Right-Justified
Total Payment	8 digits + ₫	Numeral	Purple	Right-Justified
Recipient	50 characters	Text	Black	Left-justified

Phone	10 digits	Numeral	Black	Left-justified
Address	100 characters	Text	Black	Left-justified
Delivery method	50 characters	Text	Purple	Left-justified
Notes	30 characters	Text	Black	Left-justified
Transaction code	16 characters	Text	Black	Right-Justified
Method	20 characters	Text + relevant method symbol	Black	Right-Justified
Payment time	15 characters	Text	Black	Right-Justified
Transfer note	30 characters	Text	Black	Right-Justified

Screen Specification for “SuccessOrderScreen”



Date of creation	Approved by	Reviewed by	Person in charge
20/4/2026	Phạm Đức Phước	Nguyễn Tiến Đạt	Nguyễn Huy Diễn

1. **Controls, Operations, and Functions**

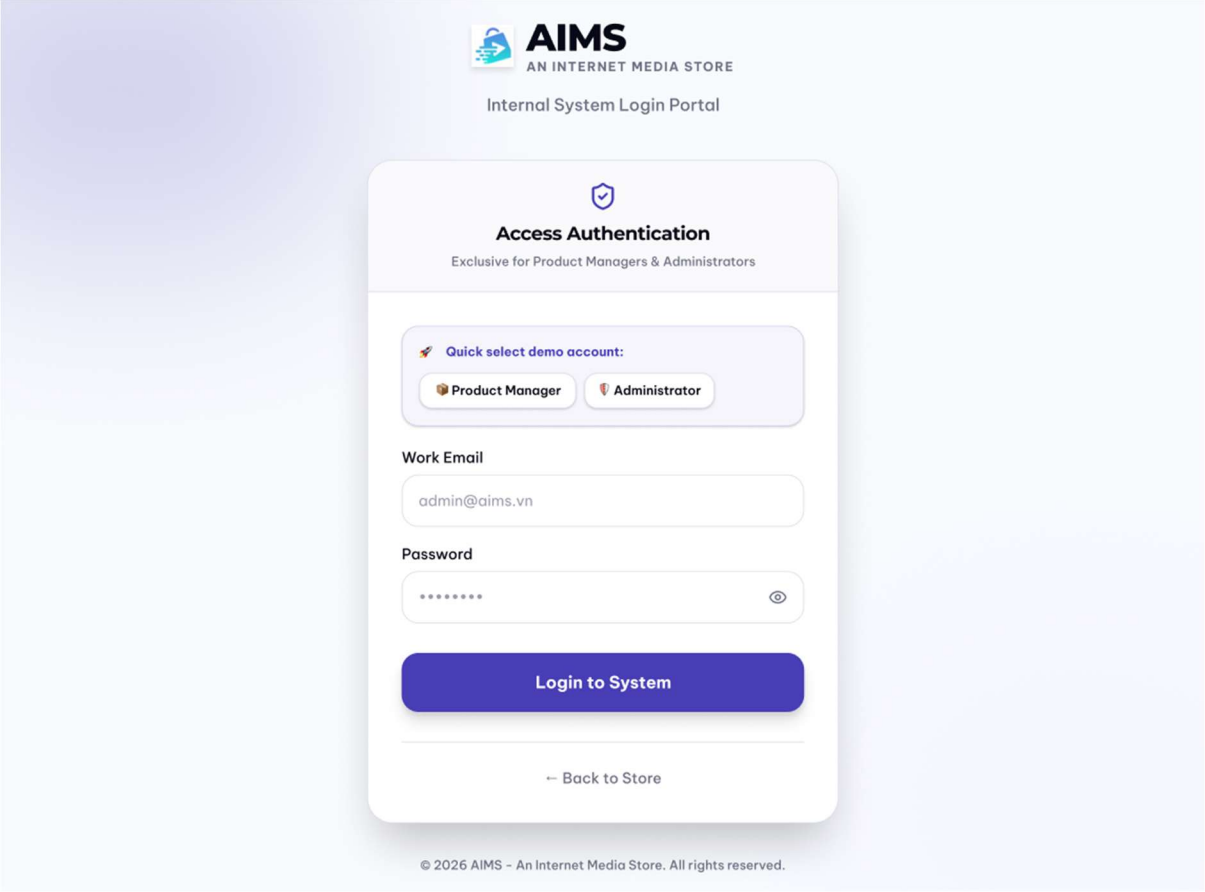
Control	**Operation**	**Function**
Success message area	Initial	Display order placement success message, along with an appreciation message
Order information area	Initial	Display general order information
Transaction receipt area	Initial	Display transaction information
View order details button	Click	Transition to Order Detail screen to show more detailed information
Continue shopping button	Click	Transition back to Homepage

2. **Field attributes**

Item	**Length**	**Type**	**Color**	**Remarks**
Order ID		Text	Purple / Indigo	Left-justified
Recipient	50 characters	Text	Black	Left-justified
Phone	10 digits	Text	Black	Left-justified
Address	100 characters	Text	Black	Left-justified
Total amount paid	8 digits + đ	Numeral	Purple / Indigo	Right-justified
Transaction ID		Text	Black	Right-justified
Transaction Content	100 characters	Text	Black	Right-justified

Transaction time		Text	Black	Right-justified
Payment method	15 characters	Text	Purple / Indigo	Right-justified

Screen Specification for “Login”



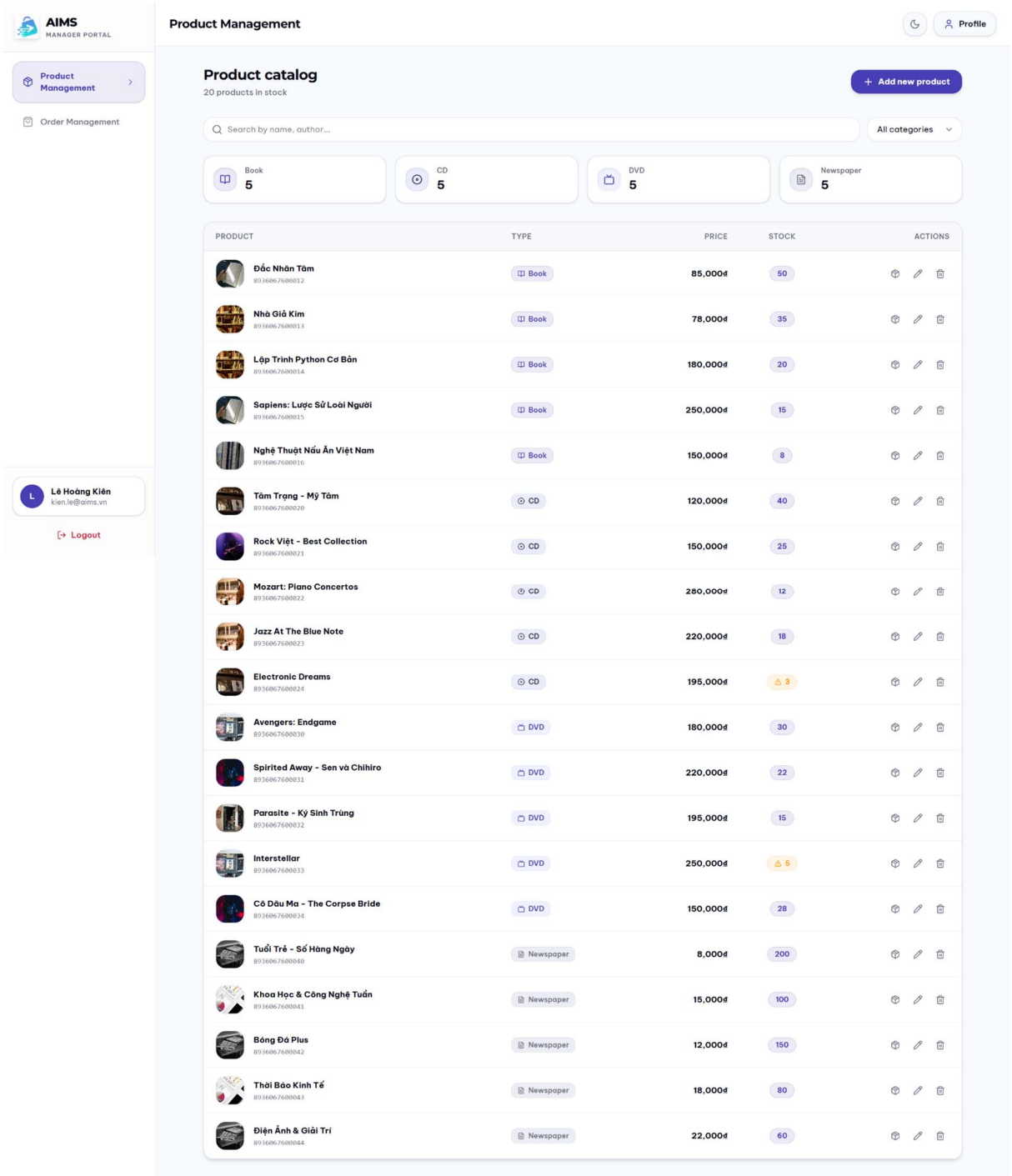
Date of creation	Approved by	Reviewed by	Person in charge
19/4/2026	Nguyễn Tiến Đạt	Nguyễn Huy Diễn	Lê Hoàng Kiên

1. Controls, Operations, and Functions

Control	Operation	Function

Area for filling email	Input	Fill in the email
Area for filling password	Input	Fill in the account password
Log in to System button	Click	Display the screen (manager/admin) based on the account role
Back to Store button	Click	Display home page

Screen Specification for “ManagerHomepage”



Date of creation	Approved by	Reviewed by	Person in charge
19/4/2026	Nguyễn Huy Diễn	Phạm Đức Phước	Lê Hoàng Kiên

1. **Controls, Operations, and Functions**

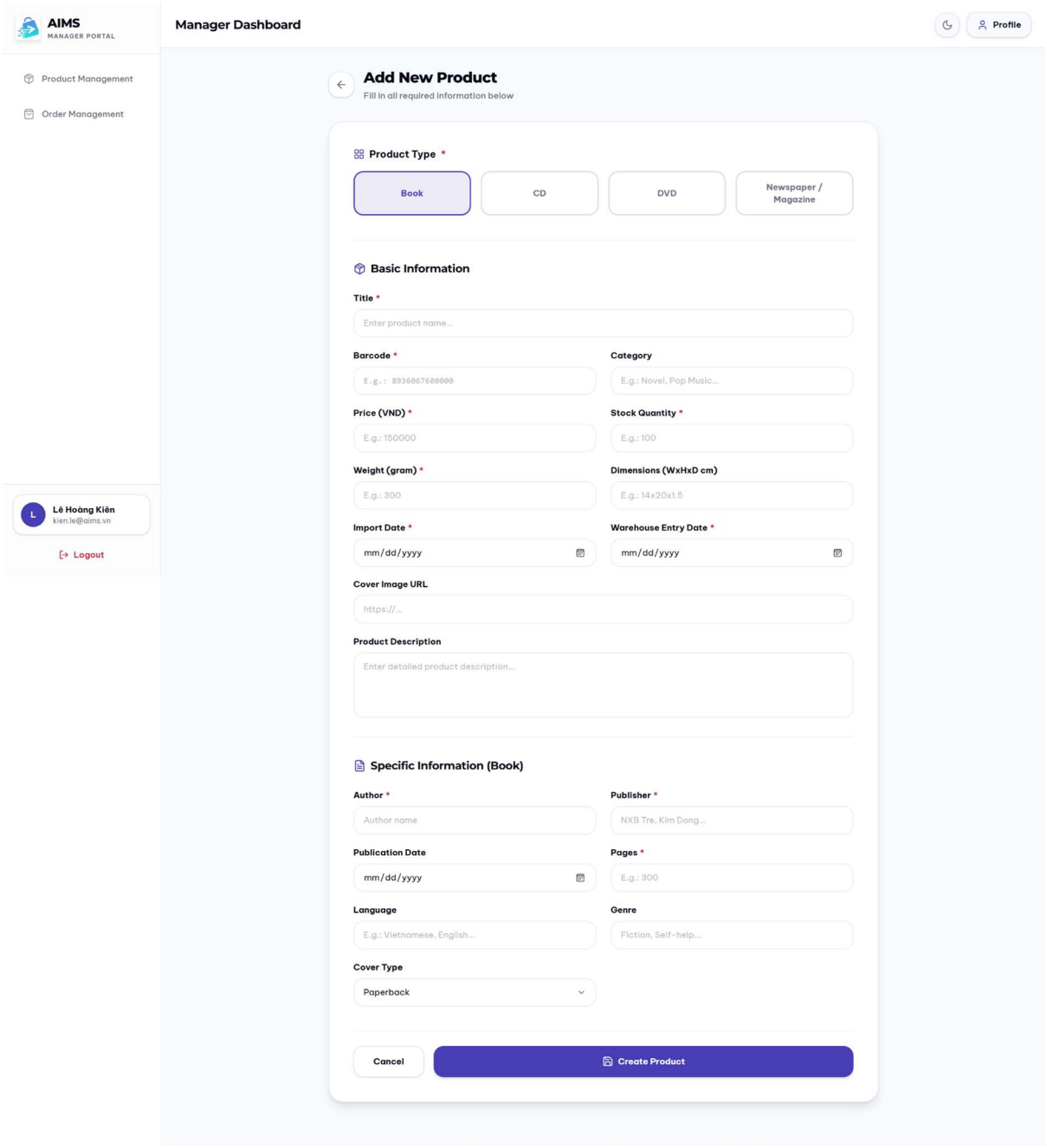
Control	**Operation**	**Function**
Profile button	Click	Displays UserProfile
Order Management button	Click	Displays OrderManagerScreen
Add new product button	Click	Displays AddProductScreen to add new product
Search product search bar	Input	Search product by name, author, …
Book/CD/DVD/Newspaper button	Click	Displays products of relevant product type
Area for displaying product	Initial/Click	Display the media with corresponding information / Click to display product details
Adjust stock button	Click	Displays AdjustStockPopup to adjust product stock level
Edit product button	Click	Displays EditProductScreen to edit product
Delete product button	Click	Displays DeleteProductPopup
Logout button	Click	Log out of the account, displays LoginSignup screen

2. Field attributes

Item	**Length**	**Type**	**Color**	**Remarks**

Media title	40 characters	Text	Black	Left-justified
Media type	12 characters	Text	Black	Left-Justified
Price	8 digits + ₫	Numeral	Purple / Indigo	Left-Justified
Stock	4 digits	Numeral	Purple / Yellow with warning sign	

Screen Specification for “AddProductScreen”

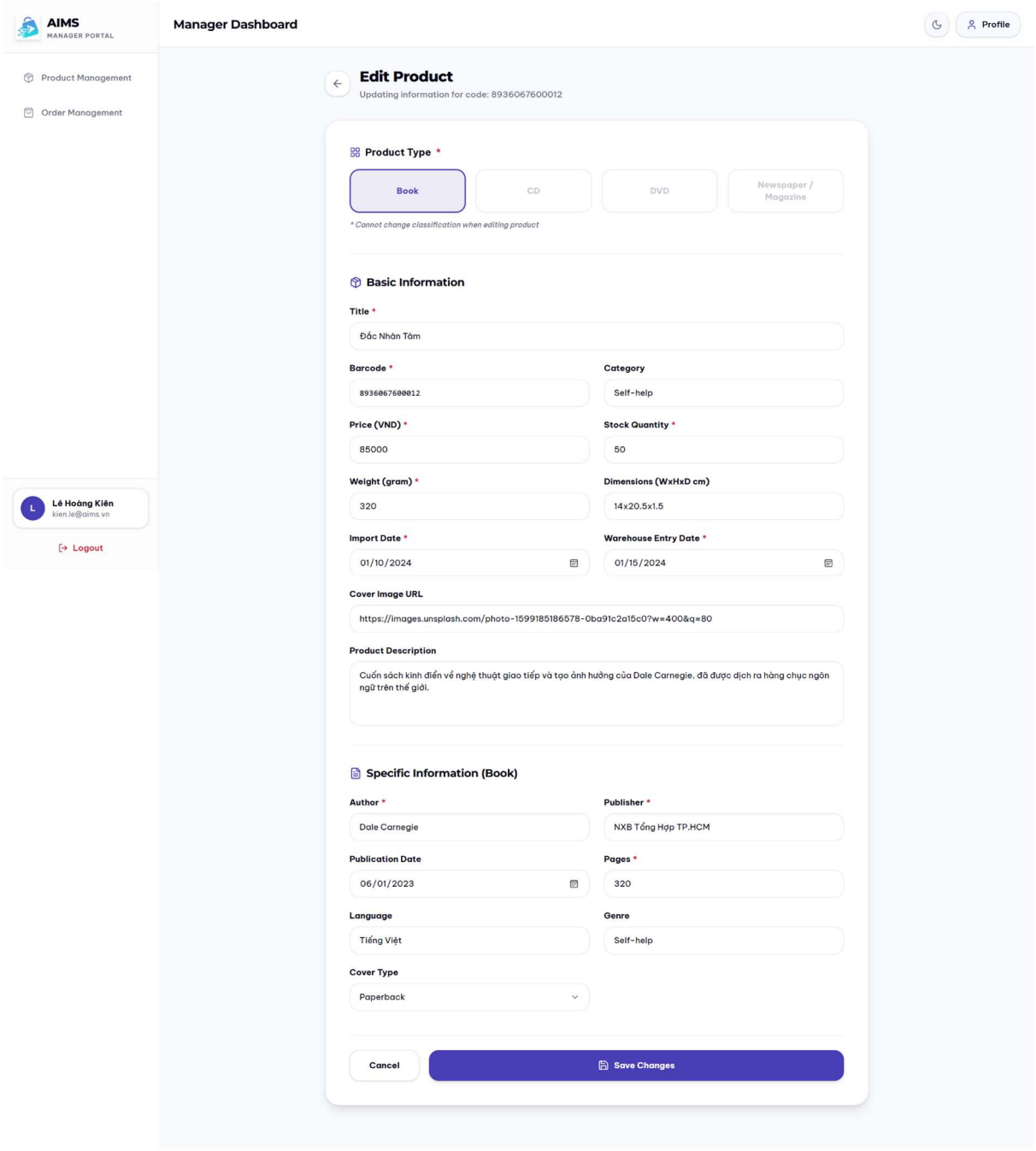


Date of creation	Approved by	Reviewed by	Person in charge
19/4/2026	Phạm Đức Phước	Lê Hoàng Kiên	Nguyễn Huy Diễn

1. Controls, Operations, and Functions

Control	Operation	Function
Book/CD/DVD/Newspaper/Magazine button	Click	Choose the product type
Area to fill in product basic and specific information	Input	Fill in product information
Product Management button	Click	Displays ManagerHomePage
Order Management button	Click	Displays OrderManagerScreen
Profile button	Click	Displays UserProfile
Logout button	Click	Log out of the account, displays LoginSignup screen
Cancel button	Click	Cancel adding new product
Create Product button	Click	Confirm creating product

Screen Specification for "EditProductScreen"

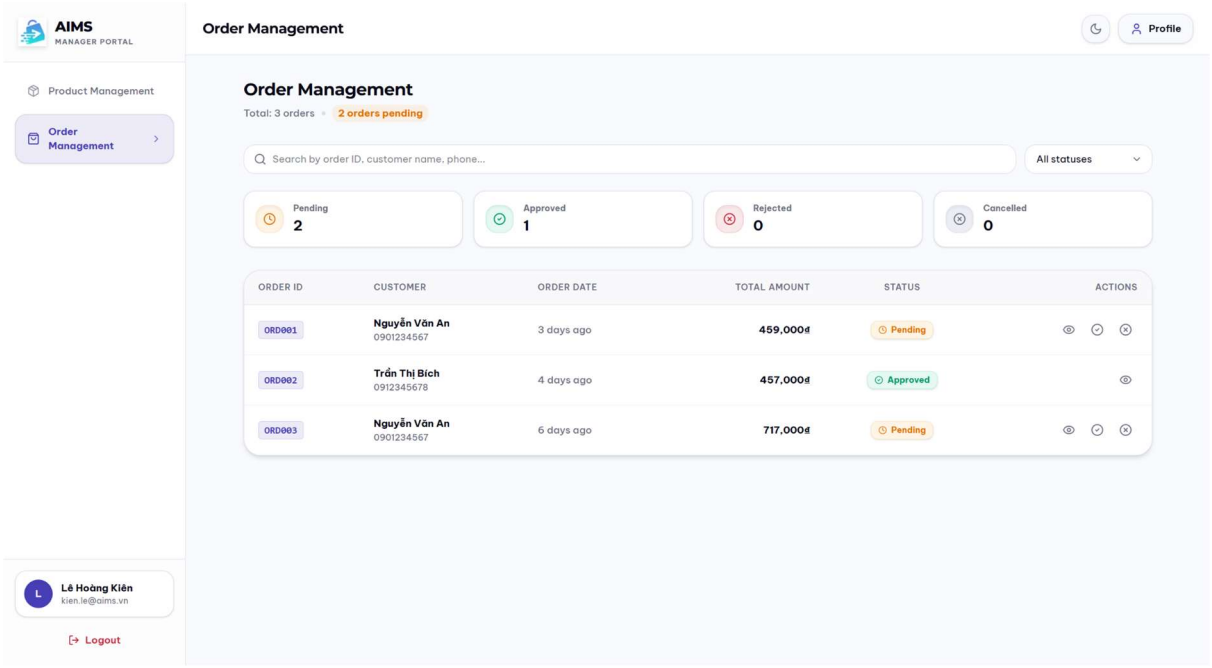


Date of creation	Approved by	Reviewed by	Person in charge
19/4/2026	Phạm Đức Phước	Nguyễn Huy Diễn	Lê Hoàng Kiên

1. Controls, Operations, and Functions

Control	Operation	Function
Book/CD/DVD/Newspaper/Magazine button	Click	Choose the product type
Product Management button	Click	Displays ManagerHomePage
Order Management button	Click	Displays OrderManagerScreen
Profile button	Click	Displays UserProfile
Area to fill in product basic and specific information	Input	Fill in product information
Logout button	Click	Log out of the account, displays LoginSignup screen
Cancel button	Click	Cancel editing product
Save Changes button	Click	Confirm editing product

Screen Specification for "OrderManagementScreen"



Date of creation	Approved by	Reviewed by	Person in charge
19/4/2026	Nguyễn Tiến Đạt	Phạm Đức Phước	Lê Hoàng Kiên

1. Controls, Operations, and Functions

Control	**Operation**	**Function**
Product Management button	Click	Displays ManagerHomePage
Profile button	Click	Displays UserProfile
Logout button	Click	Log out of the account, displays LoginSignup screen
Area to display order information	Initial	Display order information

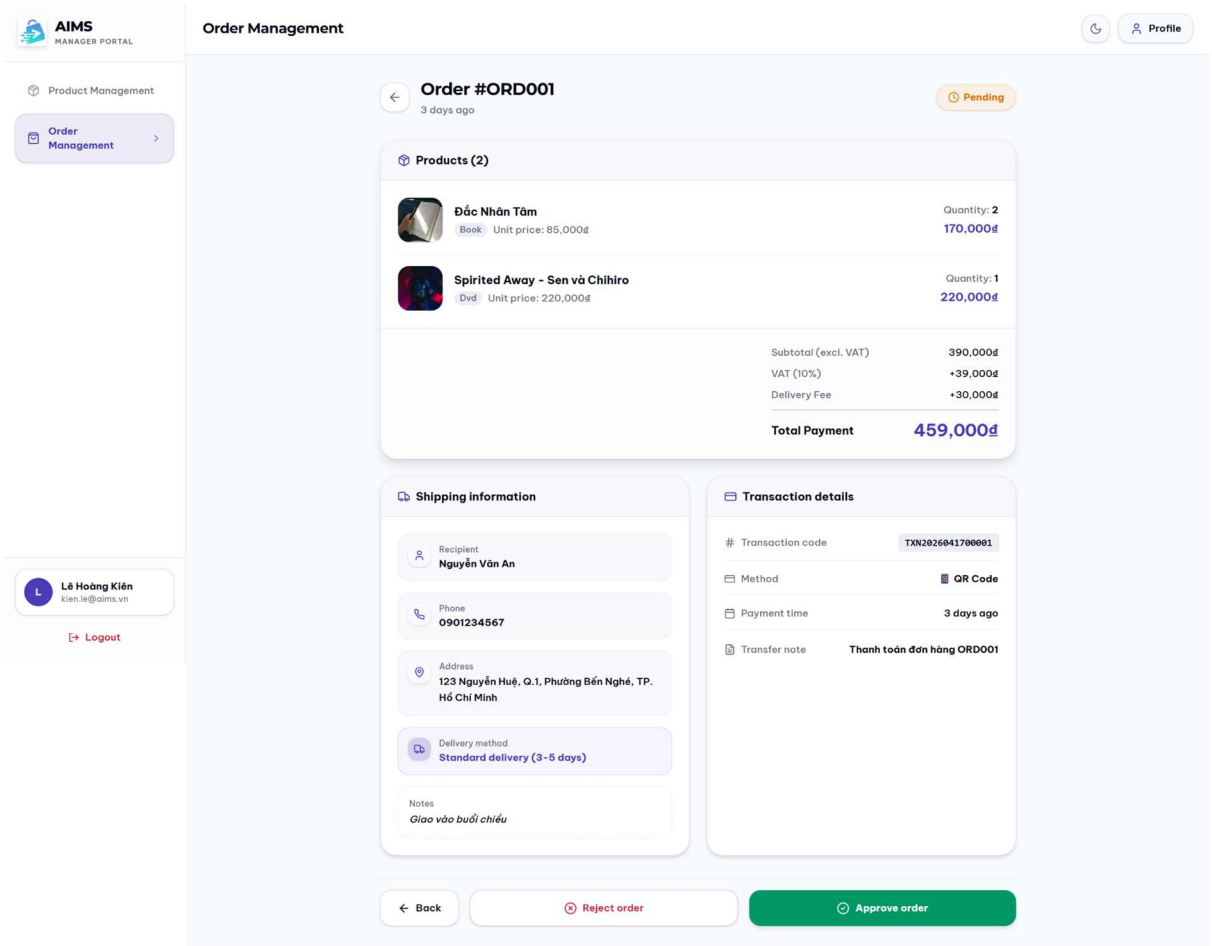
View details button	Click	Display ManagerOrderDetailScreen
Approve order button	Click	Display ApproveOrderPopup
Reject order button	Click	Display RejectOrderPopup
Pending/Approved/ Rejected/Cancelled button	Click	Show orders with corresponding status
Search bar	Input	Filter orders with corresponding field value

2. Field attributes

Item	**Length**	**Type**	**Color**	**Remarks**
Total	4 digits	Numeral	Gray	Left-justified
Pending/ Approved/ Rejected/ Cancelled amount	3 digits	Numeral	Black	Left-justified
ORDER ID	6 characters	Text	Purple	Left-justified
CUSTOMER name + phone	30 characters for name 10 digits for phone number	Text & Numeral	Black & Gray	Left-justified
ORDER DATE	15 characters	Text	Gray	Left-justified

TOTAL AMOUNT	8 digits + ₫	Numeral	Black	Left-Justified
STATUS	12 characters	Text	Yellow / Green / Red / Gray	Center alignment with corresponding symbol

Screen Specification for "ManagerOrderDetailScreen"



Date of creation	Approved by	Reviewed by	Person in charge
19/4/2026	Nguyễn Tiến Đạt	Nguyễn Huy Diễn	Lê Hoàng Kiên

1. **Controls, Operations, and Functions**

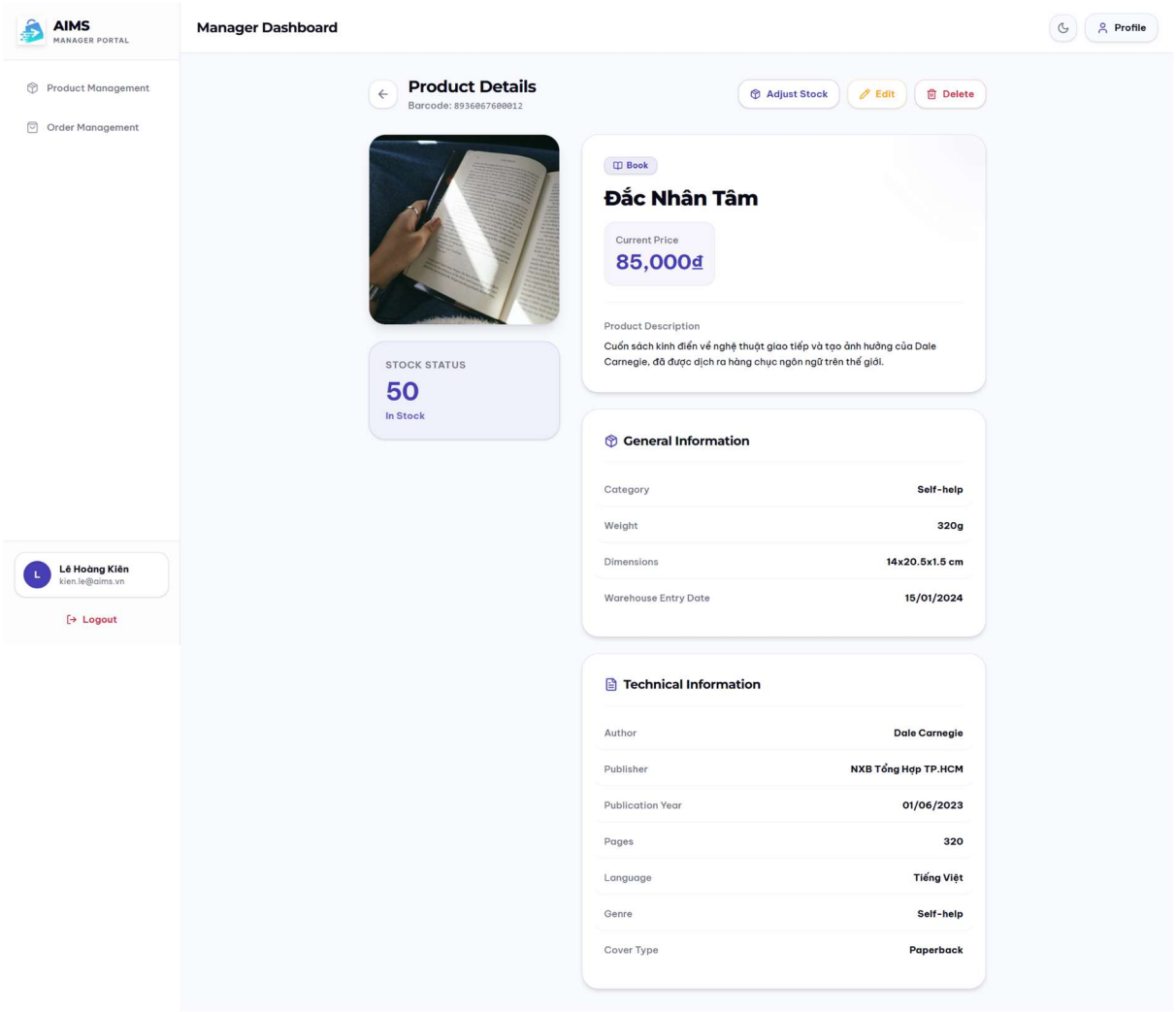
Control	Operation	Function
Product Management button	Click	Display ManagerHomePage
Profile button	Click	Display UserProfile
Logout button	Click	Log out of the account, displays LoginSignup screen
Button to return to order management screen	Click "←" or "← Back"	Return to OrderManagementScreen
Reject order button	Click	Display RejectOrderPopup
Approve order button	Click	Display ApproveOrderPopup
Area to display order information	Initial	Display order information

2. Field attributes

Item	Length	Type	Color	Remarks
Product title	40 characters	Text	Black	Left-justified
Product type	12 characters	Text	Gray	Left-justified
Unit price	8 digits + ₫	Numeral	Purple	Right-Justified
Quantity	3 digits	Numeral	Black	Right-Justified
Total Payment	8 digits + ₫	Numeral	Purple	Right-Justified

Recipient	2 characters	Text	Black	Left-justified
Phone	10 digits	Numeral	Black	Left-justified
Address	100 characters	Text	Black	Left-justified
Delivery method	50 characters	Text	Purple	Left-justified
Notes	30 characters	Text	Black	Left-justified
Transaction code	16 characters	Text	Black	Right-Justified
Method	20 characters	Text + relevant method symbol	Black	Right-Justified
Payment time	15 characters	Text	Black	Right-Justified
Transfer note	30 characters	Text	Black	Right-Justified

Screen Specification for “ManagerProductDetail”



Date of creation	Approved by	Reviewed by	Person in charge
19/4/2026	Phạm Đức Phước	Nguyễn Tiến Đạt	Lê Hoàng Kiên

1. Controls, Operations, and Functions

Control	**Operation**	**Function**
Product Management button	Click	Displays ManagerHomePage

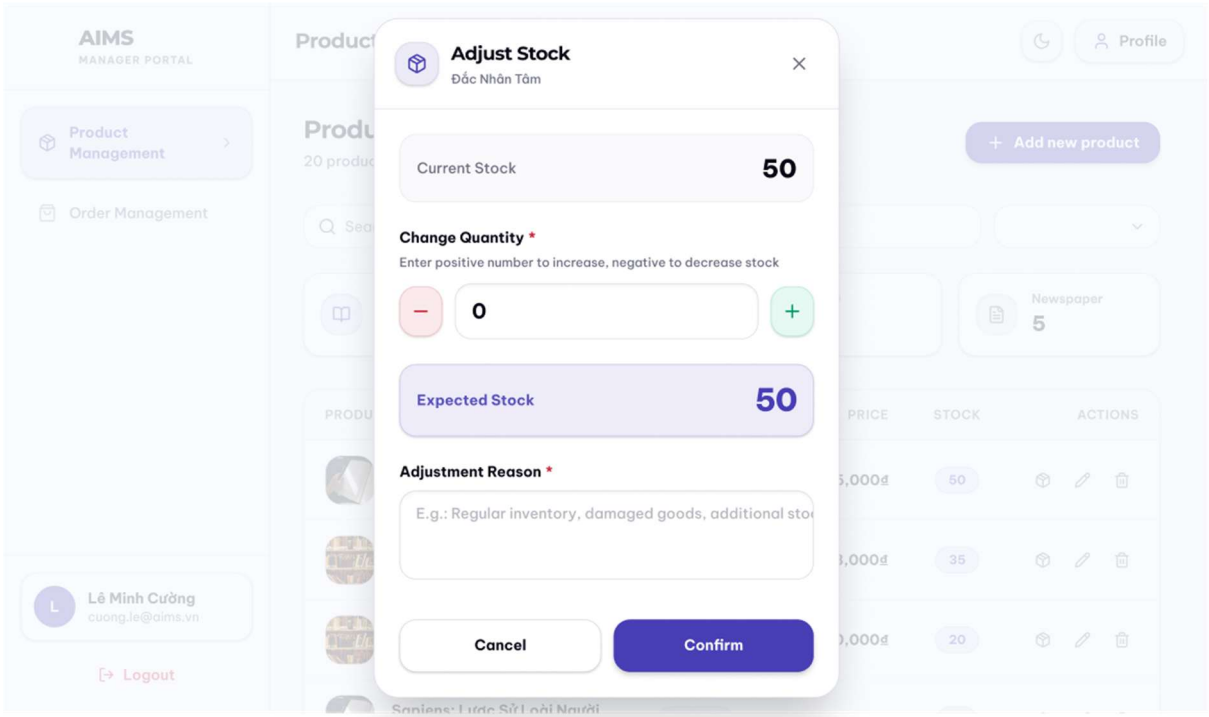
Order Management button	Click	Displays OrderManagerScreen
Profile button	Click	Displays UserProfile
Logout button	Click	Log out of the account, displays LoginSignup screen
Area to display product detail	Initial	Display product information
Adjust stock button	Click	Display AdjustStockPopup
Edit button	Click	Display EditProductScreen
Delete button	Click	Display DeleteProductPopup

2. Field attributes

Item	**Length**	**Type**	**Color**	**Remarks**
Title	40 characters	Text	Black	Right-Justified
Barcode	13 characters	Text	Gray	Left-justified
Category	30 characters	Text	Black	Right-Justified
Price	8 digits + ₫	Numeral	Purple	Left-justified
Stock Quantity	3 digits	Numeral	Purple	Left-justified
Weight	5 digits	Numeral	Black	Right-Justified
Dimensions	3 digits x 3 digits x 3 digits	Numeral	Black	Right-Justified

Warehouse Entry Date		Date	Black	Right-Justified
Product Description	1000 characters	Text	Black	Right-Justified
Author	30 characters	Text	Black	Right-Justified
Publisher	30 characters	Text	Black	Right-Justified
Publication Date		Date	Black	Right-Justified
Pages	5 digits	Numeral	Black	Right-Justified
Language	10 characters	Text	Black	Right-Justified
Genre	20 characters	Text	Black	Right-Justified
Cover Type	10 characters	Text	Black	Right-Justified
Artist/Band	30 characters	Text	Black	Right-Justified
Record Label	30 characters	Text	Black	Right-Justified
Release Date		Date	Black	Right-Justified
Number of tracks	2 digits	Numeral	Black	Right-Justified
Studio	30 characters	Text	Black	Right-Justified
Director	30 characters	Text	Black	Right-Justified
Runtime	4 digits	Numeral	Black	Right-Justified
Disc Type	15 characters	Text	Black	Right-Justified
Main Topic	30 characters	Text	Black	Right-Justified

Screen Specification for “AdjustStockPopup”



Date of creation	Approved by	Reviewed by	Person in charge
20/4/2026	Nguyễn Huy Diễn	Nguyễn Tiến Đạt	Lê Hoàng Kiên

1. Controls, Operations, and Functions

Control	**Operation**	**Function**
Current Stock	Initial	Current stock level of the product
Area to adjust quantity	Click - / + or Input the number of adding / subtracting stock level	Adjust the stock quantity
Textbox input	Input	Reason for adjusting stock

Cancel button	Click	Back to the current page
Confirm button	Click	Confirm adjusting stock level

2. Field attributes

Item	Length	Type	Color	Remarks
Current Stock	3 digits	Numeral	Black	Right-justified
Expected Stock	3 digits	Numeral	Purple / Indigo	Right-justified

4.2 Data Modeling

4.2.1 Conceptual Data Modeling

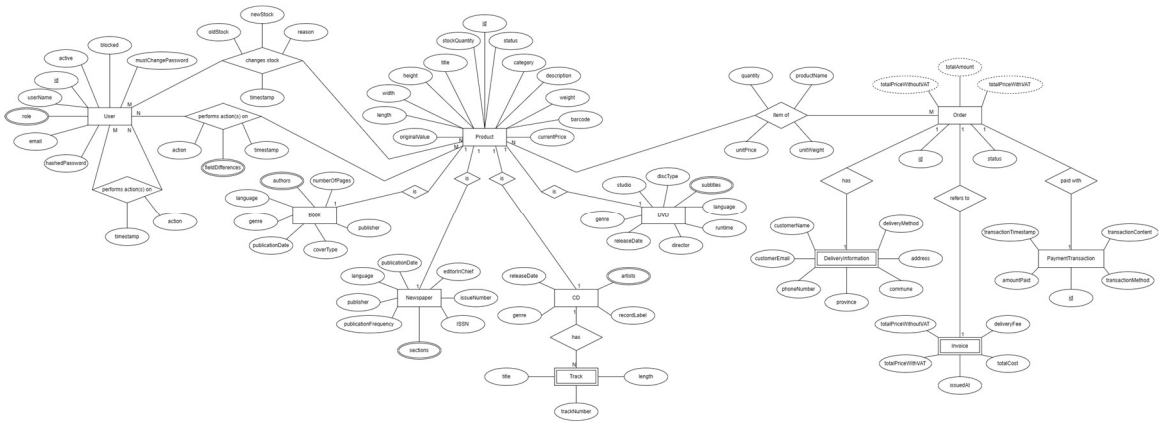


Figure 23. Entity-Relationship Diagram for AIMS System

Major entities of the AIMS system:

- User: The User entity represents all authenticated system actors: Product Managers and Administrators. Key attributes include id, username, email, role, active, blocked, and mustChangePassword. Two multivalued audit trails hang off User: a changeStock history recording stock adjustment actions with reason and timestamp and a “performs action(s)

on" history recording product CUD actions with action and timestamp, both linking back to the affected Product. There is also another multivalued audit trails hanging off User: another "performs action(s) on" history recording Admin administrative actions with action and timestamp, linking back to the User entity itself.

- Product (and subtypes): Product is the central entity of the catalog. It carries shared attributes applicable to all media types: id, title, category, description, weight, barcode, originalValue, currentPrice, stockQuantity, status, and ratio. Four specialised subtypes extend Product via an IS-A relationship:
    - Book: adds authors, numberOfPages, publisher, genre, language, coverType, and publicationDate.
    - CD: adds artist, genre, releaseDate, recordLabel, and a "has" relationship to a weak Track entity (with attributes title, length, backNumber).
    - DVD: adds studio, discType, subtitles, language, genre, releaseDate, and director.
    - Newspaper: adds publisher, language, publicationDate, publicationFrequency, issueNumber, ISSN, localNumber, editionChief, genre, and sections.
- Order: The central aggregate root for the order-placing, purchasing workflow, containing an id and a status. It has a many-to-many relationship with the Product entities that represents the Item of an Order, with attributes like productName, unitPrice, unitWeight, and quantity at the time of checkout. It also has a one-to-one relationship with 2 weak entities that has its lifecycle tied to Order:
    - DeliveryInformation: Stores shipping and recipient information required to fulfill an order.
    - Invoice: Stores billing information associated with an order, generated during checkout.
- PaymentTransaction: Represents payment records generated during the checkout process and stores transaction-specific information such as payment method, transaction status, and gateway references. It is currently modeled as having a one-to-one relationship with Order.

4.2.2 Database Design

4.2.2.1 Database Management System

The AIMS system uses PostgreSQL as its relational database management system. PostgreSQL is a powerful, open-source object-relational DBMS known for its standards compliance, extensibility, and robustness under concurrent workloads. It supports full ACID transactions, row-level locking, rich indexing strategies, and a mature ecosystem of tooling, all of which align with the reliability and data integrity requirements of AIMS.

The backend communicates with PostgreSQL via Spring Data JPA backed by Hibernate as the JPA provider. Entity-to-table mapping, relationship management, and query execution are all

handled through this stack, with schema generation managed by Hibernate's DDL auto-update capability during development.

For this project, each team member hosts a PostgreSQL instance independently on their own development machine. There is no shared or centralised database server. Database schema consistency across team members is maintained through Hibernate's schema generation and a shared set of JPA entity definitions in the codebase, any entity-level change automatically propagates to each member's local schema on the next application startup. Seed data scripts are maintained separately and run manually when needed.

This setup is appropriate for a course project context where parallel development occurs on isolated machines. It does introduce the constraint that integration testing across modules requires each developer to ensure their local schema is up to date with the latest entity definitions before running cross-module tests.

4.2.2.2 Database Diagram

The relational schema is derived from the conceptual ER model following standard ER-to-relational mapping rules. The key design decisions are described below.

IS-A specialization (Product subtypes)

The Product IS-A hierarchy is mapped using the joined table strategy: a central product table holds all shared attributes, and each subtype (book, cd, dvd, newspaper) has its own table containing only subtype-specific attributes, joined to product by a shared primary key. This preserves normalization, avoids sparse columns, and allows the shared product table to serve as the single foreign key target for all relationships involving products (orders, audit logs, stock adjustments).

Multivalued attributes and weak entities

Multivalued attributes and weak entities from the ER model are each promoted to their own table:

- book_authors and cd_artists: each storing one value per row, keyed by the owning product's id.
- cd_tracks: the weak Track entity, storing track_number, title, and length, keyed by cd_id.
- dvd_subtitles and newspaper_sections: similarly promoted to separate tables.
- user_roles: also similarly promoted to separate table, supporting the Administrator's ability to assign and modify roles independently of user identity

Many-to-many: Order ↔ Product

The many-to-many relationship between Order and Product (representing order line items) is resolved into an order_item junction table, carrying the attributes captured at checkout time: product_name, unit_price, unit_weight, and quantity. These are denormalized snapshot values, which means they record the product state at the moment of purchase and are independent of any subsequent changes to the product. The table order_item also contains a surrogate key id instead

of making (order_id, product_id) pair as primary key due to the possibility of product "vanishing" from the system for good.

One-to-one weak entities: DeliveryInformation and Invoice

Both DeliveryInformation and Invoice are lifecycle-dependent on Order. They are mapped as separate tables whose primary key is also a foreign key referencing order, enforcing the one-to-one relationship and the existence dependency at the database level.
PaymentTransaction is mapped as an independent table with its own surrogate primary key, linked to order via a foreign key. This accommodates the structural differences between VietQR and PayPal transaction records within a single table.

Audit Log tables

The three audit trail relationships from the ER model are each materialized as a dedicated table:

- product_log: records product CUD actions performed by a Product Manager, with a snapshot of the product title and a reference to both the managing user and the affected product.
- product_update_log: records field-level changes for each product update, storing field_name, old_value, and new_value per changed field. It is a child of product_log, linked by log_id.
- stock_adjust_log: records stock adjustment actions, capturing old_stock, new_stock, and reason.
- admin_log: records administrative actions performed by an Administrator, storing the action, the acting admin, and the affected user.

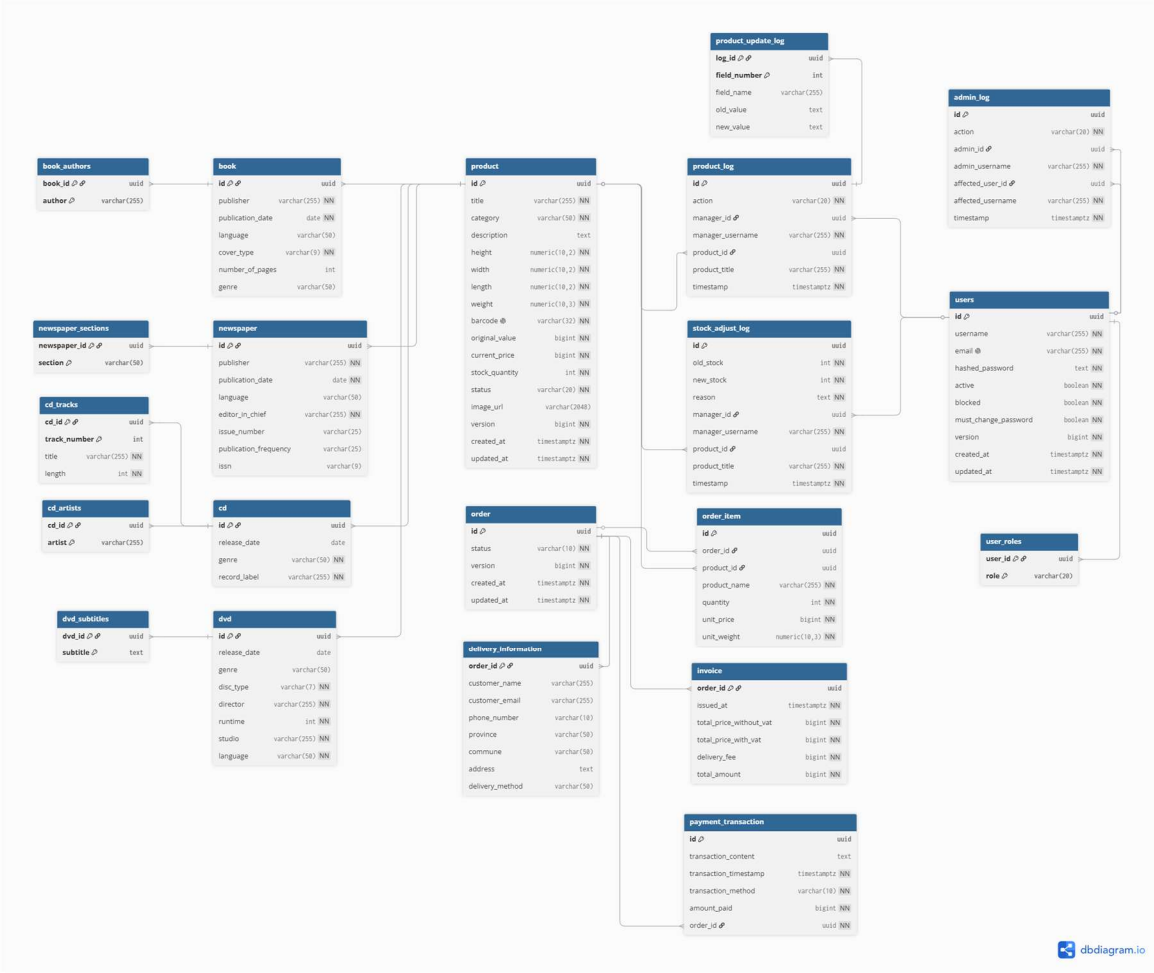


Figure 24. Database Design for AIMS System

4.2.2.3 Database Detail Design

product

#	PK	FK	Column Name	Data type	Default Value	Mandatory	Description
1	x		id	UUID	Random UUID	Yes	Unique product ID
2			title	VARCHAR(255)		Yes	Product title
3			height	NUMERIC(10,2)		Yes	Product height
4			width	NUMERIC(10,2)		Yes	Product width
5			length	NUMERIC(10,2)		Yes	Product length
6			weight	NUMERIC(10,3)		Yes	Product weight
7			category	VARCHAR(50)		Yes	Product category

8			description	TEXT		No	General product description
9			barcode	VARCHAR(32)		Yes	Unique product barcode
10			original_value	BIGINT		Yes	Original value
11			current_price	BIGINT		Yes	Current selling price
12			stock_quantity	INT		Yes	Current quantity in stock
13			status	VARCHAR(20)		Yes	Product status
14			image_url	VARCHAR(2048)		No	Product image URL
15			created_at	TIMESTAMPTZ	Current Time	Yes	Product creation time
16			updated_at	TIMESTAMPTZ	Current Time	Yes	Product update time
17			version	BIGINT	0	Yes	Product version (for optimistic locking purpose)

book

#	PK	FK	Column Name	Data type	Default Value	Mandatory	Description
1	x	x	id	UUID		Yes	ID, same as Product ID
2			language	VARCHAR(50)		No	Book language
3			genre	VARCHAR(50)		No	Book genre
4			publication_date	DATE		Yes	Book publication date
5			cover_type	VARCHAR(9)		Yes	Cover type, e.g., paperback/hardcover
6			publisher	VARCHAR(255)		Yes	Book publisher
7			number_of_pages	INTEGER		No	Number of pages

book_authors

#	PK	FK	Column Name	Data type	Default Value	Mandatory	Description
1	x	x	book_id	UUID		Yes	ID of the book
2	x		author	VARCHAR(255)		Yes	Author name

newspaper

#	PK	FK	Column Name	Data type	Default Value	Mandatory	Description
1	x	x	id	UUID		Yes	ID, same as Product ID
2			editor_in_chief	VARCHAR(255)		Yes	Editor-in-chief
3			issue_number	VARCHAR(25)		No	Issue number
4			issn	VARCHAR(9)		No	ISSN code
5			publication_frequency	VARCHAR(25)		No	Publication frequency
6			publisher	VARCHAR(255)		Yes	Newspaper publisher
7			language	VARCHAR(50)		No	Newspaper language
8			publication_date	DATE		Yes	Publication date

newspaper_sections

#	PK	FK	Column Name	Data type	Default Value	Mandatory	Description
1	x	x	newspaper_id	UUID		Yes	ID of the newspaper
2	x		section	VARCHAR(50)		Yes	Newspaper section

cd

#	PK	FK	Column Name	Data type	Default Value	Mandatory	Description

1	x	x	id	UUID		Yes	ID, same as Product ID
2			record_label	VARCHAR(255)		Yes	CD record label
3			genre	VARCHAR(50)		Yes	CD genre
4			release_date	DATE		No	CD release date

cd_artists

#	PK	FK	Column Name	Data type	Default Value	Mandatory	Description
1	x	x	cd_id	UUID		Yes	ID of the CD
2	x		artist	VARCHAR(255)		Yes	Artist name

cd_tracks

#	PK	FK	Column Name	Data type	Default Value	Mandatory	Description
1	x	x	cd_id	UUID		Yes	ID of the CD containing this track
2	x		track_number	INTEGER		Yes	The track order number
3			title	VARCHAR(255)		Yes	Track title
4			length	BIGINT		Yes	Track length/duration

dvd

#	PK	FK	Column Name	Data type	Default Value	Mandatory	Description
1	x	x	id	UUID		Yes	ID, same as Product ID
2			studio	VARCHAR(255)		Yes	DVD studio
3			disc_type	VARCHAR(7)		Yes	Disc type, e.g., Blu-ray
4			language	VARCHAR(50)		Yes	DVD language
5			runtime	BIGINT		Yes	Runtime duration

6			director	VARCHAR(255)		Yes	Director name
7			release_date	DATE		No	DVD release date
8			genre	VARCHAR(50)		No	DVD genre

dvd_subtitles

#	PK	FK	Column Name	Data type	Default Value	Mandatory	Description
1	x	x	dvd_id	UUID		Yes	ID of the DVD
2	x		subtitle	TEXT		Yes	Subtitle language/content

order

#	PK	FK	Column Name	Data type	Default Value	Mandatory	Description
1	x		id	UUID		Yes	Unique order ID
2			status	VARCHAR(10)		Yes	Order status
3			created_at	TIMESTAMPTZ	Current timestamp	Yes	Order creation time
4			updated_at	TIMESTAMPTZ	Current timestamp	Yes	Order update time
5			version	BIGINT	0	Yes	Order version (for optimistic locking purpose)

order_item

#	PK	FK	Column Name	Data type	Default Value	Mandatory	Description
1	x		id	UUID		Yes	Surrogate ID of the order item
2		x	order_id	UUID		Yes	ID of the order
3		x	product_id	UUID		No	ID of the ordered product
4			quantity	INTEGER		Yes	Ordered quantity

5			product_name	VARCHAR(255)		Yes	Product name at order time
6			unit_price	BIGINT		Yes	Unit price at order time
7			unit_weight	NUMERIC(10,3)		Yes	Unit weight at order time

delivery_information

#	PK	FK	Column Name	Data type	Default value	Mandatory	Description
1	x	x	order_id	UUID		Yes	ID of the related order
2			customer_name	VARCHAR(255)		Yes	Customer name
3			customer_email	VARCHAR(255)		Yes	Customer email
4			phone_number	VARCHAR(10)		Yes	Customer phone number
5			province	VARCHAR(50)		Yes	Customer province/city
6			commune	VARCHAR(50)		Yes	Customer commune/ward
7			address	VARCHAR(255)		Yes	Customer address
8			delivery_method	VARCHAR(255)		Yes	Delivery method

invoice

#	PK	FK	Column Name	Data type	Default value	Mandatory	Description
1	x	x	order_id	UUID		Yes	ID of the related order
2			total_price_without_vat	BIGINT		Yes	Total product price before VAT
3			total_price_with_vat	BIGINT		Yes	Total product price after VAT
4			delivery_fee	BIGINT		Yes	Delivery fee
5			total_amount	BIGINT		Yes	Final amount to be paid

6			issued_at	TIMESTAMPTZ		Yes	The issue time of the invoice

payment_transaction

#	PK	FK	Column Name	Data type	Default value	Mandatory	Description
1	x		id	UUID		Yes	Unique payment transaction ID
2		x	order_id	UUID		Yes	ID of the related order
3			amount_paid	BIGINT		Yes	Amount paid by customer
4			transaction_method	VARCHAR(10)		Yes	Payment method
5			transaction_content	TEXT		Yes	Payment transaction content
6			transaction_timestamp	TIMESTAMPTZ		Yes	Payment transaction time

user

#	PK	FK	Column Name	Data type	Default value	Mandatory	Description
1	x		id	UUID		Yes	Unique user ID
2			username	VARCHAR(255)		Yes	User display name
3			email	VARCHAR(255)		Yes	User email, must be unique
4			hashed_password	TEXT		Yes	Securely hashed password
5			active	BOOLEAN		Yes	Whether user is active or not
6			blocked	BOOLEAN		Yes	Whether user is blocked or not
7			must_change_password	BOOLEAN		Yes	Whether user is required to

							change password on their next login
5			created_at	TIMESTAMPTZ	Current timestamp	Yes	User creation time
6			updated_at	TIMESTAMPTZ	Current timestamp	Yes	User update time
7			version	BIGINT	0	Yes	User version (for optimistic locking purpose)

user_roles

#	PK	FK	Column Name	Data type	Default value	Mandatory	Description
1	x	x	user_id	UUID		Yes	ID of the user
2	x		role	VARCHAR(20)		Yes	Role of the user

product_log

#	PK	FK	Column Name	Data type	Default value	Mandatory	Description
1	x		id	UUID		Yes	Unique product log ID
2		x	manager_id	UUID		No	Product manager who performed the action
3			manager_username	VARCHAR(255)		Yes	Product manager username
4		x	product_id	UUID		No	Product affected by the action
5			product_title	VARCHAR(255)		Yes	Product title
6			timestamp	TIMESTAMP	Current timestamp	Yes	Time of the action

7			action	VARCHAR(20)		Yes	Action type, e.g., add, edit, delete

product_update_log

#	PK	FK	Column Name	Data type	Default value	Mandatory	Description
1	x	x	log_id	UUID		Yes	Unique ID product log whose action is “UPDATE”
2	x		field_number	UUID		Yes	The order number of the field
3			field_name	VARCHAR(255)		Yes	Name of the field
4			old_value	TEXT		Yes	Old value of the field
5			new_value	TEXT		Yes	New value of the field

stock_adjust_log

#	PK	FK	Column Name	Data type	Default value	Mandatory	Description
1	x		id	UUID		Yes	Unique stock adjustment log ID
2		x	manager_id	UUID		No	Product manager who performed the action
3			manager_username	VARCHAR(255)		Yes	Product manager username
4		x	product_id	UUID		No	Product affected by the action
5			product_title	VARCHAR(255)		Yes	Product title

4			old_stock	INT		Yes	Stock before adjustment
5			new_stock	INT		Yes	Stock after adjustment
6			reason	TEXT		Yes	Reason for stock adjustment
7			timestamp	TIMESTAMP	Current timestamp	Yes	Time of adjustment

admin_log

#	PK	FK	Column Name	Data type	Default value	Mandatory	Description
1	x		id	UUID		Yes	Unique admin log ID
2		x	admin_id	UUID		No	Administrator who performed the action
3			admin_username	VARCHAR(255)		Yes	Administrator username
4		x	affected_user_id	UUID		No	User affected by the action
5			affected_username	VARCHAR(255)		Yes	Affected username
6			timestamp	TIMESTAMP	Current timestamp	Yes	Time of action
7			action	VARCHAR(20)		Yes	Action type, e.g., block, unblock, reset password

4.3 Non-Database Management System Files

AIMS uses one non-DBMS file storage system: Supabase Storage, a cloud-based object storage service used to host product images.

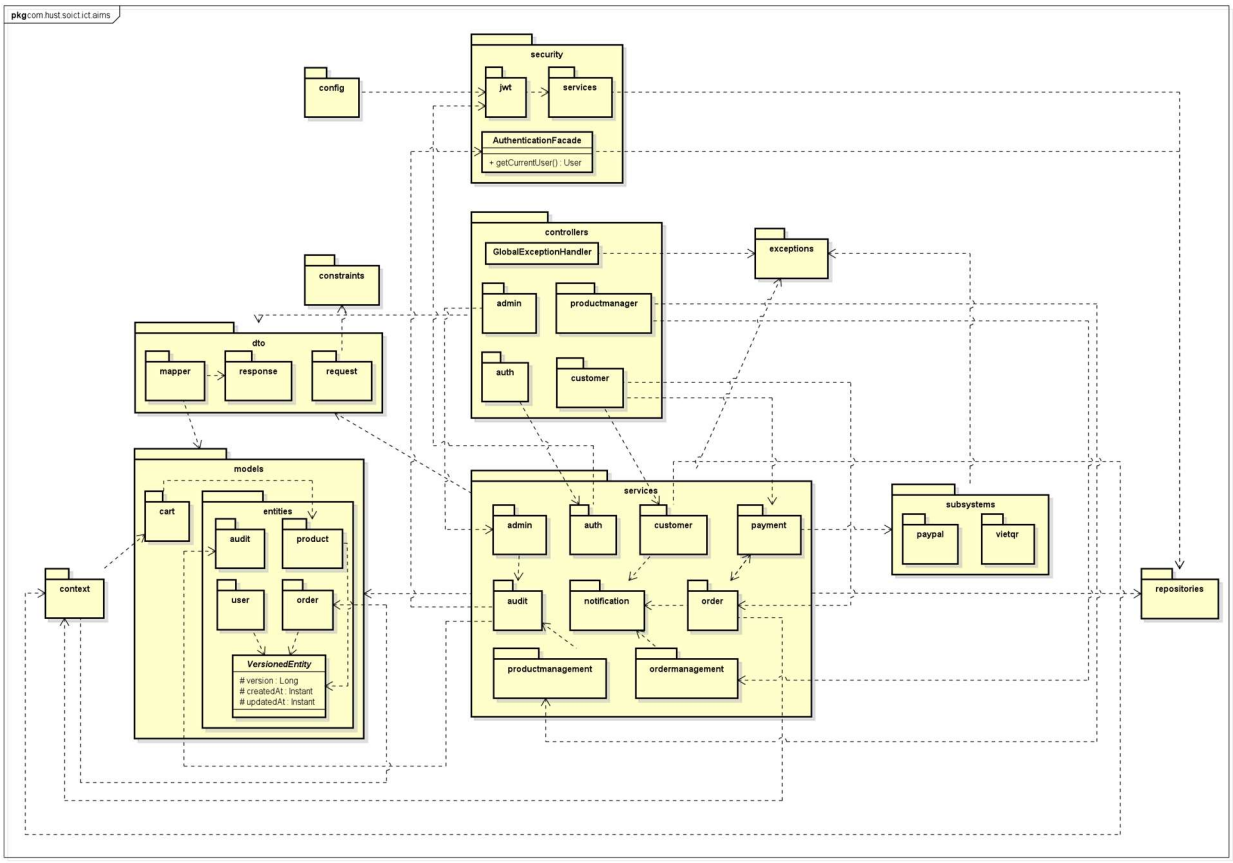
- Usage: Output only (read by the frontend). The storage bucket holds image files for each product type. The frontend retrieves images by their public URL, which is stored as a text

field (image_url) in the product table in PostgreSQL. Supabase Storage itself is therefore not queried directly by the backend; the backend stores and serves only the URL string, while the browser fetches the image directly from Supabase's CDN.

- Access method: Random access via public HTTPS URL. Each image is independently addressable by its unique URL and requires no sequential traversal of the bucket.
- Record structure: Unstructured binary files (JPEG/PNG/WebP). No record keys or indexes are maintained within the storage layer itself; the image_url column in the product table serves as the application-level reference.
- Update frequency: At the current stage of development, image upload functionality is out of scope. All image_url values have been manually assigned, meaning URLs were copied from publicly available sources and inserted directly into the database as seed data, with one representative image selected per product type. Consequently, the storage bucket is not written to at runtime and no upload transactions occur during normal system operation.
- File size estimate: Minimal. The bucket currently holds a small, fixed set of representative images, each in the range of tens to hundreds of kilobytes.
- Backup and recovery: Supabase Storage provides built-in redundancy and availability guarantees as part of its managed cloud infrastructure. No additional backup procedure is implemented at the application level. Since all image URLs are also stored in the PostgreSQL database, recovery of the URL references is covered by the standard database backup strategy.
- Planned extension: The architecture anticipates a future image upload feature in which the frontend would upload a file directly to Supabase Storage and receive a public URL in return, which would then be submitted to the backend as part of a product create or update request. This flow requires no changes to the backend or database schema; only a frontend upload component needs to be added.

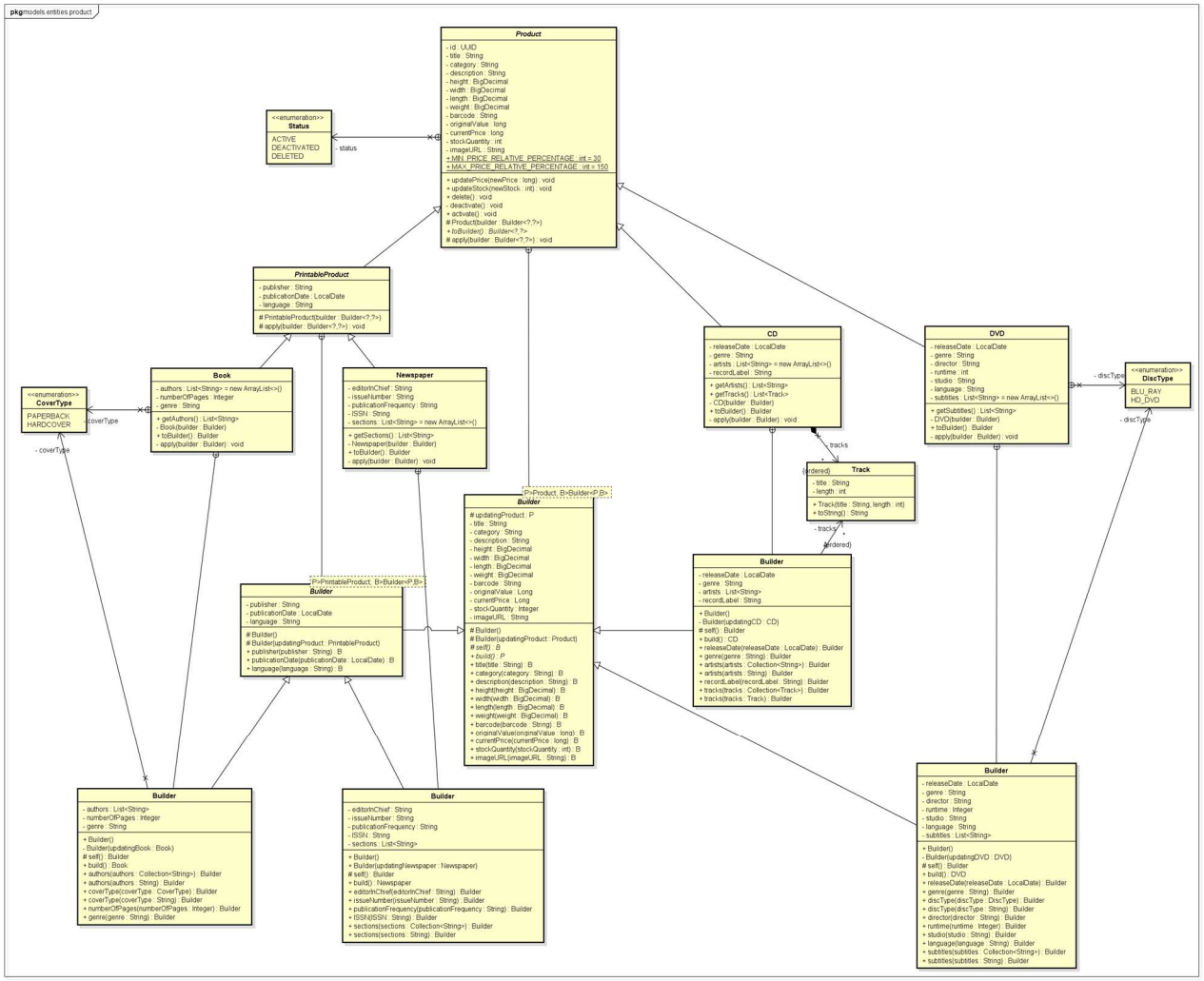
4.4 Class Design

4.4.1 General Class Diagram

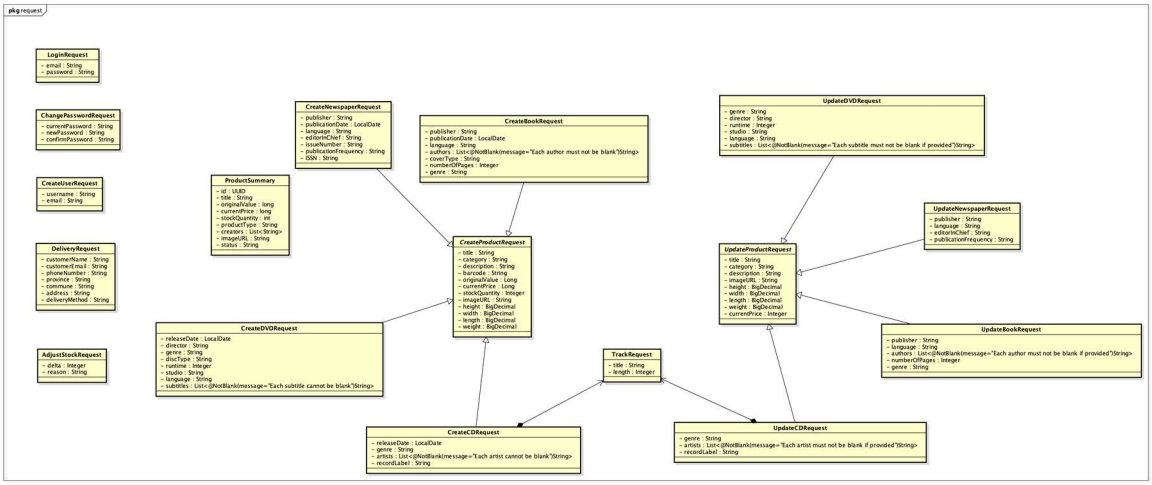


4.4.2 Class Diagrams

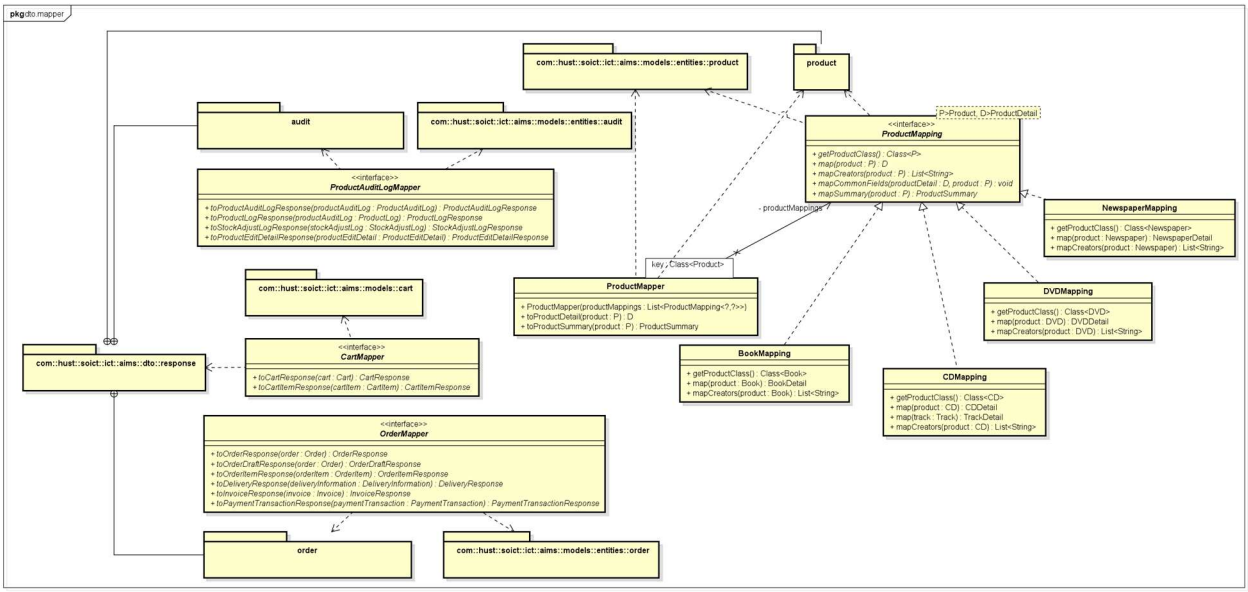
4.4.2.1 Class Diagram for Package models.entities.product



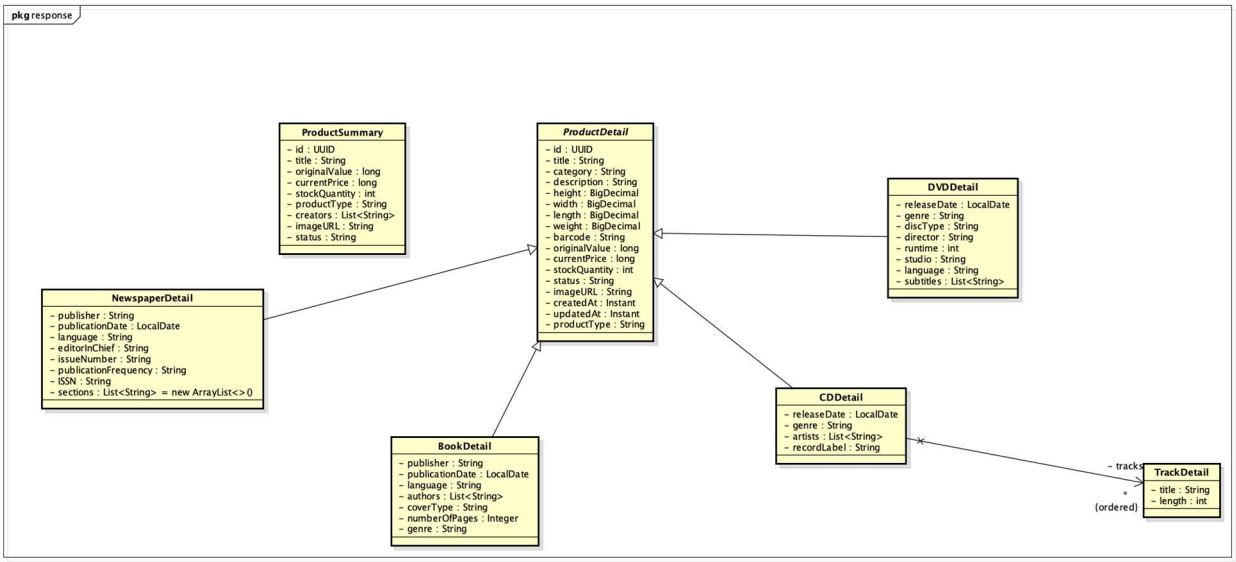
4.4.2.2 Class Diagram for Package dto.request



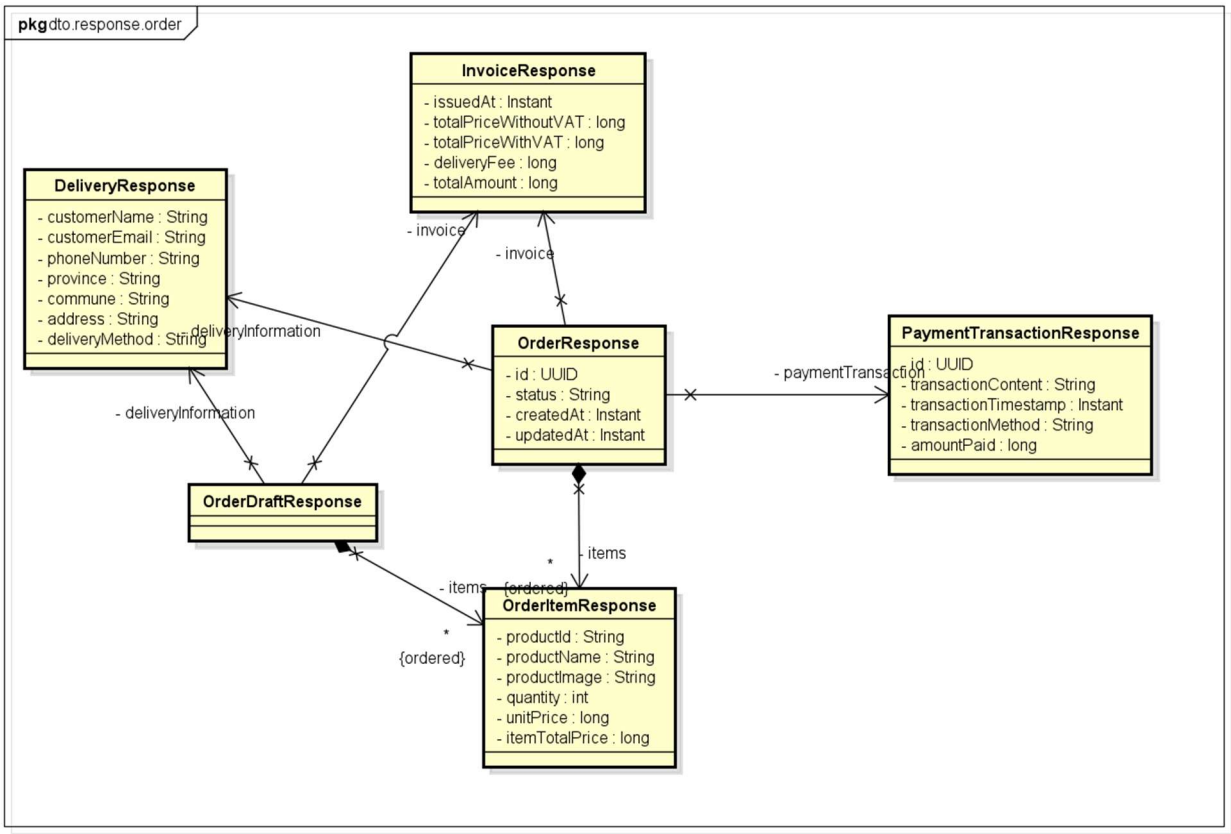
4.4.2.3 Class Diagram for Package dto.mapper



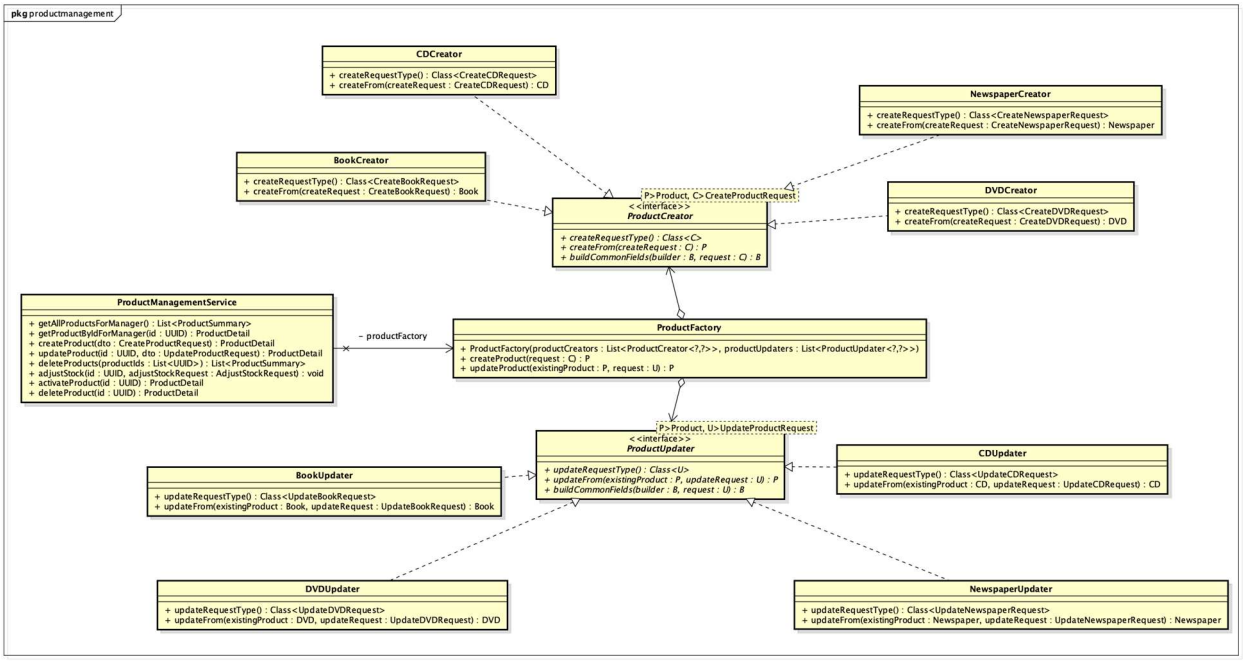
4.4.2.4 Class Diagram for Package dto.response.product



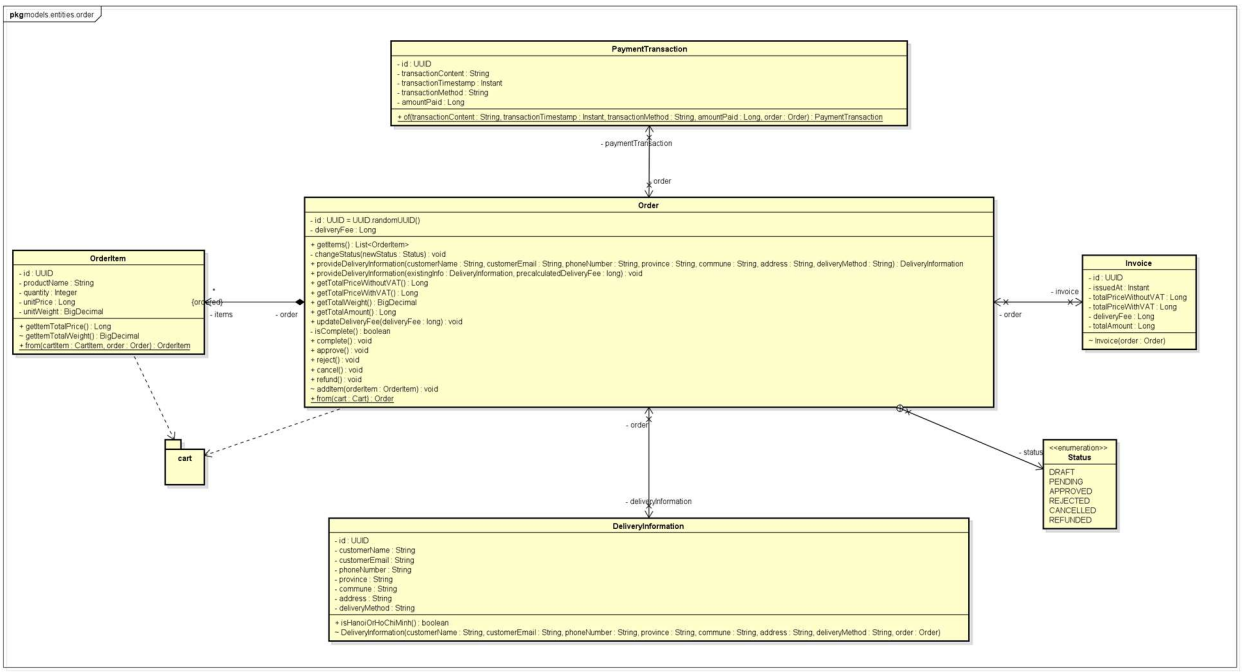
4.4.2.5 Class Diagram for Package dto.response.order



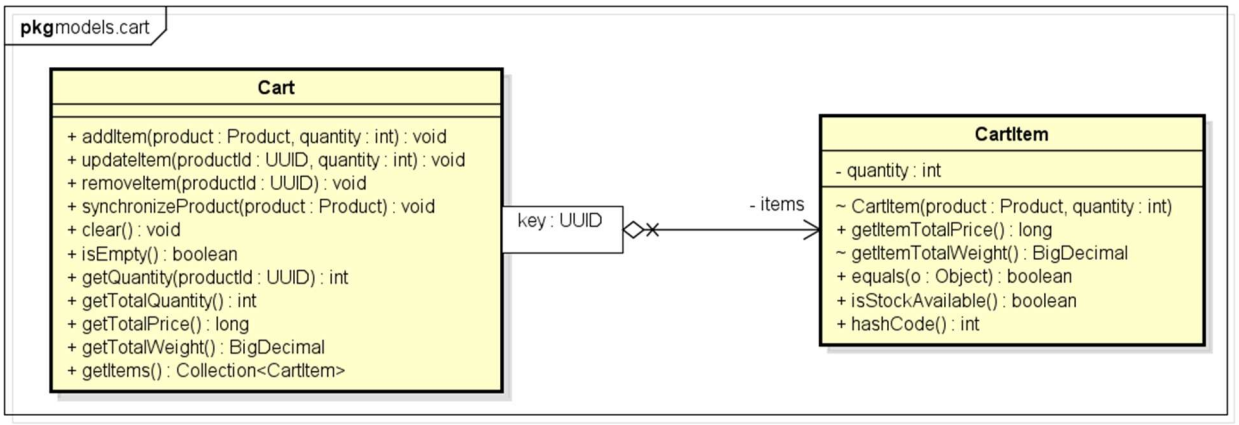
4.4.2.6 Class Diagram for Package services.productmanagement



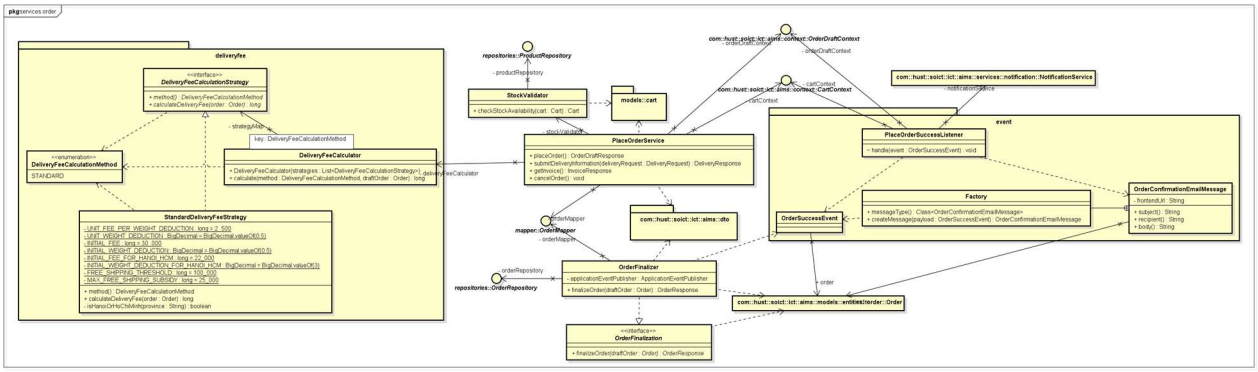
4.4.2.7 Class Diagram for Package models.entities.order



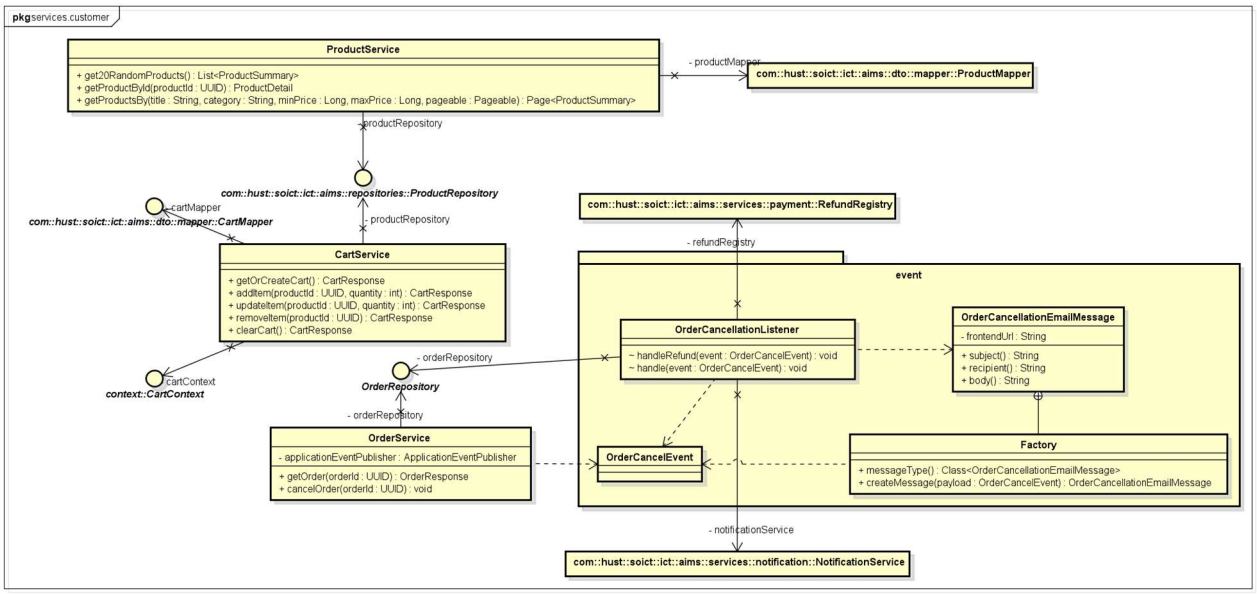
4.4.2.8 Class Diagram for Package models.cart



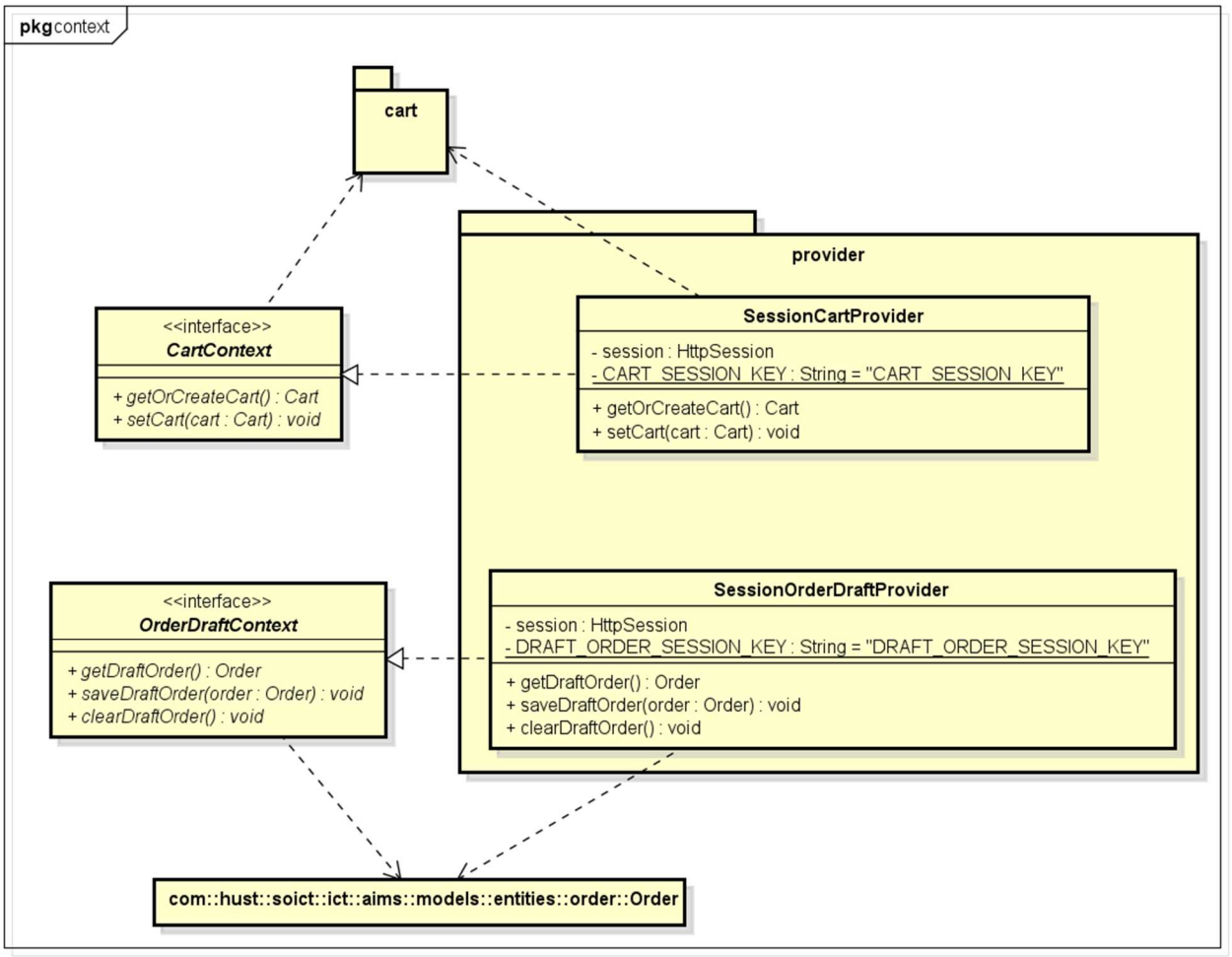
4.4.2.9 Class Diagram for Package services.order



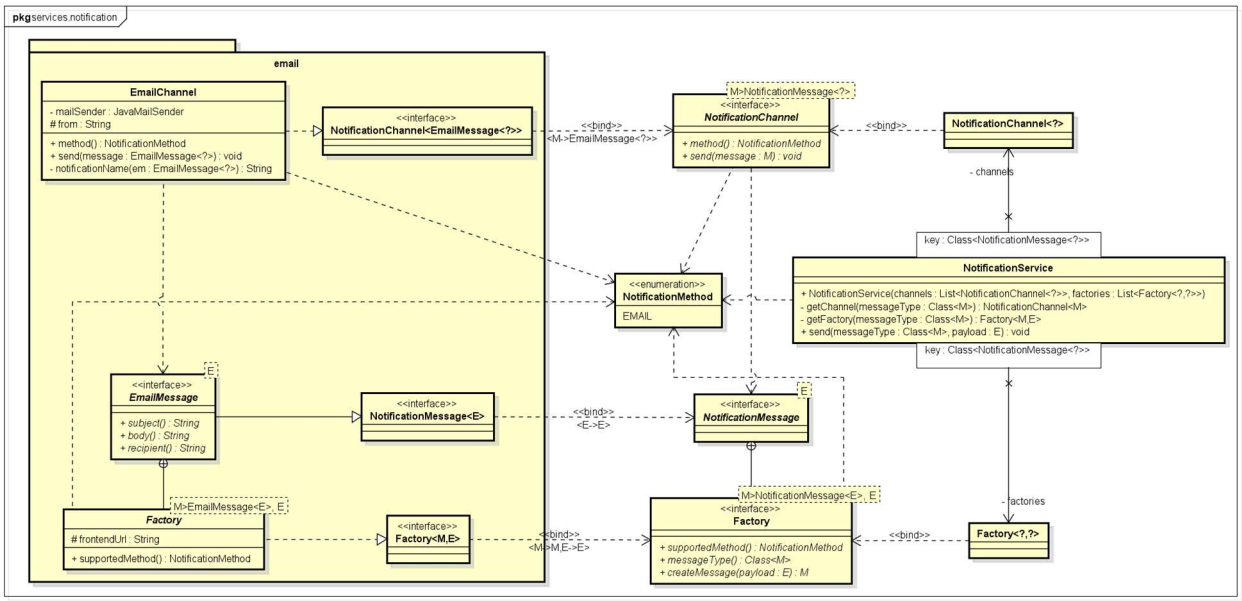
4.4.2.10 Class Diagram for Package services.customer



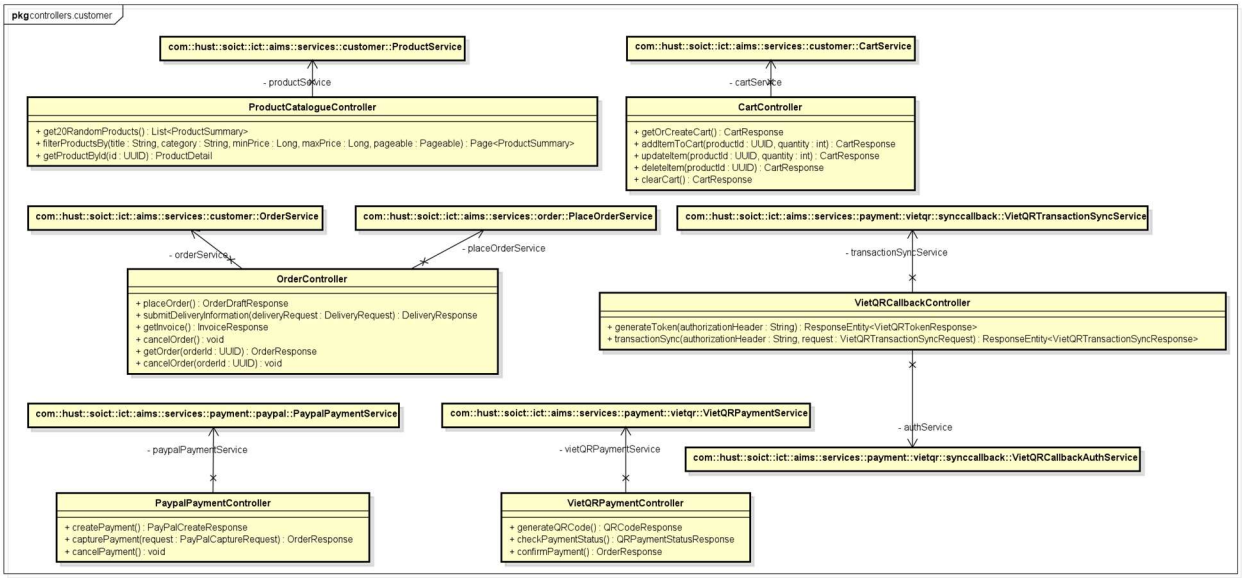
4.4.2.11 Class Diagram for Package context



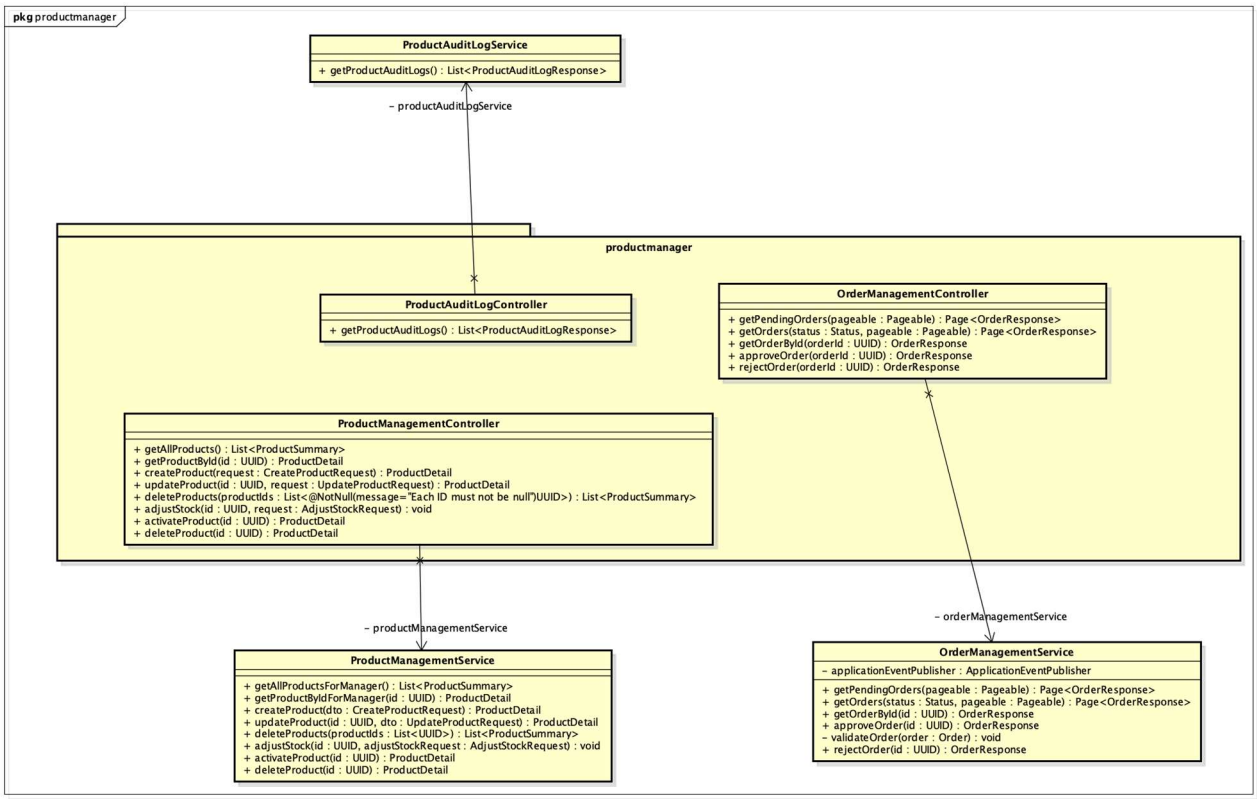
4.4.2.12 Class Diagram for Package services.notification



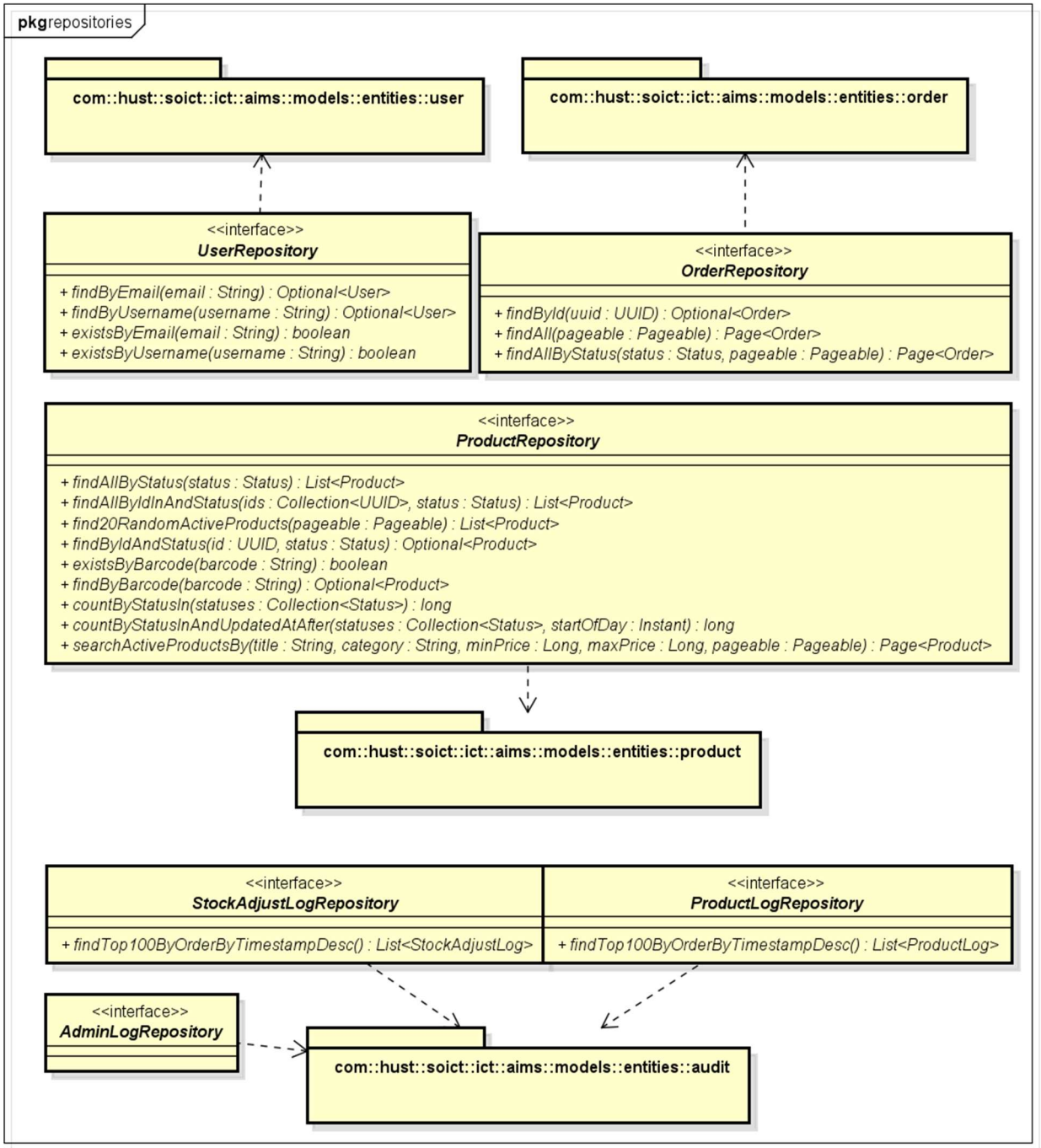
4.4.2.13 Class Diagram for Package controllers.customer



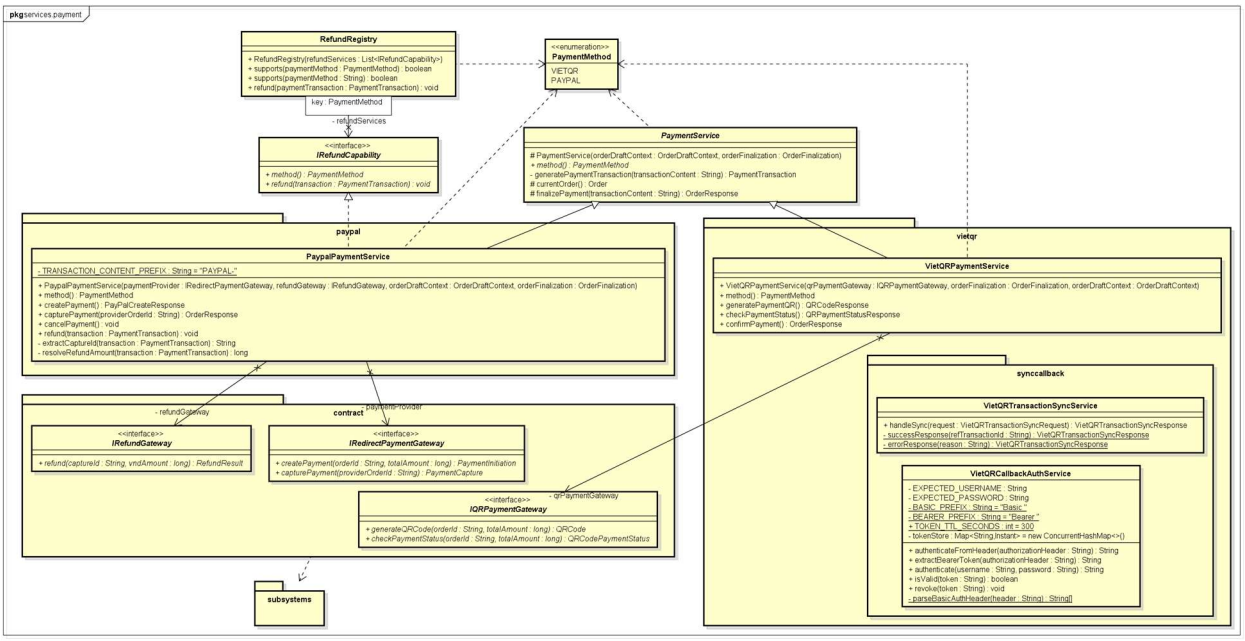
4.4.2.14 Class Diagram for Package controllers.productmanager



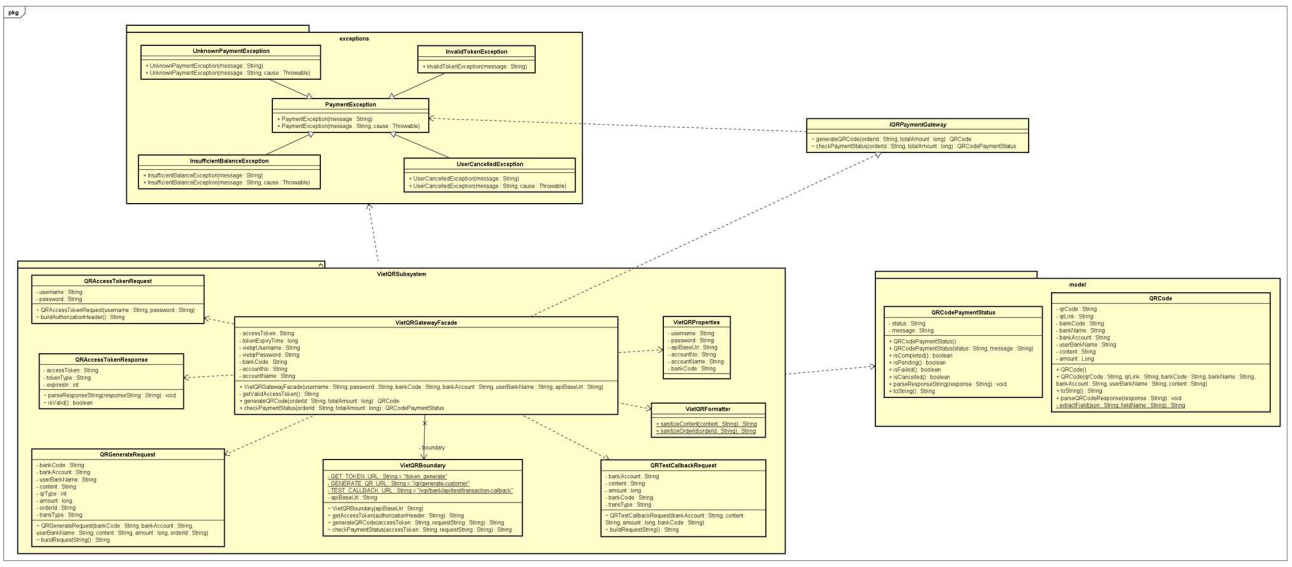
4.4.2.15 Class Diagram for Package repositories



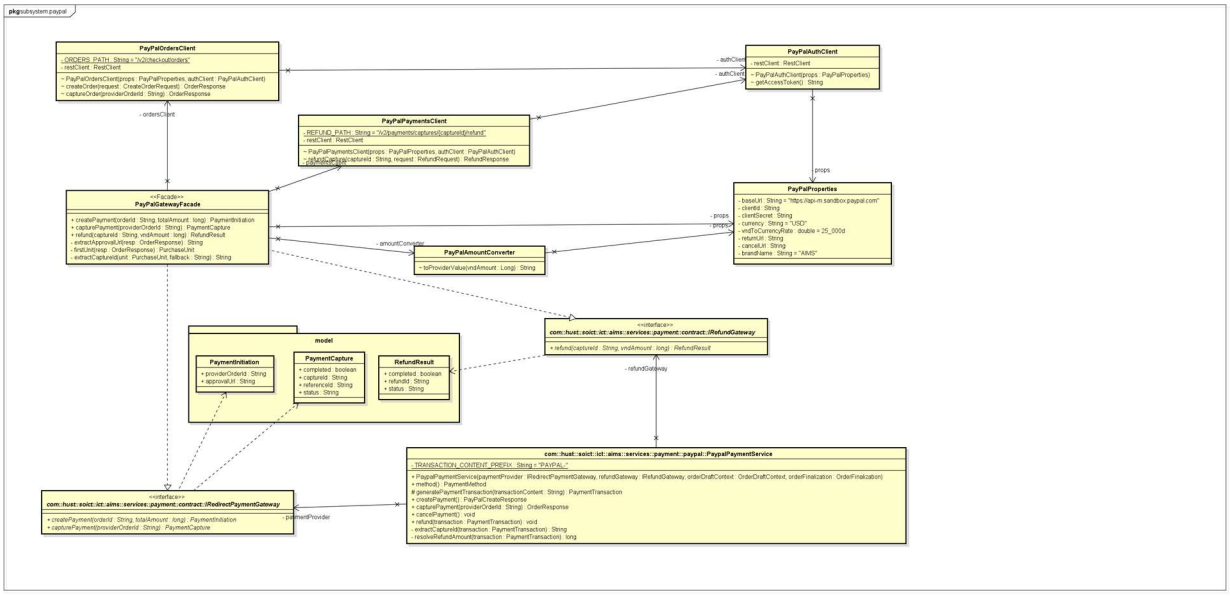
4.4.2.16 Class Diagram for Package services.payment



4.4.2.17 Class Diagram for Subsystem subsystems.vietqr

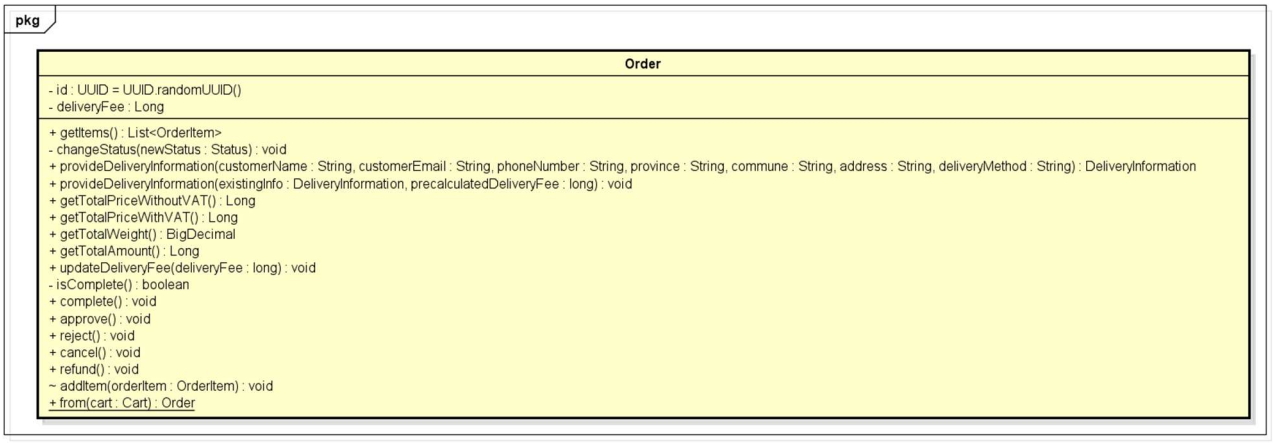


4.4.2.18 Class Diagram for Subsystem subsystems.paypal



4.4.3 Class Design

4.4.3.1 Class "Order"



Attributes

#	*Name*	*Data type*	*Default value*	*Description*
1	id	UUID	UUID.randomUUID()	Unique identifier, generated on construction
2	items	List<OrderItem>	new ArrayList<>()	Line items belonging to this order
3	status	Status (enum)	—	Current lifecycle state of the order

4	deliveryInformation	DeliveryInformation	null	Shipping and recipient data (set during checkout)
5	deliveryFee	Long (@Transient)	null	Calculated delivery fee, not persisted (captured in Invoice)
6	invoice	Invoice	null	Billing snapshot generated after delivery fee is set
7	paymentTransaction	PaymentTransaction	null	Payment record set by the payment subsystem

Operations

#	*Name*	*Return type*	*Description (purpose)*
1	from(cart)	Order	Static factory: creates a DRAFT order from a Cart
2	provideDeliveryInformation(...)	DeliveryInformation	Creates and attaches a DeliveryInformation instance
3	updateDeliveryFee(deliveryFee)	void	Sets delivery fee and generates the Invoice snapshot
4	complete()	void	Validates completeness and transitions DRAFT → PENDING
5	approve()	void	Transitions PENDING → APPROVED
6	reject()	void	Transitions PENDING → REJECTED
7	cancel()	void	Transitions PENDING → CANCELLED
8	refund()	void	Transitions REJECTED/CANCELLED → REFUNDED
9	getTotalPriceWithoutVAT()	Long	Sums all OrderItem prices (@Transient computed)
10	getTotalPriceWithVAT()	Long	Applies 10% VAT to subtotal (@Transient computed)

11	getTotalWeight()	BigDecimal	Sums all OrderItem weights (@Transient computed)
12	getTotalAmount()	Long	Returns invoice total if placed, otherwise delivery fee + VAT total
13	addItem(orderItem)	void	Package-private: adds an OrderItem and back-sets its order reference

Parameter:

- from(cart) cart: **Cart** — the customer's active cart; must be non-empty and stock-validated
- provideDeliveryInformation(...) — customerName, customerEmail, phoneNumber, province, commune, address, deliveryMethod: all String, all @NonNull
- updateDeliveryFee(deliveryFee) — deliveryFee: **long** — pre-calculated fee in VND; throws UnsupportedOperationException if order is not DRAFT

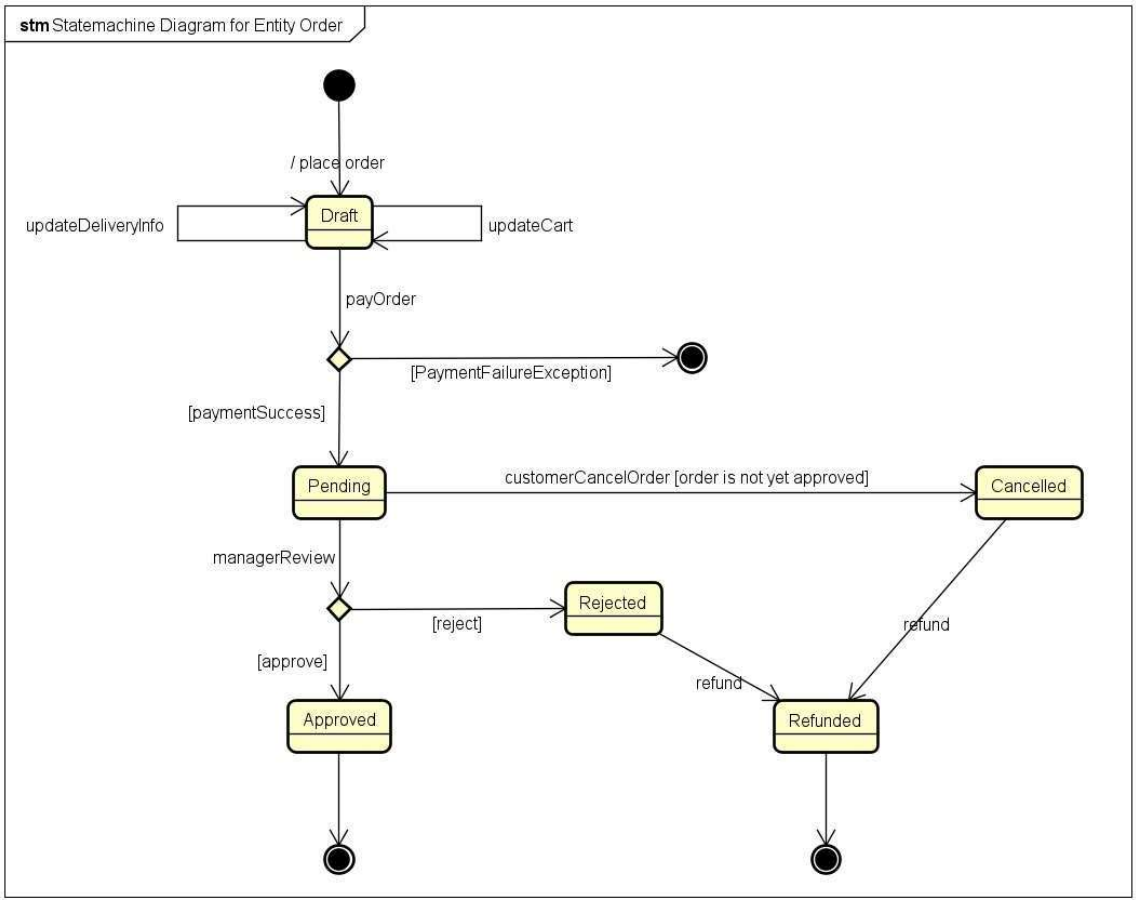
Exception:

- OrderStateTransitionException — if changeStatus() is called with an invalid transition
- OrderNotCompleteException — if complete() is called before deliveryInformation, deliveryFee, invoice, and paymentTransaction are all set
- UnsupportedOperationException — if updateDeliveryFee() is called on a non-DRAFT order

Method:

- The Order.Status enum encodes a fixed transition table: DRAFT → PENDING → APPROVED / REJECTED → REFUNDED / CANCELLED → REFUNDED. Transitions outside this table throw OrderStateTransitionException.
- The transition map is declared as a static constant inside the Status enum, so adding a new state requires only modifying the enum — no changes to service logic.

State machine diagram


stm Statemachine Diagram for Entity Order
/ place order
Draft
updateDeliveryInfo
updateCart
payOrder
[PaymentFailureException]
[paymentSuccess]
Pending
customerCancelOrder [order is not yet approved]
Cancelled
managerReview
Rejected
[reject]
refund
[approve]
refund
Approved
Refunded

4.4.3.2 Class "OrderItem"

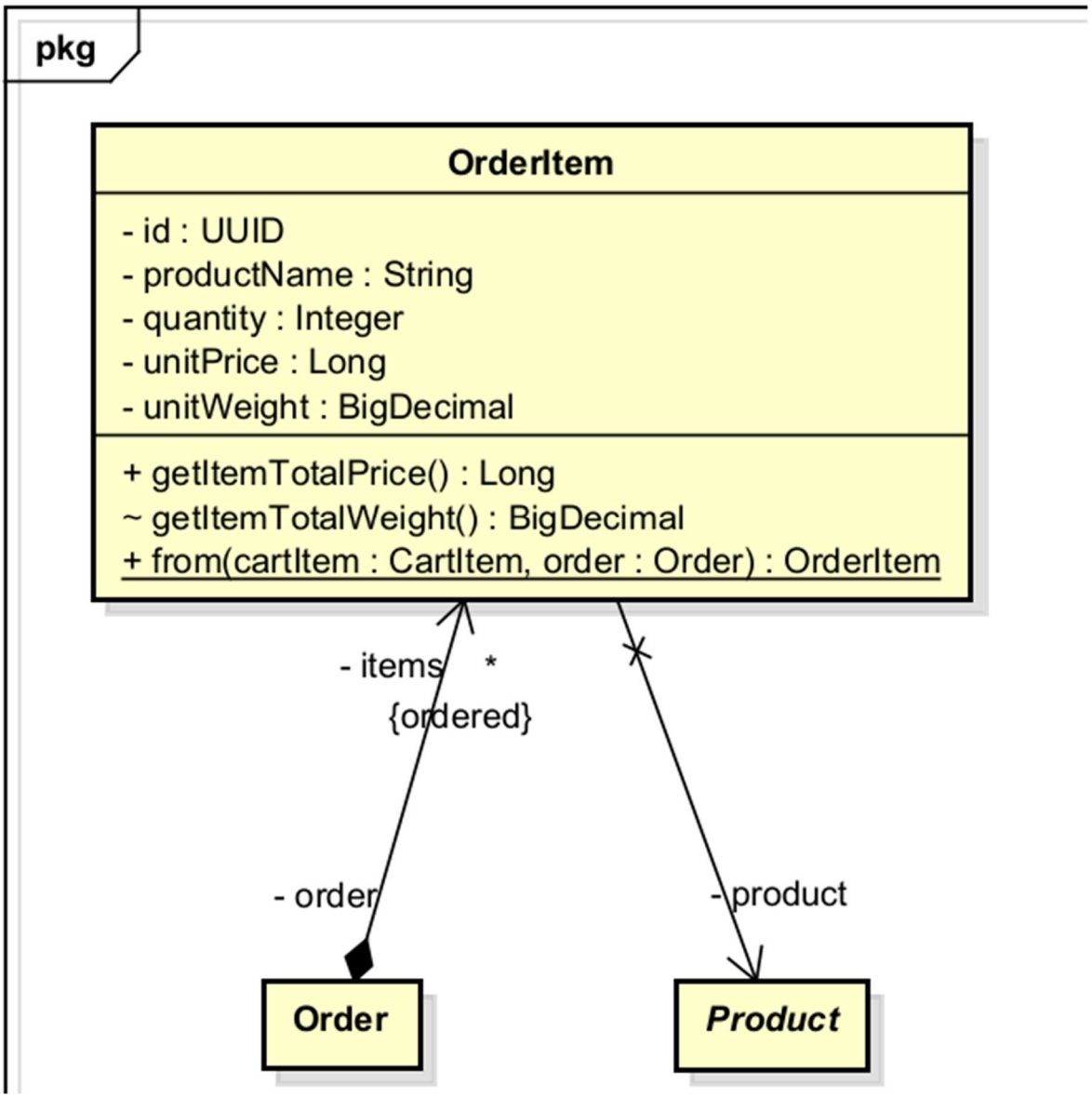


Table 1. Attribute design

#	*Name*	*Data type*	*Default value*	*Description*
1	id	UUID	generated	Surrogate primary key
2	order	Order	—	Parent order (FK)

3	product	Product	—	Reference to the product at time of purchase
4	productName	String	—	Snapshot of product title at checkout
5	quantity	Integer	—	Quantity purchased
6	unitPrice	Long	—	Snapshot of product price at checkout (VND)
7	unitWeight	BigDecimal	—	Snapshot of product weight at checkout (kg)

Table 2. Operation design

#	*Name*	*Return type*	*Description (purpose)*
1	from(cartItem, order)	OrderItem	Static factory: creates an OrderItem from a CartItem, snapshotting current product values
2	getItemTotalPrice()	Long	@Transient: unitPrice × quantity
3	getItemTotalWeight()	BigDecimal	@Transient: unitWeight × quantity

Parameter:

- from(cartItem, order) — cartItem: **CartItem** — source item; product reference and quantity are copied
- from(cartItem, order) — order: **Order** — the parent order being constructed

Exception:

Method:

- productName, unitPrice, and unitWeight are copied from the live product at the moment from() is called. Subsequent product updates do not affect existing OrderItems (snapshot pattern enforced at the factory level).

4.4.3.3 Class "Product"

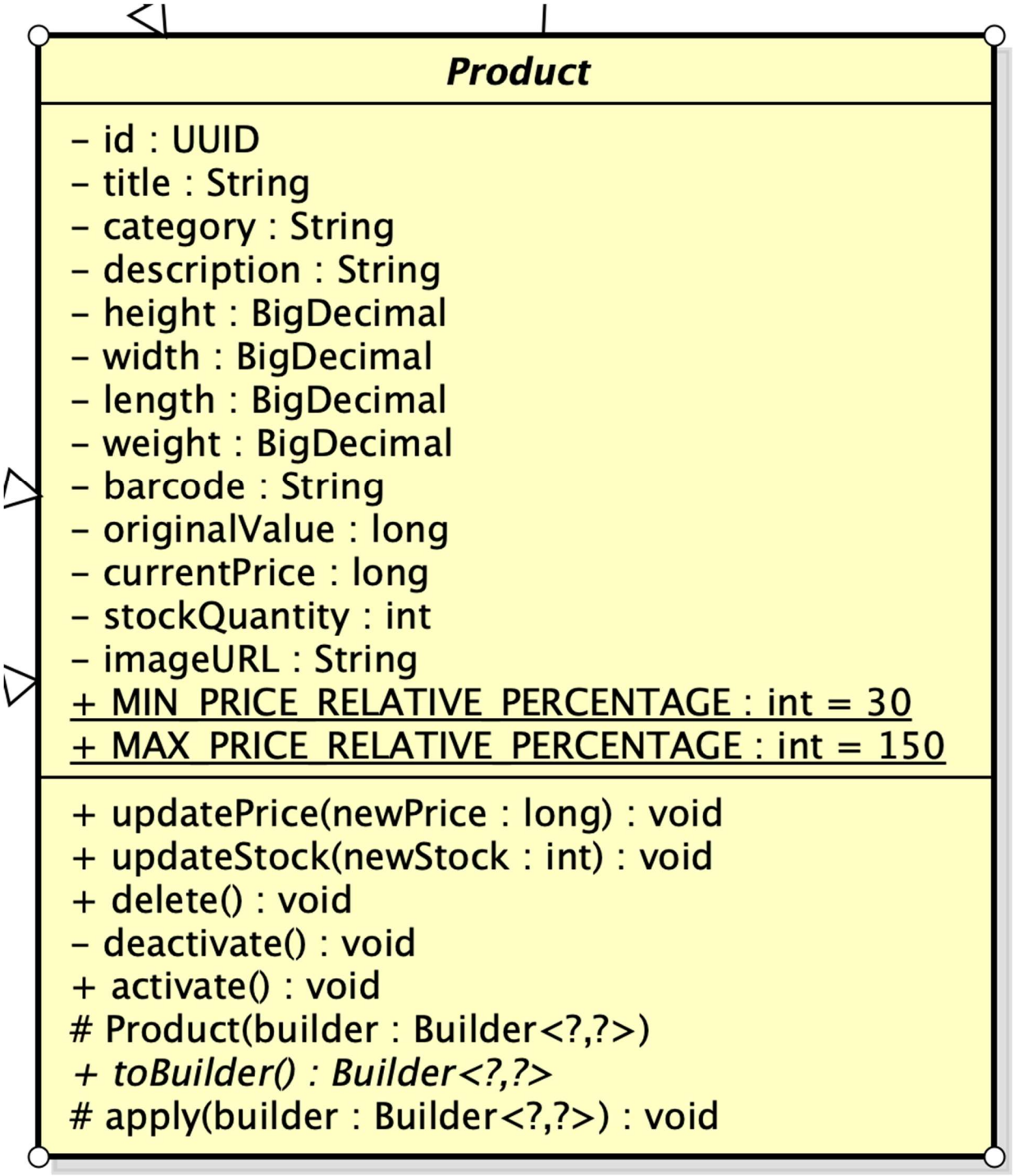


Table 1. Attribute design

#	Name	Data type	Default value	Description

1	productid	String		Unique ID of the product
2	title	String		Title of the product
3	category	String		Category of the product
4	description	String		Product description
5	height	BigDecimal		height
6	width	BigDecimal		width
7	length	BigDecimal		length
8	weight	BigDecimal		weight
9	barcode	String		Unique barcode of the product
10	originalValue	long		Product original value
11	currentPrice	long		Current price of product
12	stockQuantity	int		Product quantity in stock
13	imageURL	String		URL to load product's image
14	MIN_PRICE_RELATIVE	int	30	Min current price
15	MAX_PRICE_RELATIVE	int	150	Max current price

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Product		Contructor of the product entity
2	toBuilder	Builder<?, ?>	Construct and return a Builder object with the current state of Product entity
3	apply	void	Update information field of the Product
4	updatePrice	void	Change product price and validate the price range
5	updateStock	void	Update stock level
6	delete	void	Delete a product
7	deactivate	void	Deactivate a product
8	activate	void	Activate a product

4.4.3.4 Class "PrintableProduct"

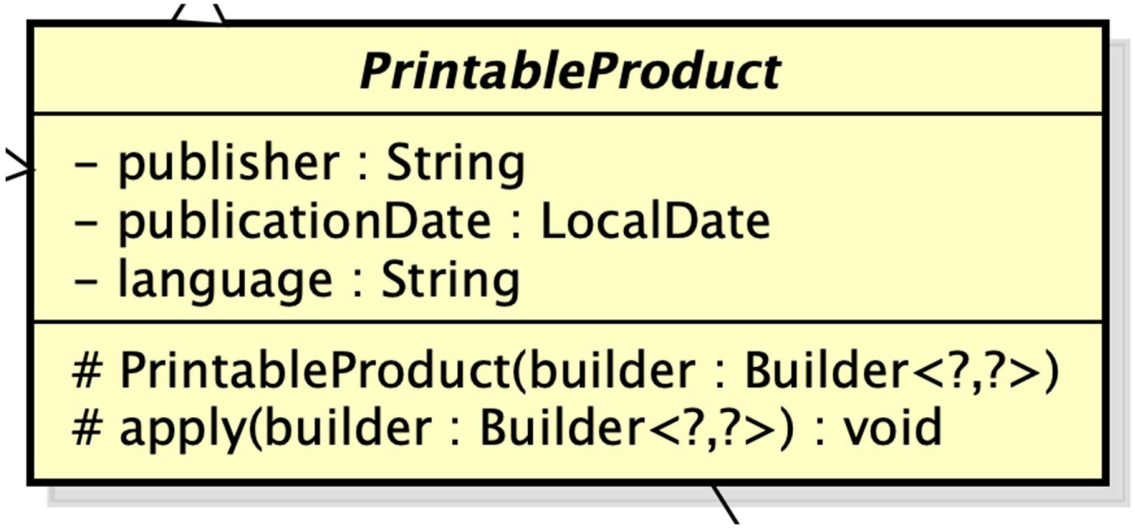


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	publisher	String		Publusher name
2	publicationDate	LocalDate		Publication date
3	language	String		Language written in the product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	PrintableProduct		Constructor of printable product (Book, Newspaper) entity
2	apply	void	Update information field

4.4.3.5 Class "Book"

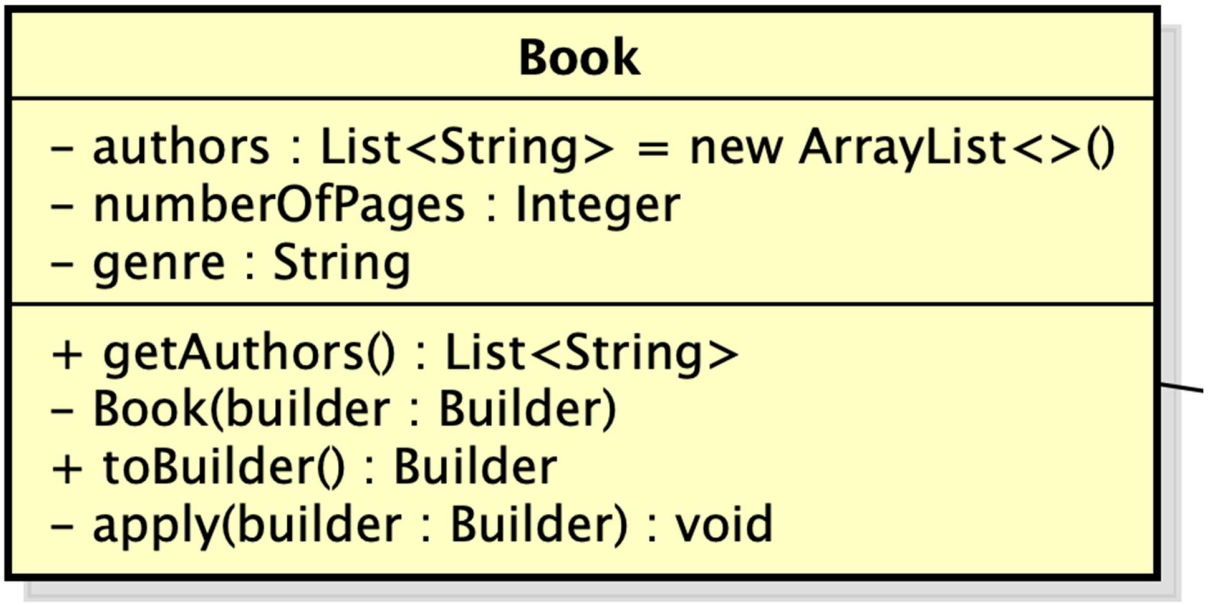


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	authors	List<String>		Name of author(s)
3	numberOfPages	int		Number of pages
4	genre	String		Genre of product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Book		Constructor of Book entity
2	toBuilder	Builder	Construct and return a Builder object with the current state of Book entity
3	getAuthors	List<String>	Return the list of authors
4	apply	void	Update information field

4.4.3.6 Class "Newspaper"

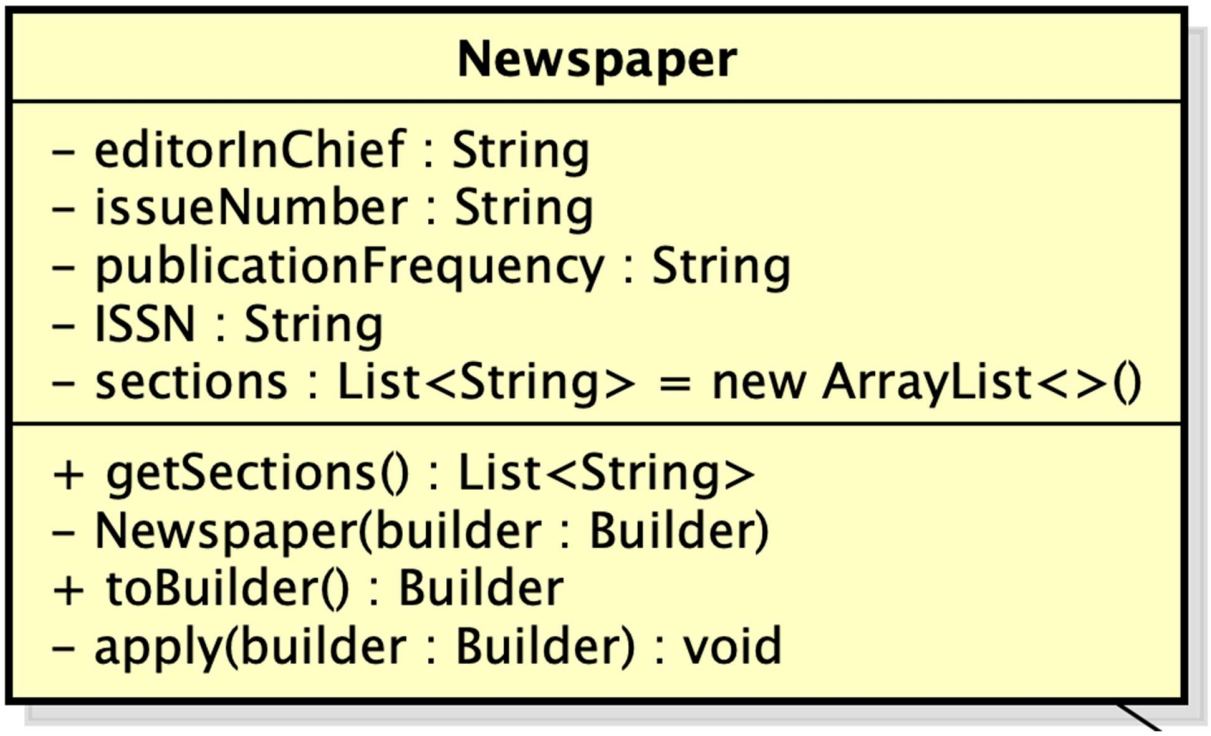


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	editorInChief	String		Name of Chief editor
2	issueNumber	String		Issue number
3	publicationFrequency	double		Publication frequency
4	ISSN	String		ISSN identification
5	sections	List<String>		Sections included in the newspaper

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Newspaper		Constructor of Newspaper entity
2	toBuilder	Builder<?, ?>	Construct and return a Builder object with the current state of Newspaper entity
3	apply	void	Update information field of the Newspaper
4	getSections	List<String>	Return Newspaper's section

4.4.3.7 Class "CD"

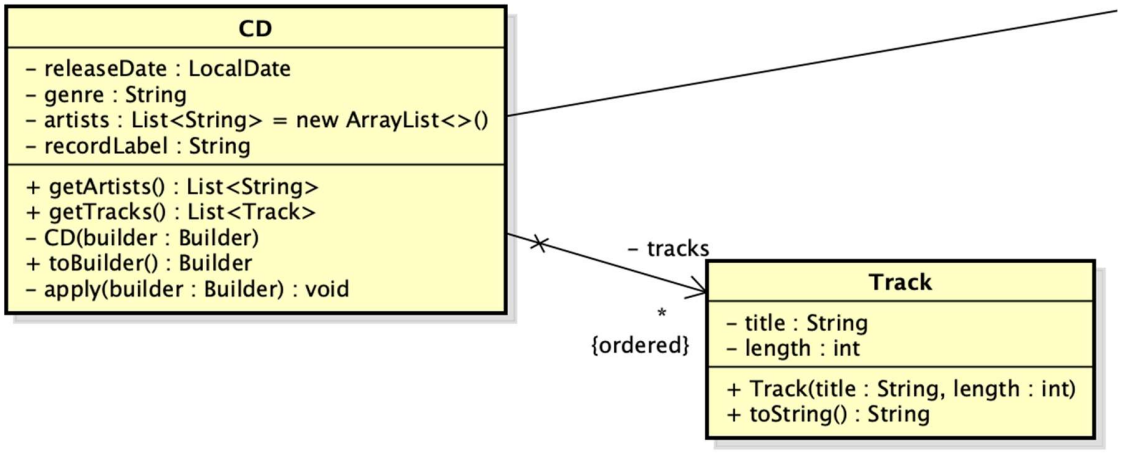


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	releaseDate	LocalDate		Release date
2	genre	String		Genre
3	artists	List<String>		Artist(s) in the CD
4	recordLabel	String		Record label
5	tracks	List<Tracks>		CD's Tracks

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	CD		Constructor of CD entity
2	toBuilder	Builder<?, ?>	Construct and return a Builder object with the current state of CD entity
3	apply	void	Update information field of the CD
4	getArtists	List<String>	Return CD's artists
5	getTracks	List<Track>	Return CD's Tracks

4.4.3.8 Class "Track"

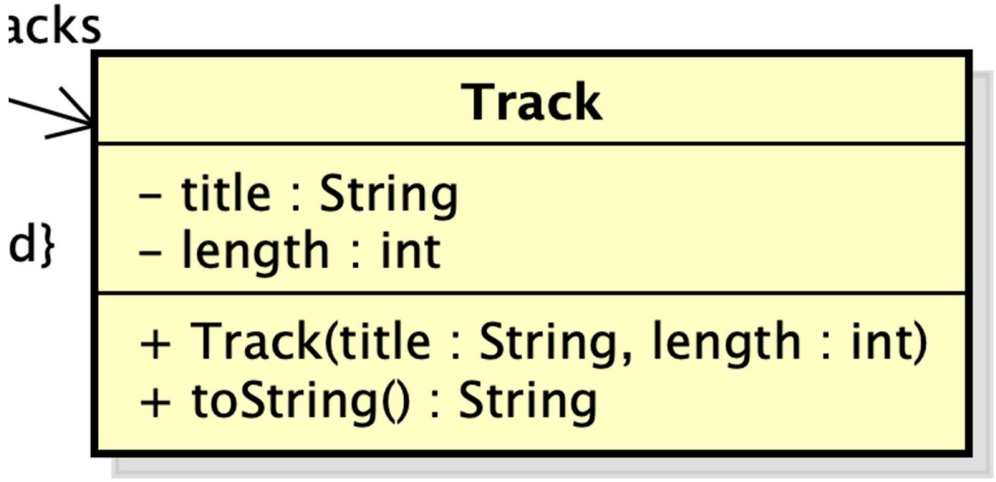


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	title	String		Track title
2	length	int		track duration

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Track		Constructor of Track entity
2	toString	String	Display Track's info

4.4.3.9 Class "DVD"

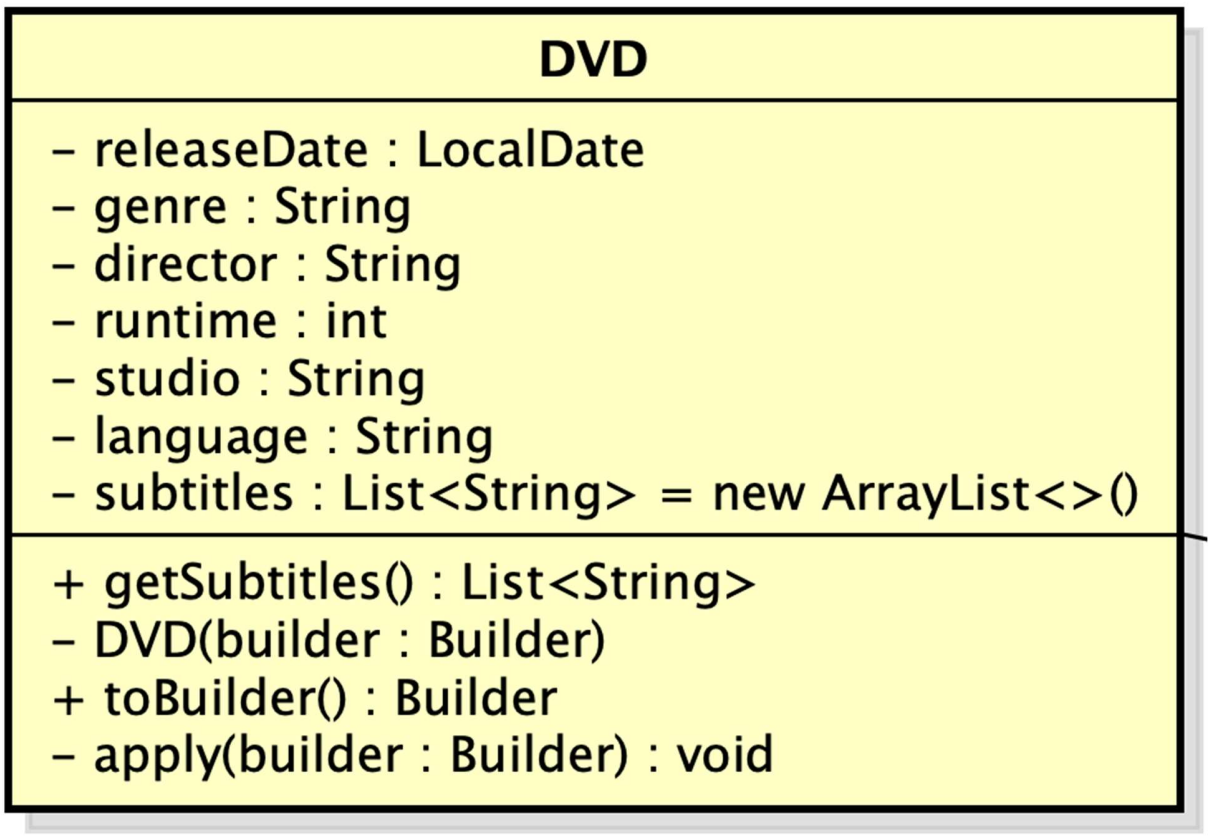


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	releaseDate	LocalDate		Release date
2	genre	String		Genre
3	director	String		Director name
4	runtime	int		Duration
5	studio	String		Studio
6	language	String		Language
7	subtitles	List<String>		Subtitles

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	DVD		Constructor of DVD entity
2	toBuilder	Builder<?, ?>	Construct and return a Builder object with the current state of DVD entity
3	apply	void	Update information field of the DVD
4	getSubtitles	List<String>	Return subtitles

4.4.3.10 Class "VersionedEntity"

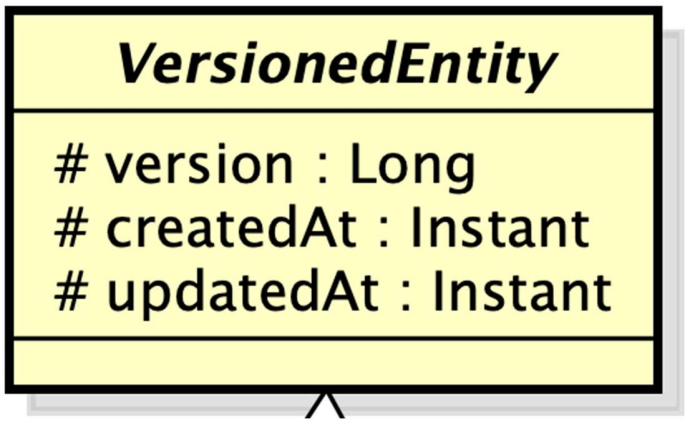


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	version	Long	0	Entity version
2	createdAt	Timestamp	Current time	Time of creation
3	updatedAt	Timestamp	Current time	Time of update

4.4.3.11 Interface "ProductRepository"

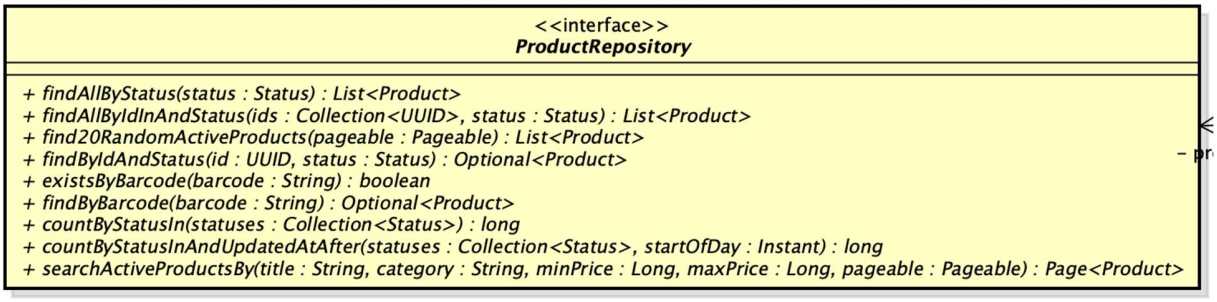


Table 1. Attribute design

#	Name	Data type	Default value	Description
1				

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	findAllByStatus	List<Product>	Return list of products based on

			status
2	findAllByIdInAndStatus	List<Product>	Return list of products based on id and status
3	find20RandomActiveProducts	List<Product>	Return 20 random active products
4	findByIdAndStatus	Optional<Product>	Return a specific product and verifies its current state
5	existsByBarcode	Boolean	Check for existence of a product with the same barcode
6	findByBarcode	Optional<Product>	Return product based on its barcode
7	countByStatusIn	long	Counts the total number of products that match specified statuses
8	countByStatusInAndUpdatedAtAfter	long	Counts the total number of products that match specified statuses after a timestamp
9	searchActiveProductsBy	Page<Product>	Return a paginated object

Parameter:

- findAllByStatus(Product.Status status): The status of the products to retrieve
- findAllByIdInAndStatus(Collection<UUID> ids, Product.Status status)
  ids (Collection<UUID>): A collection of product IDs.
  status (Product.Status): The status of the products.
- find20RandomActiveProducts(Pageable pageable): Pagination configuration
- findByIdAndStatus(UUID id, Product.Status status)
  - id (UUID): The unique ID of the product.
  - status (Product.Status): The status of the product.
- existsByBarcode(String barcode): The barcode to check for existence.

- findByBarcode(String barcode): The barcode of the product to find.
- countByStatusIn(Collection<Product.Status> statuses): A collection of statuses to count.
- countByStatusInAndUpdatedAtAfter(Collection<Product.Status> statuses, java.time.Instant startOfDay)
  - statuses (Collection<Product.Status>): Statuses to count (usually used for counting deleted/deactivated items).
  - startOfDay (Instant): The timestamp to start counting from (e.g., the beginning of today).
- searchActiveProductsBy(...)
  - title, category (String): Search strings for pattern matching (LIKE).
  - minPrice, maxPrice (Long): Value ranges for filtering.
  - pageable (Pageable): Pagination and sorting information.

Exception:

- Spring Data JPA itself might throw a `DataAccessException` (a runtime exception) if there are database connection errors or query syntax issues. There are no custom exceptions explicitly declared or thrown from this interface.

4.4.3.12 Class "ProductManagementService"

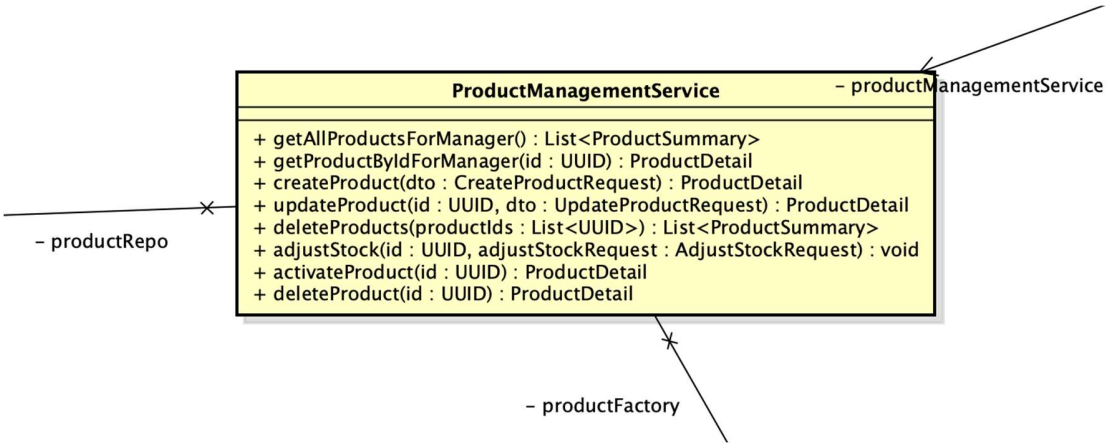


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productRepo	ProductRepository		Handles direct database operations for the Product entity.

2	productFactory	ProductFactory		Delegates the creation and updating of Product objects.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getAllProductsForManager	List<ProductSummary>	Retrieves and maps all products into summary representations for the manager dashboard
2	getProductByIdForManager	ProductDetail	Fetches specific product details by ID or throws an exception if not found
3	createProduct	ProductDetail	Validates barcode uniqueness, instantiates via factory, saves to database, and returns product details
4	updateProduct	ProductDetail	Updates existing product attributes and pricing via factory, saving changes to the database
5	deleteProducts	List<ProductSummary>	Performs batch deletion of products while strictly enforcing the daily deletion quota limit
6	adjustStock	void	Adjusts the product's inventory stock quantity based on the provided positive or negative delta
7	activateProduct	ProductDetail	Restores a previously deactivated product back to the active business status
8	deleteProduct	ProductDetail	Soft-deletes or deactivates a single product depending on its current stock quantity

Parameter:

- getProductByIdForManager(UUID id): The ID of the product to retrieve.
- createProduct(CreateProductRequest dto): The Data Transfer Object containing all the information needed to create a new product.
- updateProduct(UUID id, UpdateProductRequest dto)
  - id (UUID): The ID of the product to update.
  - dto (UpdateProductRequest): The DTO containing the fields to update.
- deleteProducts(List<UUID> productIds): A list of product IDs to delete.
- adjustStock(UUID id, AdjustStockRequest adjustStockRequest)
  - id (UUID): The product ID.

 adjustStockRequest (AdjustStockRequest): DTO containing the adjustment amount (delta) and the reason.

- activateProduct(UUID id): The ID of the product to reactivate.
- deleteProduct(UUID id): The ID of the product to delete.

Exception:

- ProductNotFoundException: Thrown if no product is found with the provided ID in the database.
- ProductAlreadyExistedException: Thrown if the barcode provided in the dto already exists in the system.
- ExceededDeletionQuotaException: Thrown if the total number of deleted products for the day (already deleted + currently requested) exceeds the daily allowed limit (20 products/day).

4.4.3.13 Class "ProductFactory"

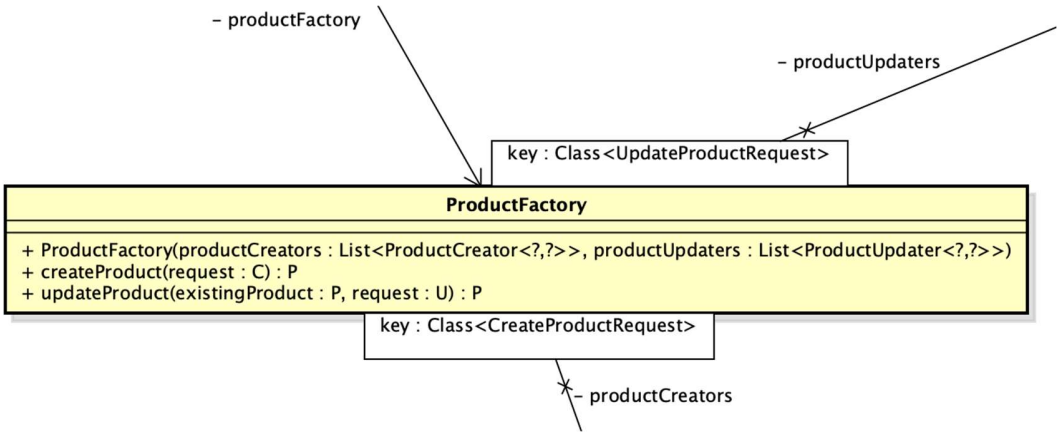


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productCreators	List<ProductCreator<?, ?>>		Maps creation requests to their specific creator strategies.
2	productUpdaters	List<ProductUpdater<?,		Maps update requests to their

		?>>		specific updater strategies.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	ProductFactory		Constructor of ProductFactory
2	createProduct		Delegates product instantiation to the specific creator strategy matching the provided request class type
3	updateProduct		Locates the matching updater strategy and delegates the modification of the existing product entity

Parameter:

- createProduct(C request): The creation request DTO.
- updateProduct(P existingProduct, U request)
  - existingProduct (P extends Product): The current entity object retrieved from the database.
  - request (U extends UpdateProductRequest): The DTO containing the update data.

4.4.3.14 Interface “ProductCreator”

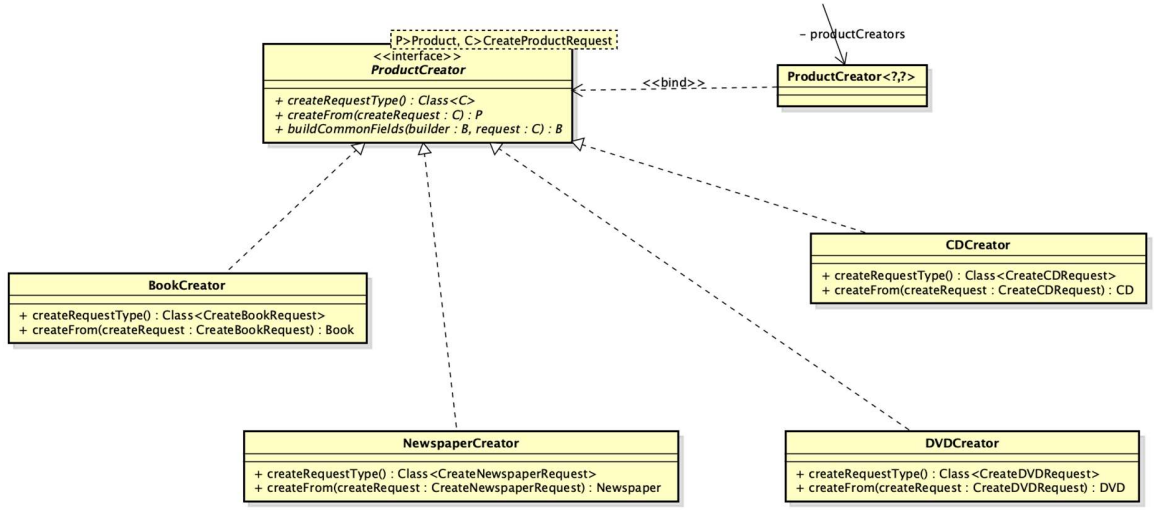


Table 1. Attribute design

#	Name	Data type	Default value	Description

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	createRequestType		Identifies and returns the specific creation request class type handled by this creator strategy
2	createFrom		Instantiates a new specific product entity using data from the provided creation request object
3	buildCommonFields		Populates the product builder with common base attributes extracted from the given creation request

Parameter:

- createProduct(C request): The creation request DTO.

- updateProduct(P existingProduct, U request)
  - existingProduct (P extends Product): The current entity object retrieved from the database.
  - request (U extends UpdateProductRequest): The DTO containing the update data.
- buildCommonFields(B builder, C/U request)
  - builder (B): The Builder for the Product entity.
  - request (C or U): The DTO (Create or Update).

4.4.3.15 Interface "ProductUpdater"

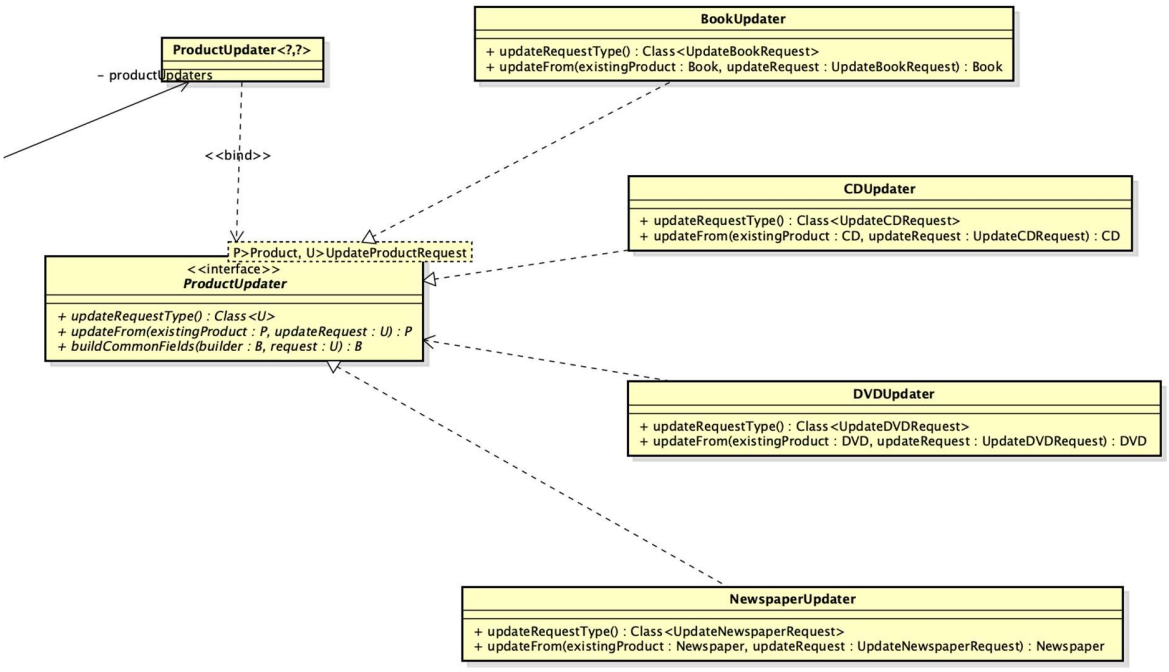


Table 1. Attribute design

#	Name	Data type	Default value	Description

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	updateRequestType		Identifies and returns the specific update request class type handled by this updater strategy

2	updateFrom		Modifies an existing specific product entity using data from the provided update request object
3	buildCommonFields		Populates the product builder with common updatable base attributes extracted from the update request

Parameter:

- createProduct(C request): The creation request DTO.
- updateProduct(P existingProduct, U request)
  - existingProduct (P extends Product): The current entity object retrieved from the database.
  - request (U extends UpdateProductRequest): The DTO containing the update data.
- buildCommonFields(B builder, C/U request)
  - builder (B): The Builder for the Product entity.
  - request (C or U): The DTO (Create or Update).

4.4.3.16 Class "ProductManagementController"

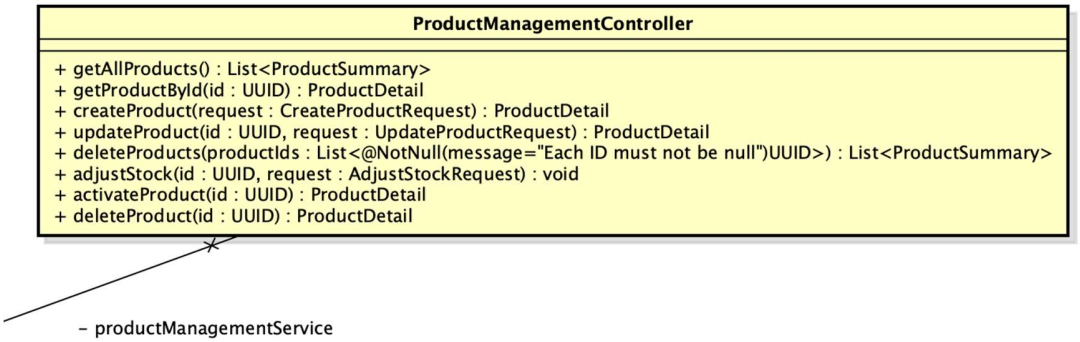


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productManagementService	ProductManagementService		Handles product management core business

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getAllProducts	List<ProductSummary>	Exposes an endpoint to retrieve summary data of all products for the manager dashboard
2	getProductById	ProductDetail	Provides an endpoint to fetch and return comprehensive details of a specific product
3	createProduct	ProductDetail	Handles HTTP requests to validate payload and create a new product in the system
4	updateProduct	ProductDetail	Accepts validated partial update payloads to modify specific attributes of an existing product.
5	deleteProducts	List<ProductSummary>	Processes batch deletion requests for multiple products while enforcing validation on the payload size
6	adjustStock	void	Provides an endpoint to modify the inventory stock quantity for a specific product
7	activateProduct	ProductDetail	Handles requests to reactivate a previously deactivated product, restoring it to active status
8	deleteProduct	ProductDetail	Exposes an endpoint to soft-delete or deactivate a single product based on its ID

Parameter:

- getProductById(@PathVariable UUID id): Extracted from the URL path.

- createProduct(@Valid @RequestBody CreateProductRequest request): Extracted from the HTTP Request Body. It is checked for validity (@Valid)
- updateProduct(@PathVariable UUID id, @RequestBody @Valid UpdateProductRequest request)
  - id (UUID): From the URL path.
  - request (UpdateProductRequest): From the HTTP Body, validated.
- deleteProducts(@RequestBody @Size(...) List<@NotNull UUID> productIds): The list of IDs to delete, extracted from the Body.
- adjustStock(@PathVariable UUID id, @Valid @RequestBody AdjustStockRequest request): id (from URL path) and request (from Body, containing the adjustment delta).
- activateProduct(@PathVariable UUID id) and deleteProduct(@PathVariable UUID id): From the URL path. Both methods call the Service layer to execute the logic.

Exception:

- MethodArgumentNotValidException thrown if the request payload violates the validation annotations defined within the CreateProductRequest class

4.4.3.17 Class "DeliveryInformation"

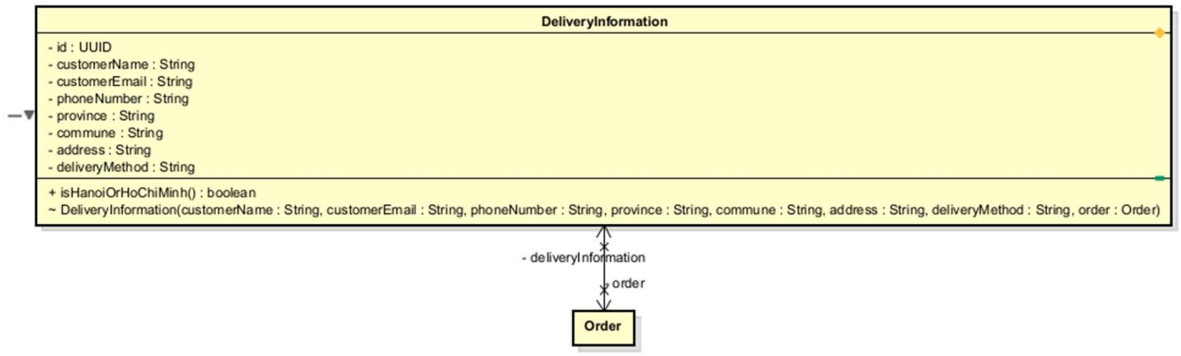


Table 1. Attribute design

#	*Name*	*Data type*	*Default value*	*Description*
1	id	UUID	—	Shared PK with Order (via @MapsId)

2	order	Order	—	Parent order (bidirectional @OneToOne)
3	customerName	String	—	Recipient full name
4	customerEmail	String	—	Recipient email address
5	phoneNumber	String	—	Recipient phone (max 10 chars)
6	province	String	—	Delivery province (max 50 chars)
7	commune	String	—	Delivery commune (max 50 chars)
8	address	String	—	Full street address (TEXT)
9	deliveryMethod	String	—	Delivery method label (max 50 chars)

Table 2. Operation design

#	*Name*	*Return type*	*Description (purpose)*
1	isHanoiOrHoChiMinh()	boolean	Returns true if province is Hà Nội or Hồ Chí Minh; used by delivery fee strategy

Parameter:

Exception:

Method:

- Constructor is package-private — instantiation is exclusively through Order.provideDeliveryInformation(...), enforcing that a DeliveryInformation always belongs to a valid Order.

4.4.3.18 Class “Invoice”

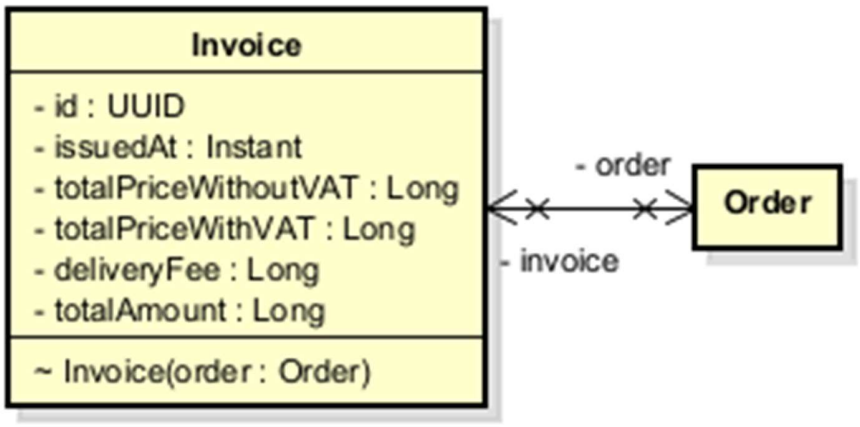


Table 1. Attribute design

#	*Name*	*Data type*	*Default value*	*Description*
1	id	UUID	—	Shared PK with Order (via @MapsId)
2	order	Order	—	Parent order
3	issuedAt	Instant	Instant.now()	Timestamp of invoice generation
4	totalPriceWithoutVAT	Long	—	Subtotal snapshot at time of issue (VND)
5	totalPriceWithVAT	Long	—	VAT-included total snapshot (VND)
6	deliveryFee	Long	—	Delivery fee snapshot (VND)
7	totalAmount	Long	—	Grand total: totalPriceWithVAT + deliveryFee

Table 2. Operation design

#	*Name*	*Return type*	*Description (purpose)*
—	Invoice(order)	—	Package-private constructor; snapshots all financial values from Order at instantiation time

Parameter:

- Invoice(order) — order: **Order** — the order whose financial values are snapshotted

Exception:

Method:

- All financial values are copied from the Order at construction time. The Invoice is immutable after creation — it is a point-in-time billing record. Constructor is package-private; only Order.updateDeliveryFee() creates an Invoice.

4.4.3.19 Class "PaymentTransaction"

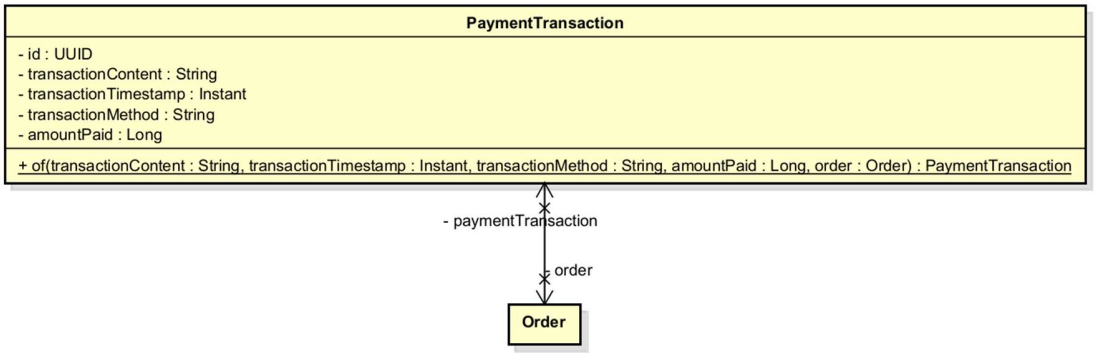


Table 1. Attribute design

#	*Name*	*Data type*	*Default value*	*Description*
1	id	UUID	generated	Surrogate primary key

2	transactionContent	String (TEXT)	—	Gateway-supplied transaction memo/reference
3	transactionTimestamp	Instant	—	Time of payment confirmation
4	transactionMethod	String	—	Payment method label (e.g. "VIETQR", "PAYPAL")
5	amountPaid	Long	—	Amount actually charged (VND)
6	order	Order	—	Associated order (FK)

Table 2. Operation design

#	*Name*	*Return type*	*Description (purpose)*
1	of(...)	PaymentTransaction	Static factory: constructs a PaymentTransaction and links it to the Order

Parameter:

- of(...) — transactionContent: String — gateway reference string
- of(...) — transactionTimestamp: Instant — time of confirmation
- of(...) — transactionMethod: String — payment method identifier
- of(...) — amountPaid: Long — confirmed payment amount in VND
- of(...) — order: @NonNull Order — the order being paid; back-sets order.paymentTransaction

Exception:

Method:

4.4.3.20 Class "StockValidator"

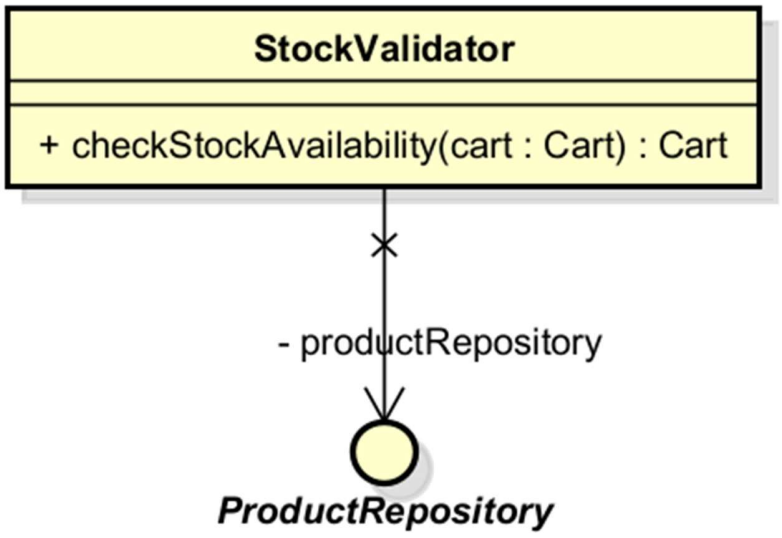


Table 1. Attribute design

#	*Name*	*Data type*	*Default value*	*Description*
1	productRepository	ProductRepository	injected	Repository for live product lookup

Table 2. Operation design

#	*Name*	*Return type*	*Description (purpose)*
1	checkStockAvailability(cart)	Cart	Validates all cart items against live stock; synchronises product references; returns the validated cart

Parameter:

- checkStockAvailability(cart) — cart: **Cart** — the customer's current cart; must be non-null

Exception:

- EmptyCartException — if cart contains no items
- NotEnoughStockException — if any item's requested quantity exceeds available stock; carries a map of productId → availableQuantity and a list of missing product IDs
- ProductNotFoundException — if a product has been deactivated since being added to cart

Method:

- Fetches all products in the cart in a single batch query (findAllByIdInAndStatus(ids, ACTIVE)). Products absent from the result (deactivated) are removed and flagged. Products present are synchronised back into the cart's item references. Stock insufficiency is collected across all items before throwing, so the error message is complete.

4.4.3.21 Class "DeliveryFeeCalculator"

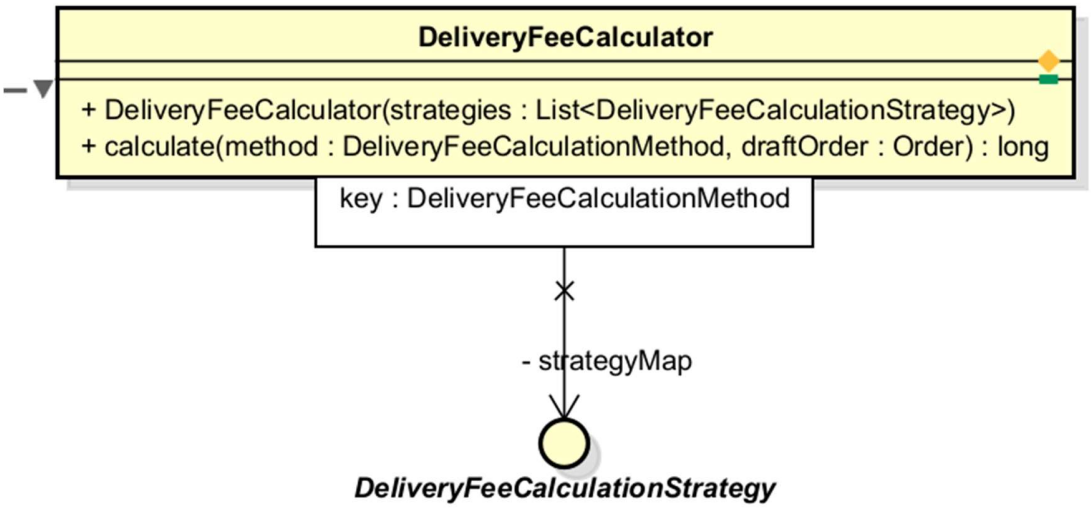


Table 1. Attribute design

#	*Name*	*Data type*	*Default value*	*Description*

1	strategyMap	EnumMap<DeliveryFeeCalculationMethod, DeliveryFeeCalculationStrategy>	populated at construction	Registry of all available strategies, keyed by method enum

Table 2. Operation design

#	*Name*	*Return type*	*Description (purpose)*
1	calculate(method, draftOrder)	long	Looks up strategy by method and delegates fee calculation

Parameter:

- calculate(...) — method: **DeliveryFeeCalculationMethod** — identifies which strategy to use
- calculate(...) — draftOrder: **Order** — the draft order containing items and delivery information

Exception:

- UnsupportedDeliveryFeeStrategyException — if no strategy is registered for the given method

Method:

- All DeliveryFeeCalculationStrategy beans are injected as a list at construction time and self-register via strategy.method(). Callers are fully decoupled from concrete strategy classes.

4.4.3.22 Class "StandardDeliveryFeeStrategy"

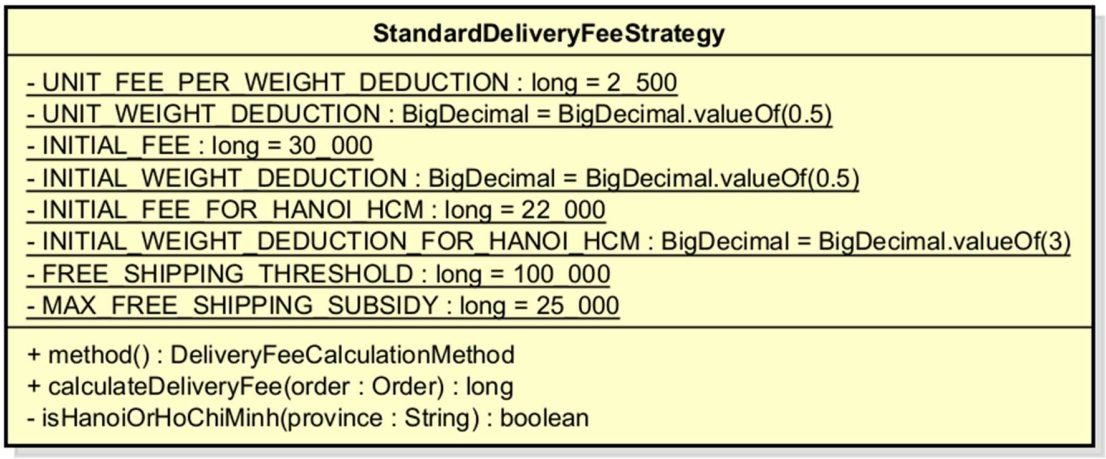


Table 1. Attribute design

#	*Name*	*Data type*	*Default value*	*Description*
1	INITIAL_FEE	long (const)	30,000	Base fee for non-Hanoi/HCM provinces (VND)
2	INITIAL_WEIGHT_DEDUCTION	BigDecimal (const)	0.5	Free weight allowance for non-Hanoi/HCM (kg)
3	INITIAL_FEE_FOR_HANOI_HCM	long (const)	22,000	Base fee for Hanoi/HCM (VND)
4	INITIAL_WEIGHT_DEDUCTION_FOR_HANOI_HCM	BigDecimal (const)	3.0	Free weight allowance for

				Hanoi/HCM (kg)
5	UNIT_FEE_PER_WEIGHT_DEDUCTION	long (const)	2,500	Additional fee per 0.5 kg over allowance (VND)
6	UNIT_WEIGHT_DEDUCTION	BigDecimal (const)	0.5	Weight increment unit (kg)
7	FREE_SHIPPING_THRESHOLD	long (const)	100,000	Order subtotal above which subsidy applies (VND)
8	MAX_FREE_SHIPPING_SUBSIDY	long (const)	25,000	Maximum subsidy deducted from fee (VND)

Table 2. Operation design

#	*Name*	*Return type*	*Description (purpose)*
1	method()	DeliveryFeeCalculationMethod	Returns STANDARD; used for self-registration in registry
2	calculateDeliveryFee(order)	long	Computes fee based on province, weight, and order subtotal

Parameter:

Exception:

Method:

- Selects base fee and weight allowance by province (Hanoi/HCM vs. other). Calculates excess weight in 0.5 kg increments (ceiling division). Applies free-shipping subsidy (up to 25,000 VND) if order subtotal exceeds 100,000 VND.

4.4.3.23 Class "PlaceOrderService"

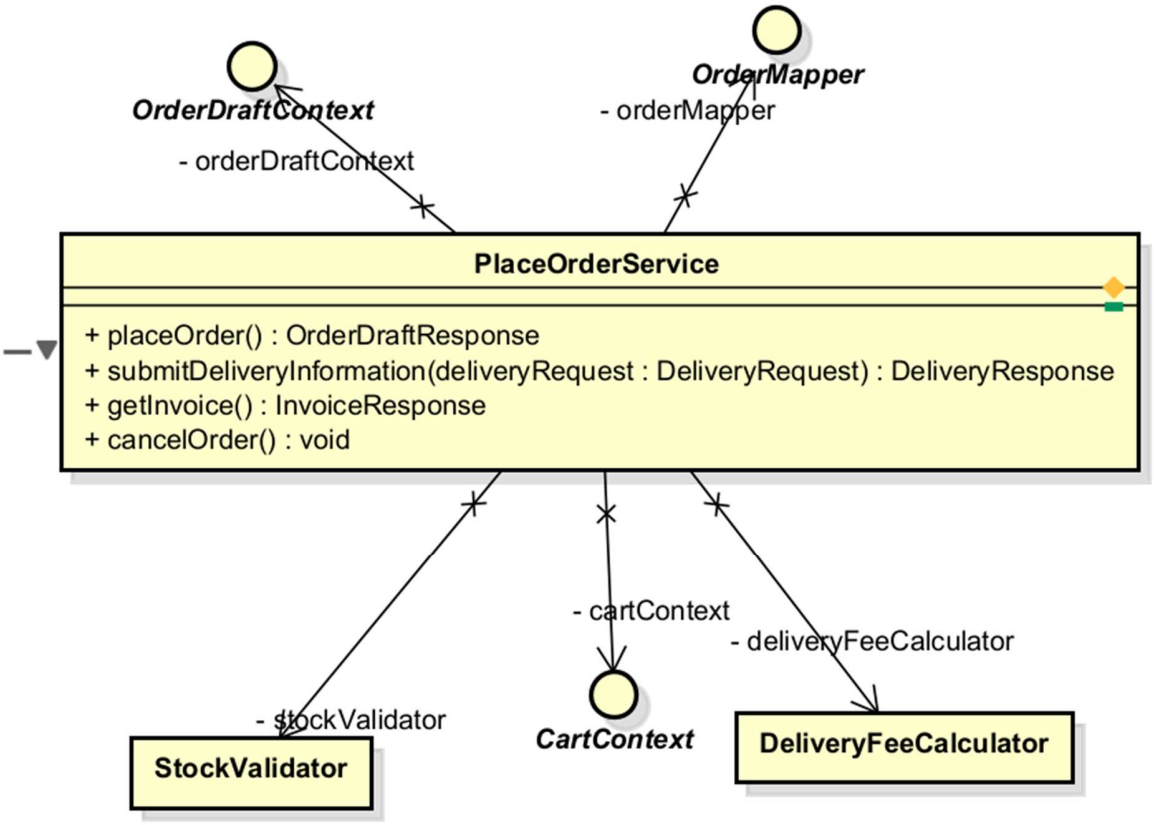


Table 1. Attribute design

#	*Name*	*Data type*	*Default value*	*Description*

1	stockValidator	StockValidator	injected	Validates cart stock before order creation
2	deliveryFeeCalculator	DeliveryFeeCalculator	injected	Calculates delivery fee for the order
3	cartContext	CartContext	injected	Provides access to the current session's cart
4	orderDraftContext	OrderDraftContext	injected	Stores and retrieves the in-progress draft order
5	orderMapper	OrderMapper	injected	Maps domain objects to response DTOs

Table 2. Operation design

#	*Name*	*Return type*	*Description (purpose)*
1	placeOrder()	OrderDraftResponse	Creates a draft order from cart; carries over delivery info if re-placing
2	submitDeliveryInformation(request)	DeliveryResponse	Attaches delivery info and calculates+sets delivery fee and invoice
3	getInvoice()	InvoiceResponse	Returns the current draft order's invoice
4	cancelOrder()	void	Clears the draft order context (pre-payment cancellation)

Parameter:

- submitDeliveryInformation — deliveryRequest: **DeliveryRequest** — validated DTO containing all delivery fields

Exception:

- EmptyCartException, NotEnoughStockException, ProductNotFoundException — from placeOrder()
- OrderNotPlacedException — if getInvoice() or submitDeliveryInformation() is called without a current draft
- OrderNotCompleteException — if invoice is missing when getInvoice() is called

Method:

4.4.3.24 Class "OrderFinalizer"

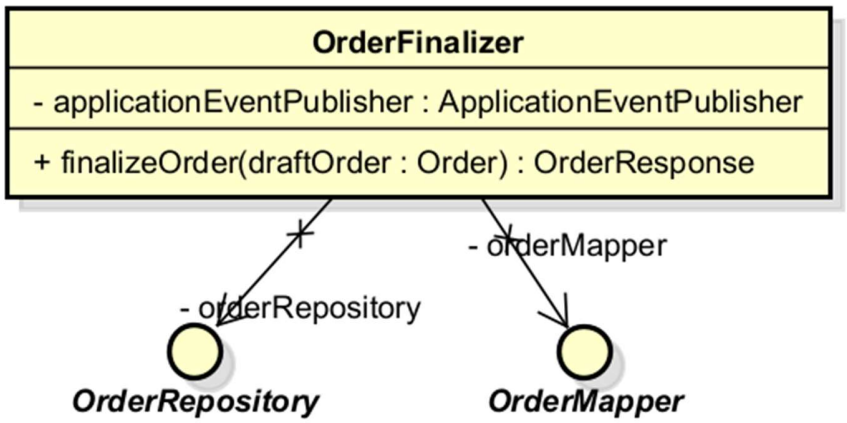


Table 1. Attribute design

#	*Name*	*Data type*	*Default value*	*Description*
1	orderRepository	OrderRepository	injected	Persists the completed order
2	orderMapper	OrderMapper	injected	Maps saved order to response DTO
3	applicationEventPublisher	ApplicationEventPublisher	injected	Publishes OrderSuccessEvent after commit

Table 2. Operation design

#	*Name*	*Return type*	*Description (purpose)*
1	finalizeOrder(draftOrder)	OrderResponse	Completes order, persists it, publishes OrderSuccessEvent

Parameter:

- finalizeOrder — draftOrder: **Order** — a fully populated DRAFT order ready for completion

Exception:

- OrderNotCompleteException — if order.complete() fails due to missing required fields

Method:

- Calls order.complete() (state transition DRAFT → PENDING), saves via repository, then publishes OrderSuccessEvent. The event is handled post-commit by PlaceOrderSuccessListener. OrderFinalizer is invoked by the payment service after payment is confirmed, not directly by PlaceOrderService.

4.4.3.25 Class "PlaceOrderSuccessListener"

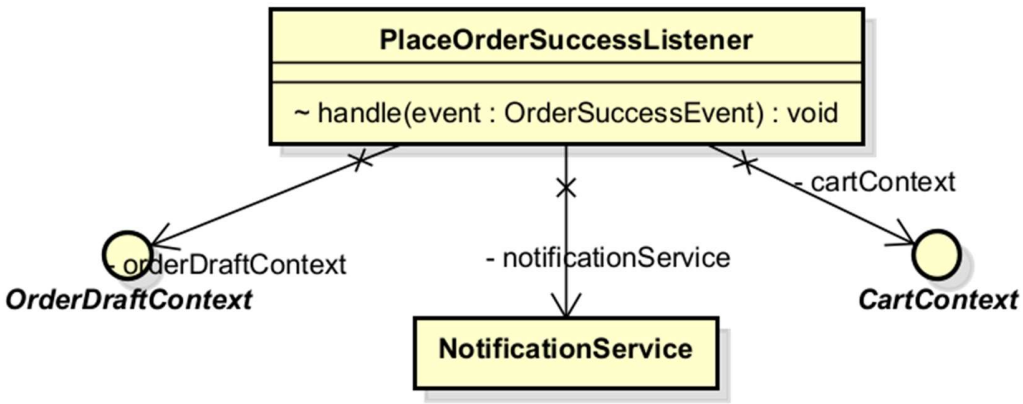


Table 1. Attribute design

#	*Name*	*Data type*	*Default value*	*Description*
1	cartContext	CartContext	injected	Clears the cart post-order
2	orderDraftContext	OrderDraftContext	injected	Clears the draft order post-order
3	notificationService	NotificationService	injected	Dispatches the confirmation email

Table 2. Operation design

#	*Name*	*Return type*	*Description (purpose)*
1	handle(event)	void	Reacts to OrderSuccessEvent: clears cart, clears draft, sends confirmation email

Parameter:

- handle — event: **OrderSuccessEvent** — record carrying the fully persisted Order

Exception:

Method:

- Annotated with @TransactionalEventListener(phase = AFTER_COMMIT). Fires only after the originating transaction commits successfully. Clears both the cart and draft order contexts, then dispatches an OrderConfirmationEmailMessage via NotificationService. A failure here does not roll back the placed order.

4.4.3.26 Class "OrderSuccessEvent"

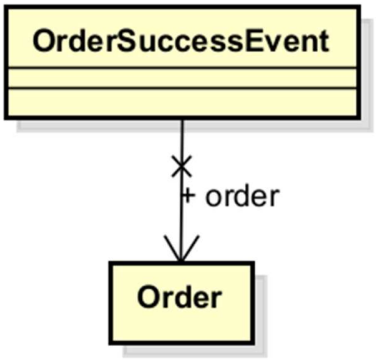


Table 1. Attribute design

#	*Name*	*Data type*	*Description*
1	order	Order	The fully persisted, PENDING-status order

Method:

- Java record — immutable carrier. All associations required by listeners (deliveryInformation, invoice, paymentTransaction, items) must be eagerly loaded before this event is published, as the Hibernate session is closed by the time listeners fire.

4.4.3.27 Class "OrderConfirmationEmailMessage"

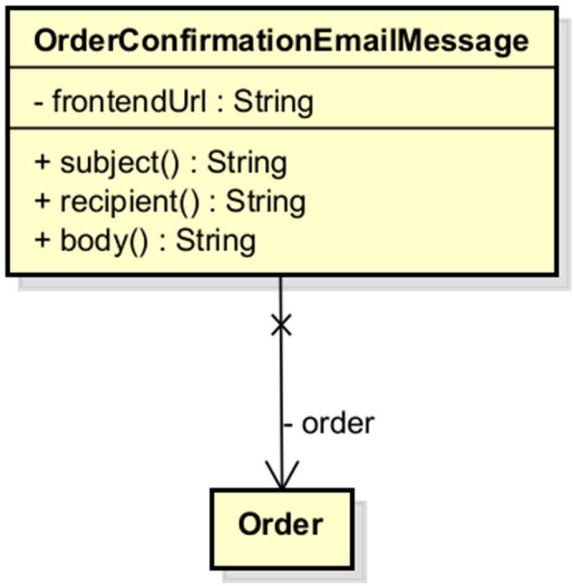


Table 1. Attribute design

#	*Name*	*Data type*	*Default value*	*Description*
1	order	Order	—	The order whose details are rendered into the email
2	frontendUrl	String	—	Base URL for the 'View Order' link in the email body

Table 2. Operation design

#	*Name*	*Return type*	*Description (purpose)*
1	subject()	String	Returns the email subject line including order ID
2	recipient()	String	Returns the customer email from DeliveryInformation
3	body()	String	Builds and returns the full HTML email body

Method:

- Nested class Factory: a Spring @Component extending EmailMessage.Factory<OrderConfirmationEmailMessage, OrderSuccessEvent>. Implements messageType() returning OrderConfirmationEmailMessage.class (routing key in notification registry), and createMessage(event) constructing the message from the event payload.

4.4.3.28 Class "OrderController"

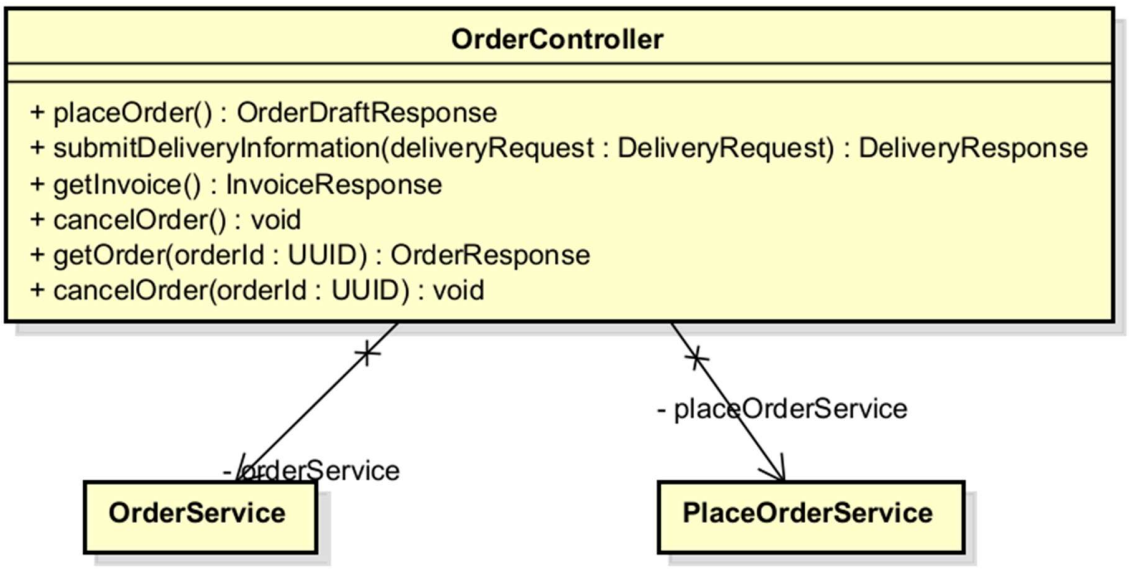


Table 1. Attribute design

#	*Name*	*Data type*	*Default value*	*Description*
1	placeOrderService	PlaceOrderService	—	Handles order placement workflow
2	orderService	OrderService	—	Handles order retrieval and post-placement cancellation

Table 2. Operation design

#	*Name*	*Return type*	*Description (purpose)*

1	placeOrder()	OrderDraftResponse	POST /order — creates draft order from cart
2	submitDeliveryInformation(request)	DeliveryResponse	POST /order/delivery — submits delivery info
3	getInvoice()	InvoiceResponse	GET /order/invoice — retrieves current invoice
4	cancelOrder()	void	DELETE /order — cancels in-progress draft
5	getOrder(orderId)	OrderResponse	GET /order/{orderId} — retrieves a placed order
6	cancelOrder(orderId)	void	POST /order/{orderId}/cancel — cancels a placed order

Exception:

- NotEnoughStockException, EmptyCartException, ProductNotFoundException — from placeOrder()
- OrderNotFoundException — from cancelOrder(orderId) if order not found
- OrderStateTransitionException — from cancelOrder(orderId) if order is not cancellable

4.4.3.29 Class "PaymentService"

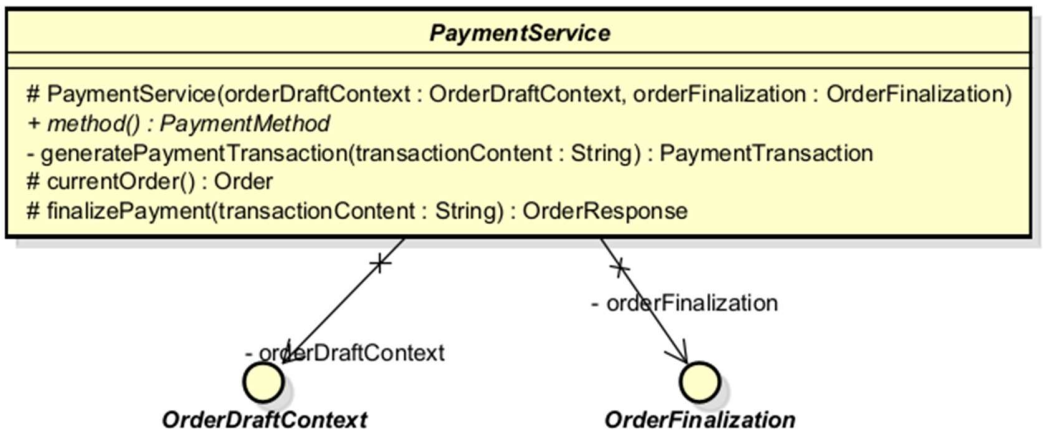


Table 1. Attribute design

#	Name	Data Type	Default Value	Description
1	orderDraftContext	OrderDraftContext	injected	Provides access to the current session's draft order
2	orderFinalization	OrderFinalization	injected	Handles order completion, persistence, and event publishing

Table 2. Operation Design

#	Name	Return type	Description
1	method()	PaymentMethod	Abstract, each subclass declares which payment method it owns; return value is consumed by generatePaymentTransaction()
2	currentOrder()	Order	Protected final, resolves and returns the current draft order from session context; cannot be overridden

3	generatePaymentTransaction(transactionContent)	PaymentTransaction	Private, constructs a PaymentTransaction from shared fields; invisible to subclasses and callers
4	finalizePayment(transactionContent)	OrderResponse	Protected final, calls generatePaymentTransaction() then delegates to orderFinalization; the single sanctioned entry point for completing a payment

Parameters:

- generatePaymentTransaction(transactionContent): transactionContent — String — gateway-supplied transaction reference or memo string; consumed as the transactionContent field of the resulting PaymentTransaction
- finalizePayment(transactionContent): transactionContent — String — passed directly to generatePaymentTransaction(); the only input subclasses need to provide to complete a payment

Method — generatePaymentTransaction:

Constructs a PaymentTransaction by calling PaymentTransaction.of(...) with the following field values, all resolved internally without subclass input:

- transactionContent — passed in by the caller
- transactionTimestamp — Instant.now() at the moment of call
- transactionMethod — method().name(), resolved from the subclass's own identity declaration
- amountPaid — currentOrder().getTotalAmount()
- order — currentOrder()

This method is private rather than protected because the PaymentTransaction schema is stable and uniform across all payment methods. No subclass has a legitimate reason to observe or override this construction logic. Subclasses interact with it exclusively through finalizePayment().

Method — finalizePayment:

Called by concrete subclasses at the terminal point of their own payment flow — after all payment-method-specific verification or capture logic has completed successfully. The method generates the transaction record, links it to the order via PaymentTransaction.of(), then hands the order to OrderFinalization.finalizeOrder(), which transitions the order to PENDING, persists it, and publishes OrderSuccessEvent.

Method — method():

Abstract — declared with no implementation. Each concrete subclass returns its own PaymentMethod enum value (VIETQR, PAYPAL). The return value is consumed directly by generatePaymentTransaction() to populate the transactionMethod field, tying the subclass's identity declaration into the shared transaction construction without requiring the subclass to pass its method name as an explicit parameter.

4.4.3.30 Class "VietQRGatewayFacade"

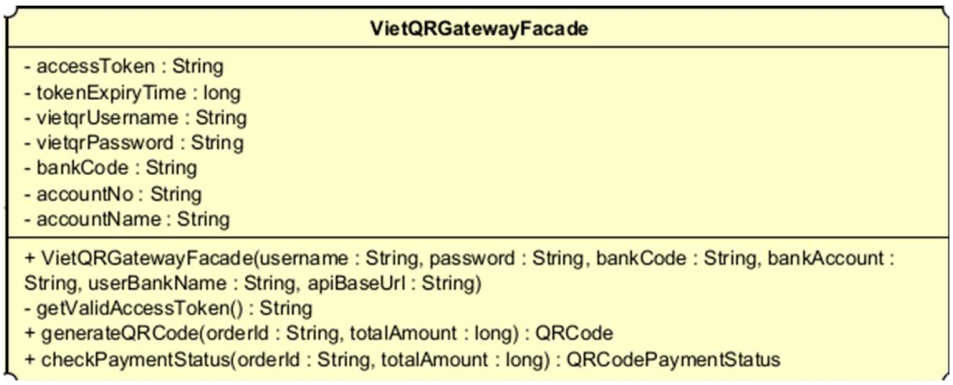


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	boundary	VietQRBoundary		Boundary object used to send HTTP requests to VietQR APIs.
2	accessToken	String		Current VietQR access token used for authenticated API calls.
3	tokenExpiryTime	long		Expiration time of the current access token in milliseconds.
4	vietqrUsername	String		Username of VietQR sandbox

5	vietqrPassword	String		Password of VietQR sandbox
6	bankCode	String		Bank code of the receiver account used to generate QRpayment.
7	accountNo	String		Bank account number used to receive customer payment.
8	accountName	String		Bank account owner name

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	generateQRCode	QRCode	Generate a QR code for the given order information and return the QRCode
2	checkPaymentStatus	PaymentStatus	Check the payment status. of the given order and return the Status.
3	getValidAccessToken	String	Return a valid VietQR access token, reusing the current one if it is still valid

Parameter:

generateQRCode(orderId : String, totalAmount : long) : QRCode

- orderId: Unique identifier of the order for which the QR code is generated.
- totalAmount: Total payment amount of the order.

checkPaymentStatus(orderId : String, totalAmount : long) : QRCodePaymentStatus

- orderId: Unique identifier of the order whose payment status is being checked.
- totalAmount: Expected payment amount used to verify the transaction.

Exception

- InvalidTokenException: if VietQR returns an invalid access token.
- PaymentException: if QR generation or payment status checking cannot be completed.
- UnknownException: if an unexpected error occurs during VietQR API communication or response handling.

Method
getValidAccessToken()

• Checks whether the current access token is still valid.
• Requests a new token from VietQR if the current token has expired.
• Returns a valid access token for API communication.

generateQRCode(orderId : String, totalAmount : long)

• Uses a valid access token to call the VietQR QR generation API.
• Generates a payment QR code for the specified order and amount.
• Returns a QRCode object containing QR payment information.

checkPaymentStatus(orderId : String, totalAmount : long)

• Uses a valid access token to call the VietQR payment status API.
•Verifies the payment information for the specified order.
• Returns a QRCodePaymentStatus object containing the current payment status.

4.4.3.31 Class "VietQRPaymentService"

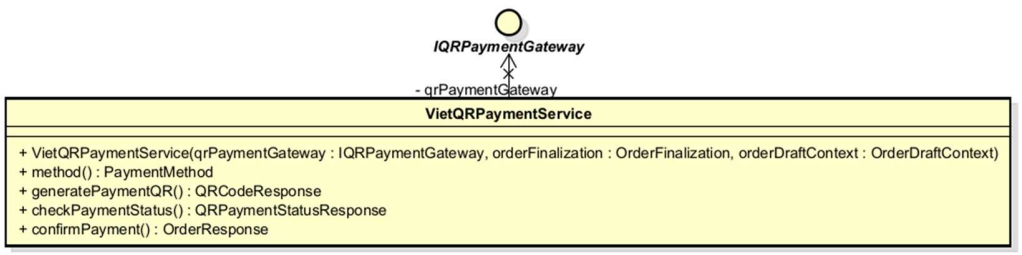


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	qrPaymentGateway	IQRPaymentGateway		Gateway interface used to communicate with the QR payment subsystem (e.g., VietQR).

Table 2. Operation design

#	Name	Return type	Description
1	generatePaymentQR	QRCodeResponse	Generate QR payment information for the current order.
2	checkPaymentStatus	QRPaymentStatusResponse	Check and return the current payment status.
3	confirmPayment	OrderResponse	Confirm payment completion and finalize the order.

Parameter:
generatePaymentTransaction(transactionContent : String)
• transactionContent: description or content associated with the payment transaction.

Method

generatePaymentQR()

• Retrieves the current order information.
• Requests QR code generation through the payment gateway.
• Returns a QRCodeResponse object for customer payment.

checkPaymentStatus()

• Retrieves the current order information.
• Checks the payment status through the payment gateway.
• Returns a QRPaymentStatusResponse object.

confirmPayment()

• Verifies that payment has been completed successfully.
• Finalizes the order and records the payment transaction.
• Returns an OrderResponse object containing the completed order information.

4.4.3.32 Class "QRCode"

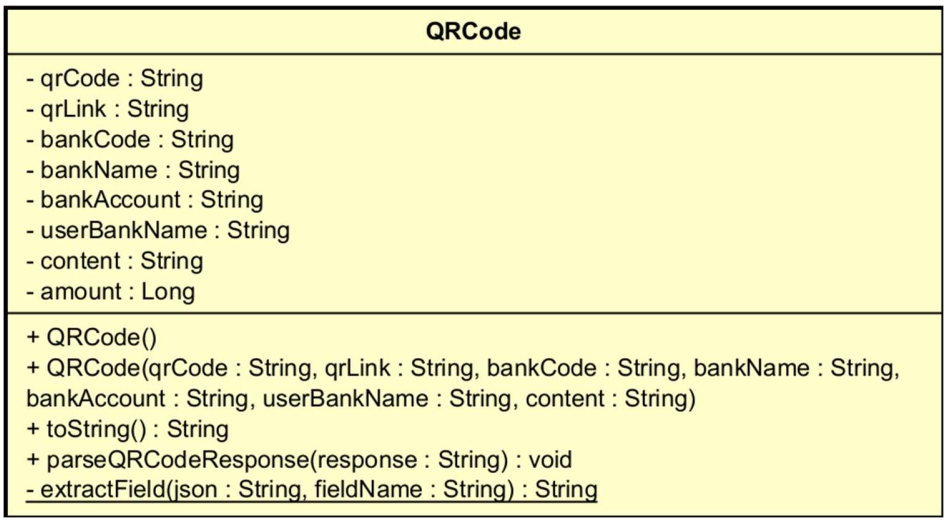


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	qrCode	String		QR code string returned from VietQR API, used for displaying or generating the QR image.
2	qrLink	String		Link to view the generated QR code image.
3	bankCode	String		Code of the receiving bank.
4	bankName	String		Name of the receiving bank.
5	bankAccount	String		Bank account number used for payment.
6	userBankName	String		Name of the bank account owner.
7	content	String		Payment transfer content or transaction

				description.
8	amount	Long		Payment amount of the order.

Table 2. Operation design

#	Name	Return type	Description
1	toString	String	Convert QR code information into a readable string format.
2	parseQRCodeResponse	void	Parse the VietQR QR generation response and extract QR information.
3	extractField	String	Extract a specified field value from a response string.

Method

toString()

Converts QR code information into a readable string format.
•Includes important QR and bank information.
• Returns the formatted string.

parseQRCodeResponse(response : String)

• Reads the raw response returned by the VietQR API.
• Extracts QR code information from the response.
• Stores the extracted values in the corresponding attributes.

extractField(json : String, fieldName : String)

• Searches for the specified field in the response string.
• Extracts the corresponding field value.
• Returns the extracted value.

4.4.3.33 Class "QRGenerateRequest"

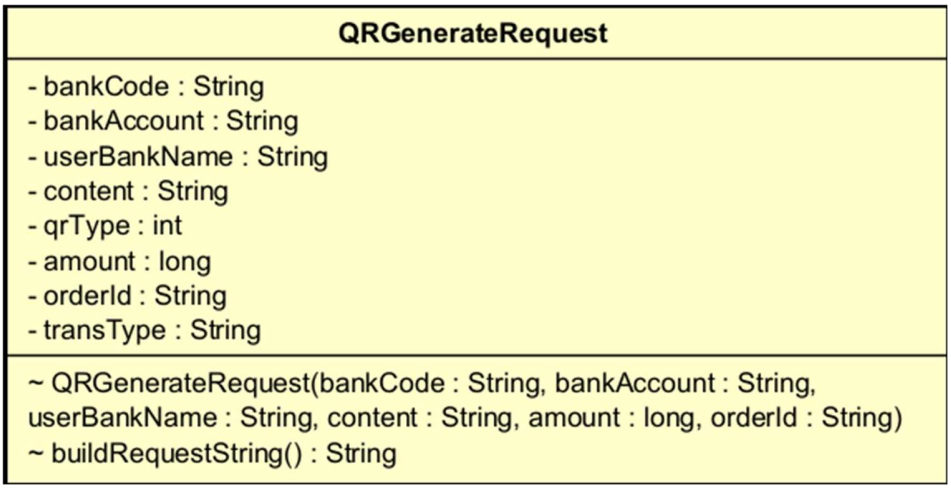


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	bankCode	String		Code of the receiving bank used for VietQR payment generation.
2	bankAccount	String		Bank account number used to receive payment.
3	userBankName	String		Name of the bank account owner.
4	content	String		Payment transfer content / transaction description.
5	qrType	int		Type of QR code (dynamic, static, or semi-dynamic).
6	amount	long		Payment amount of the order.
7	orderId	String		ID of the order associated with the payment.

8	transType	String		Type of transaction (e.g. credit transaction).

Table 2. Operation design

#		Name	Return type	Description (purpose)
1		QRGenerateRequest	constructor	Initialize request object with payment information for VietQR API.
2		buildRequestString	String	Convert request attributes into request string / JSON format for VietQR API call.

Parameter:

- QRGenerateRequest(bankCode : String, bankAccount : String, userBankName : String, content : String, amount : long, orderId : String)

- bankCode: code of the receiving bank
- bankAccount: bank account number for payment
- userBankName: account owner name
- content: payment transfer content
- amount: payment amount

• orderId: ID of the corresponding order

Method:

buildRequestString() : String

- Converts all stored attribute values into the request format required by the VietQR API.
- Constructs a formatted JSON / request string containing payment and bank information.
- Returns the final request string to be sent by the boundary layer.

4.4.3.34 Class "QRAccessTokenResponse"

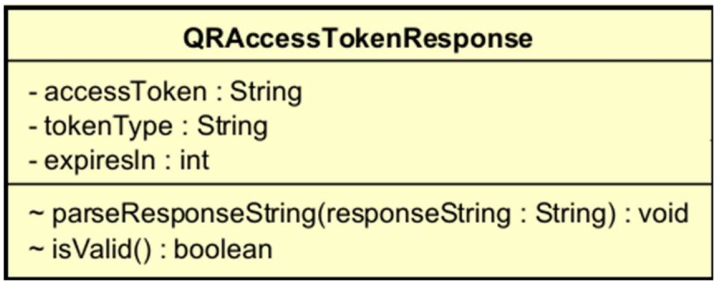


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	accessToken	String		Access token returned from VietQR API, used for authenticated requests.
2	tokenType	String		Type of token returned by VietQR (e.g. Bearer).
3	expiresIn	int		Token expiration time returned by VietQR API, usually in seconds.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	parseResponseString	void	Parse raw VietQR token response string and extract token information.
2	isValid	boolean	Check whether the parsed token response is valid.

Parameter:

- parseResponseString(response : String) : void
  • response: response string returned from VietQR token API

4.4.3.35 Class "QRAccessTokenRequest"

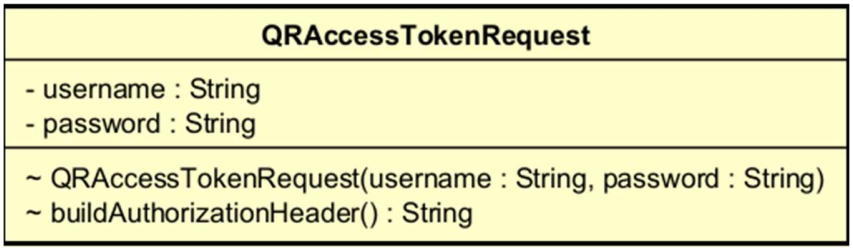


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	username	String		VietQR account username used to request access token.
2	password	String		VietQR account password used to request access token.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	QRAccessTokenRequest	constructor	Initialize token request object with username and password.
2	buildAuthorizationHeader	String	Build authorization header string for VietQR token API request.

Parameter:

QRAccessTokenRequest(username : String, password : String)

- username: VietQR account username
- password: VietQR account password

4.4.3.36 Class "VietQRBoundary"

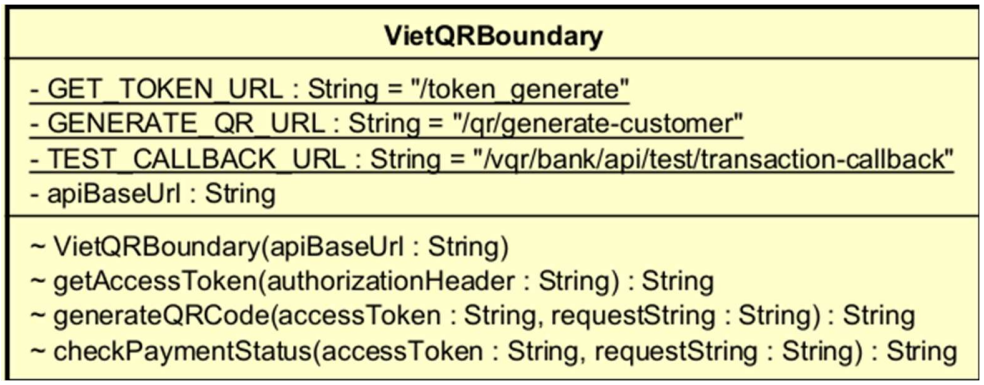


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	GET_TOKEN_URL	String		URL of the VietQR API endpoint used to request an access token.

2	GENERATE_QR_URL	String		URL of the VietQR API endpoint used to generate a payment QR code.
3	TEST_CALLBACK_URL	String		URL of the VietQR API endpoint used to simulate or check payment callback status.
4	httpClient	HttpClient		HTTP client used to send requests to VietQR APIs.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getAccessToken	String	Send a request to VietQR to obtain an access token and return the raw response string.
2	generateQRCode	String	Send a QR generation request to VietQR and return the raw response string.
3	checkPaymentStatus	String	Send a payment status or callback request to VietQR and return the raw response string.

Parameter:

getAccessToken(authorizationHeader : String) : String

- authorizationHeader: authorization header built from VietQR username and password, used to request an access token.

generateQRCode(accessToken : String, requestString : String) : String

- accessToken: valid VietQR access token used for authenticated API calls.
- requestString: formatted request payload containing bank and payment information for QR generation.

checkPaymentStatus(accessToken : String, requestString : String) : String

- accessToken: valid VietQR access token used for authenticated API calls.
- requestString: formatted request payload used to query or simulate payment status.

Method:

getAccessToken(authorizationHeader : String)

- Sends an HTTP request to the VietQR token API using the provided authorization header.
- Receives the raw response containing access token information.
- Returns the response string for further parsing.

generateQRCode(accessToken : String, requestString : String)

- Sends an HTTP request to the VietQR QR generation API using the access token and request payload.
- Receives the raw response containing generated QR information.
- Returns the response string for further parsing.

checkPaymentStatus(accessToken : String, requestString : String)

- Sends an HTTP request to the VietQR payment status or callback API using the given token and payload.
- Receives the raw response containing transaction status information.
- Returns the response string for further parsing.

4.4.3.37 Class "IQRPaymentGateway"

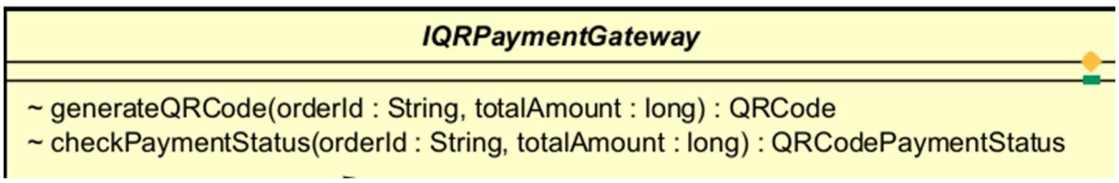


Table 2. Operation design

#	Name	Return type	Description
1	generateQRCode	QRCode	Generate a QR code for the specified order and payment amount.
2	checkPaymentStatus	QRCodePaymentStatus	Check the payment status of the specified order.

Parameter

generateQRCode(orderId : String, totalAmount : long)

- *orderId: unique identifier of the order for which the QR code is generated.*
- *totalAmount: total payment amount of the order.*

checkPaymentStatus(orderId : String, totalAmount : long)

- *orderId: unique identifier of the order whose payment status is checked.*
- *totalAmount: expected payment amount used to verify the transaction.*

Method:

generateQRCode(orderId : String, totalAmount : long)

- *Defines the operation for generating a payment QR code.*
- *Must be implemented by a concrete QR payment gateway.*
- *Returns a QRCode object containing payment QR information.*

checkPaymentStatus(orderId : String, totalAmount : long)

- *Defines the operation for checking payment status.*
- *Must be implemented by a concrete QR payment gateway.*
- *Returns a QRCodePaymentStatus object containing the current payment status.*

4.4.3.38 Class "QRTestCallbackRequest"

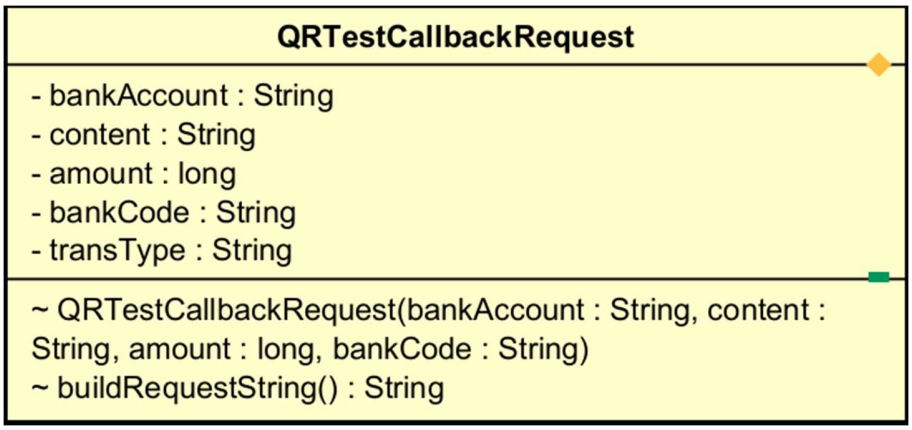


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	bankAccount	String		Bank account number used in the callback simulation request.
2	content	String		Payment content or transaction description associated with the order.
3	amount	long		Payment amount used in the callback request.
4	bankCode	String		Bank code of the receiving account.
5	transType	String		Type of transaction used in the callback simulation (e.g., incoming transfer).

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	QRTestCallbackRequest	Constructor	Initialize the callback request object with payment and bank information.

2	buildRequestString	String	Build the request payload string for the VietQR test callback API.

Parameter:

QRTestCallbackRequest(bankAccount : String, content : String, amount : long, bankCode : String)

- bankAccount: bank account number used in the callback request.
- content: payment description or order content.
- amount: transaction amount used for callback simulation.
- bankCode: bank code of the receiving account.

Method:

QRTestCallbackRequest(bankAccount : String, content : String, amount : long, bankCode : String)

- Initializes the callback request object with all required transaction information.
- Stores the provided values into the corresponding attributes.
- Prepares the object to be converted into a request payload string.

buildRequestString() : String

- Converts all stored attribute values into the format required by the VietQR callback API.
- Builds a formatted JSON / request string containing callback simulation data.
- Returns the final request string to be sent by the boundary layer.

4.4.3.39 Class "VietQRPaymentController"

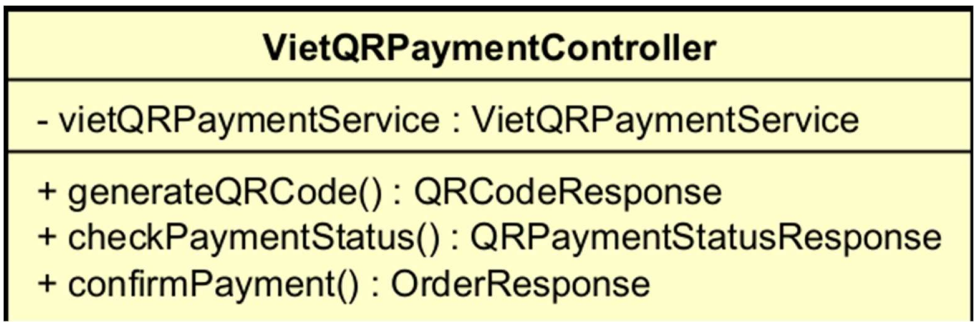


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	vietQRPaymentService	VietQRPaymentService		Service object used to handle VietQR payment business logic and payment processing operations.

Table 2. Operation design

#	Name	Return type	Description
1	generateQRCode	QRCodeResponse	Generate QR payment information for the current order.
2	checkPaymentStatus	QRPaymentStatusResponse	Check and return the current payment status.
3	confirmPayment	OrderResponse	Confirm payment completion and finalize the order.

Method:
generateQRCode()

- Calls the payment service to generate QR payment information.
- Retrieves the QR code details for the current order.

- Returns a QRCodeResponse object.

checkPaymentStatus()

- Calls the payment service to check the payment status
- Retrieves the latest transaction status information.
- Returns a QRPaymentStatusResponse object.

confirmPayment()

- Calls the payment service to confirm successful payment.
- Finalizes the order after payment verification.
- Returns an OrderResponse object containing the completed order information.

4.4.3.40 Class "QRCodePaymentStatus"

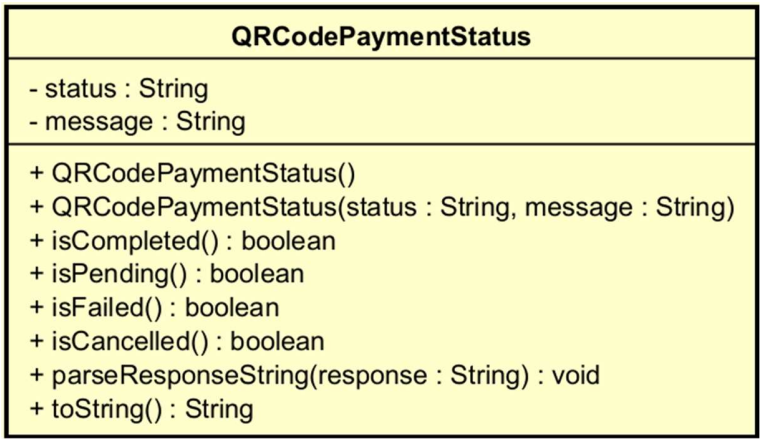


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	status	String		Current payment status returned by VietQR
2	message	String		Additional message describing the payment status returned by VietQR.

Table 2. Operation design

#	Name	Return type	Description

1	isCompleted	boolean	Check whether the payment has been completed successfully.
2	isPending	boolean	Check whether the payment is currently pending.
3	isFailed	boolean	Check whether the payment has failed.
4	isCancelled	boolean	Check whether the payment has been cancelled.
5	parseResponseString	void	Parse the raw payment status response returned from VietQR and extract status information.

Parameter:
parseResponseString(response : String)
• response: raw payment status response returned from the VietQR API.

Method:
parseResponseString(response : String)

- Reads the raw response returned by the VietQR payment status API.
- Extracts the payment status and message information.
- Stores the extracted values in the corresponding attributes (status and message).

4.4.3.41 Class “PaypalPaymentController”

Table 1. Attribute design

#	Name	Data type	Default value	Description
1				
2				
…				

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	createPayment()	PayPalCreateResponse	POST /order/payment/paypal/create. Starts a PayPal payment for the current draft order and returns the approval URL plus the provider order id.
2	capturePayment(request)	OrderResponse	POST /order/payment/paypal/capture. Captures the approved PayPal payment and finalizes the order.
3	cancelPayment()	void	POST /order/payment/paypal/cancel. Records a best-effort customer cancellation; the draft order is kept for retry.

Parameter:

- request : PayPalCaptureRequest — carries the PayPal token (“provider order id”)

PayPal appended to the return URL once the customer approved the payment.

–

Method:

- Thin boundary layer (REST controller): holds no business logic of its own.
- Exchanges only DTOs with PaypalPaymentService (DATA coupling), mirroring the existing VietQR controller for consistency across payment methods.

Other diagrams:

– Sequence diagram — PayPal create/capture lifecycle (see updated SD for this use case).

4.4.3.42 Class "PaypalPaymentService"

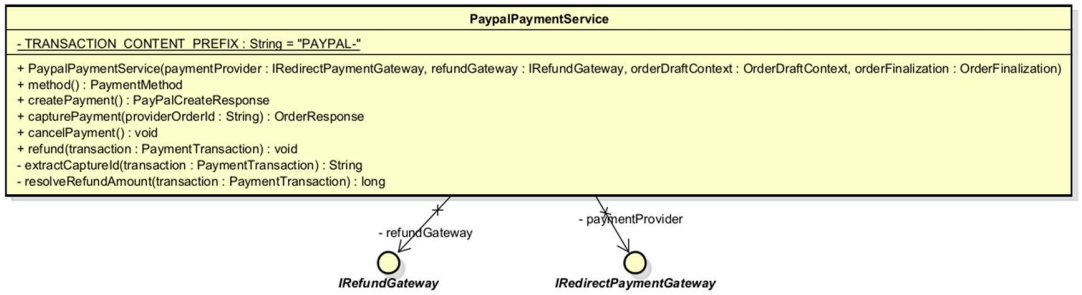


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	TRANSACTION_ CONTENT_PREFIX	String	"PAYPAL-"	Static final. Prefix stamped onto PaymentTransaction.transactionContent at capture time so the write path (capture) and the read path (refund) cannot drift apart.
2	paymentProvider	IRedirectPaymentGateway	(injected)	Abstraction used to create and capture PayPal payments; implemented by PayPalGatewayFacade.
3	refundGateway	IRefundGateway	(injected)	Abstraction used to refund a previously captured payment; implemented by PayPalGatewayFacade.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	method()	PaymentMethod	Identifies this service to the rest of the system as the PAYPAL payment method.

2	createPayment()	PayPalCreateResponse	Step 1: creates a PayPal order for the current draft order and returns the approval URL the frontend must open.
3	capturePayment(providerOrderId)	OrderResponse	Step 2: captures the approved PayPal payment, verifies it belongs to the current order, then finalizes the order (persist + email).
4	cancelPayment()	void	Step 2′: best-effort log when the customer cancels on PayPal; the draft order is kept intact.
5	refund(transaction)	void	Refunds a previously captured PayPal payment, invoked asynchronously from the order-cancellation/rejection flow.

Parameter:

- providerOrderId : String — the PayPal token returned to the frontend on redirect, used to capture the matching PayPal order.
- transaction : PaymentTransaction — the persisted record used to recover the original PayPal capture id and the amount to refund.

Exception:

- PaymentException if the PayPal capture is not completed, if the captured order does not match the current draft order (anti-tampering check), or if the refund does not complete.

Method:

- capturePayment defends against token reuse across orders by comparing capture.referenceId() with the current order id before finalizing — a forged or leaked token cannot finalize someone else's order.
- refund() reconstructs the raw PayPal capture id by stripping the "PAYPAL-" prefix recorded at capture time, keeping the write path and the read path in sync.
- Shares its order-draft and order-finalization behaviour with the abstract base class PaymentService (also used by VietQRPaymentService), so only the PayPal-specific steps are implemented here.

Other diagrams:

- Sequence diagram — PayPal create/capture lifecycle (redirect flow),

replacing the previous “card form” sequence diagram.

- Sequence diagram — refund flow, triggered from the order-cancellation event handler (shared with the cancellation use case).

4.4.3.43 Class “PayPalGatewayFacade”

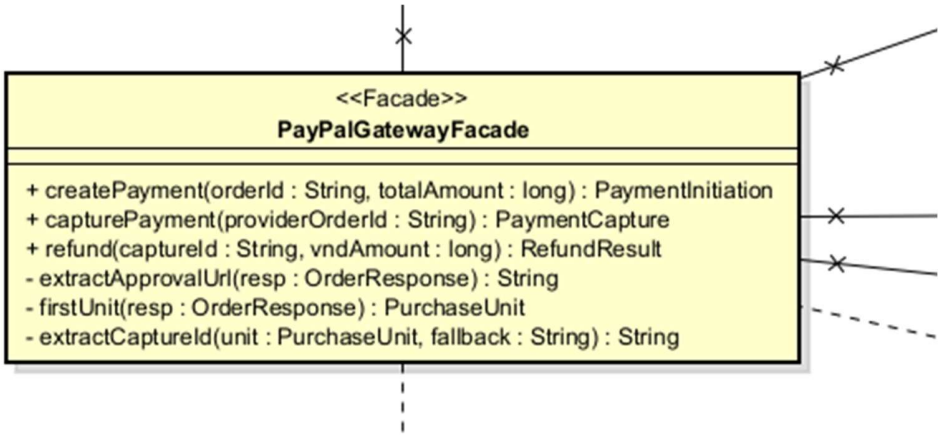


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	ordersClient	PayPalOrdersClient	(injected)	Thin wrapper over the PayPal v2 Orders endpoints (create-order, capture-order).
2	paymentsClient	PayPalPaymentsClient	(injected)	Thin wrapper over the PayPal v2 Payments refund endpoint.
3	amountConverter	PayPalAmountConverter	(injected)	Converts AIMS VND amounts into the 2-decimal value PayPal expects in the configured currency.
4	props	PayPalProperties	(injected)	Typed configuration: base URL, currency, VND conversion rate, return/cancel URLs, brand name.

Table 2. Operation design

#	Name	Return type	Description (purpose)

1	createPayment(orderId, totalAmount)	PaymentInitiation	Builds the PayPal create-order request, stamps the AIMS order id as reference_id/custom_id, and returns the provider order id plus the approval URL.
2	capturePayment(providerOrderId)	PaymentCapture	Captures the PayPal order, detects payer cancellation, and returns a provider-agnostic capture result.
3	refund(captureId, vndAmount)	RefundResult	Validates input, converts the VND amount to the gateway currency, and refunds the capture via the Payments client.

Parameter:

- orderId : String — the AIMS order id, stamped on both reference_id and custom_id so the capture can later be verified against the right order.
- vndAmount : long — amount to refund expressed in VND; currency conversion stays inside the subsystem.

Exception:

- PaymentException — translated from the internal PayPalApiException so no PayPal-specific exception ever crosses the subsystem boundary.
- UserCancelledException — thrown when PayPal reports a VOIDED or CANCELLED status on capture.

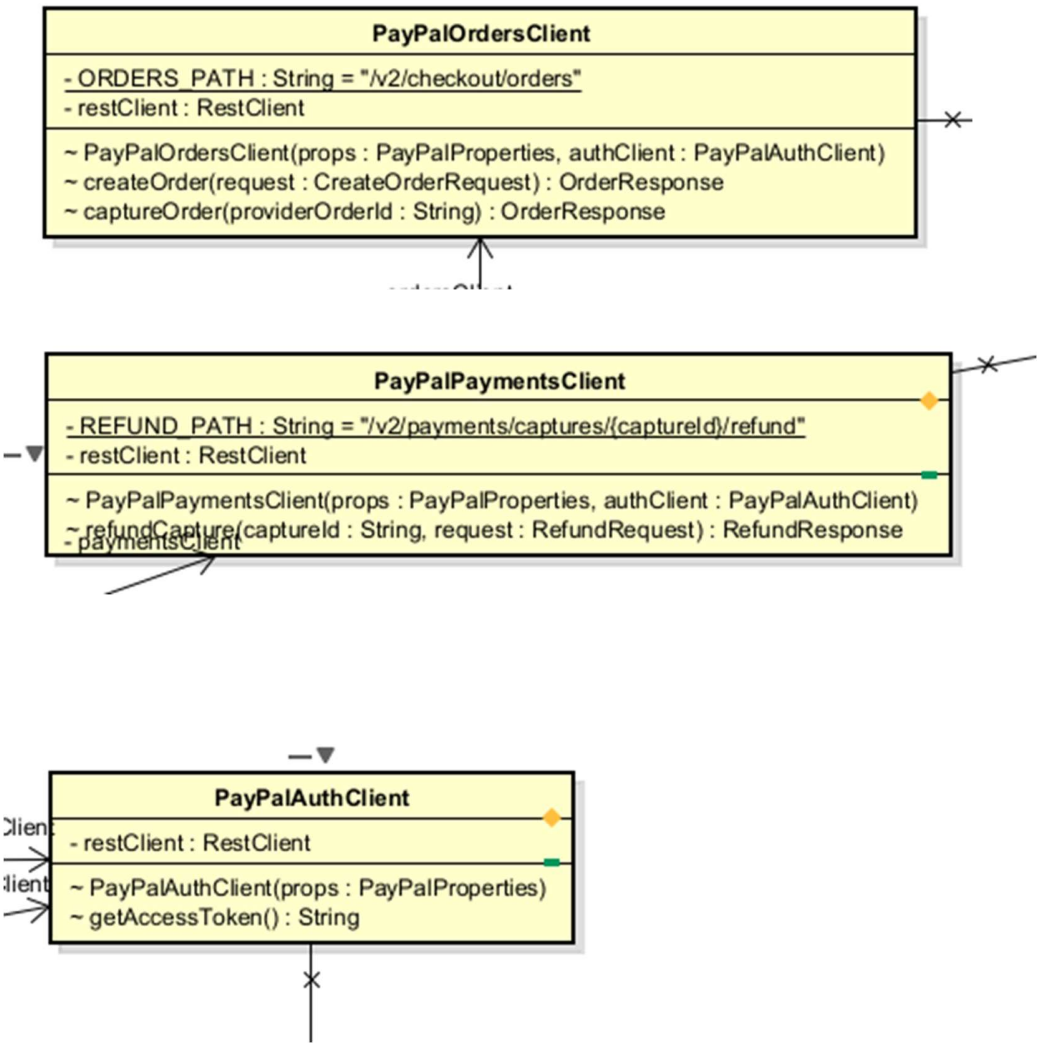
Method:

- Implements the Facade pattern (Section 5): hides PayPalOrdersClient, PayPalPaymentsClient, PayPalAmountConverter and the PayPal JSON/link-parsing details behind two narrow, provider-agnostic interfaces.
- Only PaymentInitiation / PaymentCapture / RefundResult records cross the boundary outward — never a raw PayPal type (DATA coupling outward).

Other diagrams:

- Subsystem sequence diagram — Facade → PayPalOrdersClient / PayPalPaymentsClient → external PayPal REST API, replacing the previous ProcessPayPalPayment.png.

4.4.3.44 Class "PayPalOrdersClient", "PayPalPaymentsClient", "PayPalAuthClient", "PayPalAmountConverter"



These four classes are small, single-purpose, package-private helpers inside subsystems.paypal. They are grouped in one section because none of them has more than one or two operations.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	PayPalOrdersClient.authClient	PayPalAuthClient	(injected)	Used to obtain a bearer token before every Orders API call.

2	PayPalOrdersClient.restClient	RestClient	(built from baseUrl)	Spring RestClient bound to the PayPal base URL.
3	PayPalPaymentsClient.authClient	PayPalAuthClient	(injected)	Used to obtain a bearer token before the refund call.
4	PayPalAuthClient.props	PayPalProperties	(injected)	Supplies clientId / clientSecret for the OAuth2 client-credentials grant.
5	PayPalAmountConverter.props	PayPalProperties	(injected)	Supplies the configured currency and the VND-to-currency rate.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	PayPalOrdersClient.createOrder(request)	PayPalApiModel.OrderResponse	POST /v2/checkout/orders — creates a PayPal order.
2	PayPalOrdersClient.captureOrder(providerOrderId)	PayPalApiModel.OrderResponse	POST /v2/checkout/orders/{id}/capture — captures an approved order.
3	PayPalPaymentsClient.refundCapture(captureId, request)	PayPalApiModel.RefundResponse	POST /v2/payments/captures/{id}/refund — refunds a settled capture.
4	PayPalAuthClient.getAccessToken()	String	POST /v1/oauth2/token — obtains an OAuth2 access token (client-credentials grant).
5	PayPalAmountConverter.toProviderValue(vndAmount)	String	Converts a VND amount into the 2-decimal string PayPal expects in the configured currency.

Parameter:

- request : PayPalApiModel.CreateOrderRequest / RefundRequest — built by PayPalGatewayFacade, never by the AIMS core.
- vndAmount : Long — e.g. 250 000 VND with the default rate (25 000) converts to “10.00”.

Exception:

- PayPalApiException (package-private) if PayPal returns a non-2xx HTTP response or an empty access token; caught and re-thrown as PaymentException by PayPalGatewayFacade only.

Method:

- Each client targets exactly one PayPal API group (Orders / Payments / OAuth) instead of one monolithic client — preserves functional cohesion and SRP (see Cohesion & SRP table, item 1).
- PayPalAmountConverter keeps the VND↔currency conversion a gateway-only concern; the AIMS core continues to think purely in VND.

Other diagrams: Covered by the PayPalGatewayFacade subsystem sequence diagram above; no separate diagram needed.

4.4.3.45 Class “”

5 Design Considerations

5.1 Goals and Guidelines

The design of the AIMS system is guided by a set of architectural goals, software engineering principles, and implementation guidelines intended to improve maintainability, extensibility, reliability, and long-term sustainability of the system. These goals were established not only to satisfy the functional requirements specified in the SRS but also to support future requirement changes introduced by the additional project requirements.

Goal 1: Separation of concerns is non-negotiable

Every class must have one clearly stateable reason to change. This is the Single Responsibility Principle applied not just as a SOLID checkbox but as the primary filter for every design decision made during the project. Concretely:

- Controllers own only the HTTP boundary. They parse requests, delegate to the service layer, and serialise responses. No business logic, not even a conditional, belongs in a controller.
- Services own all business rules and orchestration. They do not construct HTTP responses, format strings for display, or directly build JPA queries beyond what the repository interface exposes.
- Repositories own all database access. Named query methods and @Query annotations are acceptable; raw SQL embedded in service methods is not.
- Frontend screen components orchestrate layout and user interaction. They do not contain data transformation logic or direct API calls, those belong in the TypeScript service layer.

Violations of this principle were the most common source of coupling problems found during the lab assignments. Enforcing it as a hard guideline prevents the service layer from becoming a dumping ground for all logic that does not fit elsewhere.

Goal 2: Interfaces are defined by the consumer, not the implementor

Whenever the service layer depends on an external capability (a payment gateway, a notification channel, a refund mechanism), the interface contract is defined in the application layer and the external implementation adapts to it, not the reverse. No Spring Boot service class imports a class from a payment SDK directly; it imports an interface that the SDK adapter implements.

This is the Dependency Inversion Principle applied as a coding rule. It keeps the core business logic independent of third-party library details and makes the system testable with stub implementations without modifying production code.

Goal 3: Extension by addition, not modification

When a new product type, payment method, delivery zone, or notification channel needs to be supported, the correct response is to add a new class and register it, not to open an existing class and add a branch. Switch statements and long if-else chains over type enumerations are treated as design defects requiring refactoring, not as acceptable implementation shortcuts.

Reason: The Open/Closed Principle. The product type hierarchy and payment method set are explicitly identified in the requirements as open to future extension. The registry-based dispatch pattern applied throughout the service layer is the direct implementation of this guideline.

Goal 4: Abstractions are earned, not speculated

A shared abstraction (superclass, interface, template) is introduced only when at least two concrete cases exist that genuinely share the abstracted behavior. Speculative abstractions, designed for requirements that do not yet exist, are rejected in favor of a deferred refactor note documented explicitly in the SDD.

Reason: YAGNI (You Aren't Gonna Need It). Premature abstraction adds indirection without benefit, makes the codebase harder to read, and often produces the wrong abstraction because there is insufficient evidence to triangulate the correct shape. Two concrete examples of this guideline in practice:

- The PhysicalProduct / DigitalProduct split was considered and explicitly deferred: AIMS currently sells only physical media, so introducing a PhysicalProduct intermediate layer serves no present purpose.
- The Template Method for payment finalization was introduced only after both VietQRPaymentService and PayPalPaymentService were implemented and the shared skeleton became observable.

Goal 5: Cross-cutting concerns are handled outside business logic

Concerns that apply uniformly across multiple operations (such as audit logging, transaction management, security enforcement) are implemented once at the infrastructure level and woven in declaratively, not repeated inside each business method.

- Audit logging uses AOP.
- Transaction boundaries are declared with @Transactional on service methods, not managed manually.
- Security constraints are enforced on the configuration level, not duplicated inside service methods.

Embedding cross-cutting logic inside business methods violates SRP and produces brittle code where a change to the logging format or transaction strategy requires touching every affected method individually.

Goal 6: Post-commit side effects are never in the main transaction

Notification emails, refund triggers, and other side effects that follow an order state transition are always dispatched after the originating transaction has committed successfully. They may run in their own transaction and must not cause the main transaction to roll back if they fail.

A notification failure must not roll back a successfully placed or cancelled order. Mixing side effects into the main transaction couples unrelated concerns and produces incorrect rollback behaviour. This guideline has a concrete implementation consequence: all associations required by event listeners must be eagerly loaded inside the main transaction before the event is published, because the Hibernate session will have closed by the time the listener fires.

Goal 7: Validated input at the boundary, trusted data inside

All input validation occurs at the controller boundary using Spring Jakarta Validation, along with some additional custom constraint annotations. Once a request has passed validation and entered the service layer, the data is treated as structurally correct. The service layer enforces only business rule violations (e.g., price out of allowed range, stock insufficient), not structural ones (e.g., null checks for fields already validated as non-null).

Duplicating structural validation inside the service layer adds noise without safety, since the service is not a public API and always called through the controller. Business rule enforcement belongs in the service because it requires domain knowledge; structural enforcement belongs at the boundary because it requires no domain knowledge.

Goal 8: Naming conventions

The following naming conventions are applied uniformly across the backend codebase:

- Entities: PascalCase nouns (Product, OrderItem, PaymentTransaction).
- Service interfaces and implementations: I{Name}Service for the interface (where an interface exists); {Name}Service for the implementation.
- Gateway interfaces: I{Method}PaymentGateway (e.g., IQRPaymentGateway, IRedirectPaymentGateway).
- Registry classes: {Concern}Registry (e.g., ProductCreatorRegistry, NotificationChannelRegistry).
- Strategy implementations: {Variant}{Concern}Strategy or {Variant}{Concern} (e.g., HanoiDeliveryFeeCalculator, EmailNotificationChannel).

- Event classes: {Subject}{Outcome}Event (e.g., OrderSuccessEvent, OrderCancelledEvent).
- AOP annotations: @{Concern} in lowercase package (@AuditLog).
- REST endpoints: lowercase hyphenated paths following REST resource conventions (/api/products/{id}, /api/orders/{id}/cancel).

On the frontend:

- React components: PascalCase (ProductFormSection, OrderSummaryCard).
- TypeScript services: PascalCase with Service suffix (ProductManagementService, OrderService).
- API call functions: camelCase verbs describing the action (fetchProductById, submitOrder).

Goal 9: No business logic on the frontend

The frontend is responsible for display and interaction only. Price calculations (VAT, delivery fee, total), order eligibility checks, and stock availability are computed by the backend and returned in API responses. The frontend renders these values; it does not recompute them independently.

Duplicating business logic on the frontend creates a synchronization risk. If the backend rule changes, the frontend silently diverges. Treating the backend as the single source of truth for all computed values eliminates this class of bug.

5.2 Architectural Strategies

This section describes the major design decisions that shaped the overall organization of AIMS, the choices that, had they been made differently, would have produced a structurally different system. For each decision, the reasoning and any trade-offs made are explained explicitly.

Strategy 1: Technology stack selection

Decision: Java Spring Boot (backend) + React/TypeScript (frontend) + PostgreSQL (database).

These choices were mandated by the course, but their implications shaped every subsequent design decision. Spring Boot's annotation-driven programming model, @Transactional, @EventListener, @Aspect, made it natural to implement cross-cutting concerns declaratively rather than programmatically. Spring Data JPA reduced the repository layer to interface declarations for the majority of queries. React's component model aligned well with the OOP renderer registry pattern applied on the frontend.

Trade-off: Spring Boot's convention-over-configuration approach introduces a significant framework footprint. For a project of this scale, much of that infrastructure is overhead. However, the course evaluation criteria require demonstrable application of design patterns and SOLID principles, and Spring's ecosystem provides first-class support for AOP, event-driven decoupling, and dependency injection, all of which are central to the design.

Strategy 2: REST API as the sole frontend-backend communication mechanism

Decision: All frontend-backend communication occurs over stateless RESTful HTTP APIs returning JSON. No WebSocket, server-sent events, GraphQL, or RPC mechanism is used.

Each major domain resource (/api/products, /api/orders, /api/users) is addressable by a stable URL. State-changing operations use the appropriate HTTP verb (POST for creation, PUT/PATCH for update, DELETE for removal). The frontend holds no persistent connection to the backend.

Trade-off: A stateless REST model means the frontend must poll for order status updates (e.g., awaiting VietQR payment confirmation) rather than receiving a server push. This is acceptable at the expected scale and simplifies the backend considerably, no WebSocket session management, no event streaming infrastructure.

Strategy 3: Monolithic deployment with internal modular boundaries

Decision: AIMS is deployed as a single Spring Boot application (monolith). There is no microservices decomposition, no message broker, and no inter-process communication.

Internally, the codebase is organised into well-defined packages by domain concern (order, product, payment, notification, audit), with explicit interface contracts between them. This provides the cognitive benefits of modular separation without the operational complexity of a distributed system.

Trade-off: A monolith limits independent scaling and deployment of subsystems. However, at the current scale (course demonstration, up to 1,000 concurrent users on a single server), the operational simplicity of a monolith outweighs any scalability benefit. The internal modular structure means that, if decomposition were needed in future, the boundaries are already defined and the contracts already expressed as interfaces.

Strategy 4: Vertical slice architecture for the payment subsystem

Decision: Rather than a single unified payment abstraction, each payment method (VietQR, PayPal) is a self-contained vertical slice. Two separate interface contracts (IQRPaymentGateway, IRedirectPaymentGateway) represent the two structurally distinct interaction models.

This decision was reached after an earlier attempt to unify both payment methods behind a single PaymentGateway interface was rolled back due to structural incompatibility.

VietQR is poll-based (generate QR → wait for bank callback); PayPal is redirect-based (redirect user → capture on return). Forcing a common interface required one or both implementations to expose methods irrelevant to their flow, an ISP violation.

Trade-off: Two separate interfaces means the service layer cannot treat all payment methods identically through a single reference. This is accepted because the alternative, a leaky unified abstraction, is worse. The PaymentService abstract base class handles the shared post-payment finalization sequence via Template Method, preserving the DRY principle for the parts that genuinely are shared.

Strategy 5: Event-driven decoupling for order lifecycle side effects

Decision: Post-state-transition side effects (notification emails, refund triggers) are published as Spring application events after the originating transaction commits and handled by independent @TransactionalEventListener methods.

This means OrderService has zero direct dependencies on NotificationService or any refund service. It publishes a fact (OrderCancelledEvent, OrderSuccessEvent) and is done. Downstream handlers, today an email notification and a refund stub, subscribe independently.

Trade-off: Event-driven decoupling introduces an implicit contract between publisher and subscriber that is harder to trace statically than a direct method call. A developer reading OrderService cannot immediately see what happens after an event is published without searching for listeners. This is mitigated by consistent naming conventions (Section 5.1, G8) and documentation in this SDD. The coupling reduction benefit, OrderService does not need to know how many things happen after an order is cancelled, or what they are, outweighs the traceability cost at this scale.

Strategy 6: AOP for audit logging as a cross-cutting concern

Decision: Product CUD audit logging is implemented via @Around AOP advice on a custom @AuditLog annotation, entirely outside the business methods being logged.

The alternative, calling auditLogService.log(...) at the end of every createProduct, updateProduct, and deleteProduct method, would have required every service method to carry a logging dependency and a logging call, mixing two unrelated responsibilities in one method body.

Trade-off: AOP advice is non-obvious to developers unfamiliar with the pattern. The logging behavior is invisible when reading the service method in isolation. This is accepted because the alternative (manual logging calls) scales poorly, every new auditable operation requires a developer to remember to add the call. The AOP approach is opt-in via annotation and failure-safe: forgetting to add @AuditLog produces a missing log entry (a visible omission) rather than a runtime error.

Strategy 7: JWT-based stateless authentication

Decision: Authentication is implemented using JSON Web Tokens (JWT). The backend issues a signed JWT on successful login; the frontend attaches it as a Bearer token on every subsequent request. The backend validates the token on each request without consulting the database for session state.

No server-side session store, no cookie-based session management. Role-based access control (PRODUCT_MANAGER, ADMINISTRATOR) is enforced at the controller level via Spring Security's @PreAuthorize annotations, reading claims directly from the validated token.

Trade-off: Stateless JWTs cannot be invalidated before expiry without a token blacklist. For this project, token expiry is set to a short window and no blacklist is implemented. This is an accepted limitation within the course scope, a production system would require either short-lived tokens with refresh token rotation, or a server-side session store for immediate revocation.

Strategy 8: Joined table strategy for the Product inheritance hierarchy

Decision: The Product IS-A hierarchy (Book, CD, DVD, Newspaper) is mapped to the database using JPA's InheritanceType.JOINED strategy. Each subtype has its own table containing only subtype-specific columns, joined to the shared product table by primary key.

The alternatives considered were:

- Single table: All columns in one table with null columns for inapplicable subtypes. Rejected because the subtypes have significantly different attribute sets, producing a wide table with many nullable columns and no per-subtype constraints.
- Table per class: Each subtype carries all shared columns. Rejected because it duplicates shared product data across four tables and makes catalog-wide queries (e.g., search across all product types) require a UNION.

Trade-off: Joined table requires a JOIN for every subtype query. At the expected data volumes (product catalog in the hundreds to low thousands), this cost is negligible. The normalisation and constraint benefits outweigh it.

Strategy 9: Denormalized snapshots in order line items and audit logs

Decision: order_item records store product_name, unit_price, and unit_weight as snapshot values at checkout time. product_log and admin_log records store manager_username and affected_username as denormalised strings rather than enforcing foreign key relationships to the users table.

This means order line items and audit records are immutable historical facts, they do not change if the referenced product is later updated or the referenced user is later deactivated.

Trade-off: Denormalization sacrifices strict referential integrity in these specific tables. This is a deliberate choice: an audit log that changes when the audited user changes is not a reliable audit log. The snapshot model is semantically correct for both use cases. Foreign keys to product and user are retained as non-enforced references (stored as UUID columns) for traceability, but the displayed values are self-contained.

Strategy 10: Supabase Storage for product images, with URL-only backend coupling

Decision: Product images are stored in Supabase Storage. The backend stores only the public image URL as a varchar field in the product table. The backend never reads or writes binary image data, it handles only the URL string. The browser fetches images directly from Supabase's CDN.

Trade-off: This creates an external dependency on Supabase availability for image rendering. If the Supabase bucket becomes unavailable, product images break even though the rest of the application is functional. This is accepted for the course scope. A more robust production design would store images behind a backend-controlled endpoint that could fall back to a default image, abstracting the storage provider from the frontend entirely.

Strategy 11: Registry-based polymorphic dispatch over switch statements

Decision: Wherever the system needs to select behaviour based on a type discriminator (product type, payment method, delivery zone, notification channel), the selection mechanism is a registry (EnumMap or Map<Class<?>, Handler>) populated at startup, not a switch statement or chain of if-else branches evaluated at runtime.

Each variant registers itself (or is registered by a configuration class) under its key. The caller looks up the handler by key and invokes it through a shared interface. The caller has no knowledge of which concrete handler it receives.

Trade-off: Registry-based dispatch requires that all variants be registered at startup. A missing registration produces a runtime lookup failure rather than a compile-time error. This is mitigated by fail-fast validation at application startup: if a required handler is absent from the registry, the application fails to start rather than failing silently on the first request.

5.3 Design and Program Evaluation

5.3.1.1 Cohesion & SRP

Explain in detail for each class in the table.

Table 8. Cohesion & SRP of AIMS

#	*Class Name*	*PIC*	*Cohesion*	*SRP*	*Solution*
1	Order	Huy Diễn	Communicational — all fields and methods operate on the same Order aggregate and its directly related state (items, delivery, invoice, payment, status transitions)	Yes — single responsibility: represent and protect the lifecycle of one order aggregate	—
2	OrderItem	Huy Diễn	Functional — all attributes and methods contribute solely to representing one purchased line item and computing its price/weight	Yes — single responsibility: snapshot and represent one order line item	—
3	DeliveryInformation	Huy Diễn	Communicational — all fields and the single method operate on the same delivery-related data for one Order	Yes — single responsibility: hold shipping and recipient data for an order	—
4	Invoice	Huy Diễn	Functional — represents billing data and the logic to capture it as a snapshot from an Order	Yes — single responsibility: produce and hold a point-in-time billing record	—
5	PaymentTransaction	Huy Diễn	Communicational — all fields operate on payment record data for a single transaction	Yes — single responsibility: store the outcome of one payment event	—
6	StockValidator	Huy Diễn	Functional — all logic contributes solely to validating stock availability	Yes — single responsibility: validate that a cart's contents can be	—

			and cart consistency	fulfilled by current stock	
7	DeliveryFeeCalculator	Huy Diễn	Functional — all logic contributes to resolving and delegating to the correct delivery fee strategy	Yes — single responsibility: dispatch to the correct fee strategy by method key	—
8	StandardDeliveryFeeStrategy	Huy Diễn	Functional — all constants and methods contribute solely to computing the standard delivery fee	Yes — single responsibility: implement the standard delivery fee calculation rule	—
9	PlaceOrderService	Huy Diễn	Procedural — coordinates sequential steps in the order placement workflow (stock check → draft → delivery info → invoice retrieval → cancellation)	Partial — currently owns multiple workflow steps. Changes to any one step may require modifying this class.	If the application grows significantly, consider separating into OrderDraftService, OrderDeliveryService, and OrderQueryService. For the current project scope, the cohesion within the workflow justifies the current grouping.
10	OrderFinalizer	Huy Diễn	Functional — all logic contributes to the single act of finalizing and persisting a completed order and publishing its event	Yes — single responsibility: transition the order to PENDING, persist it, and publish the success event	—
11	PlaceOrderSuccessListener	Huy Diễn	Communicational — all operations react to the same OrderSuccessEvent and handle related post-order cleanup and notification	Yes — single responsibility: handle post-commit side effects of a successful order placement	—

12	OrderSuccessEvent	Huy Diễn	Functional — a pure data carrier for the order success fact	Yes — single responsibility: carry the placed order to post-commit event listeners	—
13	OrderConfirmationEmailMessage	Huy Diễn	Functional — all methods contribute to producing one specific email message for one specific event	Yes — single responsibility: produce the order confirmation email content	—
14	OrderController	Huy Diễn	Procedural — coordinates the HTTP surface of the order placement workflow; methods participate in the same sequential order procedure	Yes — single responsibility: expose the order placement workflow as REST endpoints and delegate to service layer	—
15	VietQRGatewayFacade	Nguyễn Tiến Đạt	Functional — all methods contribute solely to orchestrating the VietQR payment lifecycle (authenticate, generate QR, check status)	Yes — single responsibility: act as the facade that hides all VietQR API complexity behind two clean operations	—
16	VietQRBoundary	Nguyễn Tiến Đạt	Functional — all methods contribute solely to sending HTTP requests to VietQR API endpoints	Partial — each method mixes HTTP transport logic	—
17	QRGenerateRequest	Nguyễn Tiến Đạt	Functional — all attributes and methods contribute solely to constructing the QR generation request payload for VietQR API	Yes — single responsibility: represent and build the QR code generation request	—

18	QRTestCallbackRequest	Nguyễn Tiến Đạt	Functional — all attributes and methods contribute solely to constructing the test callback request payload	Yes — single responsibility: represent and build the VietQR test callback / payment status request	—
19	QRAccessTokenRequest	Nguyễn Tiến Đạt	Functional — all attributes and the single method contribute solely to building the Basic Auth header for token requests	Yes — single responsibility: encode VietQR credentials into an authorization header	—
20	VietQRPaymentController	Nguyễn Tiến Đạt	Functional — all methods contribute solely to routing HTTP requests for VietQR QR payment operations	Yes — single responsibility: receive HTTP requests, with no business logic in the controller	—
21	PayPalOrdersClient PayPalPaymentsClient PayPalAuthClient	Pham Duc Phuoc	Previous design used one PaypalAPIClient for the Orders call, the refund call and the OAuth token call together → communicational cohesion only.	The old single client had three independent reasons to change (three different PayPal endpoint groups).	Split into three single-purpose clients, one per PayPal endpoint group (Orders v2, Payments/refund, OAuth2 token). Each class now has functional cohesion and one reason to change.
22	PaypalPaymentService (formerly CreditCardPaymentService)	Pham Duc Phuoc	Old design held PaymentGateway, INotificationService, PaymentTransactionRepository and IOrderRepository directly inside one class → low cohesion (4 unrelated concerns).	The old class changed for payment, notification, persistence AND order-state reasons — four responsibilities in one place.	Narrowed to orchestrate only the PayPal create / capture / cancel / refund lifecycle. Draft-order access and order finalization are delegated to the shared OrderDraftContext / OrderFinalization abstractions,

					restoring functional cohesion.
23	PayPalGatewayFacade	Pham Duc Phuoc	Functional cohesion: the class exposes exactly the create / capture / refund lifecycle of the PayPal gateway, nothing else.	Single responsibility: translate provider-agnostic calls into PayPal API calls and back.	Already compliant. Kept as a documented example of the Facade pattern applied to this use case (see Section 5).
24	Product		Communicational — all fields and methods (updatePrice, updateStock, delete, activate, apply) operate on the same Product aggregate and its directly related state (price, stock, status lifecycle)	Yes — single responsibility: represent and protect the lifecycle invariants of one Product aggregate	
25	Book / CD / DVD / Newspaper		Communicational — all fields and the Builder belong to the same Book / CD / DVD / Newspaper - specific data aggregate, extending Product	Yes — single responsibility: represent the product-specific domain state and its construction via Builder	
26	PrintableProduct		Communicational — all fields (publisher, publicationDate, language) are tightly related shared attributes for printable products	Yes — single responsibility: factor out common printable product domain state shared by Book & Newspaper	
27	Track		Functional — a simple immutable value object where all fields (title, length) serve the single purpose of	Yes — single responsibility: represent one CD track as a value object	

			representing one CD track		
28	ProductManagementService		Communicational — all methods operate on the same core domain data (Product) through the same collaborators (ProductRepository, ProductFactory, ProductMapper)	No — the class mixes write orchestration (create, update, delete, activate, adjustStock) with read orchestration (getAllProductsForManager, getProductByIdForManager), violating SRP	Segregate into ProductCommandService (CUD + activate + adjustStock) and ProductQueryService (read operations) following CQRS principles
29	ProductManagementController		Communicational — groups all REST endpoint routing for the same domain (Product Management)	Yes — single responsibility: HTTP protocol translation, request validation routing, and delegation to the service layer	
30	ProductFactory		Functional — every method and field exists solely to dispatch product creation/update to the correct type-specific creator/updater	Yes — single responsibility: act as a type-discriminating dispatcher (Abstract Factory) for product creation and updating	
31	ProductCreator (interface)		Functional — the interface and its default method serve only one purpose: define the contract for creating a Product from a request DTO	Yes — single responsibility: define the product creation contract and common field mapping	
32	ProductUpdater (interface)		Functional — the interface and its default method serve only one purpose: define the contract for updating a Product	Yes — single responsibility: define the product update contract and common field mapping	

			from a request DTO		
33	Book/CD/DVD/NewspaperCreator		Functional — sole purpose is to map Create request into a Book / CD / DVD / Newspaper entity via the Builder	Yes — single responsibility: create a Book entity from its creation DTO	
34	Book/CD/DVD/NewspaperUpdater		Functional — sole purpose is to apply Update request to an existing ntity via the Builder	Yes — single responsibility: update a Book entity from its update DTO	
35	CreateProductRequest		Communicational — all fields carry the data required for one product creation request	Yes — single responsibility: serve as the polymorphic DTO carrying validated creation input data	
36	UpdateProductRequest		Communicational — all fields carry the data required for one product update request	Yes — single responsibility: serve as the polymorphic DTO carrying validated update input data	
37	ProductMapper		Functional — every method and field exists solely to dispatch entity-to-DTO mapping to the correct ProductMapping implementation	Yes — single responsibility: type-discriminating dispatcher for product entity-to-response mapping	
38	ProductMapping (interface)		Functional — defines the contract and shared default logic for mapping a Product entity to its detail/summary response DTOs	Yes — single responsibility: define the mapping contract from Product entity to response DTOs	
39	ProductRepository		Communicational — all query	Yes — single responsibility:	

			methods operate on the same Product aggregate root	provide data access operations for the Product aggregate	
40	ProductDetail		Communicational — groups all common product fields needed in a detailed response	Yes — single responsibility: transport detailed product information to the API consumer	
41	ProductSummary		Communicational — groups the minimal product fields needed in a summary/list response	Yes — single responsibility: transport summary product information to the API consumer	
42	PriceRangeValidator		Functional — its only method validates the price-to-original-value ratio for a CreateProductRequest	Yes — single responsibility: validate the price range constraint at the DTO level	

5.3.2 Coupling & Other SOLID

Explain in detail for each problem in the table. Each problem, you may provide a class diagram before and after the improvement.

Table 9. Coupling & other SOLID of AIMS

#	*Problem*	*PIC*	*Location*	*Solution*
1	Stamp coupling — PlaceOrderService and OrderFinalizer receive full Order objects and traverse their internal structure	Huy Diễn	PlaceOrderService, OrderFinalizer, StockValidator	Accepted as appropriate: these classes are part of the same domain layer and the Order aggregate is their shared subject. Passing primitive data instead would require reconstructing the aggregate, which is worse. Stamp coupling at domain boundaries is the expected trade-off in aggregate-oriented design.

2	Stamp coupling — OrderSuccessEvent carries the full Order object	Huy Diễn	OrderSuccessEvent, PlaceOrderSuccessListener, OrderConfirmationEmailMessage	Accepted with a documented constraint: since the event fires post-commit (Hibernate session closed), all required associations (deliveryInformation, invoice, paymentTransaction, items) must be eagerly loaded before the event is published. This is enforced by @NamedEntityGraph("Order-aggregate") on the repository fetch used in the listener path.
3	DIP violation — OrderController depends on concrete classes PlaceOrderService and OrderService rather than abstractions	Huy Diễn	OrderController	Improvement direction: introduce use-case interfaces (PlaceOrderUseCase, OrderQueryUseCase) that the concrete services implement, and inject those interfaces into the controller. For the current project scope with a single implementation per use case, this is noted as a known minor violation.
4	OCP — PlaceOrderService completion logic is hardcoded — adding a new required field requires modifying Order.isComplete() and potentially PlaceOrderService	Huy Diễn	Order.isComplete(), PlaceOrderService	Improvement direction: introduce an OrderCompletionRule interface and a registry of rules checked by isComplete(). Each new completion requirement is a new rule implementation, not a modification to Order. Deferred for current scope.
5	Data coupling (positive) — DeliveryFeeCalculator and StandardDeliveryFeeStrategy communicate through the DeliveryFeeCalculationStrategy interface using only the Order and a return value	Huy Diễn	DeliveryFeeCalculator → DeliveryFeeCalculationStrategy → StandardDeliveryFeeStrategy	This is the target coupling level. Adding a new strategy requires only a new class implementing DeliveryFeeCalculationStrategy and a new enum value — no changes to DeliveryFeeCalculator or any caller.
6	ISP — DeliveryFeeCalculationStrategy is minimal and cohesive	Huy Diễn	DeliveryFeeCalculationStrategy	The interface exposes exactly two methods: method() (for self-registration) and calculateDeliveryFee(order)

				(the calculation). No implementor is forced to implement methods it does not use. ISP is satisfied.
7	Event-driven OCP — PlaceOrderSuccessListener isolates post-commit side effects from OrderFinalizer	Huy Diễn	OrderFinalizer → ApplicationEventPublisher → PlaceOrderSuccessListener	Adding a new post-order side effect requires only a new @TransactionalEventListener method — OrderFinalizer and PlaceOrderService are not modified. OCP is satisfied at the event boundary.
8	LSP — Order status transition encapsulation	Huy Diễn	Order.Status, Order.changeStatus()	All status transitions go through changeStatus(), which validates against the transition map before applying. No external class can bypass this — changeStatus() is private. LSP holds: any code that expects an Order in PENDING state will always find it transitioned correctly by the defined operations.
9	Earlier subsystem design (PayPalSubsystem.png / ProcessPayPalPayment.png) had AIMS build its own request DTO carrying the raw card number, CVV and expiration date, so the core was content-coupled to sensitive cardholder data and the application itself sat inside PCI-DSS scope.	Pham Duc Phuoc	Old PayPal subsystem class diagram and subsystem sequence diagram (superseded).	Replaced with PayPal's redirect / Orders v2 flow: AIMS never receives card data. Only an opaque providerOrderId/token and the provider-agnostic PaymentInitiation / PaymentCapture records cross the subsystem boundary (data coupling only).
10	A leftover code comment on IRedirectPaymentGateway claims the interface “lives inside the low-level subsystems.paypal package” (a DIP ownership-inversion problem), but the interface already sits in the core-owned services.payment.contract package.	Pham Duc Phuoc	services/payment/contract/ IRedirectPaymentGateway.java	Verified the structure is already correct: the core owns the abstraction, subsystems.paypal only provides the package-private implementation (PayPalGatewayFacade). The stale comment should be corrected so the documentation matches the code.

11	A comment in PaypalPaymentService describes OrderDraftContext as injected as a “concrete class” and the file keeps an	Pham Duc Phuoc	services/payment/paypal/ PaypalPaymentService.java	OrderDraftContext and OrderFinalization are already interfaces, so DIP is respected for both collaborators. Removed the dead import
12	DIP violation: ProductManagementService depends on concrete ProductFactory		ProductManagementService — private final ProductFactory productFactory;	Extract an IProductFactory interface. Have ProductFactory implement it. Inject the interface into ProductManagementService
13	DIP violation: ProductManagementService depends on concrete ProductMapper		ProductManagementService — private final ProductMapper productMapper;	Extract an IProductMapper interface with toProductDetail(Product) and toProductSummary(Product) methods. Have ProductMapper implement it. Inject the interface

5.4 Design Patterns

AIMS applies several design patterns across its backend subsystems. Each pattern was selected to solve a concrete, recurring structural problem, not speculatively. This section describes the problem each pattern addresses, the solution structure, and how it is concretely implemented in the codebase.

5.4.1 Strategy Pattern with Registry

This pattern is by far the most applied design pattern, used to solve issues with many different functionalities that were considered a non-extensible, non-maintainable design flaw that violated several SOLID principles, lowering cohesion while tightening coupling across modules. Each of the following subsections describes the problem that this pattern addresses.

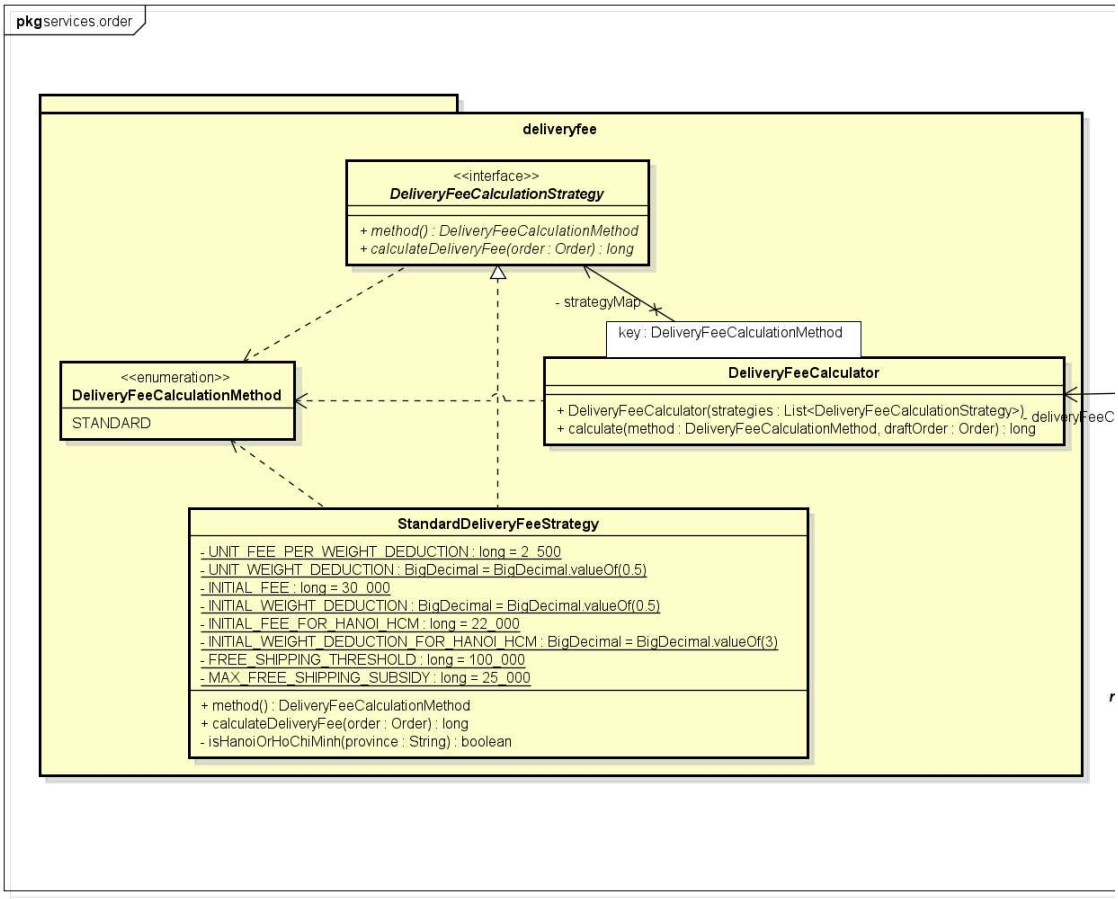
5.4.1.1 Delivery Fee Calculation

The delivery fee calculation logic differs based on the delivery method (e.g., standard, express). Without a pattern, the calling service would need a switch statement or if-else chain over the method type to select the correct logic:

```
if (method == STANDARD) { ... }
else if (method == EXPRESS) { ... }
```

Adding a new delivery method requires opening and modifying this existing logic, an OCP violation. The caller is also control-coupled to the fee calculation logic.

The solution is to Define a DeliveryFeeCalculationStrategy interface with two methods: method() for self-identification and calculateDeliveryFee(Order order) for the computation. Each concrete strategy implements this interface and self-registers into a DeliveryFeeCalculator registry backed by an EnumMap<DeliveryFeeCalculationMethod, DeliveryFeeCalculationStrategy>. The registry is populated at Spring startup by injecting all strategy beans as a list and calling strategy.method() on each to obtain the key.



SOLID benefits:

- OCP: Adding a new delivery method requires only a new @Component class, DeliveryFeeCalculator and PlaceOrderService are untouched.
- SRP: Each strategy owns exactly one fee calculation rule.
- DIP: PlaceOrderService depends on DeliveryFeeCalculator (an abstraction over all strategies), not on any concrete strategy class.
- ISP: The interface has exactly two methods; no implementor is forced to expose anything it does not use.

Trade-off & Assumption

The current StandardDeliveryFeeStrategy calculates the chargeable weight using the actual weight of the order items alone. The Additional Requirements document identifies a future strategy where the chargeable weight would instead be MAX(actual weight, volumetric weight), where volumetric weight is derived from package dimensions (length × width × height / 6000). This new strategy shares the majority of its fee calculation logic with StandardDeliveryFeeStrategy, the province-based base fee selection, the per-increment excess weight charge, and the free-shipping subsidy, differing only in how the chargeable weight is computed.

This raises a design question: should the two strategies be entirely separate implementations, or should the shared calculation skeleton be extracted and the weight computation step treated as a variable sub-step via Template Method?

The current design keeps them as entirely separate Strategy implementations. This is a deliberate application of YAGNI: with only one strategy currently implemented, there is insufficient evidence to correctly triangulate the shared skeleton. Introducing a Template Method abstract base class (AbstractDeliveryFeeStrategy) with a computeChargeableWeight() hook prematurely assumes that all future strategies will share the same base fee, province logic, and subsidy rules, which may not hold for an express or same-day delivery strategy added later.

The recommended refactoring path, once a second concrete strategy exists, is:

- Extract an AbstractStandardDeliveryFeeStrategy base class implementing the shared province-based fee, per-increment excess weight charge, and subsidy logic.
- Declare computeChargeableWeight(Order order): BigDecimal as an abstract hook method.
- Let StandardDeliveryFeeStrategy override it by returning order.getTotalWeight(), and VolumetricDeliveryFeeStrategy override it by returning MAX(actualWeight, volumetricWeight).

This refactoring is deferred until the second strategy is implemented, at which point the correct shape of the abstraction will be verifiable against two real cases rather than speculated from one.

5.4.1.2 Product Type Dispatch (DTO Mapping, Creation, & Update)

The system handles four product subtypes: Book, CD, DVD, and Newspaper. Without a pattern, creating or updating a product requires type-switching:

```
switch (productType) {
```

```
    case BOOK: return createBook(dto); break;
    case CD:   return createCD(dto);   break;
    ...
}
```

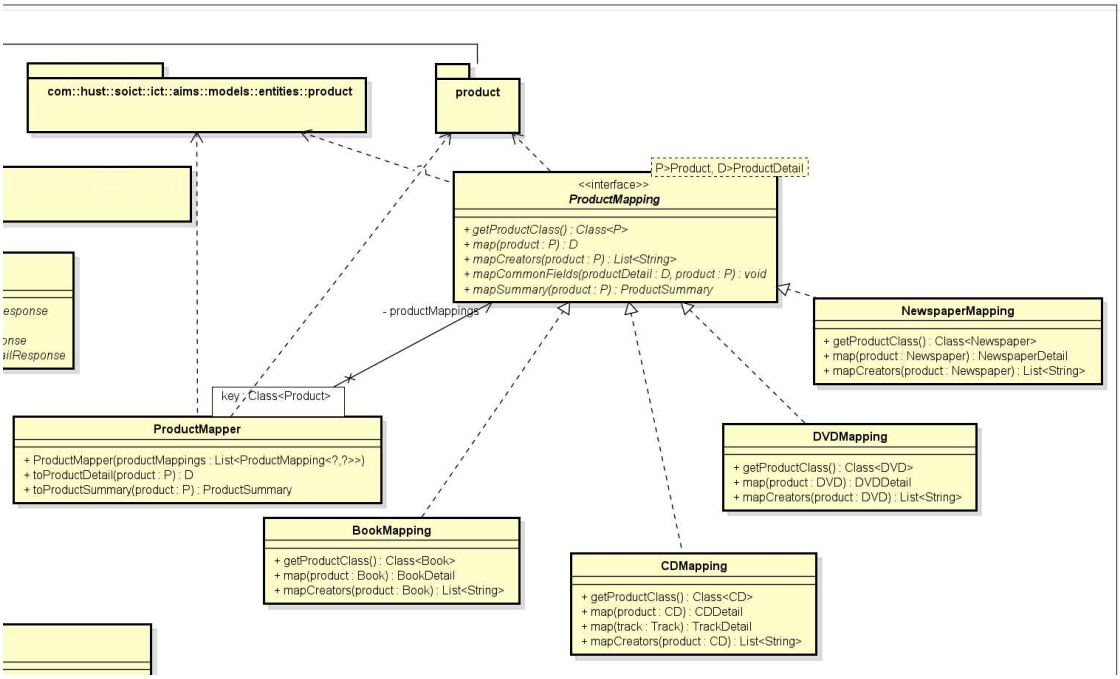
This switch appears independently for creation, update, and mapping, three separate places that must all be modified whenever a new product type is introduced.

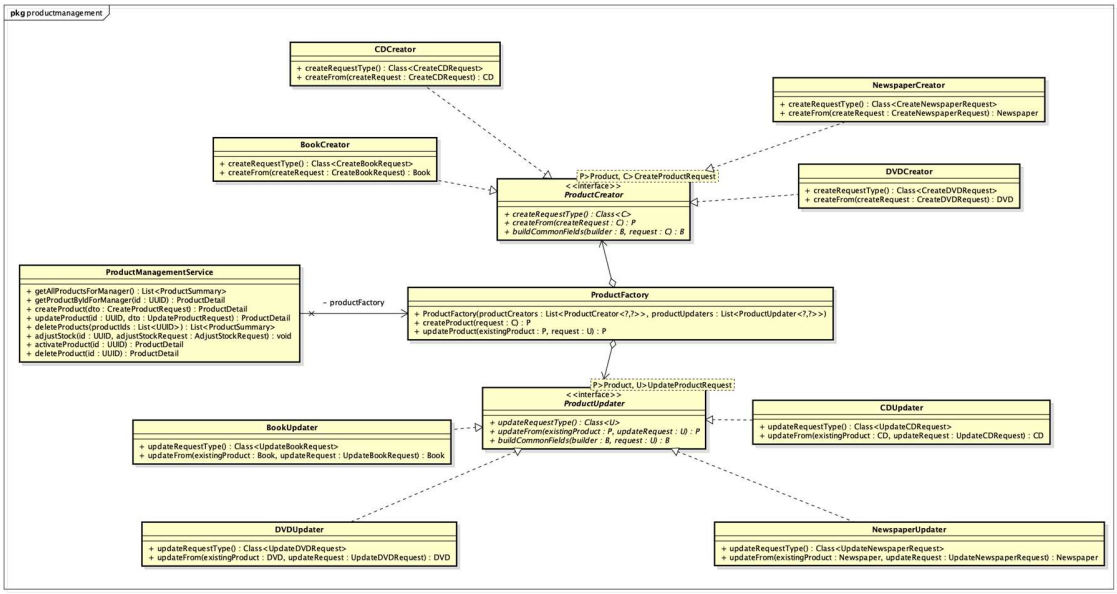
The solution is to define separate registry interfaces for each operation axis:

ProductCreator<P extends Product, C extends CreateProductRequest>: maps a creation DTO to a new Product entity

ProductUpdater<P extends Product, U extends UpdateProductRequest>: applies an update DTO to an existing Product entity

ProductMapping<P extends Product, D extends ProductDetail>: maps a Product entity to its response DTO





Each subtype provides one implementation of each interface and registers itself under its Class<?> type key. A ProductFactory registry and ProductMapper Registry resolve the correct implementation at runtime by type key, replacing all switch statements.

Note on MapStruct[1]: MapStruct is used for all non-polymorphic DTO mapping. For the Product hierarchy, MapStruct's code generation cannot resolve the correct subtype mapper at compile time because type erasure prevents a class from implementing the same raw generic interface twice with different type arguments. The manual ProductMapping registry replaces MapStruct specifically for this polymorphic case, while MapStruct continues to handle all other mapping (Order, DeliveryInformation, Invoice, etc.).

SOLID benefits

- OCP: Adding a new product type (e.g., Vinyl) requires three new classes (VinylCreator, VinylUpdater, VinylMapping) and zero changes to existing registries or service logic.
- SRP: Each creator, updater, and mapper owns exactly one product type's logic.
- DIP: The service layer depends on registry interfaces, not on concrete per-type classes.

Trade-offs and Assumptions

[1] MapStruct is a code generator that greatly simplifies the implementation of mappings between Java bean types based on a convention over configuration approach. https://mapstruct.org/

The current registry design assumes a flat product type hierarchy: every product type is a direct subtype of Product. This assumption holds for all four current types (Book, CD, DVD, Newspaper) and for the E-Book Reader future requirement described in the Additional Requirements document.

However, a future product type could extend an existing concrete product type rather than Product directly. For example, an AudioBook could be considered a specialized Book inheriting all Book attributes and adding audio-specific ones (narrator, duration, audio format). This breaks the current registry design in two ways:

- Java type erasure: A class cannot implement the same raw generic interface twice with different type arguments. AudioBookCreator cannot simultaneously implement both ProductCreator<AudioBookDTO> and inherit BookCreator's implementation of ProductCreator<BookDTO> without introducing an intermediate abstract class.
- Registry key collision: The registry works correctly for creation and update. But if the intent is for AudioBook to reuse BookCreator with minor extensions, the flat registry offers no inheritance-aware fallback, it would simply not find a registered creator for AudioBook and throw an exception.

The recommended resolution, should concrete-extends-concrete inheritance materialize, is to extract an intermediate abstract base class for the shared subtype. However, this refactoring is deferred under YAGNI: with no concrete-extends-concrete case currently in the codebase, introducing the intermediate layer prematurely would be speculative abstraction.

5.4.2 Builder Pattern (for Product Creation / Update)

The Product entity and its subtypes have a large number of fields, many optional, many subtype-specific. Without a Builder, construction requires either a large constructor (fragile, order-sensitive) or a sequence of setter calls on a mutable object (allows partially-constructed instances to escape before all required fields are set). Partial updates (e.g., updating only the title and price of a Book) are especially problematic: a naive approach would require fetching the entity, calling individual setters, and saving, exposing the entity to inconsistent intermediate states.

The solution is to have each Product subtype implement a Builder that provides:

- A static Builder class nested inside the class for constructing a new instance from scratch.
- A toBuilder() method that produces a pre-populated builder from an existing instance

- An apply() method that commits the builder's accumulated changes back to the entity

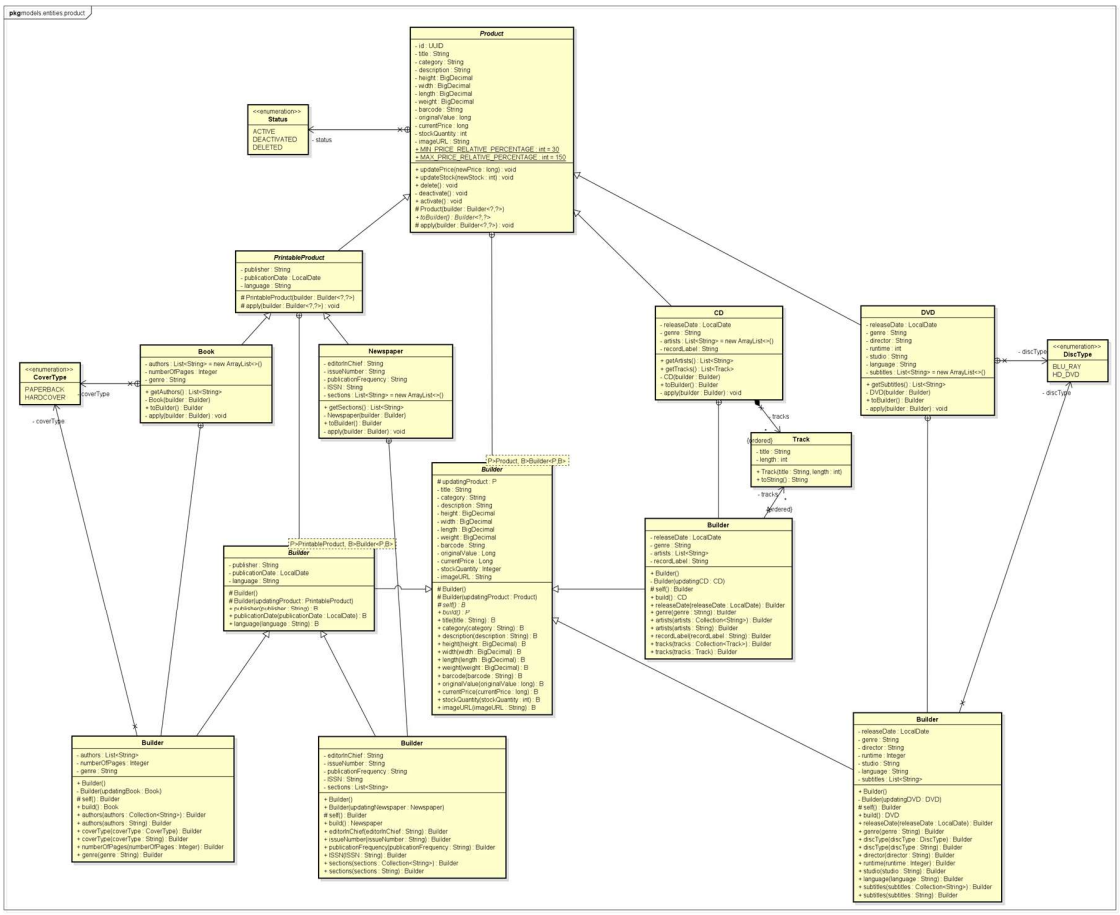
This allows partial updates to be expressed cleanly as product.toBuilder().title("New Title").currentPrice(newPrice).build() without touching fields not explicitly set.

Important implementation constraint

- Generic Builder: The builder hierarchy uses a generic self-referential (recursive) type parameter to preserve the concrete builder type throughout the method chain, eliminating the LSP tension that arises in naïve builder inheritance. The type parameter is used on Builder of an abstract Product class, which effectively also means that Builder of it is also abstract. An additional type parameter to refer to the product type is introduced to enforce stricter typing where Builder is used (e.g. in ProductCreator and ProductUpdater)

Product.Builder<P extends Product, B extends Builder<P, B>>

- Creation / Update encapsulation: All Product entity constructors are private or protected (the protected form exists solely for JPA's reflection-based instantiation and carries no fields, it cannot produce a usable instance). No public setters exist anywhere in the hierarchy; getters are implemented without a corresponding setters. This means the Builder is not merely a convenience, it is the only permitted path for constructing or mutating a product entity from outside the aggregate.



SOLID benefits

- SRP: Construction and mutation logic is separated from business logic in ProductService. The Builder owns how a product is assembled; the service owns when it is assembled.
- OCP: Adding a new field to BookBuilder does not affect ProductBuilder or any other subtype's builder.
- LSP: BookBuilder extends ProductBuilder and is fully substitutable wherever a ProductBuilder is expected, returning a valid Book from build().

Trade-off: null semantics in partial update DTOs

- The partial update pattern relies on the assumption that `null` in an update DTO always means "leave unchanged", never "explicitly clear this field". For fields that are genuinely nullable in the domain, a product description that a manager might want to deliberately remove, this assumption breaks down silently: the service would interpret a null description as "no change" when the intent was "clear".
- The resolution is to wrap intentionally nullable DTO fields in `Optional<T>`: `Optional.empty()` means "leave unchanged"; `Optional.ofNullable(null)`

means "clear this field". This refinement is noted as a future improvement. For the current product schema, no field is intentionally clearable after initial creation, so the null-means-unchanged assumption holds without exception for now.

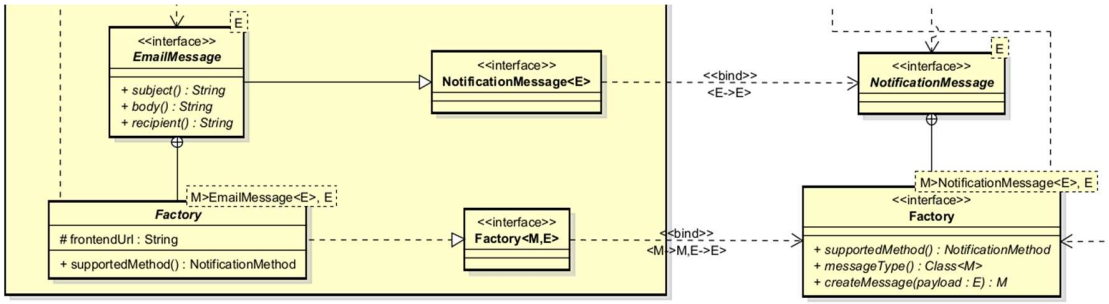
5.4.3 Abstract Factory Pattern (Notification Message Creation)

The notification subsystem must send different types of messages (OrderConfirmationEmailMessage, OrderCancellationEmailMessage, etc.) in response to different application events (OrderSuccessEvent, OrderCancelledEvent, etc.). The NotificationService must be able to produce the correct message type from the correct event payload, without being coupled to every concrete message class.

A naive solution would require a long if-else chain inside NotificationService:

```
if (messageClass == OrderConfirmationEmailMessage.class) {
    return new OrderConfirmationEmailMessage(event.order(),
frontendUrl);
}
else if (messageClass ==
OrderCancellationEmailMessage.class) { ... }
```

The solution is to define a nested abstract Factory<M extends NotificationMessage<E>, E> class inside the NotificationMessage interface, where M is the message type and E is the event payload type that the notification message can be created from. Each concrete message class contains a corresponding concrete Factory as a static nested @Component. The NotificationService holds a registry of all factories keyed by Class<M> (the message type). When send(messageClass, event) is called, the registry resolves the correct factory and calls createMessage(event). Note that all NotificationMessage classes have private constructor, totally encapsulating its internal details; sender can only pass the class type and the event payload without being able to see how it is created.



SOLID benefits

- OCP: Adding a new notification type requires only a new message class with its own nested Factory @Component, NotificationService is not modified.

- SRP: Each Factory is responsible for creating exactly one message type from exactly one event type.
- DIP: NotificationService depends on the abstract Factory base class; it never references any concrete message class directly.

Trade-off and Assumptions

The conventional Factory pattern places the factory in a separate, standalone class (e.g., OrderConfirmationEmailMessageFactory). The design here deliberately departs from this convention by nesting the Factory as a static inner @Component inside the message class it produces. This co-location is intentional: the factory and the message it creates are so tightly coupled, the factory exists for no purpose other than constructing that one message type, that separating them into two files would add navigational overhead without any cohesion benefit. A developer reading OrderConfirmationEmailMessage sees its construction logic immediately; there is no second file to find.

The trade-off is that the message class file becomes slightly larger and carries a Spring annotation (@Component) on its nested class, which is unusual and may surprise developers unfamiliar with the pattern. This is accepted as a deliberate design choice, documented here to prevent future refactoring that separates the factory without understanding the rationale.

- **Assumption 1: Class<M> as the routing key is stable and unambiguous**

The NotificationService registry is keyed on Class<M>, the concrete message type. When notificationService.send(OrderConfirmationEmailMessage.class, event) is called, the registry resolves the factory by exact class match. This assumes that each message type has exactly one factory and that the class reference is stable across the application lifecycle.

This assumption holds correctly for all current message types. However it introduces a subtle constraint: message class identity must not be obfuscated. If the codebase were processed by a class-renaming obfuscator (common in Android or heavily optimized JVM deployments), Class<M> references would break silently at runtime. For a Spring Boot backend in a course project context this is a non-issue, but it is noted as a reason why string-based keys (e.g., a messageTypeId() method returning a stable string constant) would be more robust in a production system.

- **Assumption 2: One factory produces exactly one message type from exactly one event type**

The generic signature Factory<M extends NotificationMessage<E>, E> binds each factory to exactly one message type M and one payload type E. This means a factory cannot produce different message variants from the same event, for example, producing a short SMS summary vs. a full HTML email both triggered by the same OrderSuccessEvent.

If this requirement arose, the current design would need a secondary dispatch dimension: either a second registry keyed on (messageClass, channelType), or a factory that internally branches on the target channel. For the current scope, one message type per event, one channel per message type, the single-dimension registry is both sufficient and clean. The two-dimension case is deferred under YAGNI.

- **Assumption 3: The notification channel registry is separate from the message factory registry**

The design cleanly separates two concerns that are easy to conflate:

- What to say: the message factory registry, keyed on Class<M>, resolves how to construct the message content from an event payload.
- How to deliver it: the notification channel registry, keyed on Class<M> (the message type), resolves which delivery channel (Email, SMS, Zalo, …) handles that message type.

Both registries use Class<M> as their key but serve entirely different purposes. This separation means a future requirement to send the same OrderConfirmationEmailMessage over both Email and Zalo simultaneously would require changes only to the channel registry (registering multiple channels per message type, or changing the value type from a single channel to a list), with no changes to the factory registry or the message class itself.

The assumption is that these two concerns remain independently extensible. If they were merged, a combined (factory + channel) registry entry, extending one axis would always require touching the entry for the other, violating SRP at the registry level.

- **Trade-off: NotificationMessage as a marker interface with private constructor enforcement**

NotificationMessage is a marker interface, it carries no methods of its own. Its sole purpose is to serve as the upper bound on the M type parameter throughout the factory and channel registry infrastructure. This is a deliberate minimalism: forcing all message types to implement a method-bearing interface would require every message type to expose methods that may be irrelevant to it (e.g., an SMS message has no body() in the HTML sense), which would violate ISP.

The concrete message type interfaces, EmailMessage<E>, extend NotificationMessage and add channel-specific methods (subject(), recipient(), body()). Each concrete message class implements the appropriate channel-specific interface, not NotificationMessage directly. This layering keeps the type hierarchy clean: NotificationMessage is the universal bound; EmailMessage is the delivery-channel-specific contract; the concrete class is the content.

The trade-off is that NotificationMessage itself carries no behavioral contract, making it purely structural. A future developer might question why it exists at all and remove it, breaking the generic bounds throughout the registry infrastructure. This is mitigated by this documentation and by the fact that removing it would produce immediate compilation errors at every registry and factory declaration.

5.4.4 Observer Pattern (Order Lifecycle)

When an order is successfully placed, approved, rejected, or cancelled, multiple independent side effects must occur: the cart must be cleared, the draft order context must be cleaned up, a confirmation, cancellation, or rejection email must be sent, and (for cancellations of paid orders) a refund must be triggered. Implementing all of these directly in OrderFinalizer or OrderService would:

- Violate SRP: the order service would own notification and refund logic
- Violate OCP: adding a new side effect (e.g., loyalty points) would require modifying the order service
- Create fan-out coupling: OrderService would depend on NotificationService, RefundService, CartContext, and OrderDraftContext simultaneously.

The solution is to apply the Observer pattern using Spring's ApplicationEventPublisher and @TransactionalEventListener. The order service plays the role of the Subject: it publishes a fact about what happened (OrderSuccessEvent, OrderCancelledEvent, OrderApprovedEvent, OrderRejectedEvent) without knowing who is listening. Independent listener classes play the role of Observers, each reacts to the event with its own side effect, in its own transaction.

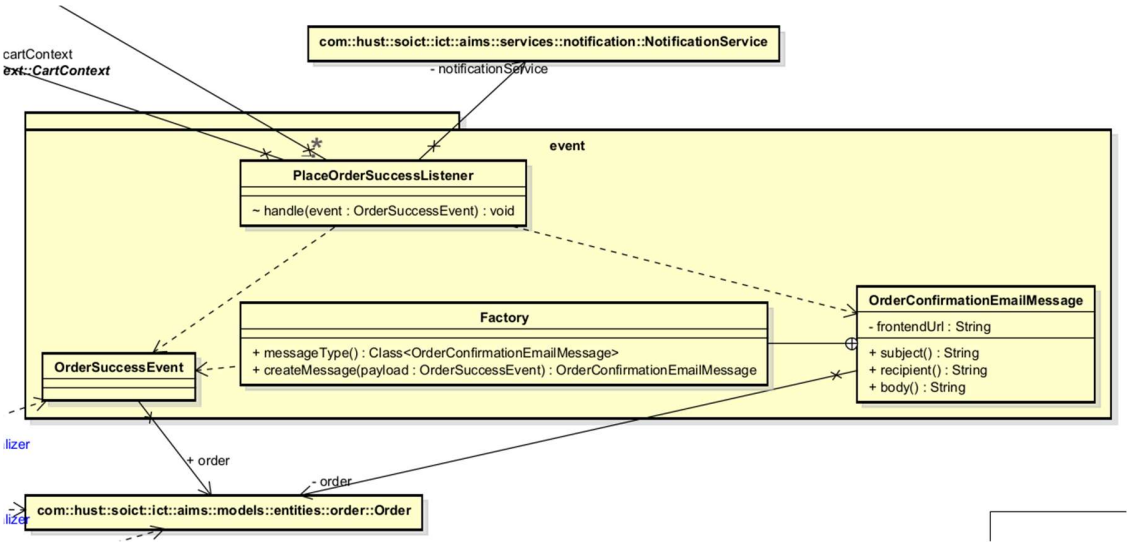


Figure 25. An example of event listener in AIMS (Order Placed Success Event)

Note that the listener always lives alongside the event object, and an implementation of NotificationMessage, and is package-private.

Each listener is annotated with @TransactionalEventListener(phase = AFTER_COMMIT) and runs in a new transaction (Propagation.REQUIRES_NEW). This ensures:

- Side effects only fire if the originating transaction committed successfully, a failed save does not trigger notifications.
- A failure in a listener (e.g., email sending fails) does not roll back the placed order.

Important implementation constraint: Because listeners fire after the Hibernate session closes, all associations needed by listeners (deliveryInformation, invoice, paymentTransaction, items) must be eagerly loaded within the originating transaction before the event is published. This is enforced by using @NamedEntityGraph("Order-aggregate") on the relevant repository query method.

SOLID benefits

- SRP: OrderFinalizer owns only order completion and persistence. Each listener owns exactly one category of side effect.
- OCP: Adding a new post-order side effect requires only a new @TransactionalEventListener, OrderFinalizer and OrderService are not modified.
- DIP: OrderFinalizer depends on ApplicationEventPublisher (a Spring abstraction), not on any concrete listener. Listeners depend on NotificationService and RefundRegistry interfaces (if listener is for events that require refund functionality), not on concrete implementations.

Trade-off and Assumptions

- **Trade-off 1: Implicit coupling replaces explicit coupling**

The most significant trade-off of the event-driven Observer pattern is that it replaces explicit, traceable method call coupling with implicit, convention-based subscription coupling. When reading OrderFinalizer.finalizeOrder(), there is no direct indication of what happens after applicationEventPublisher.publishEvent(new OrderSuccessEvent(order)) is called. A developer must search the codebase for all @TransactionalEventListener methods that accept OrderSuccessEvent to understand the full consequence of finalising an order.

This is the fundamental tension of the Observer pattern: the OCP benefit (adding listeners without modifying the publisher) comes at the cost of traceability. In a large codebase with many listeners spread across many packages, this implicit contract can become a significant maintenance burden.

In AIMS this cost is accepted and mitigated by two measures. First, the number of listeners per event is small and bounded, the system currently has one listener per event, with a second (refund) anticipated. Second, the event classes (OrderSuccessEvent, OrderCancelledEvent) are named as explicit facts about what happened, making them discoverable by name search. The traceability cost is therefore low at the current scale.

- **Trade-off 2: Event-driven decoupling solves fan-out, not optionality**

A common misapplication of the Observer pattern is to use it to make a side effect "optional", reasoning that if the listener is absent, nothing bad happens. This reasoning is incorrect in AIMS and must not be applied. For instance, the side effects handled by PlaceOrderSuccessListener, cart clearing, draft context clearing, and confirmation email dispatch, are mandatory consequences of a successful order placement, not optional enhancements. Their implementation as listeners is a structural choice to reduce coupling, not a signal that they may be omitted.

This distinction matters for testing and deployment: if a listener bean fails to register (due to a misconfiguration, a missing @Component, or a conditional bean exclusion), the order placement flow will silently succeed without performing its mandatory side effects. There will be no compilation error and no startup failure. The mitigation is integration testing that verifies the full event chain, not just that OrderFinalizer saves the order, but that the cart is cleared and the email is dispatched after the transaction commits.

- **Trade-off 3: AFTER_COMMIT listeners cannot participate in the originating transaction**

All listeners in AIMS use TransactionPhase.AFTER_COMMIT, which means they fire after the originating Hibernate session has closed. This has two concrete consequences:

First, lazy-loaded associations are unavailable inside listeners. Any association not eagerly loaded before the event is published will throw a LazyInitializationException when accessed inside the listener. This is mitigated by the @NamedEntityGraph("Order-aggregate") defined on Order, which eagerly loads items, deliveryInformation, invoice, and paymentTransaction within the originating transaction before the event is published. This is a mandatory implementation discipline, forgetting to use the entity graph on any query that precedes an event publication is a silent runtime defect.

Second, listener failures cannot roll back the order. A REQUIRES_NEW transaction in the listener is independent of the originating transaction. If the notification email fails to send, the order remains in PENDING state with no email dispatched. This is the correct behavior, a notification failure must not undo a successfully placed order, but it means the system has no built-in retry or dead-letter mechanism for failed listeners. For the current project

scope this is accepted. A production system would require a persistent outbox pattern or a message broker with retry semantics to guarantee eventual delivery of side effects.

- **Assumption: Spring's synchronous event bus is sufficient**

Spring's ApplicationEventPublisher dispatches events synchronously within the same JVM process by default. There is no message broker, no queue, and no persistence of events. If the application crashes between the transaction commit and the listener execution, an extremely narrow window but a real one, the event is lost and the side effects never fire.

This assumption is appropriate for the current project scope: AIMS runs as a single-process monolith with no high-availability requirement. In a production deployment with multiple application instances or strict delivery guarantees, the event bus would need to be replaced with a persistent outbox (e.g., writing the event to a database table inside the same transaction, then dispatching asynchronously) or an external message broker. The internal structure of OrderFinalizer and the listener classes would not change, only the publishing and subscription infrastructure would be replaced, which is a clean substitution at the framework level.

5.4.5 Template Method pattern (Payment Service abstraction)

VietQR and PayPal represent two structurally incompatible payment flows:

- VietQR: generates a QR code, waits for an asynchronous bank callback, then confirms payment.
- PayPal: redirects the customer to an external page, captures payment on return via the PayPal API.

These two flows are different enough that forcing them into a shared abstract skeleton, where the template method calls one or more abstract hook methods, would produce an artificial abstraction. Any hook abstract enough to accommodate both flows would either carry parameters irrelevant to one of them, or be so coarse-grained that it inlines the entire flow, making the template method a trivial pass-through.

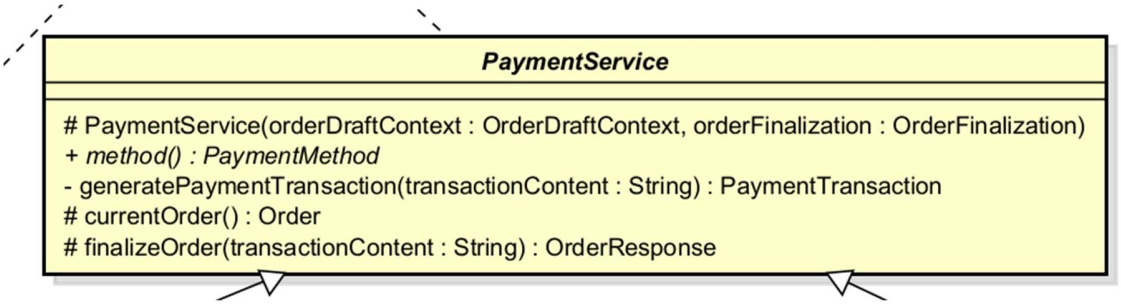
At the same time, several responsibilities genuinely recur across both flows:

- Identifying the current draft order from session context
- Constructing a PaymentTransaction record, which has the same fields (transactionContent, transactionTimestamp, method, amountPaid, order) regardless of payment method
- Invoking order finalization after payment is confirmed

Without any shared abstraction, these responsibilities would be duplicated in VietQRPaymentService and PayPalPaymentService independently, a DRY violation and a

maintenance risk if the PaymentTransaction construction logic or finalization sequence ever changes.

The solution is to apply Template Method as shared utility, not shared skeleton. Rather than defining a coarse abstract skeleton with hook methods, PaymentService applies a minimal, utility-oriented Template Method: it provides final and private methods that subclasses call at the points in their own flow where shared behaviour is needed, without dictating the overall sequence of the flow. The two concrete subclasses implement entirely independent flows and call currentOrder() and finalizePayment() at the appropriate points in their own logic.



This is a deliberate minimal application of the Template Method pattern. The textbook form defines a fixed algorithmic skeleton in the abstract class with abstract hook methods that subclasses fill in. That form is not applied here, because the two payment flows have no meaningful shared skeleton, the steps between "start" and "finalise" are entirely different for VietQR and PayPal.

What PaymentService provides instead is a set of protected and private utility methods that centralise the shared responsibilities without imposing a shared sequence. The subclasses own their own sequence entirely; they borrow the shared utilities at the points they need them. This is a lighter, more honest application of the pattern's intent, avoiding duplication in shared sub-steps, without overclaiming that the flows share a skeleton they do not.

The method() abstract method serves a secondary purpose beyond identification: it is consumed by generatePaymentTransaction() to populate the transactionMethod field of the PaymentTransaction record. This means the subclass's identity declaration directly participates in the shared transaction construction, tying the two together without requiring the subclass to pass its method name as a parameter.

PaymentTransaction construction is made private rather than protected or abstract for a concrete reason: the PaymentTransaction schema is stable and its fields are shared identically across all payment methods. Every payment transaction, regardless of method, records the same information, content, timestamp, method name, amount, and order reference. There is no legitimate reason for a subclass to override or extend this construction logic.

Making it private enforces this: subclasses cannot see it, cannot call it directly, and cannot override it. They interact with it only indirectly through finalizePayment(transactionContent), which is the single sanctioned entry point for completing a payment. This is a deliberate application of the principle of least privilege at the class hierarchy level, subclasses are given exactly the access they need (currentOrder(), finalizePayment()) and nothing more.

If the PaymentTransaction schema were to diverge across payment methods in the future, for example, if PayPal required storing a captureId field not present in VietQR transactions, then generatePaymentTransaction() would need to become protected or abstract. Until that divergence materializes, keeping it private is both safer and more expressive of the current design intent.

SOLID benefits

- SRP: generatePaymentTransaction() owns PaymentTransaction construction in one place. Each concrete service owns only its own payment flow. OrderFinalizer owns order completion. No responsibility bleeds across boundaries.
- OCP: Adding a third payment method (e.g., MoMo) requires only a new subclass of PaymentService that implements method() and calls currentOrder() and finalizePayment() at the appropriate point in its own flow. PaymentService itself is not modified.
- LSP: Both VietQRPaymentService and PayPalPaymentService are valid subtypes of PaymentService. Neither weakens any postcondition of the base class, both correctly call finalizePayment() to complete the order, and both correctly declare their PaymentMethod identity via method().
- DIP: PaymentService depends on OrderFinalization, an interface, not on OrderFinalizer directly. This allows the finalization behaviour to be substituted (e.g., in tests) without touching PaymentService or either concrete subclass.
- Additionally, DRY: PaymentTransaction construction and order finalisation are written once in PaymentService, not duplicated across two concrete services. A change to either, for example, adding a new field to PaymentTransaction, requires modification in one place only.

Trade-off and Assumptions

- **Trade-off 1: Vertical Slice Architecture creates tension with inheritance**

The payment subsystem uses Vertical Slice Architecture: each payment method is a self-contained slice with its own controller and service, communicating with the application layer through IQRPaymentGateway and IRedirectPaymentGateway interfaces. PaymentService inheritance cuts across this vertical organization, it introduces a horizontal shared layer that both slices depend on.

This tension is accepted because the shared responsibilities (PaymentTransaction construction, order finalisation) are genuinely cross-cutting and stable. The alternative, duplicating these in each slice, would be worse. The key boundary is maintained: VietQRPaymentService and PayPalPaymentService each still own their own controller, their own gateway interface implementation, and their own payment flow logic. PaymentService provides only the terminal shared steps, not the flow itself.

- **Trade-off 2: The pattern is minimal and may not be recognized as Template Method**

Because there is no abstract hook method and no dictated skeleton, this implementation departs visibly from the textbook Template Method form. A developer unfamiliar with the design intent might question whether the pattern is applied at all, or might attempt to refactor the shared utilities into a helper service (composition) rather than an abstract base class (inheritance).

The choice of inheritance over composition here is deliberate: PaymentService grants subclasses protected access to currentOrder() and finalizePayment(), which would not be expressible through composition without making these methods public and exposing them to all callers. The protected visibility is an intentional encapsulation boundary, these utilities are for subclasses only, not for controllers or other services.

- **Assumption: PaymentTransaction schema remains uniform across all payment methods**

The centralization of generatePaymentTransaction() as a private method rests on the assumption that all payment methods produce a transaction record with the same field structure. This holds for VietQR and PayPal as currently implemented. If a future payment method required storing method-specific fields, for example, a PayPal captureId for refund dispatch, the private method would need to be redesigned, either as a protected overridable method or as a method that accepts additional optional parameters. This is the primary extensibility risk in the current design and is noted as the first refactoring trigger if a third payment method with a divergent transaction schema is introduced.

5.4.6 Bonus Design Pattern in Frontend: Product Type Dispatch with Registry

The same registry-based dispatch applied in the backend for product mapping, product creation, and product update is mirrored on the frontend in two places, both solving the same problem: rendering product-type-specific content without switch statements or if-else chains over product.productType.

Two frontend concerns require product-type-specific behaviour:

- Product detail rendering: displaying the correct subtype-specific fields (authors for Book, tracks for CD, runtime for DVD, ISSN for Newspaper) on both the customer product detail screen and the Product Manager's product view screen.
- Product form rendering: displaying the correct input fields and building the correct request payload for product creation and update forms.

Without a pattern, both concerns would require a switch over product.productType at every call site, duplicated across the customer screen, the manager screen, the create form, and the edit form, producing the same OCP violation present in the backend before the registry pattern was applied.

Solution 1: Field Renderer Registry

An abstract ProductFieldRenderer<P extends Product> class defines a single abstract method render(product: P): ReactNode. Each product subtype provides a concrete renderer implementing this method with its own JSX structure. A RENDERER_MAP object — typed as { [K in ProductTypeName]: ProductFieldRenderer<Product> } — maps each ProductTypeName string literal key to a singleton renderer instance. The ProductFieldRendererRegistry wraps this map and exposes a single renderFields(product) method that resolves and delegates in one line:

```
export abstract class ProductFieldRenderer<P extends Product> {
    abstract render(product: P): ReactNode;
}
```

```
class ProductFieldRendererRegistry {  Show usages  diennguyenhuy
    renderFields(product: Product): ReactNode {  Show usages  diennguyenhuy
        return RENDERER_MAP[product.productType].render(product);
    }
}

export const productFieldRendererRegistry = new ProductFieldRendererRegistry();
```

Call sites on both the customer and manager screens reduce to a single expression:

```
{productFieldRendererRegistry.renderFields(product)}
```

No screen component knows which renderer it is invoking. Adding a new product type requires only a new renderer class and a new entry in RENDERER_MAP, no screen component is modified.

Solution 2: Form Section Registry

An abstract ProductFormSection<C, U> class defines the full contract for a product type's form behaviour across five abstract methods:

- populateFromProduct(product): pre-populates form state from an existing product for edit mode
- renderCreateFields(...): returns JSX for the creation form fields
- renderEditFields(...): returns JSX for the edit form fields
- buildCreatePayload(): constructs the typed creation request body
- buildUpdatePayload(): constructs the typed partial update request body

Each subtype provides a concrete ProductFormSection implementation. Because form sections hold local React state (controlled inputs), they are instantiated as React hooks rather than plain singletons. The registry is assembled per render via useAllFormSections().

```
export function useAllFormSections(): Record<ProductTypeName, ProductFormSection<any, any>> {
    return {
        Book: useBookFormSection(),
        CD: useCDFormSection(),
        DVD: useDVDFormSection(),
        Newspaper: useNewspaperFormSection()
    };
}
```

The ProductCreation and ProductUpdate components call useAllFormSections(), select the section for the currently chosen product type, and delegate all field rendering and payload construction to it. No form component contains any product-type-specific logic. The create and edit forms are identical in structure regardless of which product type is active.

This pattern is the direct frontend counterpart of the backend Product Type Dispatch registry. The structural parallel is intentional:

Concern	Backend	Frontend
Registry Key	Class<?> type	ProductTypeName enum
Registry mechanism	Map<Class<?>, ?> populated at startup	Object literal, compiled statically
Extension point	New ProductCreator / ProductUpdater / ProductMapping	New ProductFieldRenderer / ProductFormSection
OCP guarantee	New product type = new classes only	New product type = new classes + new map entry only

The frontend registry is simpler structurally because it uses a compiled object literal rather than a runtime-populated map, and TypeScript's mapped type { [K in ProductTypeName]: ... } enforces exhaustiveness at compile time. If a new ProductTypeName value is added

to the union but its renderer or form section is not added to the map, TypeScript raises a compile error immediately.

Trade-off: useAllFormSections() instantiates all sections on every render

Because form sections hold React state and must be instantiated as hooks, useAllFormSections() constructs all four form sections on every render of the create or edit form, including the three sections that are not currently active. This is a deliberate trade-off: the alternative of lazily instantiating only the active section would require conditional hook calls, which violates React's Rules of Hooks.

The cost is negligible at the current scale, four lightweight hook instances per render, but would need revisiting if the number of product types grew large or if individual form sections became significantly more expensive to instantiate. At that point the resolution would be to lift form section state into a reducer or context provider outside the registry, separating instantiation from selection.